

简易而严谨的人工智能讲义

前言

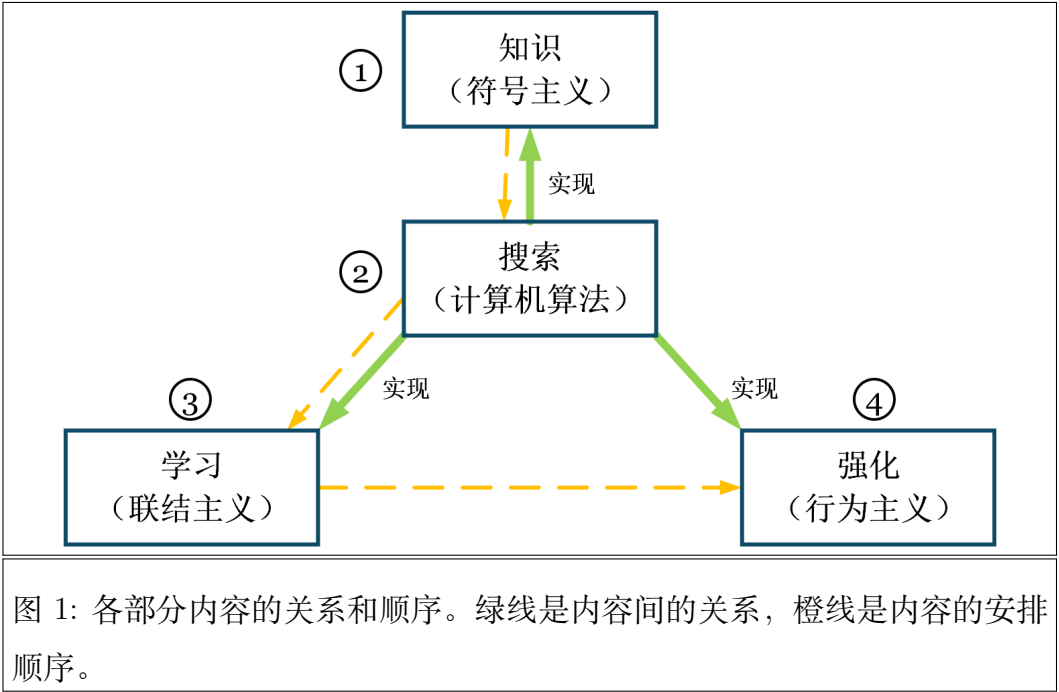
2014 年秋，我第一次为中国科学院大学的本科生和研究生开设人工智能课程。十年过去，选课人数不断增加，教材却始终未变——Russell & Norvig 的《Artificial Intelligence: A Modern Approach》。这本“千页圣经”将整座 AI 森林铺陈在学生面前，却鲜有人能一眼看出林中的路径。面面俱到地讲授，往往把一门本应扎实的硬课，讲成了空泛的概念堆砌。我于是动念写一本不一样的讲义：

一本简易到可轻松通读，又严谨到足以支撑透彻的原理理解的教材。

出于这样的初衷，本书的内容选择只有两句话：要么是在工业界广泛使用，要么是足以代表人类智慧的“某一处”跃迁。虽然这些内容都是 AI 发展史上的里程碑工作，但把知识碎片拼成体系，需要一条主线。本书把 AI 六十年的思潮压缩成四个关键词：知识、搜索、学习、强化。

- 知识：符号主义 AI 的核心，关注形式化的表示知识，并在形式语言上进行自动推理。
- 学习：联结主义 AI 的主战场，让机器从数据中自动提取规律并持续优化性能。
- 强化：行为主义 AI 的当代延续，通过与环境交互、最大化长期回报来获得行动策略。
- 搜索：上述三个部分的公共“实现底座”。绝大多数 AI 算法最终都归结为搜索的问题；可以说，搜索是连接思想与可执行代码的桥梁。

四者关系如图 1 所示，其中：绿线揭示概念耦合，橙线给出四个部分在本书中的呈现顺序，依次为：第一部分“知识”，第二部分“搜索”，第三部分“学习”，第四部分“强化”。先讲符号主义——给出知识表示与自



动推理的“纸面模型”。随即揭示：任一自动推理算法，归根结底都是状态空间里的路径搜索问题。于是，第二部分聚焦搜索，让读者完整体验“从问题定义到可执行代码”的第一次闭环。其后，第三部分学习与第四部分强化将再次使用第二部分讲授的各种搜索算法，完成从算法到场景的反向映射。一正一反，正是本讲义的刻意安排。

我们希望，这本《简易而严谨的人工智能讲义》能让你在翻页之间，体会到 AI 独有的乐趣：当符号、梯度与策略最终汇成一行可运行的代码，思想便照进现实。

愿它成为你理解 AI 的第一本书，也是你的一本 AI 随身指南。

罗平
2025 年 2 月 24 日

目录

第一部分 知识：符号主义	1
第一章 逻辑的基本概念	3
1.1 逻辑：定义、目的和研究内容	3
1.2 语义蕴含 (Entailment)	5
1.2.1 模型 (Model)	5
1.2.2 语义蕴含 (Entailment)	7
1.3 本章小结	9
第二章 命题逻辑 (Propositional Logic)	10
2.1 命题逻辑的语法	10
2.1.1 命题逻辑的基本元素	11
2.1.1.1 原子命题	11
2.1.1.2 连接词	11
2.1.2 命题逻辑的语法 (Syntax)	12
2.2 命题逻辑的语义和逻辑蕴含	13
2.2.1 命题逻辑的语义	13
2.2.2 命题逻辑的知识表示：实例	16
2.2.3 证明逻辑蕴含的自动算法	19
2.2.3.1 实例 1：抓小偷	20
2.2.3.2 实例 2：骑士与无赖之岛	21

目录	II
2.2.4 关于逻辑蕴含的更多性质	24
2.2.5 局限性和展望	27
2.3 命题逻辑的形式推演	27
2.3.1 第一套推演系统: 11 条规则	29
2.3.2 第二套推演系统: 归结原理	31
2.3.2.1 基本概念	31
2.3.2.2 将任意句子转换成合取范式	33
2.3.2.3 归结规则	34
2.3.2.4 归结原理的可靠性	37
2.3.2.5 归结原理的完备性	38
2.3.2.6 归结原理的用途与局限性	39
2.3.3 第三套推演系统: Modus Ponens 规则	40
2.3.3.1 Modus Ponens 规则的可靠性	43
2.3.3.2 Modus Ponens 规则的完备性	44
2.3.3.3 Modus Ponens 规则的局限性	45
2.4 命题逻辑的总结与局限性	45
第三章 一阶谓词逻辑	46
3.1 一阶谓词逻辑的语法	47
3.1.1 一阶谓词逻辑的基本元素	47
3.1.2 一阶谓词逻辑的语法	48
3.2 用一阶谓词逻辑表示知识	52
3.3 一阶谓词逻辑的蕴含推理	55
3.3.1 一阶谓词逻辑句子的模型	55
3.3.2 一阶谓词逻辑的蕴含推理	56
3.4 一阶谓词逻辑的形式推演	56
3.4.1 命题逻辑化 (Propositionalization)	56
3.4.2 一阶谓词逻辑的归结原理	57

目录	III
----	-----

3.4.2.1	一阶谓词逻辑的合取范式	57
3.4.2.2	合一算子 (Unification)	59
3.4.2.3	一阶谓词逻辑的归结原理	61
3.4.3	一阶谓词逻辑的 Modus Ponens 规则	66
3.5	Prolog 逻辑编程	67
3.5.1	Prolog 的基本原理	67
3.5.2	Prolog 程序的基本语法	67
3.5.3	Prolog 编程实例	71
3.5.4	Prolog 的局限性	72
3.6	定理自动证明的历史回顾	75
3.6.1	定理自动证明的背景	75
3.6.2	定理自动证明的研究现状	77
3.6.2.1	Logic Theorist 项目	77
3.6.2.2	吴文俊的工作	77
3.6.3	一阶谓词逻辑研究回顾	78
3.7	一阶谓词逻辑的总结	80

第二部分 搜索：AI 算法的实现基础	84
---------------------------	-----------

第四章 状态搜索	86
-----------------	-----------

4.1	问题定义	86
4.2	算法框架	88
4.2.1	树搜索 (Tree Search)	88
4.2.2	图搜索 (Graph Search)	90
4.3	搜索策略	92
4.3.1	无信息搜索 (Uninformed Search/Blind Search)	93
4.3.2	有信息搜索 (Informed Search/Heuristic Search)	97

4.4	状态搜索：归结原理的计算机实现	112
4.5	状态搜索的总结和局限性	113
第五章	最优化搜索	114
5.1	贪婪算法	114
5.1.1	爬山算法	115
5.1.2	梯度法	117
5.1.3	束搜索	118
5.2	元启发式算法	119
5.2.1	模拟退火算法	120
5.2.2	演化计算	121
5.3	最优化搜索的总结与局限性	125
第六章	对抗搜索	126
6.1	对抗搜索的问题定义	126
6.2	极小极大搜索算法	129
6.2.1	算法流程	130
6.2.2	算法性质	132
6.2.3	算法拓展	133
6.3	Alpha-Beta 剪枝	135
6.3.1	基本概念	137
6.3.2	算法流程	137
6.3.3	算法性质	141
6.4	对抗搜索的总结	143
第三部分	学习：联结主义	145
第七章	经典机器学习方法	149

7.1	决策树	149
7.1.1	决策树的基本假设	149
7.1.2	决策树的优化目标	151
7.1.2.1	信息熵	151
7.1.2.2	条件信息熵	152
7.1.2.3	决策树的优化目标	153
7.1.3	决策树的训练方法	154
7.1.4	决策树的预测	155
7.1.5	决策树的几何解释：在特征空间中划分超矩形	155
7.1.6	决策树的小结	156
7.2	回归树	157
7.2.1	回归树的基本假设	158
7.2.2	回归树的优化目标	158
7.2.3	回归树的训练方法	160
7.2.4	回归树的预测	160
7.2.5	回归树的小结	160
7.3	渐进梯度回归森林 GBRT	161
7.3.1	GBRT 的基本假设	161
7.3.2	GBRT 的优化目标	162
7.3.3	GBRT 的训练方法	162
7.3.4	GBRT 的预测	163
7.3.5	GBRT 的小结	163
7.4	逻辑回归 (Logistic Regression)	165
7.4.1	逻辑回归的基本假设	165
7.4.2	逻辑回归的优化目标	167
7.4.3	逻辑回归的训练方法	167
7.4.4	逻辑回归的预测	168

目录	VI
7.4.5 逻辑回归的正则化	169
7.4.6 逻辑回归与线性回归的比较	170
7.4.7 逻辑回归的小结	171
7.5 集成学习 (Ensemble)	172
7.5.1 自举汇聚法 (Bagging)	173
7.5.2 随机森林 (Random Forest)	173
7.5.3 提升法 (Boosting)	174
7.5.4 集成学习的小结	180
7.6 如何用机器学习方法解决实际问题	181
7.7 经典机器学习方法的总结	186
第八章 深度学习基础	188
8.1 前馈神经网络	188
8.1.1 前馈神经网络模型基本假设	189
8.1.1.1 神经元	189
8.1.1.2 全连接神经网络	189
8.1.2 神经网络的优化目标	193
8.1.2.1 分类任务	193
8.1.2.2 回归任务	194
8.1.3 全连接神经网络的训练方法	195
8.1.3.1 反向传播 (BP, Back Propagation) 算法	195
8.1.3.2 神经网络的训练技巧	198
8.1.4 全连接神经网络的预测	200
8.1.5 纪念 Frank Rosenblatt: 神经网络的曲折发展史	200
8.2 一般的反向传播算法	203
8.2.1 计算图	203
8.2.2 计算图上的梯度计算	204
8.2.2.1 链状路径上的梯度	204

8.2.2.2	具有多个孩子的节点的计算图上的梯度 . . .	205
8.2.2.3	任意计算图上的梯度计算	207
8.2.3	全连接神经网络的计算图以及梯度计算	209
8.2.4	反向传播算法的历史回顾和深度学习编程框架	211
8.2.5	神经网络模型的四要素	212
8.3	卷积神经网络	213
8.3.1	卷积神经网络的前向计算	214
8.3.1.1	卷积层	214
8.3.1.2	池化层	217
8.3.1.3	激活函数	218
8.3.2	卷积神经网络在计算机视觉中的应用	219
8.3.3	卷积神经网络研究工作的回顾	221
8.3.4	卷积神经网络的小结	223
8.4	循环神经网络	223
8.4.1	循环神经网络的前向计算	224
8.4.1.1	朴素的 f 函数	227
8.4.1.2	f 函数: 长短期记忆单元	228
8.4.2	循环神经网络的应用	229
8.4.2.1	词性标注和中文分词	229
8.4.2.2	机器翻译	231
8.4.3	循环神经网络小结	237
8.5	大语言模型	237
8.5.1	大语言模型的基本原理	238
8.5.2	大语言模型常用的 Transformer 网络结构	240
8.5.3	大语言模型中的幻觉问题和缓解方法	242
8.5.4	大语言模型小结	244

目录	VIII
第四部分 强化：行为主义	249
第九章 强化学习：基本问题和性质	253
9.1 强化学习问题中的专业术语	253
9.2 强化学习问题的形式化定义和基本性质	257
9.2.1 更多强化学习的基本概念和性质	257
9.2.2 强化学习问题的形式化定义	261
第十章 白盒环境模型下的强化学习	263
10.1 贝尔曼方程及其用途	263
10.2 白盒环境模型下获取最优策略的算法	267
10.3 贝尔曼最优性方程及其用途	268
10.4 连续状态的贝尔曼方程及其求解	269
10.5 小结：白盒环境模型下的强化学习	271
第十一章 灰盒环境模型下的强化学习问题及其各种变体	273
11.1 多臂老虎机问题	274
11.2 围棋	275
11.3 训练具有思考能力的大语言模型	276
11.4 小结	277
第十二章 灰盒环境模型下的无状态的在线强化学习问题	278
12.1 多臂老虎机在线学习算法	278
12.2 多臂老虎机问题的应用	280
第十三章 灰盒环境模型下有状态的在线强化学习问题	282
13.1 价值学习	283
13.2 策略学习	290
13.3 强化学习在现实中的应用：围棋	298

13.4 强化学习在现实中的应用：训练具有思考能力的大语言模型 . 309

第五部分 AI 的回顾与总结：苦涩的教训324

第十四章 AI 发展的历史回顾326

14.1 计算世界的双城记：效率与智能的相爱相杀 326

14.2 计算机科学的浪漫主义：人工智能里程碑回顾与流派交融 . . 329

第十五章 人工智能发展的“苦涩的教训”336

第一部分

知识：符号主义

符号主义也被称为逻辑主义，其起源可以追溯到 20 世纪 50 年代，是人工智能学科最早的流派之一。符号主义认为人类认知的基本单元是符号，其认知过程本质上是符号操作的过程。该学派进一步提出，由于人与计算机同属物理符号系统，因此认为人类智能可以通过计算机进行模拟。具体而言，人类在现实世界中通过理解和运用符号，遵循严格的逻辑规则从现有知识体系解决现实中存在的问题；而计算机系统则通过接收符号和问题输入，进而运用预设算法对已知符号进行处理，从而求解问题。

本部分会先介绍在人工智能学科中逻辑的定义和实现逻辑的重难点——知识表示和知识推理。接下来会分别介绍两种逻辑体系——命题逻辑和一阶谓词逻辑，展示在这两种逻辑体系下如何通过知识表示和知识推理来解决实际问题。然后会介绍 Prolog 编程语言，讲述计算机自动推理的实践过程。最后介绍定理自动证明的历史，带大家回顾符号主义在现实中的发展历程。

第一章 逻辑的基本概念

1.1 逻辑：定义、目的和研究内容

什么是逻辑？ 逻辑 (Logic) 并非陌生概念，其本质是研究论证规律的学科，核心在于揭示如何从已知前提推导出有效结论的方法论。对人类而言，逻辑思维是大脑通过推理分析解决问题的内在机制——小到判断“如果下雨，就要带伞”的生活场景，大到攻克数学难题的学术研究，这种思维模式始终贯穿其中。

从计算机视角来看，逻辑学与计算机算法又存在深刻的内在关联。具体讲，算法是解决一类问题的通用的方法，而逻辑研究的是关于证明、推理等一切思维活动的通用的算法。如果要用计算机来实现“逻辑”的功能，只需要有如下的输入和输出：

输入：

知识库：当前的问题假设，前提条件等

问题查询：想要解决的问题

输出：

答案：该查询在此知识库上的正确答案

试想，如果我们得到了这个通用问题求解器 (General Problem Solver)

的算法，就可以利用计算机自动求解任何问题，诸如黎曼猜想、哥德巴赫猜想、P/NP 问题等困扰人类的难题。因此，设计通用问题求解器是一个伟大的算法问题。

但是，设计出这样算法的难度很大。首先，计算机只能处理形式化的语言，也就是说，这个终极算法的输入，包括：知识库和问题查询，都需要用遵循某种语法的形式语言来表示。例如，我们需要把用自然语言表达的基本的数学公理表达为形式化的符号语言。这个过程，我们称为知识表示。随后，以形式化语言表述的知识库和查询为输入，我们需要一个自动化的算法，自动运行以得到最终的答案；这一步称为知识推理。

逻辑的研究内容 聚焦人工智能领域，我们在逻辑方面重点研究的是符号逻辑 (Symbolic Logic)。它通过符号系统来表达逻辑学中的概念及其关系，为分析与推理提供了形式化、精确化的方法，因此适用于实现知识表示和知识推理。

在任意逻辑系统的研究中，我们都需要解决以下的问题：

- 1) 逻辑的形式语言该怎么定义：不同的逻辑需要设计不同的形式语言，从简单到复杂，表达能力也在不断扩充；
- 2) 设计在该语言上的语义推理算法：定义形式语言的语义，并以语义蕴含 (Entailment) 为基础，实现“正确推理”的自动化算法；
- 3) 设计在语言上的形式推演的算法：不断优化算法的复杂度，提升效率；以及该算法正确性的严格证明。

接下来，我们会针对两个逻辑系统，即命题逻辑 (Propositional Logic) 和谓词逻辑 (Predicate Logic)，分别讲述这三个方面的内容。

1.2 语义蕴含 (Entailment)

在讲具体的逻辑系统之前，我们首先得定义清楚什么是所谓“正确的推理”，即逻辑蕴含的概念。“正确的推理”是指从前提必然导出结论的过程，想要更加充分的理解这一概念，我们可以通过构造可检验的情景来分析推理的正确性。以很多人都耳熟能详的三段论式推理为例，已知前提“所有人都会死”和“苏格拉底是人”，那么我们能够轻易推论出“苏格拉底会死”。这一推理之所以正确，是因为在所有可能满足前提的情景下，结论也必然成立。或者我们可以理解成，“苏格拉底会死”这一结论是能被已知前提**蕴含**的。

然而，上述的例子是基于自然语言的描述的，很难直接被计算机理解。想要在计算机上完成自动化的推理过程，我们就需要像逻辑学家一样，用模型 (Model) 来思考问题。

1.2.1 模型 (Model)

我们在这一小节中会介绍模型的定义及其相关衍生概念，并且使用例子来帮助读者理解模型的思考方式是如何与实际问题对应的。

模型是可以评估的结构化世界

简单来说，模型是对现实世界问题的数学抽象，象征着一种可能存在的世界情况，用于判断在这种“世界”的条件下，某件事情是否会真实发生。例如，气象学家使用大气模型来预测天气，虽然大气模型并不是真实的大气层，但通过预先设定明确的规则（如气压的概念、风的概念、气压与风的关系等已知气象学概念及规律）和参数（如温度、气压、空气湿度等），它能告诉我们“如果某条件满足，那么某现象必然发生”（例如：如果两个地方气压不同，就会产生风）。那么，根据不同的模型，我们可以对一件事情有不同的判断。例如，要判断明天是否会下雨，在“空气湿度很小，气温

高”的大气模型中不会成立，但在“空气湿度大，气温低”的模型中可能会成立。

逻辑体系是用于判断结论是否在某一个模型中成立的语言体系

要深入理解逻辑体系的概念，让我们先从逻辑体系中的基本构成元素——句子 (Sentence) 入手。句子是由符号按照特定规则构成的表达式，表达的内容为需要被判定真假的信息。

在一个逻辑体系中，每一个句子都需要根据语法 (Syntax) 来表达，语法定义了句子的表达结构和规则。例如：在代数中， $x + 2 > y$ 是一个句子，如果输入到计算机中，计算机也能明白这个句子想要表达“未知数 x 加上 2 大于未知数 y ”这个信息，而 $x2y > +$ 不是一个句子，因为它不符合数学表达式的规则，其信息也无法被计算机理解。

除了语法之外，语义 (Semantics) 也是逻辑体系的重要部分，它定义了这个句子是否能在某一种可能的情况下成立，即判断该句子在此情况下的真假。例如， $x + 2 > y$ 这个句子能在 $x = 1, y = 2$ 的情况下成立 (True)，但在 $x = 0, y = 3$ 的情况下不成立 (False)。

总结来看，逻辑体系是整合语法和语义的规则体系，使每个句子被合理表示，被正确判断，让知识表示和知识推理能准确的进行。

在明确了逻辑体系后，我们可以对模型和句子的关系有如下定义：

定义 1.2.1.

如果有一个句子 α 在模型 m 中为真，则称 m 满足 α ，或 m 是 α 的一个模型，并用符号 $M(\alpha)$ 表示所有满足 α 的模型的集合。

在实际的推理过程中，我们可以将模型视为一种情景，如果说模型 m 是句子 α 的一个模型，就说明我们能在该情景判断这个句子表达的信息是成立的。举个例子，令 α 为句子 $x + y = 4$ ，对于 α ， $x = 0, y = 4$ 是它的一个模型，但 $x = 1, y = 4$ 不是它的一个模型。同时， $x = 0, y = 4$ 只是能

让 $x + y = 4$ 成立的一种可能。 $x = 1, y = 3$ 也可以是 $x + y = 4$ 的模型, $x = 2, y = 2$ 也可以。我们把所有能使句子 α 成立的模型的集合记作 $M(\alpha)$, 在这个例子中可以理解为 $\{x, y | x + y = 4, x, y \in N\}$ 。

在了解完模型的概念和用法后, 我们可以正式定义语义蕴含的概念。

1.2.2 语义蕴含 (Entailment)

逻辑的语义蕴含实质是定义了一组逻辑句子与一个新句子之间的所谓的“正确推理”的关系。此时, 这组逻辑句子, 我们称为知识库 (Knowledge Base), 记作 KB , 它是一个包含了多个逻辑句子的集合; 这个新的句子, 记作 α 。那么, KB 与 α 之间的语义蕴含关系定义为:

定义 1.2.2.

KB 蕴含句子 α , 记为 $KB \models \alpha$, 当且仅当, 在所有 KB 为真的世界中, α 也为真。或者说, 对于任意的模型, 如果它使 KB 为真的同时使 α 也为真, 则此时 KB 蕴含 α 。

逻辑蕴含意味着一个事实可以从一个知识库中“正确推理”出来。我们通过以下两个例子来具体展示蕴含的定义。

例 1: $x + y = 4$ 和 $4 = x + y$

很容易可以看出, 因为能让 $x + y = 4$ 成立的 x, y 值一定也能让 $4 = x + y$, 因此 $x + y = 4$ 蕴含 $4 = x + y$ 。同理, 我们也可以说 $4 = x + y$ 蕴含 $x + y = 4$ 。

例 2: $x + y = 4$ 和 $4 \geq x + y$

在这个例子中, $x + y = 4$ 蕴含 $4 \geq x + y$, 但是 $4 \geq x + y$ 并不蕴含 $x + y = 4$ 。

对于满足蕴含条件的推理, 我们认为它是正确的推理。换句话说, 蕴含定义了句子间的正确推理关系。在此定义下, 我们有以下定理:

定理 1.2.1.

$$KB \models \alpha \text{ 当且仅当 } M(KB) \subseteq M(\alpha).$$
定理 1.2.1 的证明.

先证 $\Rightarrow$:

$\forall m \in M(KB)$, 因为 $KB \models \alpha$, 可推出 m 也使 α 为真, 所以 $m \in M(\alpha)$, 所以 $M(KB) \subseteq M(\alpha)$.

再证 $\Leftarrow$:

$\forall m \in M(KB)$, $m \in M(\alpha)$, 对于所有 m 来说, 使得 KB 为真, α 也为真, 根据定义即 $KB \models \alpha$.

证毕。

□

对于定理 1.2.1, 一个直观表示如图 1.1 所示。

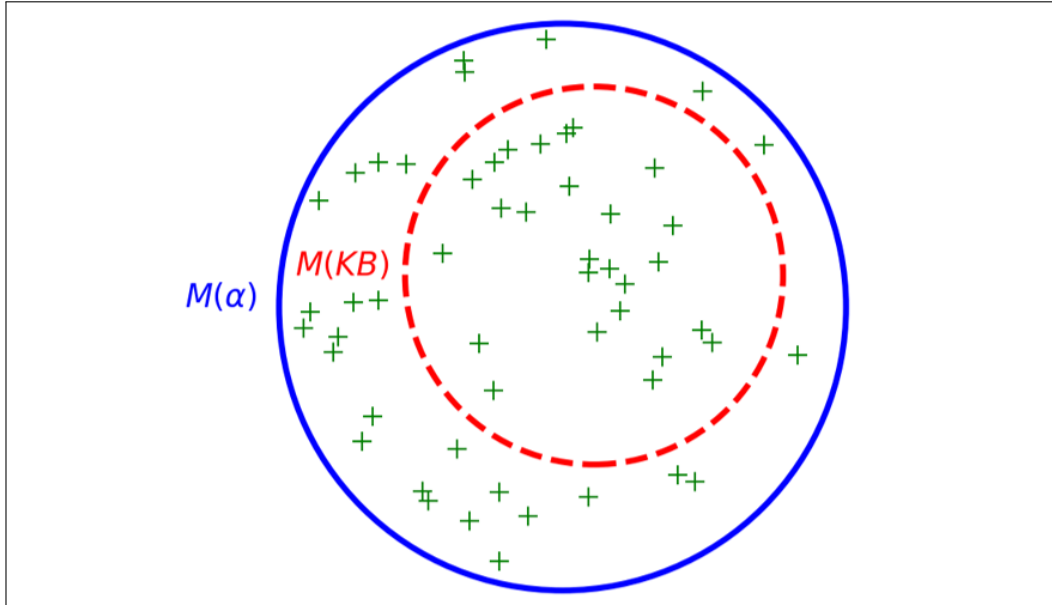


图 1.1: 定理 1.2.1 的图形表示。 $KB \models \alpha$ 直观理解就是: 在使 KB 为真的世界 (worlds) 里面 α 也全都要为真。

定理1.2.1表明了逻辑蕴含的数学含义。在后续的内容中，我们会使用这个定理直接验证逻辑蕴含关系是否成立。

1.3 本章小结

至此，我们已经介绍了人工智能学科符号主义视角下逻辑的内涵以及逻辑要研究的内容，同时也介绍了什么是正确的推断，并给出了如何用数学符号来表示语义蕴含。在接下来的章节中，我们会带大家学习具体的逻辑系统，了解它们的形式语言以及自动推理的方法。

第二章 命题逻辑 (Propositional Logic)

学习完了什么是正确的推理，我们将开始学习最基础的逻辑系统——命题逻辑。这里，命题 (Proposition)，是一个要么为真，要么为假的陈述句，不考虑模棱两可的表述。命题逻辑通过形式化的符号系统，研究命题之间的逻辑关系，其核心作用在于自动验证推理的正确性。同时，它也是形式化推理的起点，为后面更加复杂的逻辑系统奠定基础。

在之前的1.1节中我们提到，每个具体的逻辑系统都需要解决如下的三个问题：

- 1) 该逻辑的形式语言该怎么定义；
- 2) 设计在此语言上的语义推理算法；
- 3) 设计在此语言上的形式推演的算法。

在本节中，我们将按照这个顺序，介绍命题逻辑的具体内容。首先是命题逻辑的形式语言的定义，即命题逻辑的语法。

2.1 命题逻辑的语法

命题逻辑的核心在于通过原子命题和连接词构建复合命题，并研究其真值关系。首先我们会先介绍命题逻辑中的基本元素，即原子命题和连接

词的概念，然后总结命题逻辑的语法规则，以帮助读者了解命题的结构。

2.1.1 命题逻辑的基本元素

2.1.1.1 原子命题

原子命题 (Atomic Propositions) 是最小命题，指无法被进一步分解的简单陈述句，其真值为“真” (True) 或“假” (False)，且不随时间或语境变化。例如，“2025 年 10 月 10 日北京下雨”是一个原子命题，若实际下雨则其值为真，否则为假；但“今天是星期一”则不属于原子命题，因为其真值会随日期改变（这类命题被称为流命题 (Fluent)）。

原子命题通常用大写字母（如 P,Q,R）表示，以便在形式化系统中处理。

2.1.1.2 连接词

连接词的作用是将命题与命题连接起来，形成一个复合命题 (Complex Propositions)，被连接词连接的命题可以是一个简单的原子命题，也可以是一个复合命题。在任意逻辑系统中，通常有如下的连接词：

表 2.1: 逻辑系统中的连接词

符号	英文名称	中文名称	用法示例	简要说明
$\neg$	Negation	否	$\neg P$	非 P
$\wedge$	Conjunction	合取	$P \wedge Q$	P 且 Q
$\vee$	Disjunction	析取	$P \vee Q$	P 或 Q
$\Rightarrow$	Implication	条件	$P \Rightarrow Q$	若 P 则 Q
$\Leftrightarrow$	Biconditional	双条件	$P \Leftrightarrow Q$	P 当且仅当 Q

前文提到过，原子命题无法被进一步拆分成更小的命题，比如“今天下雨”这个命题，只留下“今天”或是“下雨”都无法构成一个可以被判断真

假的句子。然而，如果命题是“今天和明天都会下雨”，我们可以把它拆分成“今天下雨”和“明天下雨”这两个原子命题，那么原命题就不属于原子命题。实际上，我们可以把它看成用连接词 $\wedge$ 连接的复合句子。

连接词能扩展命题逻辑的表达能力，通过连接词，我们能更加灵活地将自然语境下的命题用符号来表达，为后续的自动化推理奠定基础。

2.1.2 命题逻辑的语法 (Syntax)

在了解了命题逻辑中所有基本元素的概念后，我们可以开始正式学习命题逻辑的语法。为了精准严谨地描述语法规则，我们采用 BNF 范式 (Backus-Naur Form) 来定义命题逻辑的语法。BNF 范式是一个形式化的语法体系，优点是可以消除自然语言的二义性，提供严格的形式化描述。

基于 BNF 范式，命题逻辑的语法结构如下所示：

命题句子 $\rightarrow$ 原子命题句子 | 复合命题句子
原子命题句子 $\rightarrow$ True | False | P | Q | $R \dots$
复合命题句子 $\rightarrow$ (命题句子) | [命题句子]
| $\neg$ 命题句子
| 命题句子 $\wedge$ 命题句子
| 命题句子 $\vee$ 命题句子
| 命题句子 $\Rightarrow$ 命题句子
| 命题句子 $\Leftrightarrow$ 命题句子

这个定义的涵义是：

- 如果 S 是一个命题，那么 $\neg S$ 也是一个命题 (Negation).

- 如果 S_1 和 S_2 都是命题, 那么 $S_1 \wedge S_2$ 也是一个命题 (Conjunction).
- 如果 S_1 和 S_2 都是命题, 那么 $S_1 \vee S_2$ 也是一个命题 (Disjunction).
- 如果 S_1 和 S_2 都是命题, 那么 $S_1 \Rightarrow S_2$ 也是一个命题 (Implication).
- 如果 S_1 和 S_2 都是命题, 那么 $S_1 \Leftrightarrow S_2$ 也是一个命题 (Biconditional).

可以看到, $\neg$ 为一元连接词, 其余的都是二元连接词。除此之外, 连接词的运算顺序为: $\neg, \wedge, \vee, \Rightarrow, \Leftrightarrow$ 。

命题逻辑的语言需要遵循此严格的语法定义, 以便于后续的计算机处理。

接下来, 我们将介绍命题逻辑的句子的语义, 即在已知原子命题的真假赋值的情况下, 如何判断复杂命题的真假。

2.2 命题逻辑的语义和逻辑蕴含

2.2.1 命题逻辑的语义

命题逻辑中, 任何命题都只有两种取值, True 或者 False。其中, 我们可以直接判断原子命题的真假, 但对复合命题的取值需要进行分析。复合命题的取值会受到被连接词连接的命题 (可能是复合命题, 也可能是原子命题) 和连接词种类的影响。

在五种连接词的语义规则如下所示:

- $\neg$ 表示“否”的含义, 如果命题 P 为真, 那么 $\neg P$ 必定为假;
- $\wedge$ 表示“合取”, 只有当命题 P 和命题 Q 同时为真时, 复合命题 $P \wedge Q$ 才为真, 否则为假;
- $\vee$ 表示“析取”, 只有当命题 P 和命题 Q 同时为假时, 复合命题 $P \vee Q$ 才为假, 否则为真;

- $\Rightarrow$ 表示“条件”，只有当命题 P 为真而命题 Q 为假时，复合命题 $P \Rightarrow Q$ 才为假，否则为真；
- $\Leftrightarrow$ 表示“双条件”，只有当命题 P 和命题 Q 同时为真或同时为假时，复合命题 $P \Leftrightarrow Q$ 才为真，否则为假。

表2.2展示了在不同连接词情况下，已知命题的真值时，复杂句子的真值是什么。

表 2.2: 逻辑连接词的真值表

P	Q	$\neg P$	$P \wedge Q$	$P \vee Q$	$P \Rightarrow Q$	$P \Leftrightarrow Q$
False	False	True	False	False	True	True
False	True	True	False	True	True	False
True	False	False	False	True	False	False
True	True	False	True	True	True	True

需要强调的是，逻辑中的连接词 $\wedge, \vee$ 和集合运算中的 $\cap, \cup$ 的含义是完全不同的。不过，它们也有如下的联系：

$$\begin{aligned} M(P \wedge Q) &= M(P) \cap M(Q) \\ M(P \vee Q) &= M(P) \cup M(Q) \end{aligned} \quad (2.1)$$

从上式中可以看出，所有能同时使 P 和 Q 成立的模型的集合，是 P 所有模型的集合与 Q 所有模型的集合的交集。同样，所有能至少使 P 或 Q 其中一个命题成立的模型的集合，是 P 所有模型的集合与 Q 所有模型的集合的并集。

除此之外，蕴含符号 $\models$ 和形式逻辑连接词 $\Rightarrow$ 的区别也值得注意。 $KB \models \alpha$ 并不是一个合法的复杂句子，因为 $\models$ 不是合法的连接词（连接词只有五个）。 $\models$ 表达了两个句子之间的一种关系，而 $\Rightarrow$ 是用于知识表示的连接词。这是两个符号的本质区别。

句子的真值指派 (Truth Assignment)

给定一个命题逻辑的句子 α ，给 α 中的每个原子命题指定一个确定的真值，被指定的这个真值的组合称为一次真值指派。例如：对于句子 $A \vee B$ ， A, B 是句子中的所有的原子命题，把它们分别指派为 True 和 False，则 $A = \text{True}, B = \text{False}$ 是该句子的一次真值指派。

在命题逻辑中，真值指派的作用是为句子中的原子命题遍历所有可能的真值组合。在表2.2中，我们在前两列里对 P 和 Q 分别赋予真值，然后探索两个句子被不同连接词连接后的句子是否为真。这一过程也是构建真值表的过程。

显然的，如果一个句子中有 n 个原子命题，则它的所有的真值指派的个数为 2^n 。例如，对于命题 $A \vee (B \wedge C)$ 有三个原子命题 A, B, C ，每个原子命题都可以被指派为 True 或 False。因此，整个命题存在 8 种不同的真值指派组合，我们可以构建如下的真值表。

表 2.3: $A \vee (B \wedge C)$ 的真值表

A	B	C	$A \vee (B \wedge C)$
False	False	False	False
False	False	True	False
False	True	False	False
False	True	True	True
True	False	False	True
True	False	True	True
True	True	False	True
True	True	True	True

另外，如果在某个真值指派下，句子为真，则这个真值指派就是该句子的一个模型。表2.3中的第 4、5、6、7、8 行对应的真值组合都是该句子

$A \vee (B \wedge C)$ 的模型。因此，我们已经了解了如何通过枚举所有可能的真值指派来找出哪些模型能让句子成立。

逻辑等价 (Logical Equivalence)

接下来，我们介绍两个句子之间的逻辑等价关系。

定义 2.2.1.

两个句子 α, β 逻辑等价 (记作 $\alpha \equiv \beta$)，当且仅当 $\alpha \models \beta$ 且 $\beta \models \alpha$

显然的，我们有

$$\alpha \equiv \beta \quad \text{当且仅当} \quad M(\alpha) = M(\beta) \quad (2.2)$$

可以看出，如果两个句子逻辑等价，那么在任意的真值指派下，此两个句子的真假始终保持一致。我们在表2.4中展示了一些常用的逻辑等价的性质规律（可枚举所有的真值指派，并利用表2.2的真值表来证明）。

接下来，我们会举一个例子，展示用自然语言表达的人类知识如何表示为命题逻辑的语言。

2.2.2 命题逻辑的知识表示：实例

我们将通过一个警察抓罪犯的例子，来给大家展示如何将现实自然环境中的逻辑推理问题用命题逻辑的形式语言来表达。

推理情景——抓小偷：

某个珠宝店发生了盗窃案，经过调查，警察锁定了三个嫌疑人——小 A、小 B、小 C。真正的小偷肯定是他们三人里面的其中一个，然而这三个嫌疑人都不承认自己是犯人。警察推定哪一个真正的小偷在说谎，于是分别问这三个人案发情况。

小 A 说：“我不是小偷，我看到了，小偷是小 B。”

小 B 说：“我不是小偷，我看到了，小偷是小 C。”

表 2.4: 常用的逻辑等价性质

$\wedge$ 的交换律 (Commutativity of $\wedge$)	$(\alpha \wedge \beta) \equiv (\beta \wedge \alpha)$
$\vee$ 的交换律 (Commutativity of $\vee$)	$(\alpha \vee \beta) \equiv (\beta \vee \alpha)$
$\wedge$ 的结合律 (Associativity of $\wedge$)	$((\alpha \wedge \beta) \wedge \gamma) \equiv (\alpha \wedge (\beta \wedge \gamma))$
$\vee$ 的结合律 (Associativity of $\vee$)	$((\alpha \vee \beta) \vee \gamma) \equiv (\alpha \vee (\beta \vee \gamma))$
双重否定消除 (Double negation elimination)	$\neg(\neg\alpha) \equiv \alpha$
逆否命题 (Contraposition)	$(\alpha \Rightarrow \beta) \equiv (\neg\beta \Rightarrow \neg\alpha)$
蕴含消除 (Implication elimination)	$(\alpha \Rightarrow \beta) \equiv (\neg\alpha \vee \beta)$
双条件消除 (Biconditional elimination)	$(\alpha \Leftrightarrow \beta) \equiv ((\alpha \Rightarrow \beta) \wedge (\beta \Rightarrow \alpha))$
德摩根律 (De Morgan)	$\neg(\alpha \wedge \beta) \equiv (\neg\alpha \vee \neg\beta)$ $\neg(\alpha \vee \beta) \equiv (\neg\alpha \wedge \neg\beta)$
分配律 (Distributivity)	$(\alpha \wedge (\beta \vee \gamma)) \equiv ((\alpha \wedge \beta) \vee (\alpha \wedge \gamma))$ $(\alpha \vee (\beta \wedge \gamma)) \equiv ((\alpha \vee \beta) \wedge (\alpha \vee \gamma))$

小 C 说：“我不是小偷，虽然我什么都没看到，但是小偷肯定在另外两个嫌疑人之中。”

请判断：谁是小偷？

从这个情境中，我们能确定两个前提，首先是小偷会隐瞒自己的罪行，所以小偷会说谎，而无辜的人不会说谎，其次是这三个人里只有一个是小偷。以上两个前提一定为真，而三个人的陈述则真假难辨，因为小偷在三个人之中，三个人就都有说谎的可能，我们想要找到小偷，就需要分辨出谁在说谎。

分析完案例，我们可以看出，最小的命题就是某个人（小 A、小 B 或小 C）是小偷（根据第一个前提，小偷和说谎是等价的，最小命题也可以表示为某个人说谎），所以不妨设定 A 代表小 A 是小偷， B 代表小 B 是小

偷， C 代表小 C 是小偷，同时， $\neg A$ 代表小 A 没说谎， $\neg B$ 代表小 B 没说谎， $\neg C$ 代表小 C 没说谎，具体示例如下所示。

符号示例	
小 A 是小偷	A
小 A 没说谎	$\neg A$
小 B 是小偷	B
小 B 没说谎	$\neg B$
小 C 是小偷	C
小 C 没说谎	$\neg C$

明确了符号的含义，我们可以开始归纳案例中的其它知识。首先根据另一个前提，三个人只有一个人是小偷，另外两个人不是，可以看出 A 、 B 、 C 之中只有一个命题为真。假设小 A 是小偷，那么就可以用 $A \wedge \neg B \wedge \neg C$ 为真来表示三个人的身份。三个人都有成为那个小偷的可能，那么这个知识就可以用 $(A \wedge \neg B \wedge \neg C) \vee (\neg A \wedge B \wedge \neg C) \vee (\neg A \wedge \neg B \wedge C)$ 来表示。

然后看三个人的陈述，小 A 说小 B 是小偷，同时也在说自己不是小偷，如果他真的不是小偷，那么他表达的信息都成立，即 $\neg A$ 和 B 同时成立，然而如果他是小偷，那么他的话全是谎言， $\neg A$ 和 B 都不会成立。因此我们可以用 $\neg A \Leftrightarrow B$ 来表示小 A 的陈述。同理，小 B 的话可以用 $\neg B \Leftrightarrow C$ 表示，小 C 的话可以用 $\neg C \Leftrightarrow (A \vee B)$ 表示。

综上，以上分析结果可以总结出如下的知识库：

知识库	
只有一个人是小偷	$(A \wedge \neg B \wedge \neg C) \vee (\neg A \wedge B \wedge \neg C)$
	$\vee (\neg A \wedge \neg B \wedge C)$
小 A 说小 B 是小偷	$\neg A \Leftrightarrow B$
小 B 说小 C 是小偷	$\neg B \Leftrightarrow C$
小 C 说小偷是小 A 或小 B	$\neg C \Leftrightarrow (A \vee B)$

至此，我们已经可以把案例中的所有自然语言都表示为形式语言。然而，计算机又该如何通过自动推理来找出谁是小偷呢？这就是**逻辑蕴含**所负责的问题。

2.2.3 证明逻辑蕴含的自动算法

下面我们可以依据蕴含的定义，对于给定的知识库 KB 和某个句子 α ，设计一个自动的算法去验证 $KB \models \alpha$ 是否成立，这一步也就是让计算机实现知识推理的过程。

具体讲，我们可以遍历所有的可能的真值指派组合，检查是否存在一种真值指派能使得： KB 中所有句子为真，且 α 为假。如果存在，则 KB 不蕴含 α ，反之则说明 $KB \models \alpha$ 。由此可以得到 Algorithm 1 中的自动算法。

可以看到，这个算法的时间复杂度是 2^n ， n 是知识库中原子命题的个数。这个指数规模的算法，难以实际应用，只能适用于求解一些逻辑智力题。

自动求解逻辑谜题：实例

我们会通过实例，展示如何使用“检验逻辑蕴含”的算法进行一些逻辑谜题的自动求解。如前所述，这个过程通常包含两个步骤：

- 1) 知识表示：将题目中的已知条件和待检验的结论分别表示为命题逻辑的

Algorithm 1 验证 $KB \models \alpha$ 的算法**Require:** 知识库 KB , 句子 α **Ensure:** 布尔值

```

    计算  $KB$  中所有句子的原子命题总数为  $n$ 
    初始化  $M_0$  为  $2^n$  种可能的真值指派的集合
    for 每个真值指派  $m \in M_0$  do
        if  $m \in M(KB)$  且  $m \notin M(\alpha)$  then
            return False
        end if
    end for
    return True

```

知识库 KB 和 α ;2) 知识蕴含的推理: 使用2.2.3节中的算法检验 $KB \models \alpha$ 是否成立。**2.2.3.1 实例 1: 抓小偷**

让我们回到2.2.2中的“抓小偷”例子中来, 已经得到了知识库, 并且了解到有 A 、 B 、 C 三个原子命题, 因此我们可以构建一个真值表 (我们定义 F 为命题不成立, T 为命题成立, 具体内容在表2.5中展示)。

在表2.5中, 前三列分别为不同的真值指派, 后面四列则是已经总结出来的知识库 KB , 对于“谁是小偷”这个问题, 我们可以把它理解成要分别检测三个结论: 小 A 是小偷 (A)、小 B 是小偷 (B), 以及小 C 是小偷 (C), 这三个结论正好和前三列对应, 因此我们只需要在表中看能让 KB 中每一个句子成立的真值组合下, 哪一个结论也必定成立即可。在查表后可以看出, 只有第三行的真值组合能让 KB 中的句子全部成立, 此时 B 成立, A 、 C 不成立。所以 $KB \models B$, 可以根据所有已知条件推理出小 B 是小偷。

结合第2.2.2小节, 我们通过“抓小偷”这个例子, 分别展示了知识表示

表 2.5: “抓小偷” 案例中的真值表

A	B	C	$(A \wedge \neg B \wedge \neg C) \vee$			
			$\neg A \Leftrightarrow B$	$\neg B \Leftrightarrow C$	$\neg C \Leftrightarrow (A \vee B)$	$(\neg A \wedge B \wedge \neg C) \vee (\neg A \wedge \neg B \wedge C)$
F	F	F	F	F	F	F
F	F	T	F	T	T	T
F	T	F	T	T	T	T
F	T	T	T	F	F	F
T	F	F	T	F	T	T
T	F	T	T	T	F	F
T	T	F	F	T	T	F
T	T	T	F	F	F	F

和知识推理的过程。

下面我们会进入另一个逻辑谜题，来带大家整体梳理一遍自动求解的过程。

2.2.3.2 实例 2：骑士与无赖之岛

埃德加·阿伯克龙比是一位人类学家，他对说谎和说真话的逻辑和社会学特别感兴趣。有一天，他决定去参观骑士和无赖的岛，在那里，那些被称为骑士的人总是说真话，而无赖总是撒谎。此外，每个居民不是骑士就是无赖。

在他到达的那天，阿伯克龙比遇到了三个居民，我们称他们为 A , B 和 C 。他问 A : “你是骑士还是无赖?” A 回答了，但声音模糊，阿伯克龙比听不懂他在说什么。然后他问 B : “他说了什么?” B 回答说: “他说他是个无赖。” 这时， C 尖声说: “不要相信; 这是个谎言!”

请判断: C 是骑士还是无赖?

这个案例比较复杂，为便于读者能更好理解自动求解如何在这个案例中实现，我们先通过自然语言推理一下这个题。

对问题分析

- 1) A 或者是骑士 (只说真话)，或者是无赖 (只说假话)，二者必居其一。
- 2) 假如 A 是骑士，他不可能说自己是无赖，因为那不是事实。
- 3) 假如 A 是无赖，也同样不可能说自己是无赖，因为那是真话。
- 4) 所以，A 说的话不可能是“我是无赖”。¹
- 5) 因此，B 说的不是事实，他必然是无赖。
- 6) 因此，C 说的是事实。
- 7) 结论：C 是骑士。

以上过程是完全通过自然语言推理得到的答案。那么我们能不能使用刚刚学到的内容，先做知识表示，将自然语言转换为命题逻辑的形式语言，之后利用蕴含的定义去证明结论是否成立呢？答案当然是肯定的。

仿照“抓小偷”的例子，我们可以分别对 A、B、C 三人的身份和他们说的话进行如下知识表示：

知识表示	
第 i 个人是骑士：	A_i
第 i 个人是无赖：	$\neg A_i$
第 i 个人，说了一句话 P ：	$A_i \Leftrightarrow P$

之后对 B、C 两人说的话构建如下知识库：

¹A 可以是任何人，所以实际上到这里我们就知道这个岛上不会有任何人说自己是无赖。

当前知识库	
B 说: A 说的话是 A 是无赖:	$A_2 \Leftrightarrow (A_1 \Leftrightarrow \neg A_1)$
C 说: B 在撒谎:	$A_3 \Leftrightarrow \neg A_2$

接下来，我们遍历所有真值指派组合，构建如下真值表：

表 2.6: 推理问题 1 的真值表				
A_1	A_2	A_3	$A_2 \Leftrightarrow (A_1 \Leftrightarrow \neg A_1)$	$A_3 \Leftrightarrow \neg A_2$
F	F	F	T	F
F	F	T	T	T
F	T	F	F	T
F	T	T	F	F
T	F	F	T	F
T	F	T	T	T
T	T	F	F	T
T	T	T	F	F

可以看出，在能让知识库所有句子都为真的真值指派中，C 为骑士的结论都成立。根据蕴含的定义我们有 $KB \models A_3$ ，即 C 是骑士。

至此，我们完整梳理了一遍自动求解算法的流程，动手能力强的读者可以尝试将上述算法在计算机上实现。下面我们给出“骑士与无赖之岛”案例的两个情景延伸问题，读者可以尝试用自动的方法解决以下两个问题。

情景延伸 1

接下来，阿伯克龙比又遇到了两个居民，A 和 B。A 说：“我们两个都是无赖。”

请判断：A 和 B 分别是什么？

情景延伸 2

根据这个故事的另一个版本，阿伯克龙比没有问 A 他是骑士还是无赖 (因为他会提前知道他会得到什么答案)，而是问 A 三个人中有几个是无赖。 A 又一次含糊地回答，于是阿伯克龙比问 B A 说了什么。 B 说， A 说他们中正好有两个人是无赖。然后，像以前一样， C 声称 B 在撒谎。

请回答：现在能够判断 C 是无赖还是骑士

2.2.4 关于逻辑蕴含的更多性质

永真 (Validity) 和可满足 (Satisfiability)

最后，我们介绍关于逻辑蕴含的更多的性质。首先介绍什么是永真和可满足，这两个概念用于描述句子的属性。

永真

句子在所有真值指派下都为真时，被称为该句子是永真的。例如，对于下列几个句子，无论其中的命题 A 、 B 取值是真是假，它们都一定会为真 (具体证明过程可以参考表2.2自行完成)：

1. $A \vee \neg A$,
2. $A \Rightarrow A$
3. $(A \wedge (A \Rightarrow B)) \Rightarrow B$

我们可以把永真的句子看成无论在什么条件下，都一定会成立的事情。就像对于命题“这次考试要么及格，要么不及格”，无论你考试分数是多少，这句话都一定会成立。永真性的主要作用在于可以判断知识库和新句子是否有蕴含关系，这点我们会在定理2.2.1中详细展开。

可满足和不可满足

若存在一个真值指派，句子在这个指派中为真，那么它是可满足的 (Satisfiable)。例如，对于句子 $A \wedge B$ ，当 A 、 B 取值都是真的时候，这个句子一

定为真, 那么, $A \wedge B$ 是可满足的, 但不是永真的。值得一提的是, 任何原子命题一定是可满足的。

另外, 如果一个句子在任何真值指派中都不为真, 那么它是不可满足的 (Unsatisfiable)。例如, $A \wedge \neg A$, 无论 A 取真还是取假, 这个句子都一定不为真。不可满足是和永真完全相反的特点, 也是可以用于判断知识库和新句子是否存在蕴含关系的重要性质。

判断逻辑蕴含的更多方法

介绍完命题逻辑中句子的一些基本性质之后, 我们可以将逻辑蕴含的判断改写成更多的形式。

已知一个用命题逻辑表达的知识库 KB 后, 我们往往需要检验 KB 是否蕴含某个句子 α , 即判断 $KB \models \alpha$ 是否成立。下面给出一些 $KB \models \alpha$ 成立的等价条件。

定理 2.2.1.

$KB \models \alpha$ 当且仅当 $(KB \Rightarrow \alpha)$ 是永真的

其中, $(KB \Rightarrow \alpha)$ 表示的句子是 $(\alpha_1 \wedge \cdots \wedge \alpha_k \Rightarrow \alpha)$, 这里 $\alpha_1 \cdots \alpha_k$ 是 KB 中的所有句子。

定理 2.2.1 的证明.

先证 $\Rightarrow$:

因为 $KB \models \alpha$ 所以 $M(KB) \subseteq M(\alpha)$, $\forall m \in M(KB), m \in M(\alpha)$, 即 KB 为真, α 为真。

$KB \Rightarrow \alpha$ 永真。

再证 $\Leftarrow$:

1) 若 $M(KB) \neq \emptyset$ 则 $\forall m \in M(KB)$, 因为 $KB \Rightarrow \alpha$ 永真, 所以 $m \in M(\alpha)$, 可推出 $M(KB) \subseteq M(\alpha)$, 即 $KB \models \alpha$.

2) 若 $M(KB) = \emptyset$ 则不存在 m 能使 KB 为真, 又因为 $\emptyset \subseteq M(\alpha)$ 所以 $M(KB) \subseteq M(\alpha)$, 即 $KB \models \alpha$ 。

证毕。

□

定理2.2.1表明: 知识库 KB 蕴含 α 的充要条件是 $KB \Rightarrow \alpha$ 为永真式。除非存在模型, 能使 KB 为真, 却使 α 为假, 否则都可以说 KB 蕴含 α 。

定理 2.2.2.

$KB \models \alpha$ 当且仅当 $(KB \wedge \neg\alpha)$ 是不可满足的

定理2.2.2的证明.

先证 $\Rightarrow$:

假设 $KB \models \alpha$, 这意味着 $M(KB) \subseteq M(\alpha)$ 。换句话说, 任何使 KB 为真的模型也会使 α 为真。

如果 $KB \wedge \neg\alpha$ 是可满足的, 那么存在一个模型 m 使得 KB 和 $\neg\alpha$ 同时为真。这意味着 $m \notin M(\alpha)$ 且 $m \in M(KB)$, 与 $M(KB) \subseteq M(\alpha)$ 矛盾。因此, $KB \wedge \neg\alpha$ 一定是不可满足的。

再证 $\Leftarrow$:

假设 $(KB \wedge \neg\alpha)$ 是不可满足的。这意味着不存在任何模型 m 使得 KB 和 $\neg\alpha$ 同时为真。

因此, 任何使 KB 为真的模型 m 必须使 α 也为真, 即 $M(KB) \subseteq M(\alpha)$ 。

因此, $KB \models \alpha$ 。

综上所述, $KB \models \alpha$ 当且仅当 $(KB \wedge \neg\alpha)$ 是不可满足的。

证毕。

□

定理2.2.2表明：知识库 KB 蕴含 α 的另一个充要条件是任何模型都无法让 $KB \wedge \neg\alpha$ 为真。也就是说，如果有一个模型能够让 KB 和 $\neg\alpha$ 成立，也就说明该模型无法让 α 为真，此时的 KB 就无法蕴含 α 。

2.2.5 局限性和展望

回顾我们前面提到的任何逻辑系统都需要解决的三个问题，当前我们已经以命题逻辑为例，解决了其中的前两个，即：命题逻辑的形式语言的定义和该语言上的逻辑蕴含自动推理算法。

具体讲，我们利用逻辑蕴含的定义，可通过遍历真值指派的方法，自动的验证逻辑蕴含的正确与否。但是这种算法的时间复杂度是指数复杂度的。因此，有人想探索别的途径，试图通过符号运算，解决该问题。这就是逻辑系统需要解决的第三个问题，所谓的形式推演问题。

下面，我们就来介绍命题逻辑中的形式推演。

2.3 命题逻辑的形式推演

形式推演是符号化的机械推演过程。我们能通过一系列推演规则，从已知的结论中“推出”目标结论是否成立。形式推演和语义推理不同的地方就在于我们不用指派真值就可以完成自动化推演算法；也就是说，符号其实并不代表任何含义，只要前提（符号）符合推演规则，我们就能推演出对应的结论。因此，我们可以给出如下定义：

定义 2.3.1.

形式可推演性： A 是在命题逻辑中由 Σ 形式可推演（或形式可证明）的，记作

$$\Sigma \vdash A,$$

当且仅当 $\Sigma \vdash A$ 能由（有限次使用）命题逻辑的形式推演规则生成。

同时，对于任何形式推演算法，我们都希望它具有可靠性 (Soundness) 和完备性 (Completeness)，这样才能保证形式推演得出的结论和语义蕴含完全一致。

可靠性

对于一个形式推演系统，如果它从知识库中推导出的所有结论都能被知识库蕴含，就称这个推演体系具有可靠性，即：若 $KB \vdash \alpha$ ，则 $KB \models \alpha$ 。可靠性是一个极为重要的属性，它代表了通过这个推演体系推理出来的结论一定是正确的；而一个不可靠的推理体系会在推理过程中编造事实，可能会给出错误的结论。

完备性

对于一个形式推演体系，如果它能推导出所有被知识库蕴含的结论，就称这个推演体系具有完备性，即：若 $KB \models \alpha$ ，则 $KB \vdash \alpha$ 。这也是形式推演中的一个非常重要的属性，代表了这个推演体系可以推出所有正确的结论。一个不完备的推理系统可能会存在部分正确的结论无法被推理出来的问题。

总结一下，可靠性是说形式推演出的句子一定是正确的，就是可被逻辑蕴含的；完备性是说任意能被正确的蕴含的句子，都可以被这种算法形式推出。一个推演系统是否具有完备性和可靠性是判断一个推演体系能否使用的重要标准。如果把知识库比喻一群涉案人员和一些有关犯罪嫌疑人的线索的话，我们要找的结论比喻成藏在涉案人员中真正的罪犯，那么推

演体系就是找罪犯的流程。可靠性代表在推理过程中不会错误地得出一个无辜的人是罪犯的结论，而完备性代表不会漏下一个坏人。

在本章中，我们会介绍命题逻辑中的三种形式推演体系——11 条规则、归结原理和 Modus Ponens 规则，其中只有第二种和第三种适合作为自动化的推演算法，方便计算机实现。

2.3.1 第一套推演系统：11 条规则

这套推演系统一共有如下的 11 条推演规则²：

自反(Ref)	$A \vdash A$
增加前提 (+)	如果 $\Sigma \vdash A$, 则 $\Sigma, \Sigma' \vdash A$.
$\neg$ 消去($\neg-$)	如果 $\Sigma, \neg A \vdash B$, $\Sigma, \neg A \vdash \neg B$, 则 $\Sigma \vdash A$.
$\Rightarrow$ 消去($\Rightarrow -$)	如果 $\Sigma \vdash A \Rightarrow B$, $\Sigma \vdash A$, 则 $\Sigma \vdash B$.
$\Rightarrow$ 引入($\Rightarrow +$)	如果 $\Sigma, A \vdash B$, 则 $\Sigma \vdash A \Rightarrow B$.
$\wedge$ 消去($\wedge -$)	如果 $\Sigma \vdash A \wedge B$, 则 $\Sigma \vdash A$, $\Sigma \vdash B$.
$\wedge$ 引入($\wedge +$)	如果 $\Sigma \vdash A$, $\Sigma \vdash B$, 则 $\Sigma \vdash A \wedge B$.
$\vee$ 消去($\vee -$)	如果 $\Sigma, A \vdash C$, $\Sigma, B \vdash C$, 则 $\Sigma, A \vee B \vdash C$.
$\vee$ 引入($\vee +$)	如果 $\Sigma \vdash A$, 则 $\Sigma \vdash A \vee B$, $\Sigma \vdash B \vee A$.
$\Leftrightarrow$ 消去($\Leftrightarrow -$)	如果 $\Sigma \vdash A \Leftrightarrow B$, $\Sigma \vdash A$, 则 $\Sigma \vdash B$ 如果 $\Sigma \vdash A \Leftrightarrow B$, $\Sigma \vdash B$, 则 $\Sigma \vdash A$.
$\Leftrightarrow$ 引入($\Leftrightarrow +$)	如果 $\Sigma, A \vdash B$, $\Sigma, B \vdash A$, 则 $\Sigma \vdash A \Leftrightarrow B$.

使用形式推演体系的方法就是反复利用推演规则得到新的结论。结合定义 2.3.1 可知，当且仅当有限序列 $\Sigma_1 \vdash A_1, \dots, \Sigma_n \vdash A_n$ 中的每一项 $\Sigma_k \vdash A_k$ ($1 \leq k \leq n$) 都是使用某一形式推演规则生成，并且 $\Sigma_n \vdash A_n$ 就是 $\Sigma \vdash A$

²本节的内容来自于中国科学院数学研究所陆钟万先生编写的《面向计算机科学的数理逻辑》。

(即 $\Sigma_n = \Sigma, A_n = A$) 时, $\Sigma \vdash A$ 成立。

根据以上推论和 11 条规则, 读者可尝试证明如下结论:

$$(i) \ A \Rightarrow B, A \vdash B.$$

$$(ii) \ A \vdash A \Rightarrow A.$$

$$(iii) \ A \Rightarrow B, B \Rightarrow C \vdash A \Rightarrow C.$$

$$(iv) \ A \Rightarrow (B \Rightarrow C), A \Rightarrow B \vdash A \Rightarrow C.$$

选证 (iii):

$$(1) \ A \Rightarrow B, B \Rightarrow C, A \vdash A \Rightarrow B \quad (\text{由(自反), (增加前提)}).$$

$$(2) \ A \Rightarrow B, B \Rightarrow C, A \vdash A \quad (\text{由(自反), (增加前提)}).$$

$$(3) \ A \Rightarrow B, B \Rightarrow C, A \vdash B \quad (\text{由}(\Rightarrow -), (1), (2)).$$

$$(4) \ A \Rightarrow B, B \Rightarrow C, A \vdash B \Rightarrow C \quad (\text{由(自反), (增加前提)}).$$

$$(5) \ A \Rightarrow B, B \Rightarrow C, A \vdash C \quad (\text{由}(\Rightarrow -), (4), (3)).$$

$$(6) \ A \Rightarrow B, B \Rightarrow C \vdash A \Rightarrow C \quad (\text{由}(\Rightarrow +), (5)).$$

□

其余的证明留给读者。

从上述例子可以看出, 形式推演与2.2节中介绍的语义蕴含是完全不同的。在语义蕴含中, 我们必须考虑语义, 通过真值指派构建模型来判断知识库和句子是否存在正确的推理关系。而在形式推演过程中, 我们无需关注语义, 只需依照推演规则对知识库中的符号进行操作。

毫无疑问, 形式推演要更加简便。但如果我们想要说明某个形式推演系统是正确的, 就需要严格证明它的可靠性和完备性。不过由于此 11 条规则的推演系统包含太多推理规则, 证明过程繁琐, 我们在此略去。

值得注意的是, 这种推演系统需要不断使用十一条规则中的某一条, 来推出结论。但每次使用哪条规则, 并没有明确的方法。因此, 这种方法并不是机械化的过程, 难以写成一个算法。

2.3.2 第二套推演系统：归结原理

归结原理 (Resolution) 是一个只有一条推演规则的推演系统, 其核心思想是通过互补文字的消除推导新子句。相较于命题逻辑的 11 条规则系统, 归结原理更容易在计算机上实现, 它提供了一个机械化的形式推演算法。

在正式介绍归结原理之前, 我们需要了解一些概念。

2.3.2.1 基本概念

文字 (Literals)

文字是一个原子命题或是一个原子命题的否定。

在归结原理推演系统中, 文字是最基本的构成元素。文字包括正文字和负文字。原子命题是正文字, 而原子命题的否定是负文字。

例如, A 是正文字, $\neg B$ 是负文字。

互补文字 (Complementary Literals)

互补文字是两个互为否定的文字, 即同一个原子命题的正文字和负文字共同构成一组互补文字。

例如, A 和 $\neg A$ 为一组互补文字。

子句 (Clauses)

子句是用析取连接的文字，一般形式为：

$$\ell_1 \vee \ell_2 \vee \cdots \vee \ell_k$$

其中， ℓ_i 为文字 ($i = 1, 2, \cdots, k$)。

子句是归结原理的核心处理单元。从语义上讲，一个真值指派至少需要使得子句中的一个文字为真，则该子句为真。

合取范式 (Conjunctive Normal Form, CNF)

合取范式用合取连接的子句，一般形式为：

$$C_1 \wedge C_2 \wedge \cdots \wedge C_m$$

其中， C_i 为子句 ($i = 1, 2, \cdots, m$)。

在归结原理体系中，合取范式是所有命题的统一表现形式。从语义上讲，一个真值指派需要使得合取范式中的每个子句都为真，该合取范式才为真。

介绍完这几个重要的概念后，我们可以把归结原理的推演过程概括为两步：

- 首先，将知识库 KB 和待判断的句子 α 都转换为合取范式；
- 然后，不断使用归结规则来判断 α 是否能够通过 KB 推出。

具体的算法将在下文详细描述。

2.3.2.2 将任意句子转换成合取范式

把任何一个句子都转换为合取范式的关键是：使用逻辑等价的性质对句子进行变换。关于逻辑等价的详细内容，可以参见表2.4。下面我们会展示如何将句子 $\neg C \Leftrightarrow (A \vee B)$ 转换成合取范式。

- 1) 使用双条件消除规律消除 $\Leftrightarrow$ ：

$$(\neg C \Rightarrow (A \vee B)) \wedge ((A \vee B) \Rightarrow \neg C)$$

- 2) 使用蕴含消除规律消除 $\Rightarrow$ ：

$$(\neg \neg C \vee A \vee B) \wedge (\neg(A \vee B) \vee \neg C)$$

- 3) 使用德摩根律和双重否定消除规律，消除多余的 $\neg$ 并把 $\neg$ 挪到子句里面的文字上：

$$(C \vee A \vee B) \wedge ((\neg A \wedge \neg B) \vee \neg C)$$

- 4) 使用分配律，用 $\vee$ 把括号内为 $\wedge$ 的命题拆分：

$$(C \vee A \vee B) \wedge (\neg A \vee \neg C) \wedge (\neg B \vee \neg C)$$

因为转换成合取范式的每一步都是依据逻辑等价的性质，所以合取范式和原句子语义等价，它们在相同模型下的真假肯定是完全一致的。同时也容易发现，任何命题逻辑的句子都可以通过上述转换变成等价的合取范式。因此，可以设计算法自动完成这一过程。最坏情况下，该算法的时间复杂度仍然是指数级的。

相比于原句子，合取范式显然更加冗长，我们并不容易理解；不过对于推演系统来说，合取范式的规范形式反而便于计算机处理，具体过程我们会在归结规则中给大家介绍。

2.3.2.3 归结规则

归结规则是归结原理推演系统中唯一的一条规则，它的作用是通过两个已知的子句来推出一个新的子句。

两个子句的归结规则：

$$\frac{\ell_1 \vee \cdots \vee \ell_k, \quad m_1 \vee \cdots \vee m_n}{\ell_1 \vee \cdots \vee \ell_{i-1} \vee \ell_{i+1} \vee \cdots \vee \ell_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n} \quad (2.3)$$

其中 ℓ_i 和 m_j 是互补文字。横线上的是已知条件，横线下的是能推演出的结论。

上面的式子中，已知条件里的第一个子句中的 ℓ_i 和第二个子句中的 m_j 互为互补文字，在归结规则中能够被抵消掉，因此能让两个子句推演出一个新的子句。这个过程被称为归结。例如：

$$\frac{A \vee B, \quad \neg B}{A}$$

如果两个子句中有多对互补文字的存在，那么两个子句可能归结出很多个不同的新子句。同时，归结出来的新子句也可以和其他子句归结，得到更多新子句，直到无法归结出新子句为止。

在了解了归结的过程后，我们可以推出如下的定理：

定理 2.3.1.

若 $KB \not\models \emptyset$ ， $KB \vdash \alpha$ 当且仅当 $\{KB, \neg\alpha\} \vdash \emptyset$ 。其中 $\vdash$ 仅使用归结法则获得新子句。

证明留给读者。(提示： $\{\neg\alpha, \alpha\} \vdash \emptyset$ 。)

这里的符号 $\vdash$ 是只用归结规则进行形式推演的意思，与模型表达的语义蕴含 ($\models$) 有本质的不同。定理2.3.1的意思是，当 KB 无法使用归结原理推演得到空子句，即这个知识库是一致的，不会包含矛盾（例如同时包含 A

和 $\neg A$) 时, 那么 $KB \vdash \alpha$ 的充要条件就是 $\{KB, \neg\alpha\}$ 的合取范式能够在有限次使用归结规则后得到空子句。

因此, 我们可以总结出一个验证句子 α 是否能够被知识库 KB 推理出来的算法, 具体过程如下所示:

1) 构建合取范式:

将 KB 的所有子句与 $\neg\alpha$ 合取, 构建初始合取范式。

2) 反复迭代归结:

对合取范式中的子句使用归结规则, 生成新子句, 然后将原子句和新子句合并称新的子句集合。

3) 判断终止条件:

如果在某次归结中生成空子句 $\emptyset$, 则 $KB \vdash \alpha$ 成立;

如果无法进一步归结出新的子句且所有迭代过程中都未出现空子句, 则 $KB \vdash \alpha$ 不成立。

下面我们会展示如何通过归结原理来验证 $KB \vdash \alpha$ 。回到第2.2.2小节“抓小偷”的例子当中, 看看我们能不能通过归结原理从三个人的陈述中推理出小 B 是小偷。根据第2.3.2.2小节中的算法, 我们可以分别把小 A、小 B、小 C 的陈述表示为 $(A \vee B) \wedge (\neg B \vee \neg A)$ 、 $(B \vee C) \wedge (\neg C \vee \neg B)$ 和 $(C \vee A \vee B) \wedge (\neg A \vee \neg C) \wedge (\neg B \vee \neg C)$ 。

表 2.7: “抓小偷” 中的合取范式

KB	$(A \vee B) \wedge (\neg B \vee \neg A) \wedge (B \vee C) \wedge (\neg C \vee \neg B)$ $\wedge (C \vee A \vee B) \wedge (\neg A \vee \neg C) \wedge (\neg B \vee \neg C)$
α	B
$KB \wedge \neg\alpha$	$(A \vee B) \wedge (\neg B \vee \neg A) \wedge (B \vee C) \wedge (\neg C \vee \neg B)$ $\wedge (C \vee A \vee B) \wedge (\neg A \vee \neg C) \wedge (\neg B \vee \neg C) \wedge \neg B$

因此, 我们可以得到以下知识库 KB 和待推理结论 α , 并构建初始合

取范式 $KB \wedge \neg\alpha$ 。具体如表2.7所示。然后我们要使用定理2.3.1来判断小 B 是小偷，即证明 $KB \wedge \neg\alpha \vdash \emptyset$ ，具体过程如图2.1所示。

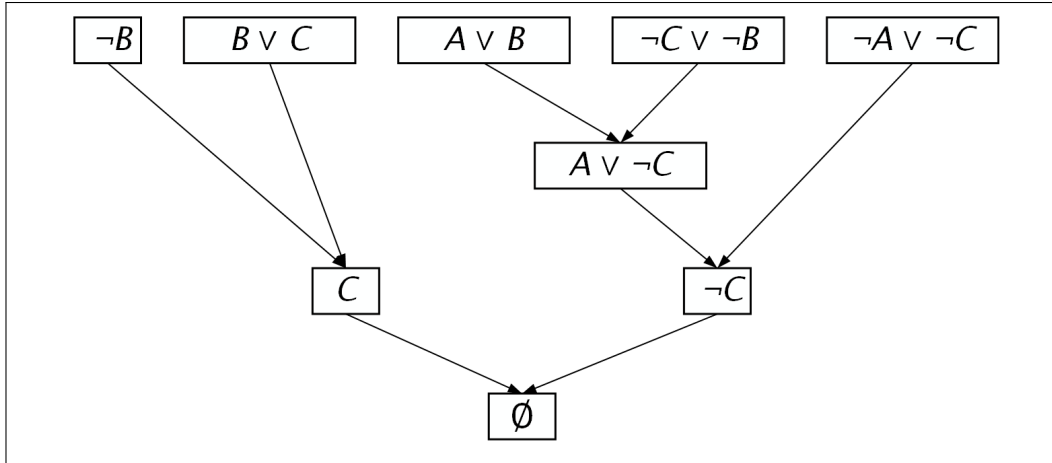


图 2.1: 在“抓小偷”案例中使用归结原理推理的过程。由于最终得到了空子句，因此根据定理2.3.1，我们可以断言 $KB \vdash \alpha$ 是正确的。

以上便是使用归结原理来推理逻辑问题的全过程。当然，这个例子中没有展示所有能够归结出来的子句，感兴趣的读者可以试着通过归结过程来证明小 B 是小偷，以及试着仿照上述过程来判断小 A 和小 C 的身份。

需要指出的是：对于包含有限数量的子句的知识库，整个归结的过程肯定会停止。也就是说，不可能永不停止的生成出新的子句。原因很简单：因为知识库的子句数量是有限的，则知识库中的原子命题数量也是有限的；那么，使用这些原子命题构建出的子句的数量也是有限的。

本节展示了归结规则以及如何通过归结原理来从知识库推出结论，下面我们会分别证明归结原理的可靠性和完备性。

2.3.2.4 归结原理的可靠性

想要证明归结原理是可靠的，只需证明每一次单独的归结步骤是可靠的，即证明

$$(l_1 \vee \cdots \vee l_k) \wedge (m_1 \vee \cdots \vee m_n)$$

$$\models (l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n),$$

其中 l_i 和 m_j 是互补文字。

证明.

不失一般性，记 A 为 $l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k$ ， B 为 $m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n$ ， C 为 l_i 。那么也就是证明：

$$(A \vee C) \wedge (B \vee \neg C) \models A \vee B. \quad (2.4)$$

表 2.8: 真值表

A	B	C	$A \vee C$	$\neg C$	$B \vee \neg C$	$(A \vee C) \wedge (B \vee \neg C)$	$A \vee B$
T	T	T	T	F	T	T	T
T	T	F	T	T	T	T	T
T	F	T	T	F	F	F	T
T	F	F	T	T	T	T	T
F	T	T	T	F	T	T	T
F	T	F	F	T	T	F	T
F	F	T	T	F	F	F	F
F	F	F	F	T	T	F	F

如表 2.8 所示，从真值表中可以看出，当 $(A \vee C) \wedge (B \vee \neg C)$ 为真时， $A \vee B$ 也为真。因此，我们证明了 $(A \vee C) \wedge (B \vee \neg C) \models A \vee B$ 。也就是说，我们证明了归结原理的可靠性。

证毕。

□

2.3.2.5 归结原理的完备性

要证明归结原理的完备性，也就是证明若 $KB \models \alpha$ ，则 $KB \vdash \alpha$ 。

设 $S = \{KB, \neg\alpha\}$ ， $RC(S)$ 是 S 经过所有可能的归结操作后，得到的所有子句集合（包含 S 中原有的所有子句）。那么 $KB \vdash \alpha$ 当且仅当 $\emptyset \in RC(S)$ 。此外，根据蕴含的性质， $KB \models \alpha$ 当且仅当 $(KB \wedge \neg\alpha)$ 是不可满足的。因此，我们只需证明：

$$\text{若 } (KB \wedge \neg\alpha) \text{ 是不可满足的, 则 } \emptyset \in RC(S) \quad (2.5)$$

考虑此命题的逆否命题，则待证结论变为了

$$\text{若 } \emptyset \notin RC(S), \text{ 则 } (KB \wedge \neg\alpha) \text{ 是可满足的} \quad (2.6)$$

证明.

针对 S 中的原子命题 $R_1, R_2, \dots, R_l$ ，我们构造如下的真值指派：

- 首先，因为 $RC(S)$ 中不包含空集，因此 $RC(S)$ 中不包含永假的子句。
- 从 $i = 1$ 到 l ，顺序的指派 $R_1, R_2, \dots, R_l$ 的真值：
 - 如果 $RC(S)$ 中包含一个子句，此子句包含 $\neg R_i$ ，且此子句的其它文字都已经被指派为 *False*（在之前的步骤中进行的）或不包含其它文字，则把 R_i 指派为 *False*；
 - 否则，把 R_i 指派为 *True*。
- 我们用反证法证明：这个真值指派使得 $RC(S)$ 中的子句都为真。假设，在此过程的第 i 步，我们这样来指派 R_i 使得某个子句 C 为 *False*，且假设这是首次出现 *False* 的子句；此时，子句 C 只能是如下两种形式之一：

$$\text{False} \vee \text{False} \vee \dots \vee \text{False} \vee R_i \quad (2.7)$$

或者

$$\text{False} \vee \text{False} \vee \cdots \vee \text{False} \vee \neg R_i \quad (2.8)$$

- 显然，如果 $RC(S)$ 中只包含以上两个子句之一，子句 C 是不可能在此真值指派中为 *False* 的。只有在此两个子句同时存在时，它们其中的一个才为 *False*。因此， $RC(S)$ 此时应同时包含了以上两个子句。
- 以上两个子句显然是满足归结条件的，也就是说，它们归结后的子句也应该在 $RC(S)$ 中；同时，该归结得到的新子句已经被指派为 *False* 了，这与我们之前的假设矛盾。

证毕。 □

通过以上步骤，我们成功给出了一个使 $RC(S)$ 为真的真值指派，这说明它是可满足的。也因此，我们证明了最初想要证明的结论，也就是归结原理是完备的。

2.3.2.6 归结原理的用途与局限性

归结原理，这一形式推演系统，是人类智慧的结晶。它仅包含一条形式推演的规则，就保证了它与语义蕴含的结论是完全一致的。即，在命题逻辑中，归结原理所形式推演出的结论，与语义蕴含的结论是完全一致的。因此，我们可以用归结原理的算法替换掉语义蕴含的自动化验证算法，来完成通用的问题求解。

通用问题求解的过程则变成：对于给定的题目，

- 1) 知识表示：将题目中的已知条件和结论表示为命题逻辑的知识库 KB 和 α ；
- 2) 归结原理的形式推演：即检验 $\{KB, \neg\alpha\} \vdash \emptyset$ 是否成立

然而，检验 $\{KB, \neg\alpha\} \vdash \emptyset$ 是否成立这个问题，实际是一个“状态搜索”问题，将在第4节中进行详细讲解。在最坏的情况下，这个算法的时间复杂度仍然是知识库规模的指数级别的。因此，我们仍然需要设计更高效的形式推演方法。

2.3.3 第三套推演系统：Modus Ponens 规则

Modus Ponens 规则也可以被称为肯定前件规则，是命题逻辑中基本的推理规则之一。它和归结原理都是以文字和子句为主要构成单位，但是为了减少时间复杂度，Modus Ponens 规则对知识库中的子句形式做出了一些限制，因此我们需要首先介绍一个概念：确定子句 (Definite Clause)。

基本概念

确定子句

确定子句是有且只有一个正文字的子句

显然，确定子句只有如下两种形式：

1. 一个正文字，比如 A 。
2. (多个正文字的合取) $\Rightarrow$ 正文字，比如 $C \wedge D \Rightarrow B$ (等价于 $\neg C \vee \neg D \vee B$)。

Modus Ponens 规则

和归结原理系统类似，Modus Ponens 规则也只有一条推出新子句的规则，即：

$$\frac{a_1, \dots, a_n, \quad a_1 \wedge \dots \wedge a_n \Rightarrow \beta}{\beta} \quad (2.9)$$

这里, $a_1, \dots, a_n$ 都是正文字。

事实上, Modus Ponens 规则相当于使用了 n 次归结原理。

当 KB 中的子句都是确定子句, 且 α 是一个正文字, 我们则可以反复使用 Modus Ponens 规则, 去验证 $KB \vdash \alpha$ 是否成立。这里的推出符号 $\vdash$ 指的是仅使用 Modus Ponens 规则。这一验证过程的计算机实现有两种方法: 前向推理 (Forward Chaining) 和后向推理 (Backward Chaining)。

前向推理 (Forward Chaining) 和后向推理 (Backward Chaining)

前向推理是从文字和子句开始, 逐步推出结论, 被称为数据驱动型推演; 后向推理则是从结论逆推, 搜寻是否有能满足结论的子句, 称为目标驱动型推演。在 Modus Ponens 规则中, 前向推理是我们先从 KB 中尽可能地去推出结论, 然后看看能不能在这些结论中找到 α , 能找到就说明 $KB \vdash \alpha$ 成立, 反之则不成立。而后向推理则是我们先假设 α 成立, 然后在 KB 中寻找能让 α 成立的条件是否都存在, 如果存在则假设成立, $KB \vdash \alpha$ 成立, 反之则不成立。

下面我们会通过一个例子来展示 Modus Ponens 规则的推理过程。

已知一个知识库由如下确定子句构成, 能否根据该知识库推理出 Q ?

- $P \Rightarrow Q, L \wedge M \Rightarrow P, A \wedge P \Rightarrow L, B \wedge L \Rightarrow M, A \wedge B \Rightarrow L$
- A, B

在这个例子中, 形式为 “(多个正文字的合取) $\Rightarrow$ 正文字” 的确定子句要多于形式为正文字的确定子句, 但已知的正文字可以通过 Modus Ponens 规则得到新的正文字, 新的正文字又可以作为其他推论的已知条件。因此, 我们可以把所有形如 “(多个正文字的合取) $\Rightarrow$ 正文字” 的确定子句提取出

来，然后遍历知识库中形式为正文字的确子句，看能否满足生成新正文字的条件；将新生成的正文字继续遍历，直到推出结论或是没有正文字可以遍历。这就是前向推理的算法实现过程，具体步骤如下所示：

第一步：列出推理条件	第二步：前提 A 被满足
$P \Rightarrow Q$ 1 $L \wedge M \Rightarrow P$ 2 $A \wedge P \Rightarrow L$ 2 $B \wedge L \Rightarrow M$ 2 $A \wedge B \Rightarrow L$ 2	$P \Rightarrow Q$ 1 $L \wedge M \Rightarrow P$ 2 $A \wedge P \Rightarrow L$ 1 $B \wedge L \Rightarrow M$ 2 $A \wedge B \Rightarrow L$ 1
初始条件：已知 A、B 成立	结论：暂无
第三步：前提 B 被满足	第四步：前提 L 被满足
$P \Rightarrow Q$ 1 $L \wedge M \Rightarrow P$ 2 $A \wedge P \Rightarrow L$ 1 $B \wedge L \Rightarrow M$ 1 $A \wedge B \Rightarrow L$ 0	$P \Rightarrow Q$ 1 $L \wedge M \Rightarrow P$ 1 $A \wedge P \Rightarrow L$ 1 $B \wedge L \Rightarrow M$ 0 $A \wedge B \Rightarrow L$ 0
结论：L 成立，纳入知识库	结论：M 成立，纳入知识库

第五步：前提 M 被满足	第六步：前提 P 被满足
$P \Rightarrow Q$ 1	$P \Rightarrow Q$ 0
$L \wedge M \Rightarrow P$ 0	$L \wedge M \Rightarrow P$ 0
$A \wedge P \Rightarrow L$ 1	$A \wedge P \Rightarrow L$ 0
$B \wedge L \Rightarrow M$ 0	$B \wedge L \Rightarrow M$ 0
$A \wedge B \Rightarrow L$ 0	$A \wedge B \Rightarrow L$ 0
结论：P 成立，纳入知识库	结论：Q, L 成立

注：第二列代表了该子句前提中还没有被满足的正文字的数量。当数量为 0 时，新的正文字成立，纳入知识库。

以上是使用前向推理解决这个问题的过程。后向推理的过程与之相反，先判断 Q 的成立前提 P 是否成立，然后看 P 的成立条件 L 和 M 是否成立，按照这种方式一步一步的往前找条件。感兴趣的读者可以尝试用后向推理来解决这个问题。

在实际情况中，后向推理往往比前向推理更为高效，原因是：前向推理通常会产生大量与结论无关的其它结论。不管是前向推理还是后向推理，它们的时间复杂度都与知识库中的原子命题的数量线性相关，因此，都是一个高效的算法。

2.3.3.1 Modus Ponens 规则的可靠性

证明 Modus Ponens 规则是可靠的，即证明：

$$a_1 \wedge \cdots \wedge a_n \wedge (a_1 \wedge \cdots \wedge a_n \Rightarrow \beta) \models \beta.$$

证明.

不失一般性，记 $A = a_1 \wedge \cdots \wedge a_n$ ，那么也就是证明

$$A \wedge (A \Rightarrow \beta) \models \beta.$$

上式可以用真值表来证明，过程略。

证毕

□

2.3.3.2 Modus Ponens 规则的完备性

证明 Modus Ponens 规则是完备的，即证：若 $KB \models \alpha$ ，则 $KB \vdash \alpha$ 。这里， KB 中仅包含确定子句， $\vdash$ 仅使用 Modus Ponens 规则，且 α 是一个正文字。

证明.

$RC(KB)$ 是 KB 里面的子句以及所有可以根据 *Modus Ponens* 规则从 KB 推演出来的子句的集合。

1) 构造如下的真值指派 m ：对于任意的正文字 a ， a 指派为 *True* 当且仅当 $a \in RC(KB)$ 。

2) 接下来，证明：在 m 下， KB 为真，也就是反证：若此时 KB 为 *False*，那么必存在一个确定子句，在 m 下为 *False*。

i) 若该子句为 $a_1 \wedge \cdots \wedge a_k \Rightarrow b$ ，也就是说，在 m 中， $a_1, \cdots, a_k$ 均为 *True*，且 b 为 *False*。

根据 1) 中的定义， $a_i \in RC(KB)$ ($i = 1, \cdots, k$)。

又根据 *Modus Ponens* 规则， $b \in RC(KB)$ 。

根据 1) 中的定义，在 m 中， b 为 *True*。推出矛盾。

ii) 若该子句为 b ，在 m 下为 *False*，则 $b \notin RC(KB)$ ，矛盾。

3) 若 $KB \models \alpha$ ，根据蕴含的定义，在 m 中， α 为真；则根据 1) 中的定义， $\alpha \in RC(KB)$ 。

也就是说： $KB \vdash \alpha$ 。

证毕。

□

2.3.3.3 Modus Ponens 规则的局限性

Modus Ponens 规则的形式推演，既可靠又完备；同时，它的实现算法运行高效，在实际场景中可以广泛应用。不过，这个推理算法的高效，来源于对知识库句子形式的限制，即知识库的每个句子都有且只有一个正文字。这种限制，使得在某些情况下并不能完全表达领域知识。

2.4 命题逻辑的总结与局限性

在本章中，我们首先定义了命题逻辑的形式语言，并定义了命题逻辑的句子的语义。在此基础上，我们给出了在命题逻辑上验证语义蕴含的自动算法，并以两个逻辑谜题为例，展示了如何利用该算法进行自动问题求解。

与此同时，我们介绍了三套形式推演系统，并证明了它们的可靠性和完备性。也就是说，语义蕴含的自动算法完全可以被形式推演算法替换。

可以证明，在有限句子的知识库上，命题逻辑的形式推演算法总是要停止的。以归结原理为例，在有限句子的知识库上，原子命题的数量是有限的，则由这些原子命题能构造出来的子句也是有限的；因此，归结的过程不可能无限的生成新的子句。所以，命题逻辑是可判定 (Decidable) 的系统。

然而，命题逻辑的句子的表达能力是有限的。例如，我们有如下的结论：任意的双数都能被 2 整除。由于命题逻辑中没有变量的概念，我们只能枚举所有的双数，将这条结论表示为无数条命题逻辑的句子，即：2 能被 2 整除，4 能被 2 整除，6 能被 2 整除等。因此，我们需要建立一个具有更强表达能力的逻辑语言体系。

在下一章中，我们将介绍一阶谓词逻辑，它具有更强的表达能力：对于枚举所有双数这件事情，在一阶谓词逻辑中，只需要表达成一条句子；而且，它更贴近自然语言。

第三章 一阶谓词逻辑

一阶谓词逻辑 (First-Order Predicate Logic) 是命题逻辑的扩展，通过引入量词、变量、谓词和函数，在增强表达能力的同时也能够进行自动推理。

为什么要引入一阶谓词逻辑呢？虽然命题逻辑是一种非常严谨的逻辑系统，避免自然语言中的模糊性，但是它只能判断一个个的事实在模型中的真假，无法归纳各种事实之间的关联。在需要描述一类对象的性质时，命题逻辑缺乏能简明扼要的表达能力，只能对该类别中的每个对象进行穷举说明。因此我们需要一个既能避免语义模糊，又能以简洁的句子表达更多信息的语言体系。

相比于命题逻辑，一阶谓词逻辑更加贴近自然语言。在自然语言中，所有可以被名词或名词短语指称的实体都可以视为对象 (Object)，而对象与对象之间的交互，被统称为关系 (Relation)，而那些只要给定输入就能得到唯一的对应结果的映射被称为函数 (Function)。举些例子：

- 对象：在现实生活中，很多东西都可以被称为对象，无论是具体事物，像是房子，车子，信息，空气……，或是抽象的概念像颜色，时光，爱情，战争……。
- 关系：根据涉及到的对象的个数分为一元关系和多元关系。一元关系，例如：红色（车子是红色的），美好（爱情是美好的），残酷（战争是残酷的）……；多元关系，例如：……比……大（房子比车子大）。

- 函数：对象之间的映射。例如：当老张是张三的爸爸，如果输入是“张三”，*FatherOf()* 这个函数就会输出“老张”。

接下来，我们依然按照逻辑系统需要解决的三个问题来逐步介绍一阶谓词逻辑，首先是一阶谓词逻辑的语法。

3.1 一阶谓词逻辑的语法

作为命题逻辑的扩展，一阶谓词逻辑在命题逻辑的语法基础上，引入了新的元素和语法规则，从而丰富了该逻辑体系的知识表示能力。我们在本节中会依次介绍一阶谓词逻辑中的各种基本元素和句法结构。

3.1.1 一阶谓词逻辑的基本元素

一阶谓词逻辑中的基本元素包括常量、变量、函数、谓词、连接词、量词和等号。

常量 (Constants)

常量是用来指称特定对象的符号，一般用大写字母来表示。

每个常量符号在具体模型中都被解释为讨论域中的一个固定对象。

变量 (Variables)

变量是用来表示任意对象的符号，一般用小写字母来表示。

换言之，变量并不直接对应某个固定对象，而是一个占位符，在逻辑推理过程中可以取论域中的任意个体。

函数 (Functions)

函数是指称对象与对象之间的映射关系的符号。一般使用首字母大写的单词或一串字符来表示。

谓词 (Predicates)

谓词是一种特殊的函数，输出只可能为 *True* 或者 *False*，和命题逻辑中的原子命题相对应。

连接词 (Connectives)

连接词用于连接不同句子，与命题逻辑中的连接词相同。

量词 (Quantifiers)

量词包括全称量词 $\forall$ 和存在量词 $\exists$ ，对变量起修饰作用。

假设变量为任意一个中国人，就可以用 $\forall z$ 来指代；如果我们想要表示存在一个中国人，我们就可以用 $\exists z$ 来表示。

等号 (Equality)

等号代表了两件事物是完全相同的。

一阶谓词逻辑中的等号不同于数学中的等号，其具体语义会在第3.3.1节中详细描述。

相较于命题逻辑，一阶谓词逻辑的符号体系更为丰富，而多样的基本元素也会带来复杂的语法结构。下一小节我们会为大家详细介绍如何将这些基本元素组合成符合语法的句子。

3.1.2 一阶谓词逻辑的语法

一阶谓词逻辑和命题逻辑类似，都是通过句子来表达信息，而且句子的结构也必须符合一些语法规则。不过，一阶谓词逻辑的语法结构包含了更多种组成元素，其 BNF 范式如下所示：

句子 $\rightarrow$ 原子句子 | 复杂句子
 原子句子 $\rightarrow$ 谓词($项_1, \dots, 项_n$) | $项_1 = 项_2$
 复杂句子 $\rightarrow$ (句子) | [句子]
 | $\neg$ 句子
 | 句子 $\wedge$ 句子
 | 句子 $\vee$ 句子
 | 句子 $\Rightarrow$ 句子
 | 句子 $\Leftrightarrow$ 句子
 | 量词 变量, $\dots$, 变量 句子
 项 $\rightarrow$ 函数($项_1, \dots, 项_n$)
 | 常量 | 变量
 量词 $\rightarrow \forall | \exists$
 常量 $\rightarrow A | Z | LZ | \dots$
 变量 $\rightarrow a | c | z | \dots$
 谓词 $\rightarrow True | False | Parent | After | \dots$
 函数 $\rightarrow Fatherof | Motherof | \dots$

和命题逻辑相比，一阶谓词逻辑借由项 (Term) 和谓词的概念，能让句子精确表达个体之间的关系或属性；并且引入量词，让复杂句子的表达能力也得到提升。下面我们会具体介绍一阶谓词逻辑中的句子语法。

原子句子 (Atomic Sentence)

原子句子是由一个谓词，或者一个等号连接两个项组成的句子。我们可以把原子句子和命题逻辑中的原子命题对应起来，它们都是可以被判断真假的陈述句，而且无法被分解成更简单的句子。

不过从句子的语法上看，原子句子比原子命题复杂许多。原子句子分两种：一个以若干个项为输入的谓词的形式和一个等号连接两个项的形式。这里的项可以是函数、常量或变量。例如， $Fatherof(z)$ 不是原子句子，它是一个函数。 $Parent(LZ, z)$ 是一个原子句子， $Parent(Fatherof(Z_1), Z_3)$ 也是原子句子。

用等号连接两个项，即 $term_1 = term_2$ ，也是一个原子句子。例如：当 LZ 是常量， $Fatherof(z)$ 是函数时， $LZ = Fatherof(z)$ 是一个原子句子。

复杂句子 (Complex Sentence)

复杂句子是使用连接词连接的原子句子，连接词的用法和命题逻辑中的连接词用法一致，设 S, S_1, S_2 为一个句子（原子句子或复杂句子），那么下面五个句子也一定是复杂句子：

$$\neg S, \quad S_1 \wedge S_2, \quad S_1 \vee S_2, \quad S_1 \Rightarrow S_2, \quad S_1 \Leftrightarrow S_2$$

以下是几个复杂句子的示例：

$$Sibling(Z_1, Z_2) \Rightarrow Sibling(Z_2, Z_1)$$

$$>(1, 2) \vee \leq(1, 2)$$

$$>(1, 2) \wedge \neg >(1, 2)$$

(注： $>$ 、 $\leq$ 是谓词)

量词

量词是约束变量范围的词语, 可以通过修饰变量来改变句子的含义, 整个句子也会因此成为一个全新的复杂句子。

下面我们会分别介绍两种量词修饰变量后对句子产生的影响。为了更好地说明两种量词在语义上的区别, 我们会暂时用命题逻辑的思维来说明有全称量词和存在量词的句子。

全称量词

对于变量 x 和包含它的句子 P , $\forall x P$ 为真, 当且仅当 P 对于论域中每个可能的 x 都为真。

存在量词

对于变量 x 和包含它的句子 P , $\exists x P$ 为真, 当且仅当 P 对于论域中某个可能的 x 为真。

下面我们用以张三家为论域的例子来分别展示一阶谓词逻辑下全称量词修饰变量和存在量词修饰变量对句子含义产生的差别。我们以“老张家”的论域为例, 假设在此论域中一共有四个对象, 分别为 LZ, Z_1, Z_2, Z_3 。

- 对于句子 $\forall z \text{Parent}(LZ, z)$, 它可以被逻辑等价的表示为: $\text{Parent}(LZ, LZ) \wedge \text{Parent}(LZ, Z_1) \wedge \text{Parent}(LZ, Z_2) \wedge \text{Parent}(LZ, Z_3)$ 。
- 对于句子 $\exists z \text{Parent}(LZ, z)$, 它可以被逻辑等价的表示为: $\text{Parent}(LZ, LZ) \vee \text{Parent}(LZ, Z_1) \vee \text{Parent}(LZ, Z_2) \vee \text{Parent}(LZ, Z_3)$ 。

从上面的例子可以看出, 仅仅是改变一个变量的量词, 整个句子表达的内容就会截然不同。同时, 值得注意的是, 句子中的每个变量都必须有一个量词来修饰。

当句子中有多个变量时, 修饰它们的量词的顺序满足以下性质:

- $\forall x\forall y$ 和 $\forall y\forall x$ 相同,
- $\exists x\exists y$ 和 $\exists y\exists x$ 相同,
- $\exists x\forall y$ 和 $\forall y\exists x$ 不同。(例如, 定义谓词 $Loves(x, y)$ 为 x 爱 y , 则 $\exists x\forall y Loves(x, y)$ 表示“存在一个 x , 他会爱上任意的 y ”, 而 $\forall y\exists x Loves(x, y)$ 则表示“对于任意的 y , 都存在一个爱他的人”。)

此外, 对于否定符号 $\neg$, 还有以下转换规则:

- $\neg(\forall x P)$ 逻辑等价于 $\exists x \neg P$,
- $\neg(\exists x P)$ 逻辑等价于 $\forall x \neg P$ 。

在梳理了一阶谓词逻辑的形式化语法规则后, 我们就可以用它来表示现实中的各种知识。

3.2 用一阶谓词逻辑表示知识

下面我们将通过四个具体的例子, 来给大家展示如何使用一阶谓词逻辑形式化地表示现实中的知识。

实例 1: 偶数

偶数是整数体系中的数类之一, 它的定义为:

偶数是能够被 2 所整除的整数。

为了将这一定义转化为一阶谓词逻辑形式, 我们首先定义如下谓词:

$Even(x) : x$ 是偶数

然后我们定义函数:

$$\times(x, y) = x \times y$$

$$+(x, y) = x + y$$

除此之外，我们还需定义常量：

1 : 表示整数 1

则偶数的定义可形式化为：

$$\forall x \left(\text{Even}(x) \Leftrightarrow \exists y (x = \times(+ (1, 1), y)) \right)$$

实例 2：素数

素数的定义为：

素数指在大于 1 的整数中，除了 1 和自身外，无法被其他自然数整除的数。

我们定义谓词：

$\text{Prime}(x) : x$ 是素数

$<(x, y) : x$ 小于 y

除此之外，我们仍然使用**实例 1**中定义的函数和常量，则素数的定义可形式化为：

$$\forall x \left(\text{Prime}(x) \Leftrightarrow <(1, x) \wedge \forall y \forall z (x = \times(y, z) \Rightarrow (y = 1 \vee z = 1)) \right)$$

实例 3：哥德巴赫猜想

哥德巴赫猜想是数论中最著名的未解问题之一，其自然语言表述为：

任何大于 2 的偶数都可以表示为两个素数之和。

我们使用**实例 1**和**实例 2**中定义的所有谓词、函数和常量，则哥德巴赫猜想可形式化为：

$$\forall x \left(\text{Even}(x) \wedge <(2, x) \Rightarrow \exists y \exists z (\text{Prime}(y) \wedge \text{Prime}(z) \wedge x = +(y, z)) \right)$$

实例 4：理发师悖论

理发师悖论是由逻辑学家罗素提出的自指性悖论，其自然语言表述为：

理发师只给那些不给自己理发的人理发。

我们定义谓词：

$\text{Barber}(x) : x$ 是理发师

$\text{Haircut}(x, y) : x$ 给 y 理发

则理发师悖论可形式化为：

$$\forall x \left(\text{Barber}(x) \Rightarrow \forall y \left(\text{Haircut}(x, y) \Leftrightarrow \neg \text{Haircut}(y, y) \right) \right)$$

实际上，谓词、函数和常量的名称本质上只是符号，我们完全可以使用任意命名。例如，用 $\text{AA}(x)$ 表示“ x 是偶数”，在形式上与 $\text{Even}(x)$ 并无区别，或者用符号“2”表示整数 1 也是可行的。然而，在上述四个例子中，我们仍然为谓词、函数和常量采用了统一且清晰的命名方式，如谓词 Even 表示偶数、函数 $+$ 表示加法以及常量 1 表示整数 1。这样做并非多余，而是为了使一阶谓词逻辑在语义表达和实际应用中更加规范和高效。

统一且具有描述性的命名能够让公式的含义更加直观、可读性更高。谓词或函数的名称若能直接反映其所表示的属性或关系，读者便能快速理解公式的含义，而无需额外解释。例如， $\text{Even}(x)$ 一眼即可理解为“ x 是偶数”， $\text{Haircut}(x, y)$ 直观表示“ x 给 y 理发”，相比使用抽象符号如 $P(x)$ 或 $R(x, y)$ ，理解起来更自然，也更符合人类的阅读习惯。

最关键的是，统一命名还有助于公式的维护和复用。虽然符号本身没有固有意义，但在自动化推理或知识库系统中，如果同一概念被赋予了不同的名称（如表示“ x 是偶数”时，一处使用 $\text{Even}(x)$ ，另一处使用 $\text{AA}(x)$ ），系统将无法识别它们为同一谓词，从而影响推理的正确性。因此，保持统一且清晰的命名则可以有效避免这类问题，使知识表示具有复用性和可扩展性。例如实例 3 的知识表示复用了实例 1 和实例 2 中所定义的谓词、函数和常量，实现了知识的扩展。

总结来说，采用统一且语义清晰的命名方式能够显著提高公式的可读性与理解性，并确保在自动推理与知识复用中的正确性与可靠性。

3.3 一阶谓词逻辑的蕴含推理

下面，我们将聚焦一阶谓词逻辑的语义，并阐明如何通过模型进行蕴含推理。

3.3.1 一阶谓词逻辑句子的模型

回忆一下，在命题逻辑中，我们在讨论一个句子的模型的时候，我们只需要对句子中的每个原子命题都做真值指派。然后，在一阶谓词逻辑中，我们并不能直接指派原子句子的真假，我们只能对原子句子涉及的常量、函数、谓词做一次**解释** (Interpretation)。具体讲，我们需要将每个常量符号指派为论域中的一个具体的对象，把谓词符号指派为论域中的一个具体的关系，而把函数符号指派为论域中的一个具体函数。这就是对一个原子句子的解释。

还是用“张三家”的例子来说明，假设在此论域包含：

- 4 个具体对象：老张、张大、张二、张三；
- 2 个关系：父子关系和兄弟关系；
- 1 个函数：“某个人的父亲是谁”；
- 在此论域中，老张是其他三位共同的父亲。

对于给定的原子句子 $Sibling(Z_1, Z_2)$ ：

- 当我们把 Z_1 指派为“张大”，把 Z_2 指派为“张二”，把 $Sibling(,)$ 指派为现实中的兄弟关系时，该原子句子为真。上述的这个解释被称为该原子句子的一个模型。
- 当我们把 Z_1 指派为“老张”，把 Z_2 指派为“张二”，把 $Sibling(,)$ 指派为现实中的兄弟关系时，该原子句子为假。上述的这个解释则不是该原子句子的模型。

也就是说, 对于一个原子句子, 在对其中的常量、函数、谓词做完一次解释之后, 所指派的对象在该论域中确实符合所指派的关系, 那么这个解释就是该原子句子的一个模型; 否则, 此解释不是该原子句子的模型。这就是一阶谓词逻辑中原子句子的模型的定义。

需要指出的是, 如果原子句子是“ $\text{项}_1 = \text{项}_2$ ”的形式, 那么, 它在某个解释中为真, 当且仅当在该解释中等号两边的符号被指派为论域中的同一个对象。

3.3.2 一阶谓词逻辑的蕴含推理

同样的, 我们可以根据语义蕴含的一般定义去检查一阶谓词逻辑下的某个知识库是否蕴含某个句子, 即 $KB \models \alpha$ 是否成立。这里, KB, α 都是一阶谓词逻辑的句子。但是, 对于一阶谓词逻辑的句子, 可能的解释的数量非常巨大。具体讲, 我们可以把句子中的一个常量指派为论域中的任一对象, 把句子中的谓词指派为论域中的任一关系, 把句子中的函数指派为论域中的任一函数, 这样的组合数量是巨大的。因此, 去遍历每一个可能的解释来检查 KB 与 α 的真假是非常困难的。

基于这个原因, 我们需要设计一阶谓词逻辑的形式推演算法, 使得检验语义蕴含的过程更加高效。

3.4 一阶谓词逻辑的形式推演

3.4.1 命题逻辑化 (Propositionalization)

首先, 通过将一阶谓词逻辑的句子转换为命题逻辑中的句子, 我们可以使用命题逻辑的形式推演规则来推演。

具体讲, 对于全称量词, 我们需要将变量替换为所有可能的常量; 对于存在量词, 则替换为一个可能的常量。

假设“张三家”的例子的论域中有 4 个对象,则:对于句子 $\forall z \text{Parent}(LZ, z)$, 我们可以将它转化为 $\text{Parent}(LZ, LZ) \wedge \text{Parent}(LZ, Z_1) \wedge \text{Parent}(LZ, Z_2) \wedge \text{Parent}(LZ, Z_3)$; 对于句子 $\exists z \text{Parent}(LZ, z)$, 我们可以将它转化为 $\text{Parent}(LZ, C)$, 其中 C 代表常量。转化后的句子, 都可以被理解为命题逻辑的句子; 因此就可以使用命题逻辑的形式推演规则进行推演。

可是, 由于函数可以无限嵌套, 所以转换后的句子可能有无限多; 这导致在转化后的命题逻辑的句子做形式推演的复杂度非常高。同时, 这样的算法也并不一定会停机。因此, 我们说, 一阶谓词逻辑是半可判定 (Semi-Decidable) 的系统。

3.4.2 一阶谓词逻辑的归结原理

另一方面, 我们也可以把命题逻辑的归结原理直接扩充为一阶谓词逻辑下的归结原理。具体讲, 和命题逻辑相比, 一阶谓词逻辑中的归结原理的基本元素没有改变, 而且原理也是互补消解。不过因为一阶谓词逻辑引入了量词、变量、谓词等概念, 在使用归结原理的过程中, 我们需要在转化合取范式的步骤做一些调整, 并且在消除互补文字的步骤需要引入合一算子来进行操作。

3.4.2.1 一阶谓词逻辑的合取范式

如何将带有变量和量词的句子转换为适合归结原理的形式呢?

一方面, 和命题逻辑相同, 需要将句子转换为合取范式形式; 另一方面, 需要去掉存在量词, 只保留全称量词。

下面是一个转换的示例:

待转化句子:

$$\forall x \forall y [\text{Animal}(y) \Rightarrow \text{Loves}(x, y)] \Rightarrow [\exists y \text{Loves}(y, x)]$$

1) 消除 $\Rightarrow$ 和 $\Leftrightarrow$:

$$\forall x \neg [\forall y \neg \text{Animal}(y) \vee \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

2) 内化 $\neg$: 运用 $\neg \forall x p \equiv \exists x \neg p$, $\neg \exists x p \equiv \forall x \neg p$, 双重否定消除规律, 德摩根律等规律:

$$\forall x [\exists y \neg (\neg \text{Animal}(y) \vee \text{Loves}(x, y))] \vee [\exists y \text{Loves}(y, x)]$$

$$\forall x [\exists y \neg \neg \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists y \text{Loves}(y, x)]$$

3) 标准化变量 (Standardize Variables): 改变变量符号, 让每一个量词都修饰一个不同的变量:

$$\forall x [\exists y \text{Animal}(y) \wedge \neg \text{Loves}(x, y)] \vee [\exists z \text{Loves}(z, x)]$$

4) 斯科伦化 (Skolemize): 一种更一般化的实例化形式, 用包含全称量词变量的 Skolem 函数来替代所有存在量词变量:

$$\forall x [\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee [\text{Loves}(G(x), x)]$$

5) 消去全称量词符号 (保留变量, 此时所有变量都是全称量词修饰的变量):

$$[\text{Animal}(F(x)) \wedge \neg \text{Loves}(x, F(x))] \vee [\text{Loves}(G(x), x)]$$

6) 使用分配律, 用 $\vee$ 把括号内为 $\wedge$ 的句子拆分:

$$[\text{Animal}(F(x)) \vee \text{Loves}(G(x), x)] \wedge [\neg \text{Loves}(x, F(x)) \vee \text{Loves}(G(x), x)]$$

容易发现, 上述步骤通过 Skolemize 操作消除了存在量词, 同时也引入

了一些新的符号。同时，由于剩下的变量都是由全称量词修饰的，因此我们可以去掉全称量词，则原句子被转化为没有量词的句子。

同时，如果要想实现归结原理中的互补消解，我们仍然需要把合取范式中的不同函数符号、不同变量和常量匹配上，而使用合一算子可以很好解决这一问题。

3.4.2.2 合一算子 (Unification)

表 3.1: 合一算子示例

p	q	θ
Knows(John, x)	Knows(John, Jane)	$\{x/\text{Jane}\}$
Knows(John, x)	Knows(y , OJ)	$\{x/\text{OJ}, y/\text{John}\}$
Knows(John, x)	Knows(John, y)	$\{x/y\}$
Knows(John, x)	Knows(y , Mother(y))	$\{y/\text{John}, x/\text{Mother(John)}\}$
Knows(John, x)	Knows(x , OJ)	fail

在介绍合一算子之前，我们首先需要定义替换 (Substitution) 操作。

替换

一个替换是指，将一阶谓词逻辑句子中的某些变量替换为常量、函数或者其他变量的过程，事实上该过程也可以视作一次赋值。

替换通常被记作：

$$\theta = \{x_1/t_1, x_2/t_2, \dots, x_n/t_n\}$$

其中：

- x_i 是一阶谓词逻辑句子中的变量；
- t_i 是任意项，可以是常量、函数或者变量；

- 要求 $x_i \neq t_i$, 即不能将变量替换为自己, 这是冗余的。

对于某个句子 α , 被 θ 替换后的句子记作 $\alpha\theta$ 。例如对于句子 $\alpha = \text{Knows}(\text{John}, x)$, 被 $\theta = \{x/\text{Jane}\}$ 替换后的句子为 $\alpha\theta = \text{Knows}(\text{John}, \text{Jane})$ 。

替换的应用范围包括以下三种情况:

- 把变量替换为常量。例如替换 $\theta = \{x/\text{John}, y/\text{John}\}$ 会将变量 x 和 y 均替换为常量 John ;
- 把变量替换为函数。函数的返回值也是一个常量, 所以替换也可以将某些变量替换为某些函数, 例如替换 $\theta = \{y/\text{Mother}(y)\}$ 会将变量 y 替换为函数 $\text{Mother}(y)$;
- 把变量替换为其他变量。例如替换 $\theta = \{x/y\}$ 会将变量 x 替换为另一个不同的变量 y 。如前所述, 如果一阶谓词逻辑的句子都转化成了规范的合取范式, 其中所有的变量都是全称量词修饰的变量, 那么变量之间的替换也是合理的。

值得注意的是, 一阶谓词逻辑句子中被替换的对象只能是变量, 而不能是常量或者函数, 这是因为常量以及返回值为常量的函数是不可变的, 不能像变量那样被“赋值”或“替换”。例如替换 $\theta = \{\text{John}/x\}$ 和 $\theta = \{\text{Mother}(y)/y\}$ 均是不合法的替换, 因为它们分别对常量 John 和函数 $\text{Mother}(y)$ 进行了错误的替换。

合一算子

对于句子 α, β 和替换 θ , 若 $\alpha\theta$ 和 $\beta\theta$ 是两个相同的字符串, 则 θ 是 α, β 的合一算子, 即 $\text{UNIFY}(\alpha, \beta) = \theta$ 。

表3.1展示了一些使 $\text{UNIFY}(p, q) = \theta$ 成立的 p, q, θ , 其中 $\theta = \text{fail}$ 表示不存在符合要求的合一算子 θ 。

我们该如何理解合一算子呢? 事实上, 它意味着两个一阶谓词逻辑的句子在经过某种实例化后, 得到了相同的结果。有了合一算子作为工具, 我们就可以在一阶谓词逻辑中完成归结了。

3.4.2.3 一阶谓词逻辑的归结原理

和命题逻辑中的归结原理类似，一阶谓词逻辑中的归结原则如下所示：

$$\frac{l_1 \vee \cdots \vee l_k, \quad m_1 \vee \cdots \vee m_n}{(l_1 \vee \cdots \vee l_{i-1} \vee l_{i+1} \vee \cdots \vee l_k \vee m_1 \vee \cdots \vee m_{j-1} \vee m_{j+1} \vee \cdots \vee m_n)\theta} \quad (3.1)$$

其中 $\text{UNIFY}(l_i, \neg m_j) = \theta$.

可以看到，不同于命题逻辑中两个互补文字可以直接消去，一阶谓词逻辑中归结原理的规则是两个经过实例化的替换 θ 后为互补文字的句子可以消去。不过，消去的代价是，得到的结果也是需要经过 θ 的替换操作的。

下面是一个例子：

$$\frac{\neg \text{Rich}(x) \vee \text{Unhappy}(x), \quad \text{Rich}(\text{Ken})}{\text{Unhappy}(\text{Ken})}$$

在这个例子中， $\theta = x/\text{Ken}$ 。

在得到一阶谓词逻辑的归结原理后，我们就可以仿照命题逻辑中归结原理的做法，通过检查 $S = \{KB, \neg\alpha\}$ 会不会归结出空子句来判断 $KB \models \alpha$ 是否成立了。下面我们通过一个推理情景来展示如何通过一阶谓词中的归结原理来进行推理。

推理实例 1

A 国法律规定，任何把武器贩卖给敌对国家的行为都被视为犯罪，而 A 国的敌人 B 国有一批导弹，B 国所有的导弹都是从一个叫 L 的 A 国人那里购买的。

请证明：**L 是一个罪犯。**

这个结论在自然语境下非常好证明，L 卖了武器给其他国家，而 A 国法律不允许这种行为，因此 L 是罪犯。那么如何让计算机也完成这个推理过程呢？

首先,我们需要进行知识表示,把已知条件转变为形式语言,构建知识库。以下内容展示了这个知识表示的过程:

- A 国法律规定,任何把武器贩卖给敌对国家的行为都被视为犯罪:

$$\forall x \forall y \forall z [A(x) \wedge Weapon(y) \wedge Sells(x, y, z) \wedge Hostile(z) \Rightarrow Criminal(x)]$$

- B 国有一批导弹:

$$\exists m [Missile(m) \wedge Owns(B, m)]$$

- B 国所有的导弹都是从一个叫 L 的 A 国人那里购买的:

$$[\forall a (Missile(a) \wedge Owns(B, a) \Rightarrow Sells(L, a, B))] \wedge A(L)$$

- 导弹是武器:

$$\forall b [Missile(b) \Rightarrow Weapon(b)]$$

- B 国是 A 国的敌人:

$$Hostile(B)$$

经过这样的知识表示过程,我们得到了知识库 KB 。接着,我们将这些句子转化为合取范式,此步骤请读者自行完成。我们要推理的结论是 $Criminal(L)$, 记为 α 。图3.1展示了 $\{KB, \neg\alpha\}$ 的归结过程。

推理实例 2

理发师只给那些不给自己理发的人理发。

请证明: **不存在这样的理发师。**

这个问题是我们在 3.2 节中提到过的理发师悖论,它的自然语言形式本身就带有明显的矛盾色彩,下面我们用一阶谓词逻辑的形式化推理来证明它的矛盾性,即证明不存在这样的理发师。我们将按照知识表示、合取范式化以及归结推理三个步骤进行。

首先,我们假设存在这样的理发师,然后进行知识表示:

- 理发师只给那些不给自己理发的人理发：

$$\forall x [\text{Barber}(x) \Rightarrow \forall y (\text{Haircut}(x, y) \Leftrightarrow \neg \text{Haircut}(y, y))]$$

- 存在一个理发师：

$$\exists x \text{Barber}(x)$$

接着，我们将以上命题转化为合取范式：

待转化句子（命题一）：

$$\forall x [\text{Barber}(x) \Rightarrow \forall y (\text{Haircut}(x, y) \Leftrightarrow \neg \text{Haircut}(y, y))]$$

1) 消除 $\Rightarrow$ 和 $\Leftrightarrow$ ：

$$\begin{aligned} \forall x [\neg \text{Barber}(x) \vee \\ \forall y ((\neg \text{Haircut}(x, y) \vee \neg \text{Haircut}(y, y)) \wedge \\ (\text{Haircut}(y, y) \vee \text{Haircut}(x, y)))] \end{aligned}$$

2) 内化 $\forall$ ：

$$\begin{aligned} \forall x \forall y [\neg \text{Barber}(x) \vee \\ ((\neg \text{Haircut}(x, y) \vee \neg \text{Haircut}(y, y)) \wedge \\ (\text{Haircut}(y, y) \vee \text{Haircut}(x, y)))] \end{aligned}$$

3) 消除全称量词符号：

$$\begin{aligned} \neg \text{Barber}(x) \vee \\ ((\neg \text{Haircut}(x, y) \vee \neg \text{Haircut}(y, y)) \wedge \\ (\text{Haircut}(y, y) \vee \text{Haircut}(x, y))) \end{aligned}$$

4) 使用分配律, 用 $\vee$ 把括号内为 $\wedge$ 的句子拆分:

$$\begin{aligned} & (\neg \text{Barber}(x) \vee \neg \text{Haircut}(x, y) \vee \neg \text{Haircut}(y, y)) \wedge \\ & (\neg \text{Barber}(x) \vee \text{Haircut}(y, y) \vee \text{Haircut}(x, y)) \end{aligned}$$

待转化句子 (命题二):

$$\exists x \text{ Barber}(x)$$

1) 斯科伦化 (Skolemize): 因为没有外围的全称量词, 用一个 Skolem 常数 b 替换存在量词:

$$\text{Barber}(b)$$

最后, 我们进行归结推理:

1) 命题集合

$$C_1 : \neg \text{Barber}(x) \vee \neg \text{Haircut}(x, y) \vee \neg \text{Haircut}(y, y)$$

$$C_2 : \neg \text{Barber}(x) \vee \text{Haircut}(y, y) \vee \text{Haircut}(x, y)$$

$$C_3 : \text{Barber}(b)$$

令替换 $\theta = \{x/b, y/b\}$, 则有:

$$C_1\theta : \neg \text{Barber}(b) \vee \neg \text{Haircut}(b, b) \vee \neg \text{Haircut}(b, b)$$

$$C_2\theta : \neg \text{Barber}(b) \vee \text{Haircut}(b, b) \vee \text{Haircut}(b, b)$$

2) 归结推理

(i) $C_1\theta$ 与 C_3 归结得到:

$$R_1 : \neg \text{Haircut}(b, b) \vee \neg \text{Haircut}(b, b) \equiv \neg \text{Haircut}(b, b)$$

(ii) $C_2\theta$ 与 C_3 归结得到:

$$R_2 : \text{Haircut}(b, b) \vee \text{Haircut}(b, b) \equiv \text{Haircut}(b, b)$$

(iii) R_1 与 R_2 归结得到:

$\emptyset$

我们得到了空子句 ($\emptyset$), 说明该句子集合不可满足。也就是说, 假设“存在这样一位理发师”会导致逻辑矛盾, 因此不存在只给那些不给自己理发的人理发的理发师。

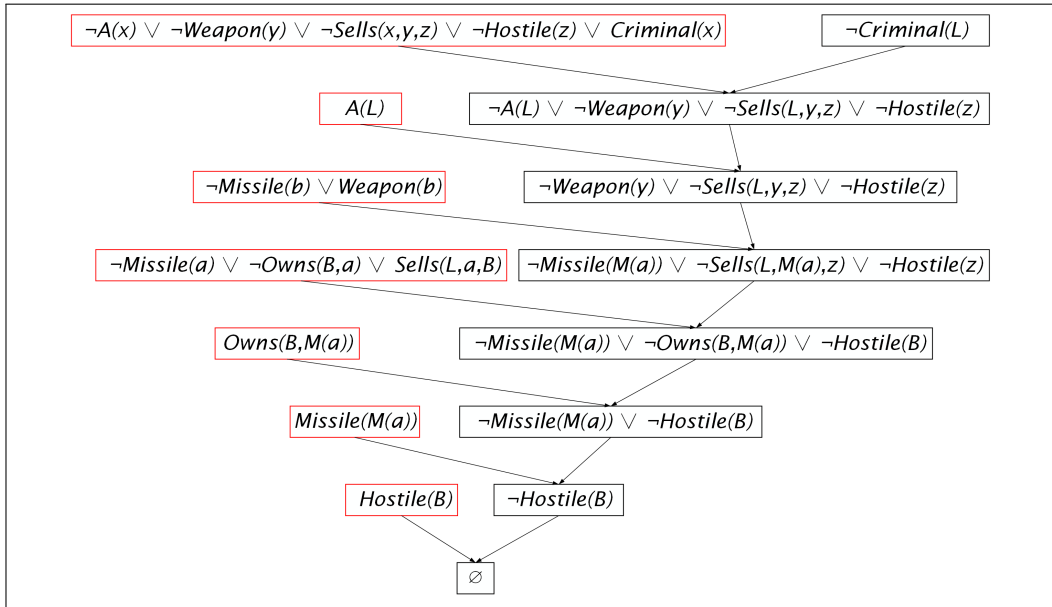


图 3.1: 一阶谓词逻辑下使用归结原理推理案例的过程, 其中 $\theta = \{x/L, y/M(a), b/M(a), a/M(a), z/B\}$; $M(a)$ 是 Skolem 函数, 用于替换存在量词修饰的变量 m ; 红框表示来自知识库的子句, 黑框表示推出的子句。

需要注意的是, 一阶谓词逻辑的归结原理有可能不停机, 比如下面这个例子:

$$KB \text{ 为 } \{\forall x P(x) \Rightarrow P(x+1)\}, \alpha \text{ 为 } \forall y \neg P(y).$$

一阶谓词逻辑的归结原理的可靠性和完备性证明过于繁琐，但大致的思想与它在命题逻辑中的证明类似，这里略过。

3.4.3 一阶谓词逻辑的 Modus Ponens 规则

一阶谓词逻辑的 Modus Ponens 规则称为 Generalized Modus Ponens (GMP) 规则，也被称为一般的肯定前件推理。具体的规则如下所示：

$$\frac{p'_1, p'_2, \dots, p'_n, (p_1 \wedge p_2 \wedge \dots \wedge p_n \Rightarrow q)}{q\theta} \quad (3.2)$$

其中 $p'_i\theta$ 和 $p_i\theta$ 是相同的字符串 ($i = 1, \dots, n$)。

下面是一个使用 GMP 进行推理的例子：

$KB :$

$\forall x \text{ King}(x) \wedge \text{Greedy}(x) \Rightarrow \text{Evil}(x)$

$\text{King}(\text{John})$

$\forall y \text{ Greedy}(y)$

推出：

$\text{Evil}(\text{John})$

这里， $\theta = \{x/\text{John}, y/\text{John}\}$ ，可以很容易推出此结论。

与命题逻辑中的 Modus Ponens 规则类似，GMP 也有前向推理和后向推理的算法。不过，该规则也只能作用在 definite 子句上。同样的，这里略过了 GMP 的可靠性和完备性的详细证明。

以上是一阶谓词逻辑体系所有理论上的内容，可以看出，一阶谓词逻辑体系不仅贴近自然语言，而且它又有形式化的语法规则，能构建精确表

达领域知识的形式化知识库。之外，一阶谓词逻辑的形式推演方法使用合一算子替换变量，赋予系统自动消解矛盾，推出新结论的能力。

下面我们会介绍 Prolog，它是基于一阶谓词逻辑中的 Modus Ponens 规则设计的编程语言，为实现自动知识推理提供了自动化工具。

3.5 Prolog 逻辑编程

Prolog 是 “Programming in Logic” 的缩写，是一种用于自动逻辑推理的编程语言。它建立在逻辑学的理论基础之上，被广泛应用于自动定理证明与人工智能领域。

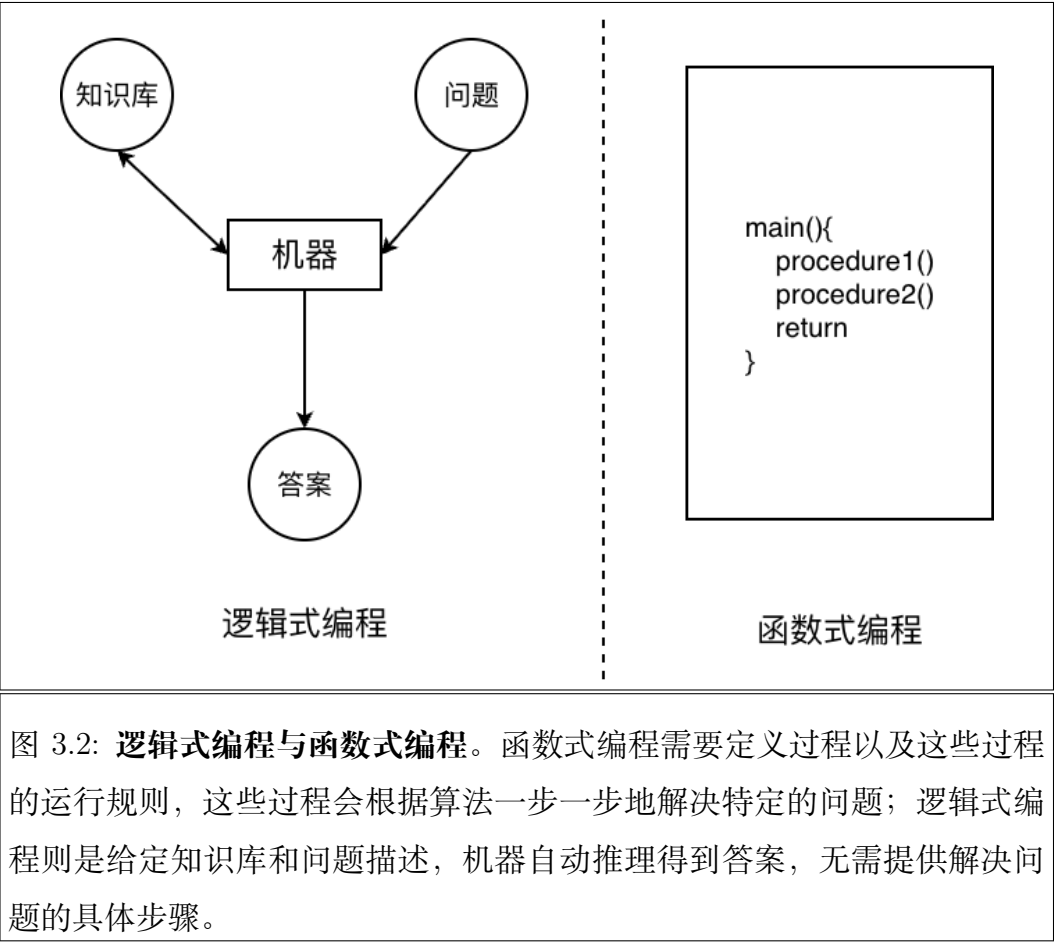
3.5.1 Prolog 的基本原理

与大家比较熟悉的函数式编程（Functional Programming）语言（例如 C、Java）不同，Prolog 是一种逻辑式编程（Logical Programming）语言，它不强调“如何执行”一个任务，而是强调“要什么”——即通过事实与规则来描述某个知识库，用户输入查询或问题后，由系统自动推理得到答案。图 3.2 清楚地展示了逻辑式编程和函数式编程的区别。

因此，当我们以 Prolog 编程时，并不是在编写程序的执行过程，而是在将自然语言或领域知识“翻译”为逻辑形式，然后交由系统推理机制来自动执行。Prolog 采用的是“后向推理”（backward-chaining）策略，即从用户提出的查询出发，逆向查找知识库中可以证明该查询的规则和事实。这一编程思想使得 Prolog 在自然语言处理、知识表示、专家系统与自动推理等领域具有独特优势。

3.5.2 Prolog 程序的基本语法

一个 Prolog 程序通常由三个基本部分组成：**事实**、**规则**和**查询**。事实用于描述已知的知识，规则用于表达逻辑关系或推理模式，事实和规则共



同构建起了知识库，而查询则用于向系统提问，让系统通过推理机制自动找出满足条件的解。

在介绍这三种结构之前，我们先了解 Prolog 中的三类基本数据对象：**常量、变量和列表**。它们构成了 Prolog 程序中一切信息表示的基础。

(1) 常量

常量用于表示具体的对象或关系名称，主要分为以下几种：

- **原子**：以小写字母开头的标识符，例如 `apple`、`tom`、`likes`。
- **数值常量**：整数或浮点数，例如 `42`、`3.14`。

- **字符串**：用单引号括起来的字符序列，例如 'Hello'。

(2) 变量

变量用于表示未知对象，在推理过程中可以与具体值进行匹配。变量名以大写字母或下划线开头，例如 X、Person、_value。

(3) 列表

列表是 Prolog 中的重要复合数据结构，用方括号表示，用逗号分隔元素。例如 [apple, banana, cherry]。列表的头尾结构可用 “|” 符号拆分：

[Head | Tail]

其中Head表示列表第一个元素,Tail为剩余所有元素的列表。例如,[a, b, c] 可以表示为 [a | [b,c]]。空列表用 [] 表示。

在了解了这些数据对象后，我们可以更好地理解 Prolog 程序的三种基本组成部分。

(4) 事实

我们可以将事实定义为对象之间明确的关系，以及这些对象可能具有的属性，事实是无条件为真的。其形式为：

predicate(term_1, term_2, ..., term_n).

例如，事实“Tom 是 Lily 的父亲”和“Alice 喜欢苹果”可以表示为：

parent(tom, lily).

likes(alice, apple).

(5) 规则

规则用于表达事实之间的逻辑关系或蕴含，形式为：

$$\text{Head} \text{ :- } \text{Body}.$$

它等价于逻辑公式 $\text{Body} \Rightarrow \text{Head}$ ，表示“如果 Body 成立，则 Head 成立”。

例如：

$$\text{grandparent}(X, Z) \text{ :- } \text{parent}(X, Y), \text{parent}(Y, Z).$$

等价于 $\text{parent}(X, Y) \wedge \text{parent}(Y, Z) \Rightarrow \text{grandparent}(X, Z)$ ，表示“如果 X 是 Y 的父母，且 Y 是 Z 的父母，那么 X 是 Z 的祖父母”。

(6) 查询

查询由用户输入，用于向 Prolog 系统提问。系统会尝试根据已知的事实和规则来回答。查询以问号和减号（?-）开头，例如：

$$\begin{aligned} &?- \text{parent}(\text{tom}, \text{lily}). \\ &\text{true}. \end{aligned}$$

查询 Tom 是否是 Lily 的父母，系统输出 true。实际上，Prolog 会尝试寻找使查询成立的所有变量取值，并列出所有可能的解。

值得注意的是，Prolog 中每个句子（事实、规则和查询）都必须以英文句号“.”结束。

综上所述，Prolog 程序的核心在于用**事实**描述已知知识，用**规则**表示推理逻辑，再通过**查询**与系统交互，从而让推理机制自动找到满足条件的答案。为了更深入理解，下面我们通过具体示例演示如何用 Prolog 来表达知识与执行推理。

3.5.3 Prolog 编程实例

Prolog 编程有两个主要步骤，首先是知识表示即编写谓词定义（事实与规则），然后通过查询自动推理得到结果。我们以计算阶乘为例：

```
% 设置一个谓词 f(N,F), 表示 F 是 N 的阶乘
f(0, 1).           % 0 的阶乘为 1
f(N, F) :-         % 若以下条件成立, 则 F 为 N 的阶乘
    N > 0,          % N 大于 0
    N1 is N - 1,    % N1 = N - 1
    f(N1, F1),      % F1 是 N1 的阶乘
    F is F1 * N.    % 递归计算
```

加载此知识库后, 可有两种常见用法:

```
% 用法一: 判断阶乘
?- f(5, 120).      % 输入: 让程序判断 5 的阶乘是不是 120
true.              % 自动输出: 判断成立
```

```
% 用法二: 计算阶乘
?- f(5, W).        % 输入: 计算 5 的阶乘, W 为变量
W = 120.           % 自动输出: W 的取值
```

这里的推理过程可视为:

- 首先尝试证明 $f(5,120)$ (或 $f(5,W)$) 为真;
- 发现规则要求 $N > 0$, $N1 = N - 1$, $f(N1,F1)$, $F = F1 * N$ 为真, 故递归调用 $f(4, F1)$;
- 继续递归, 直到基准事实 $f(0,1)$;
- 然后将 $F1$ 的值逆向代入上一层, 最终确定 F (或 W) 的值。

这种自顶向下、后向证明的方式，就是 Prolog 的典型逻辑编程流程。

另一个经典例子是“列表拼接”操作 (append):

```
% 设置谓词 app(A, B, C), 表示 C 由 A 和 B 拼接得到
app([], A, A).           % 空列表与 A 拼接仍为 A
app([H|T], B, [H|R]) :-
    app(T, B, R).
```

同样的，它也有两种用法：

```
% 用法一：求解两个列表拼接而成的列表
?- app([1,2], [3,4], C). % 输入：求列表 [1,2] 和 [3,4] 拼接而成的列表
C = [1,2,3,4].          % 自动输出：C 的取值
```

```
% 用法二：拆分列表
?- app(A, B, [1,2,3]). % 输入：求列表 [1,2,3] 可以由哪两个列表拼接而成
A = [],      B = [1,2,3] ; % 自动输出：所有情况
A = [1],     B = [2,3] ;
A = [1,2],   B = [3] ;
A = [1,2,3], B = [].
```

3.5.4 Prolog 的局限性

虽然 Prolog 是功能强大的逻辑编程语言，但为了简化实现并保证推理效率，它也存在一些局限和潜在问题。下面简要说明两方面：可靠性 (soundness) 和完备性 (completeness)。

不可靠（可能推出错误结论）

由于 Prolog 的控制机制（如回溯策略、无 occur-check 优化、剪枝操作）可能削弱其逻辑纯性，程序有时可能得出“逻辑上不应得”的结论。例如：

```
test :-  
    p(A, A).  
p(A, f(A)).
```

在某些 Prolog 实现中，执行查询 `test.` 可能会被判定为真。这是因为，当 Prolog 试图让 `p(A, A)` 与规则 `p(A, f(A))` 匹配时，它会尝试让两者“看起来一样”，也就是让 $A = f(A)$ 。从逻辑上讲，这是不可能的——因为这会导致 A 变成一个无限套娃式的结构： $A = f(f(f(\dots)))$ 。但有些 Prolog 实现并不会严格检查这种“无限递归”（即不执行 occur-check），于是就错误地认为匹配成功，从而让 `test` 被判定为真。

这说明：虽然 Prolog 是一种基于逻辑的语言，但它的实际推理过程仍然受到实现细节的影响，有时会与纯粹的逻辑推理有所偏差，并不总是“逻辑上完全可靠”的。

不完备（可能无法推出所有应该推出的结论）

因为 Prolog 通常采用深度优先搜索与回溯算法，并依据规则顺序进行匹配，若规则书写顺序不当或递归结构导致无限展开，则程序可能“卡住”或无法找到结论。

下面这个例子定义了路径 `path` 这个谓词：如果两个点之间直接相连，或者能通过其他点间接连起来，就认为它们之间有路径。

```
link(a, b).  
link(b, c).
```

% 好的顺序：先直接链接，再递归链接

```
path(A, B) :-  
    link(A, B).  
path(A, B) :-  
    path(A, C),  
    link(C, B).
```

上述代码，若查询 `path(a,c).`，Prolog 会先通过第一条直接链接规则找到 `a` 到 `b` 的路径，再通过第二条规则递归地找到从 `b` 到 `c` 的连接，最终判定为真。

但如果改为：

```
link(a, b).  
link(b, c).
```

% 错误顺序：先递归链接，再直接链接

```
path(A, B) :-  
    path(A, C),  
    link(C, B).  
path(A, B) :-  
    link(A, B).
```

在大多数 Prolog 系统中，`path(a,c).` 不但可能无法成功，还可能因为无限递归而“崩溃”。这是因为 Prolog 执行时使用“从上到下、从左到右”的搜索顺序。它会先尝试第一个规则，而第一个规则的开头又是递归调用 `path(A, C)`，所以 Prolog 深度优先进入 `path(A,C)`，导致第一个规则不断自调用，永远也到不了第二条规则，最终陷入无限循环。

这说明：虽然从逻辑上看，“`a` 到 `c` 存在路径”这件事是可以被证明的（因为 `a→b→c` 确实存在），但 Prolog 的执行过程受“规则顺序”和“搜索

策略”的影响，有时会导致推理卡死，从而“无法推出”这一结论——即完备性受限。

总结来说，Prolog 提供了一种以逻辑为核心的编程范式：你编写事实与规则，让系统负责推理。它在知识表示、人工智能、自然语言处理、逻辑查询等领域拥有独特优势。与此同时，我们也要认识到其“执行机制”所带来的局限如搜索策略、规则顺序、控制构造所影响的可靠性与完备性。在实际编程中，写好清晰的事实与规则、合理安排规则顺序、避免无限递归结构，是提高 Prolog 程序健壮性的关键。

在人工智能领域，让计算机更好地模拟人类的逻辑推理能力是无数科学家、数学家、逻辑学家都希望实现的目标。下面我们会回顾定理自动证明的历史，带大家了解过去自动逻辑推理在数学定理证明方面取得过哪些成果以及当前的研究现状。

3.6 定理自动证明的历史回顾

定理自动证明是指利用计算机程序，基于形式化的逻辑规则和数学公理，自动推导各种数学性质。这个过程，也被称为机器定理证明。定理自动证明的核心功能是通过算法化的形式推演，替代或辅助人类完成复杂的数学证明或逻辑验证任务。

很多数学家把机器定理证明当作工具，而逻辑学家则把机器定理证明当作目标¹。

3.6.1 定理自动证明的背景

使用计算机来证明数学定理

定理自动证明 (Automatic Theorem Proving) 最早起源于对逻辑演算自动化的设想。如果把一个新的数学定理看作未知命题，数学定理的证明就

¹本节内容摘录自《人工智能简史》（尼克著）

是利用已知的公理和定理进行一系列逻辑推理，判断这个未知命题是否成立的过程。然而，很多高深的数学难题往往需要非常复杂的推理过程，若能将已知的数学知识用符号表示出来，借由机器自动推演来实现逻辑推理，那无疑会给数学理论的发展带来极大的便利。不过这也涉及到了一个很深奥的数学问题，是不是所有数学定理都能通过符号逻辑推导出来？

希尔伯特的梦想：构架数学的大厦

20 世纪 20 年代，数学家大卫·希尔伯特 (David Hilbert) 提出来的“数学基础计划”（也称“希尔伯特计划”）。该计划旨在将整个数学体系构建为一座逻辑严密、无矛盾的“大厦”，通过公理化、形式化来证明数学的一致性，从而消除悖论并确立数学的绝对可靠性。在希尔伯特看来，数学的过程就是把一串符号变成另一串符号，如果能设计一个大一统的算法，那么所有数学问题都可以由这个算法来解答。

以希尔伯特计划为理论核心，数学家们尝试建立一阶谓词逻辑的完整体系，将数学中的定理和性质转化为符号表达，并采用自动化方法进行新结论的推演。其依据和想法如下：

- 依据
 - 一阶谓词逻辑是可靠和完备的。
 - 数学定理“希望”都能够被一阶谓词逻辑表达。
- 梦想：构建数学的基础的 KB，基于一阶谓词逻辑推演出所有的数学性质。

梦想的破灭：哥德尔不完备定理

在 1931 年，数学家库尔特·哥德尔 (Kurt Gödel) 提出了具有革命性的哥德尔不完备定理。该定理揭示了形式系统的局限性，深刻影响了数学、逻辑学和计算机科学，让希尔伯特的梦想破灭。

哥德尔不完备性定理指出,任意一个包含算术系统在内的形式系统中,都存在一个命题,它在这个系统中既不能被证明,也不能被否定。后来,他又证明了任何足够复杂且自洽的形式系统都存在一些不可判定的命题,表明没有完全自洽的数学系统能够包含所有的数学真理。

哥德尔的理论使数学基础研究发生了划时代的变化,更是现代逻辑史上很重要的一座里程碑。虽然哥德尔不完备定理终结了建立完全统一的数学公理体系的梦想,但仍有许多学者在此基础上探索定理自动证明在部分领域的可行性,例如塔尔斯基就在哥德尔不完备性定理提出后不久证明了存在算法可以对所有初等几何给出证明。

3.6.2 定理自动证明的研究现状

由于无法自动证明所有的数学定理,因此目前的研究一般会找到一个相对可控的子领域,然后针对这个子领域的特性,找到有效的算法。

3.6.2.1 Logic Theorist 项目

符号主义在上世纪 50 年代有一个开创性工作。在 1956 年,Allen Newell、Herbert A. Simon 和 John Cliff Shaw 合作,在卡内基梅隆大学开发了自动定理证明程序“逻辑理论家”(Logic Theorist),该程序成功证明了 Bertrand Russell 与 Alfred North Whitehead 合著的《数学原理》第一卷中命题逻辑部分中的很大一部分问题,被认为是 AI 历史上重要的原创性工作之一,标志着定理自动证明这一领域的诞生。

3.6.2.2 吴文俊的工作

吴文俊是中国著名数学家、计算机科学家,被誉为“中国人工智能之父”。他在研究中国数学史时,受到启发,将中国传统数学中的构造性思维与西方代数几何结合,针对于一大类的初等几何问题给出了高效的算法。不仅如此,他的方法还被推广到了一类微分几何问题上,开创了几何定理自

动证明的道路。1977 年，他的文章《初等几何判定问题与机器证明》发表在《中国科学》上，推动了定理自动证明领域的发展。

当时的中国还不像现在那样开放，这也使得吴文俊的学术工作从中国推广向世界的过程比较有意思。上世纪 70 年代末，吴文俊在中科院计算所开设了几何定理证明的课。当时他的书《几何定理机器证明的基本原理》还未出版，但所有参与课的学生都已拿到该书手稿，而师从唐稚松的周咏青（1978 年考入计算所）正是其中一位旁听生。后来周咏青去 UT Austin 留学，师从 Robert S. Boyer 和 Woody Bledsoe。两个逻辑定理证明的大师从周咏青那里听到了吴文俊的工作后非常兴奋，让周咏青自己规划博士论文。周咏青参考了吴文俊的方法，借由美国的硬件设备优势，取得了更为丰富的研究成果，也因此让吴文俊的名字传向自动定理证明界。

吴文俊的工作给与定理自动证明领域的学者很大的启发。在定理自动证明的早期，虽然已经有了哥德尔不完备定理说明所有数学定理无法被同一形式推演系统推演，但许多研究者仍然追求“一招鲜吃遍天”，希望找到一个超级算法能证明所有的问题，最典型的例子是归结原理。从吴文俊的工作中，我们可以看到研究定理自动证明有效的方法就是找到一个相对可控的子领域，然后针对这个子领域的特性，找到有效的算法。

3.6.3 一阶谓词逻辑研究回顾

Resolution 归结原理

归结原理是由英国哲学家、数学家、计算机科学家 John Alan Robinson 提出的。John Alan Robinson 受到 Dag Prawitz 工作的启发，拓展了他的原始合一算子，将合一算法应用到了一阶谓词逻辑体系中，并在 1965 年发表文章，提出了归结原理。以前的定理证明技术会用到很多规则，有了归结原理后，所有的证明推导只需要归结这一条规则。这是定理自动证明的重要里程碑。

John Alan Robinson 的主要贡献是奠定了定理自动证明的基础。如今，归结原理和合一算子被广泛应用于许多定理自动证明系统，成为逻辑编程语言中的底层推理机制。John Alan Robinson 也因为他的巨大贡献在 1996 年获得了 Herbrand 自动推理杰出成就奖。

Prolog

Prolog 编程语言由法国计算机科学家 [Alain Colmerauer](#) 发明，它建立在逻辑学的理论基础之上，最初被运用于自然语言等研究领域，现在它已广泛的应用在人工智能的研究中，用来建造专家系统、自然语言理解、智能知识库等。

Alain Colmerauer 和他的团队于 20 世纪 60 年代末开始开发 Prolog 语言。当时，Alain Colmerauer 和他的学生 Philippe Roussel 考虑设计一个全新的程序设计语言来研发具备逻辑推理的自然语言交流系统，Philippe Roussel 的妻子想到了 Prolog 这个名字，意为“逻辑编程” (Programming in Logic)。1972 年，Alain Colmerauer 设计实现了 Prolog 的雏形，并在 1973 年成功研发完全版本。Alain Colmerauer 教授由此被称为 Prolog 之父。

深度学习应用到定理证明中

传统的定理自动证明在归结原理之后虽然也经历了各种改进，但面对比较复杂的数学问题时，仍然无法避免中间子句指数级增长的可能性。在使用归结原理时，计算机只能依赖传统搜索算法一次次遍历子句集合，而每次归结出来的新子句又不一定会在后续的证明过程中用上，这就造成了时间和空间上的浪费。因此，如何在已经证明的子句集合中选取下一次推演的两个子句是一阶谓词逻辑中亟须解决的难题。

随着计算机科学的不断发展，一些深度学习工具也被用于定理自动证明当中。一方面，可以使用大语言模型来做知识表示，之后进行定理自动证明。例如，你可以把遇到的数理问题输入给大模型，让它用一阶谓词逻辑的

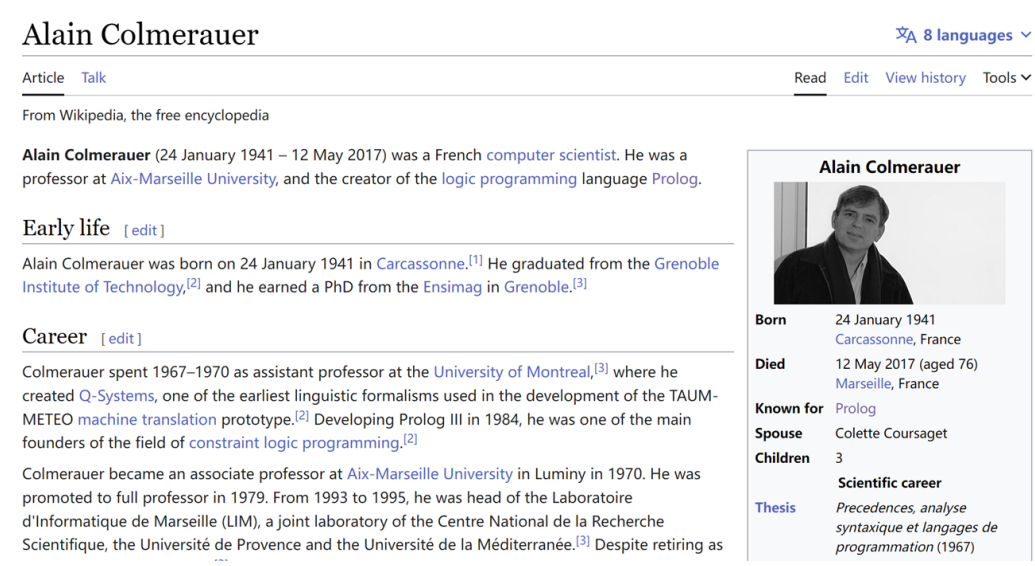


图 3.3: Alain Colmerauer

符号表示出来。另一方面，也可以使用神经网络搜索来加速推理，即通过使用神经网络的方法，自动学习选择子句的策略。这将加快证明新命题时的推演速度，也提高了计算机对于困难定理的证明能力。²

3.7 一阶谓词逻辑的总结

一阶谓词逻辑体系通过引入量词、变量、谓词和函数，显著提升了逻辑表达能力。相较于命题逻辑仅能处理原子命题的真假关系，一阶逻辑通过常量、变量、函数符号刻画对象属性，借助谓词符号定义对象间的关系，使用量词对变量范围进行约束，形成更贴近自然语言的符号体系。

在推演方面，一阶谓词的语义通过解释将符号映射到论域对象。因为解释的组合数量巨大，所以一阶谓词逻辑的推演方法以形式推演为主。通过

² 《Deep Network Guided Proof Search》, <https://arxiv.org/abs/1701.06972>, 数据集: <http://www.mizar.org/library/>

引入合一算子，一阶谓词逻辑也能使用归结原理和 Modus Ponens 规则进行推演。在人工智能符号主义的发展过程中，许多学者都选择使用一阶谓词逻辑来作为定理自动证明的工具，实现自动化推理数学定理的梦想，并且也出现了 Prolog 这样基于 Modus Ponens 规则实现自动推理的编程语言。

不过，一阶谓词逻辑也并非完美。首先是一阶谓词逻辑的半可判定性，在处理一些函数多重嵌套的问题时，一阶谓词逻辑可能会导致计算机无法停机。其次是一阶谓词逻辑的表达能力依然有限，无法表达一些高阶概念（例如，多个对象共享相似的性质， $\exists P \forall x P(x)$ ）。因此，现在的符号主义学者的主要研究方向依旧是寻找更好的知识表示和自动推理的方法。

对符号主义部分的总结

在这一部分中，我们以通用问题求解器为切入点，介绍了逻辑的基本概念，探讨了命题逻辑和一阶谓词逻辑下知识表示和知识推理的主要方法。其中一阶谓词逻辑下的归结原理和 Modus Ponens 规则无疑是使用程度最高的形式推演算法。我们也介绍了自动推理的编程语言 Prolog。

然而，通用问题求解器仍然是一个未解决的问题，在知识表示和知识推理两个方面都存在瓶颈。首先，当前的知识表示并不能完全自动化，虽然利用大语言模型可以辅助将一些知识用一阶谓词逻辑来表示，但对于大规模且复杂的背景知识，人工的参与仍然必不可少。其次，自动推理所使用的形式推演算法的复杂度也是无法逾越的障碍。具体讲，归结原理仍然是指数时间复杂度的算法，无法满足大规模知识推理的需求；而 Modus Ponens 规则虽然是线性时间复杂度的算法，但它限制了知识库中句子的形式（只能是 Definite Clause），使得有些知识无法表达。因此，这些瓶颈导致符号主义的研究并没有在现实问题中大规模的实际应用。

但不可否认的是，符号化的逻辑系统的研究，包括：逻辑语言的形式化定义、逻辑蕴含的定义、形式推演算法的设计以及对应的可靠性和完备性的证明，这些工作充满了学术的美感，它揭示了推理的本质，也确实是人类智慧的跃迁。因此，我们不能否定这些工作在 AI 发展历史中的价值。同时，需要指出的是，这部分内容的研究，包括：逻辑蕴含和形式推演等，都是在元语言中研究的，研究时所使用的推理都是直观而非形式的推理。我们期待着有那么一天，本部分的内容都可以用形式化的语言自动推理出来，

例如：形式化的自动证明归结原理的完备性。这一天也一定会到来！

可以看到，符号系统的推理能力高度依赖形式推演算法实现。面对大规模知识库的推理问题时，往往需要结合搜索策略，找到归结的最短路径，提高推演的效率。因此，搜索技术成为衔接符号推理与现实问题求解的关键桥梁，是实现人工智能的重要基础，也是本书下一部分要讲的内容。

第二部分

搜索：AI 算法的实现基础

搜索是人工智能系统的核心能力之一，指通过系统化方法在可能的状态空间里找到满足特定目标的解决方案。搜索问题的关键在于如何在更短时间、更小的算力消耗、更小的存储需求下找到满足目标的方案，而这就涉及到了搜索算法。在人工智能领域，搜索算法能构建问题状态的抽象模型，模拟人在未知环境中的决策过程，输出解决问题的方案。而根据搜索问题种类的不同，搜索算法也需要相应变化。

本部分我们将介绍三种搜索问题（其中前两类统称为经典搜索）：

第一种搜索问题：给定一个初始状态和终止状态，我们希望找到从初始状态到终止状态的一条路径。这种搜索称为状态搜索。

第二种搜索问题：给出一个函数 $f(x)$ ，我们希望找到使目标函数最小的自变量 x 。这种搜索称为最优化搜索。

第三种搜索问题：针对双（多）方博弈问题的策略搜索，这种搜索称为对抗搜索。

在本部分，我们会依次介绍以上三种搜索问题，并介绍对应的搜索算法。同时，我们也会看到，很多实际的问题，都可以转化为以上三种搜索问题之一。因此，搜索技术实际是各种 AI 问题的实现基础。

第四章 状态搜索

状态搜索是找寻一条从初始状态到目标状态的最优路径，其中状态、路径是对现实中情景的一个抽象化表达，取决于当前搜索问题是如何描述的。在状态搜索问题中，我们需要把现实情况抽象为一个由各种状态构成的世界，而从一个状态到另一个状态的动作被视为路径。举个例子，前文中我们介绍过使用归结原理进行形式推演的具体过程，在这个情景中，我们可以把归结前、后的子句的集合分别称为一个状态，把选取互补文字消除的过程称为路径。事实上，现实中很多问题都可以被抽象为状态搜索问题，像是导航中的路线规划、外卖平台给外卖员设计配送方案等。只要能够把问题的开始和结束抽象成两种明确的状态，并需要找到达到通往结束状态的路径的问题，我们都可以把它看作状态搜索问题。

在本章中我们会先介绍状态搜索问题需要包含的要素，给出状态搜索问题的具体定义，之后展示状态搜索中的算法框架，然后详细讲述不同的搜索策略以及搜索算法，最后用归结原理的计算机实现过程为例展示状态搜索的应用。

4.1 问题定义

一个状态搜索的问题可通过初始状态、动作集合、转移模型、目标测试、路径成本五个要素来定义。下面，我们以一个实际问题为例，来看看这五个要素的具体含义。

实际问题案例

假设你身处在一座陌生的国家中，手中有一张该国家的地图（如图4.1所示），现在你要从 A 市开车到 B 市，想选一条最短的路径。

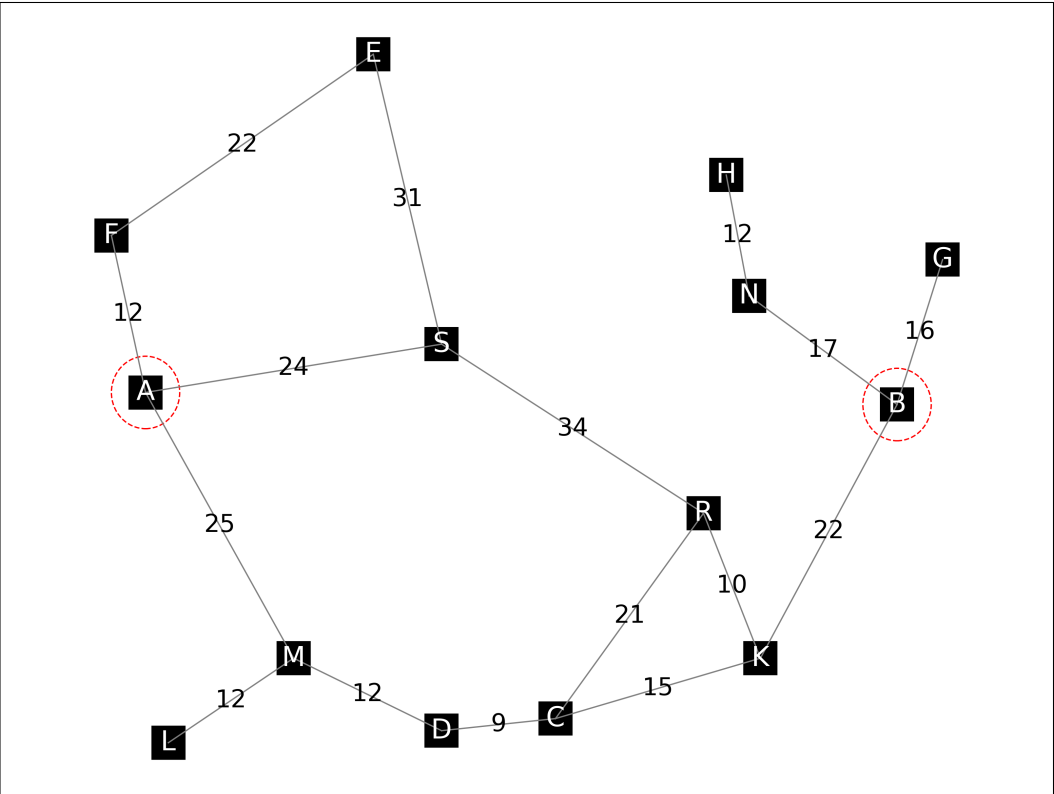


图 4.1: 一个虚构的城市间简化地图，其中字母代表城市，数字代表道路的长度

根据这个情景，我们可以提炼出以下要素：

- **初始状态 (Initial State)**
搜索的起点：A 市。
- **动作集合 (Actions)**
在某一状态下可执行的操作集合。例如：在状态 A 下，可能的动

作集合是前往 F、S 或 M 三座城市。

- **转移模型 (Transition Model)**

描述执行动作后的状态变化。例如，从状态 A，选取走长度为 24 的这条路径，则转移到状态 S。

- **目标测试 (Goal Test)**

判断当前状态是否为目标状态。例如：是否到达了 B 市。

- **路径成本 (Path Cost)**

衡量解决方案的总代价。例如：总的行驶距离。

一个符合以上定义的状态搜索问题是：根据地图4.1，找到从 A 市到 B 市的最短路径。

不过，这样的定义其实是被简化过的。在真实情况中，状态不一定是离散的，可能是连续的。转移模型可能不是确定的，而是随机的；也就是说，采取一个动作后，会以不同的概率转移到其他不同状态。状态也可能不是完全可观测的，而是部分可观测的；比如打麻将时，我只能看到自己的牌和别人打出来的牌，但是无法知道其他人手中是什么牌。

4.2 算法框架

在了解了状态搜索问题的定义后，本小节会介绍两种适用于状态搜索问题的算法框架。在同一个框架中，搜索的策略可以变化。

4.2.1 树搜索 (Tree Search)

树搜索框架是把一个状态搜索问题的求解过程抽象成了一个树状结构的生长过程。自然界中，树从根开始生长，在生长过程中会不断往不同方向分出新树枝，而每个树枝又会在长到一定程度后继续分出树枝，这个现象

很像是：在搜索过程中，在某个状态采取不同动作，然后转移到新的状态的过程。

类比到状态搜索问题上，我们将根节点视为初始状态，任意的树节点都代表一个状态，长出树枝的过程就是执行动作或扩展路径的过程。同时，我们将所有即将长出新树枝的节点称为前沿 (Fringe)，每次都前沿中选取出一个节点：首先检查它是否是目标状态，如是，则停止搜索；若不是，则扩展此节点的后续状态，将这些后续状态放入前沿。这就是树搜索框架的基本思路。

树搜索的伪代码如下所示：

```
function Tree-Search(problem, fringe) returns a solution, or
failure
fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
loop do
    if fringe is empty then return failure
    node ← Remove-Front(fringe)
    if Goal-Test(problem, State[node]) then return node
    fringe ← InsertAll(Expand(node, problem), fringe)

function Expand(node, problem) returns a set of nodes
successors ← the empty set
for each action, result in Successor-Fn(problem, State[node]) do
    s ← a new Node
    Parent-Node[s] ← node
    Action[s] ← action
    State[s] ← result
    Path-Cost[s] ← Path-Cost[node] + Step-Cost(node, action, s)
    Depth[s] ← Depth[node] + 1
    add s to successors
```

```
return successors
```

在这个框架中，前沿 (伪码中的 fringe 变量) 是一个用于储存节点的数据结构。从前沿中取出一个状态，直接做目标测试，即判断该节点是不是目标状态。如果不是目标状态，那么进行扩展 (伪码中的 Expand 函数)，把此状态下采取所有不同动作后能得到的新状态记录下来，插入前沿中。然后继续从前沿中取出下一个状态，重复上述过程，直到找到目标状态或前沿中的节点为空。值得注意的是，在扩展过程中，被扩展的节点将自己的父节点设置为当前节点。这样的操作，实际是为了记录从初始状态到该节点的整条路径。

从伪码中不难看出，搜索问题的本质就是用特定方法遍历所有可能状态，而这个遍历的次序取决于搜索策略 (Search Strategy)。在树搜索框架中，不同的搜索策略就是，按照什么顺序从前沿中取出状态。不同取出节点的顺序会对算法的复杂度产生不同影响，这一点我们会在后续内容中详细展开。

不过，虽然树搜索框架能够遍历到所有状态，但它也存在一个问题：有些状态会反复进入前沿。部分搜索问题不需要重复扩展状态，因此需要用图搜索来解决这个问题。

4.2.2 图搜索 (Graph Search)

图搜索可以看作是在树搜索框架上进行了一些改动，其伪代码如下所示。

```
function Tree-Search(problem, fringe) returns a solution, or
    failure
    fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
    loop do
        if fringe is empty then return failure
```

```
node ← Remove-Front(fringe)
if Goal-Test(problem, State[node]) then return node
fringe ← InsertAll(Expand(node, problem), fringe)
end

function Graph-Search(problem, fringe) returns a solution, or
failure
  closed ← an empty set
  fringe ← Insert(Make-Node(Initial-State[problem]), fringe)
  loop do
    if fringe is empty then return failure
    node ← Remove-Front(fringe)
    if Goal-Test(problem, State[node]) then return node
    if State[node] is not in closed then
      add State[node] to closed
      fringe ← InsertAll(Expand(node, problem), fringe)
    end
  end
```

以上伪码分别展示了树搜索框架和图搜索框架，橙色的代码是图搜索相较于树搜索增加的部分。二者的区别在于，图搜索新增了一个名为 `closed` 的数据结构，用于储存已经被扩展过的状态；并且只对不在此集合中的状态进行扩展，这样就可以避免重复探索同一个状态。

同时，需要注意的是：图搜索虽然避免重复探索同一个状态，但是这些相同状态的父节点可能不同，也就是说：这些状态相同的节点代表了不同的到达这个状态的路径。因此，图搜索并不能穷尽所有路径的可能，在某些情况下找不到到达终止状态的最优路径。也就是说，图搜索可能保证算法的完备性，但不能保证算法的最优性。这一点，我们将在4.3.2节中展开讨论。

图分离性 (Graph Separation Property)

因为图搜索框架不会探索已经被扩展的状态，因此图搜索具有图分离性，指的是：在图搜索过程中，前沿总是将已访问的节点和未访问的节点分离开。也就是说，在图搜索过程中，从任意一个的已访问的节点到任意一个未访问节点的任意一条路径上都必有一个节点在前沿中。这个结论留给读者自行证明。

以上是对于树搜索和图搜索的介绍，针对一个具体的状态搜索问题，我们可以先设计好节点的结构和连接节点之间的转移路径，然后结合状态搜索问题的五个要素，套用一种算法框架。不过如果想要进一步提高搜索效率，就必须找到适合解决该搜索问题的搜索策略，这也是我们下一节要讨论的内容。

4.3 搜索策略

一种搜索策略通过从前沿中选择节点的顺序来定义。不同的顺序直接影响算法效率与解的质量。实际应用中，需根据问题规模、路径质量要求及资源约束选择适合的搜索策略。

在本节中，我们将从以下几个维度评估不同的搜索策略：

- **完备性 (Completeness)**：如果存在解决方案，是否一定能找到。
- **时间复杂度 (Time Complexity)**：扩展/生成节点的数量。
- **空间复杂度 (Space Complexity)**：在前沿中需要存储的最大的节点数。
- **最优性 (Optimality)**：该策略是否总能找到最优（如：成本最低）的解决方案。

其中，时间和空间复杂度将通过以下变量来计算：

- b 搜索树中所有节点的最大的度。
- d 最优解决方案的深度。
- m 搜索树的最大可能的深度。

在了解了如何评估搜索策略后，我们可以正式开始学习搜索策略。根据是否需要用到将来路径中的未知节点的信息，我们可以把搜索策略分为以下无信息搜索和有信息搜索两类。

4.3.1 无信息搜索 (Uninformed Search/Blind Search)

无信息搜索也可以被称为盲目搜索，是只用到已知路径中的节点信息的搜索策略，它包含以下几种方法。

宽度优先搜索 (Breadth-First Search)

宽度优先搜索是一种简单的策略。首先扩展根节点，然后扩展根节点下一层的所有后继节点，接着是扩展这些后继节点的下一层所有后继节点，依此类推，直到检测到目标节点。我们可以把它理解成以“层序扩展”的方式遍历状态空间，优先探索当前层的所有可能状态，再深入下一层。

在算法实现上，计算机在每一次目标测试失败后，都会扩展最浅的未扩展节点。此时前沿相当于先进先出的队列。

宽度优先搜索的完备性、时间复杂度、空间复杂度、最优性如下所示：

- 完备性：如果 b 是有限的，则完备。
- 时间复杂度： $1 + b + b^2 + \dots + b^d = O(b^d)$ （按照目标所在的深度 d 指数级增长）。
- 空间复杂度： $O(b^d)$ （在找到目标节点前会存储搜索过程中每一层

的节点)。

- 最优性: 当每一步的路径成本都相同时, 可以找到最优方案; 一般情况下无法计算出成本最低的方案。

深度优先搜索 (Depth-First Search)

与宽度优先搜索相反, 深度优先搜索总是扩展当前前沿中最深的未扩展节点。首先扩展根节点的后续节点, 然后检测其中一个后续节点, 若此节点不是目标状态, 则继续扩展它的后续节点, 直到当前节点或当前节点的所有后续节点都被检测完, 才会继续以同样方式去检测当前节点的兄弟节点。我们可以把它理解成以“不撞南墙不回头”的方式探索状态空间, 优先沿着一条路径尽可能深地推进, 直到无法继续 (无未访问的后继节点或达到目标), 再回溯到上一个节点, 探索其他分支。在算法实现上, 此时前沿相当于先进后出的栈。

深度优先搜索的完备性、时间复杂度、空间复杂度、最优性如下所示:

- 完备性: 一般是完备的, 若在深度无限或存在循环的状态空间可能会无法找到解决方案。
- 时间复杂度: $O(b^m)$: 当 m 远大于 d 时, 时间复杂度将很差。
- 空间复杂度: $O(bm)$, 即线性空间存储 (在搜索过程中, 最多探索 m 层, 每层存储 b 个节点)。
- 最优性: 不能保证。

对比宽度优先搜索和深度优先搜索, 我们可以看出深度优先算法虽然会在搜索上花费较多的时间, 但能够大幅度减少存储空间, 而宽度优先算法虽然可以在搜索上节省时间, 但是需要占用大量存储空间。所以实际应用中, 当计算存储比较有限时, 我们更倾向于使用深度优先搜索方法, 并且通过以下两种改进方法来减少深度优先搜索所需要的时间。

深度限制搜索 (Depth-limited Search)

设置了最大深度的深度优先搜索，可以提高深度优先搜索的速度，但如果设置的深度太浅，可能会找不到解。

迭代深度搜索 (Iterative Deepening Search)

解决了深度限制搜索不完备的问题。迭代增加深度，对不同的深度使用深度限制搜索，直至找到解。这一算法的完备性、时间复杂度、空间复杂度、最优性如下所示：

- 完备性：和深度优先搜索一样。
- 时间复杂度： $(d+1)b^0 + db^1 + (d-1)b^2 + \cdots + b^d = O(b^d)$ 。
- 空间复杂度： $O(bd)$ 。
- 最优性：当每一步的路径成本都相同时可以找到最优方案。

迭代深度搜索可以减少深度优先搜索的时间复杂度，不过也会带来其他问题。每一轮搜索都从初始状态开始，可能存在一定的浪费。

一致代价搜索 (Uniform Cost Search)

无论是宽度优先搜索还是深度优先搜索，它们都没有办法在单步路径消耗不等的搜索问题中找到最优解。而一致代价搜索是找寻最优解的经典算法。

具体讲，对于前沿中的每个状态 n ，记 $g(n)$ 为从初始状态沿着路径到 n 所消耗的总路径成本，在扩展时，从前沿中优先选取 $g(n)$ 最小的节点进行扩展。如果所有的单步路径成本相等，此算法等价于宽度优先搜索。

值得关注的是，不管使用树搜索框架还是图搜索框架，一致代价搜索策略都能找到最优路径。此结论留给读者自行证明。同时，在学习了[4.3.2节](#)

中的一致启发式函数后，此结论也是显然的。

一致代价搜索的完备性、时间复杂度、空间复杂度、最优性如下所示：

- 完备性：如果路径成本 $\geq \epsilon$ ，则是完备的。
- 时间复杂度： $O(b^{C^*/\epsilon})$ （假设 C^* 是最优路径的成本且在找到最优路径时，所有 $g(n) \leq C^*$ 的节点都被扩展）。
- 空间复杂度： $O(b^{C^*/\epsilon})$ （假设 C^* 是最优路径的成本且在找到最优路径时，所有 $g(n) \leq C^*$ 的节点都被扩展）。
- 最优性：能找到最优解决方案。

注：每一个动作的路径成本至少是 ϵ 。

小结

了解了不同的无信息搜索策略后，我们在表4.1中展示了不同搜索策略的对比。

表 4.1: 不同无信息搜索策略的对比

评判标准	宽度优先 搜索	一致代价 搜索	深度优先 搜索	深度限制 搜索	迭代深度 搜索
完备性	Yes*	Yes*	No	Yes, 若 $l \geq d$	Yes
时间复杂度	b^d	$b^{\lceil \frac{C^*}{\epsilon} \rceil}$	b^m	b^l	b^d
空间复杂度	b^d	$b^{\lceil \frac{C^*}{\epsilon} \rceil}$	bm	bl	bd
最优性	Yes*	Yes	No	No	Yes*

* 代表单步代价相同时可找到最优解

ϵ 是最小的单步成本

C^* 是最优解的总路径成本

l 是深度限制搜索中允许的最大搜索深度

整体来看,大部分无信息搜索都是可以找到终止状态,但时间复杂度都是指数级的,意味着计算机需要消耗大量的时间来寻找解。造成这个现象的原因在于无信息搜索是对所有可能状态的一种盲目遍历,当面对终止状态深藏于状态空间深处或路径分支极多时,这种粗暴的遍历方式会将很多明显无用的路径都试一遍,十分低效。而我们下一节介绍的有信息搜索能引导计算机优先探索高潜力路径,在降低资源消耗的同时逼近最优解。

4.3.2 有信息搜索 (Informed Search/Heuristic Search)

有信息搜索也被称为启发式搜索,这种搜索策略会用到未来节点的信息,估计出一个可能离目标状态最近的路径,从而能比无信息搜索更有效地找到终止状态。

在有信息搜索策略中,我们通常以 $g(n)$ 表示从初始状态到当前状态的路径成本总和,以 $h(n)$ 表示对当前状态到目标状态的路径成本总和的估计。 $h(n)$ 被称为启发式函数 (Heuristic Function),可以根据不同的搜索问题进行不同设计。在大多数情况下,我们都无法准确地估计当前状态到目标状态的最优路径成本总和,即 $h^*(n)$ 。

一致代价搜索是使用 g 函数对待扩展的节点进行排序,那么我们能不能使用 h 函数来排序呢?下面我们介绍两种不同的搜索策略,来看启发式函数在搜索问题中的应用。

贪婪算法 (Greedy Search)

贪婪算法使用 $h(n)$ 对节点排序,并挑选 $h(n)$ 最小的节点扩展。从原理上,贪婪算法似乎总是选择扩展距离目标节点最近的状态。下面我们继续使用图4.1中的示例来展示贪婪算法的过程。

我们设定启发式函数为 $h_{\text{SLD}}(n)$ 为直线距离函数,代表了地图上某个地点到目的地的直线距离,如果 $h_{\text{SLD}}(n)$ 越小,说明状态 n 离目标状态越近,越值得被扩展。地图上每个点到 B 的 h_{SLD} 值如表4.2所示:

表 4.2: 虚构地图中各市到 B 的 h_{SLD} 值

地点	h_{SLD} 值	地点	h_{SLD} 值
A	66	B	0
C	35	D	48
E	54	F	70
G	16	H	24
K	22	L	69
M	57	N	17
R	19	S	40

我们想要从 A 市出发前往 B 市，那么根据贪婪算法，首先要去 S 市，因为在 A 可以扩展的三个市中，S 市的 h_{SLD} 值 (40) 要小于 M 市 (57) 和 F 市 (70)，同理，从 S 市开始会继续前往 R 市 ($h_{SLD}(R)$ 小于 $h_{SLD}(E)$ 和 $h_{SLD}(A)$)，然后从 R 市走向 K 市，最后到 B，搜索到了从 A 到 B 的路径。不过，这条路并不是最优的，很明显 “ $A \rightarrow M \rightarrow D \rightarrow C \rightarrow K \rightarrow B$ ” 这条路的实际总路径成本要更小一些。而这也是为什么这个算法名为“贪婪”，因为它在任何状态时都只想要找到距离目标最近的路，不过这种短视的贪婪有时反而会绕路。

同时值得注意的是，贪婪算法有时不一定能找到路径，例如在地图4.1中我们要从 H 市到 S 市，在启发式函数同样为 h_{SLD} 的情况下，我们会先转移到 N，但到了 N 市以后，根据贪婪算法，我们会继续转移到 H 市，因为此时的 H 市的 h_{SLD} 值 (24) 要小于 B 市 (40)，然后就陷入循环，找不到能到达 S 市的路了。

因此，我们对贪婪算法有如下总结：

- 完备性：不一定完备，因为可能陷入到循环之中。在有限空间且检查了重复状态时完备。

- 时间复杂度: $O(b^m)$ 。不过一个好的启发式函数能进一步降低时间复杂度。
- 空间复杂度: $O(b^m)$ 需要记录所有的节点。
- 最优性: 不能保证找到最优解。

A* 搜索 (A* Search)

A* 搜索使用 $f(n) = g(n) + h(n)$ 作为排序的函数, 其中 f, g, h 的定义如下所示:

- $g(n)$: 从初始状态到节点 n 已经消耗的总路径成本
- $h(n)$ = 预估从节点 n 到目标的总成本
- $f(n)$ = 预估一条经过节点 n 的完整路径的总路径成本

回到图4.1的例子中, 假设我们这次想从 C 市到 B 市, 仍然以 h_{SLD} 作为启发式函数。如果根据贪婪搜索, 那么我们会转移到 R 市, 因为 $h_{\text{SLD}}(R)$ 小于 $h_{\text{SLD}}(K)$ 和 $h_{\text{SLD}}(D)$ 。这样最终只会得到 “ $C \rightarrow R \rightarrow K \rightarrow B$ ” 的路, 明显不是最优解。但如果我们用 A* 搜索, 可以得到 $f(R) = 21 + h_{\text{SLD}}(R) = 40$, $f(K) = 15 + h_{\text{SLD}}(K) = 37$, $f(D) = 9 + h_{\text{SLD}}(D) = 57$, 因此我们会转移到 K 市, 最后得到最优路径 “ $C \rightarrow K \rightarrow B$ ”。由此可以看出, A* 搜索要比贪婪搜索更容易找到最优路径。

不过, 如果想要 A* 搜索能够保证完备性和最优性, 我们需要让启发式函数满足特定条件, 下面我们会介绍启发式函数的一些性质。

可采纳的启发式函数

如果一个启发式函数 $h(n)$ 是可采纳的 (Admissible), 那么它具有如下性质:

对于任意的节点 n ,

$$0 \leq h(n) \leq h^*(n),$$

其中 $h^*(n)$ 是从状态 n 到目标状态的最优路径的总路径成本。

对于可采纳的启发式函数，我们有以下性质：

定理 4.3.1.

使用树搜索和可采纳启发式函数的 A^* 搜索是最优的。

定理4.3.1的证明：

为了证明使用树搜索和可采纳启发式函数的 A^* 搜索是最优的，即需要证明此时 A^* 搜索总能找到最优解。假设存在一个次优解 G' 已经存在于前沿中，即 G' 的状态是终止状态，但它对应的路径的成本总和并不是最小的。同时，假设节点 n 在前沿中的通往最优解 G 的最短路径上的一个未扩展节点。

容易发现下列结论：

- $f(G') = g(G')$ ，因为 $h(G') = 0$ 。
- $g(G') > g(G)$ ，因为 G' 是次优的。
- $g(G) \geq f(n)$ ，因为 $g(G) = g(n) + h^*(n)$ 且 $h^*(n) \geq h(n)$ 。

因此， $f(G') > f(n)$ 。

由于 A^* 算法在每一步选择 f 值最小的节点进行扩展，因此 A^* 算法不会选择 G' 进行扩展。最终， A^* 算法将扩展到最优解 G 。

因此，使用树搜索和可采纳启发式函数的 A^* 搜索是最优的。

□

可采纳性质最大的特点就是不会高估路径成本。这一条件虽然限制了启发式函数的选择范围，但在一定程度上保证它能正确指示距离目标状态最近的搜索方向，并找到最优路径。

同时，我们还有如下的结论：使用树搜索和可采纳启发式函数的 A* 搜索，在整个搜索过程中该算法扩展到的节点 n 有如下性质：

- 当 $f(n) < C^*$ ，节点 n 肯定会被扩展
- 当 $f(n) = C^*$ ，节点 n 可能会被扩展。若存在多条最优路径，该算法可能找到其中的一条便终止搜索
- 当 $f(n) > C^*$ ，节点 n 肯定不会被扩展

其中 C^* 是最优路径的总成本。

一致的启发式函数

如果一个启发式函数 $h(n)$ 是一致的 (Consistent)，那么它具有如下性质：

对于任意的节点 n ,

$$h(n) \leq c(n, a, n') + h(n')$$

其中 n 为任意节点， n' 为节点 n 的其中一个直接后续节点， $c(n, a, n')$ 为从 n 到 n' 的真实路径成本。

图4.2展示了一个一致的启发式函数的示意图。

事实上，一致是比可采纳更严格的限制，如果一个启发式函数是一致的，那么它必定是可采纳的。证明留给读者。

对于一致的启发式函数，我们有以下性质：

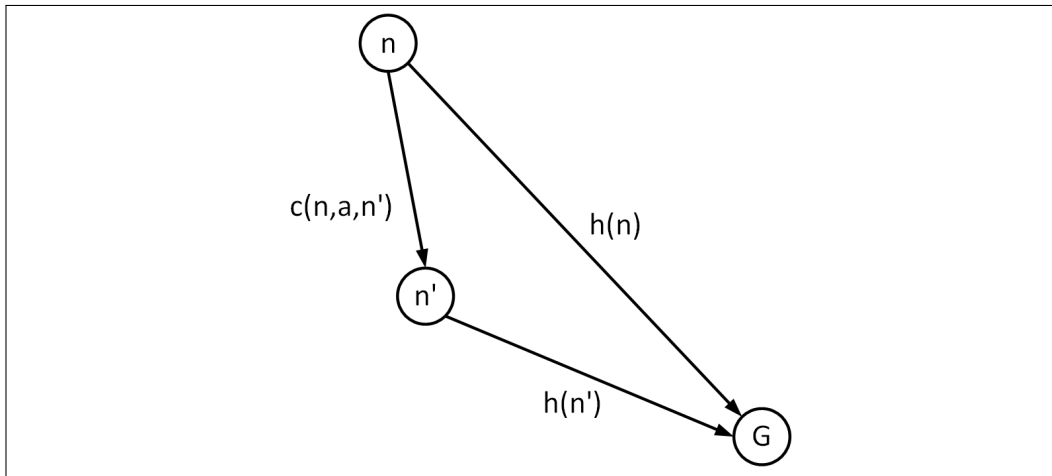


图 4.2: 一致的启发式函数示意图, 表示在启发式函数具有一致性的时候, $h(n) \leq c(n, a, n') + h(n')$ 恒成立

定理 4.3.2.

使用图搜索和一致启发式函数的 A^* 搜索是最优的。

定理 4.3.2 的证明.

考虑证明一个更强的结论: 使用图搜索和一致启发式函数的 A^* 搜索过程中, 对于任何一个被扩展的节点 n , 其被扩展时的路径是最优的 (图搜索保证任何一个节点只会被扩展一次)。

为了证明以上结论, 我们需要首先证明如下性质: 使用图搜索和一致启发式函数的 A^* 搜索中, 对于任意的搜索路径 $n_0, n_1, \dots, n_k$, 都有

$$f(n_0) \leq f(n_1) \leq \dots \leq f(n_k)$$

, 即 f 值沿路径单调不减。这个性质利用一致启发式函数的定义就可以容易证明。

那么, 使用图搜索和一致启发式函数的 A^* 搜索过程中, 前沿中存在一个节点 n , 把它视为终止状态, 但这条路径不是最优的, 即此时 $f(n) > f^*(n)$ 。此时, n 仍然是未访问的节点。又根据图搜索的图分离性质, 此时在前沿中

表 4.3: 不同搜索框架下 A* 搜索的最优性变化

	可采纳的启发式函数	一致的启发式函数
树搜索	一定找到最优路径	一定找到最优路径
图搜索	不保证找到最优路径	一定找到最优路径

必存在一个节点 n' ，它在从初始状态到 n 的最优路径上。那么，根据 f 值沿路径单调不减的性质，我们有 $f(n') \leq f^*(n)$ 。因此， $f(n') < f(n)$ ，则此时节点 n 不会被选中出来被扩展。

所以，对于任何一个被扩展的节点 n ，其被扩展时的路径肯定是最优的。也就是说，当目标节点被扩展时，到它的路径也是最优的。这也就是说，使用图搜索和一致启发式函数的 A* 搜索是最优的。

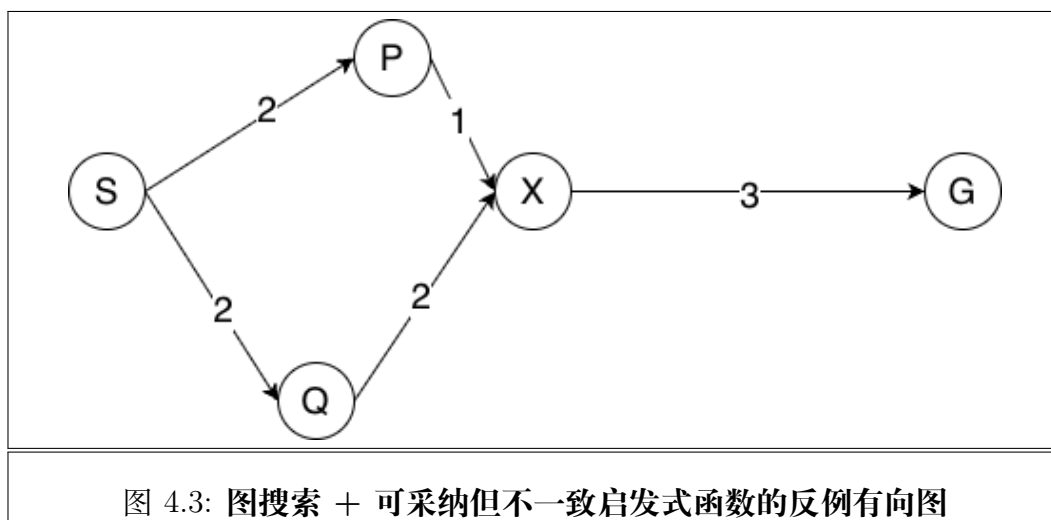
□

至此我们已经介绍了有关启发式函数的两个重要性质，并且分别展示了在不同的搜索框架下该如何选择启发式函数才能保证搜索到最优路径，具体总结如表4.3所示。

通过定理4.3.1和定理4.3.2，我们能够证明当在树搜索框架下使用可采纳的启发式函数和在图搜索框架下采用一致的启发式函数时，A* 搜索是肯定可以找到最优解的。又因为一致是比可采纳更加严格的条件，所以在树搜索框架下使用一致的启发式函数的 A* 搜索也可以找到最优解。但图搜索因为不会重复扩展已经被扩展的状态，在搜索路径的范围上比树搜索理论上会少一些，若此时还使用约束相对较少的可采纳的启发式函数进行 A* 搜索，则可能找不到最优解。下面给出一个具体的搜索实例，说明在图搜索框架下采用 A* 搜索，即使启发式函数是可采纳的，也可能找不到最优解：

图搜索 + 可采纳但不一致的启发式函数并不能找到最优路径的实例

考虑有向图4.3，起点为 S ，目标结点为 G ，中间结点为 P, Q, X 。



边及其代价如下：

$$\begin{aligned} c(S, P) &= 2, & c(S, Q) &= 2, \\ c(P, X) &= 1, & c(Q, X) &= 2, \\ c(X, G) &= 3. \end{aligned}$$

从各结点到目标结点 G 的最短路径代价分别为：

$$\begin{aligned} \text{dist}(X, G) &= 3, \\ \text{dist}(P, G) &= 1 + 3 = 4, \\ \text{dist}(Q, G) &= 2 + 3 = 5, \\ \text{dist}(S, G) &= 2 + 1 + 3 = 6, \end{aligned}$$

因此最优路径为 $S \rightarrow P \rightarrow X \rightarrow G$ ，总代价为 6；而另一条次优路径 $S \rightarrow Q \rightarrow X \rightarrow G$ 的代价为 $2 + 2 + 3 = 7$ 。

现在定义启发式函数 h 如下：

$$h(G) = 0, \quad h(X) = 1, \quad h(P) = 4, \quad h(Q) = 2, \quad h(S) = 5.$$

(1) 可采纳性

对每个结点 n 有

$$0 \leq h(n) \leq \text{dist}(n, G),$$

例如

$$h(S) = 5 < \text{dist}(S, G), \quad h(P) = 4 = \text{dist}(P, G), \quad h(Q) = 2 < \text{dist}(Q, G) = 5, \text{ 等等。}$$

因此 h 是一个可采纳的启发式函数。

(2) 不一致性

一致性要求对任意边 $u \rightarrow v$ 满足

$$h(u) \leq c(u, v) + h(v).$$

检查可知：

$$S \rightarrow Q : \quad h(S) = 5 \not\leq c(S, Q) + h(Q) = 2 + 2 = 4;$$

$$P \rightarrow X : \quad h(P) = 4 \not\leq c(P, X) + h(X) = 1 + 1 = 2.$$

因此该启发式虽可采纳，但不一致。

(3) 图搜索版 A* 的搜索过程

下面给出图搜索 A* 算法在此例子上的完整执行过程。记 fringe 为按照 $f(n) = g(n) + h(n)$ 从小到大排序的优先队列，closed 为已经被扩展过的状态集合。为了便于说明，用

$$\text{fringe} = \{ n_1(f_1), n_2(f_2), \dots \}$$

表示队列中各结点及其 f 值（按 f 从小到大排列）。

1. 初始化

起点 S 的代价为

$$g(S) = 0, \quad h(S) = 5, \quad f(S) = 5.$$

此时

$$\text{fringe} = \{ S(5) \}, \quad \text{closed} = \emptyset.$$

2. 第 1 步：取出并扩展 S

从 fringe 中取出 f 最小的结点 S 。 S 不是目标结点，将其状态加入 closed：

$$\text{closed} = \{ S \}.$$

对 S 扩展，生成后继 P 与 Q ：

$$g(P) = g(S) + c(S, P) = 0 + 2 = 2, \quad h(P) = 4, \quad f(P) = 2 + 4 = 6,$$

$$g(Q) = g(S) + c(S, Q) = 0 + 2 = 2, \quad h(Q) = 2, \quad f(Q) = 2 + 2 = 4.$$

将新结点加入 fringe，得到

$$\text{fringe} = \{ Q(4), P(6) \}, \quad \text{closed} = \{ S \}.$$

3. 第 2 步：取出并扩展 Q

取出 f 最小的 $Q(4)$ 。 Q 不是目标结点，将其状态加入 closed：

$$\text{closed} = \{ S, Q \}.$$

扩展 Q ，生成后继 X ：

$$g(X) = g(Q) + c(Q, X) = 2 + 2 = 4, \quad h(X) = 1, \quad f(X) = 5.$$

加入 fringe 后：

$$\text{fringe} = \{ X(5), P(6) \}, \quad \text{closed} = \{ S, Q \}.$$

4. 第 3 步：取出并扩展 X （第一次到达）

取出 $X(5)$ 。 X 不是目标结点，将其状态加入 closed ：

$$\text{closed} = \{ S, Q, X \}.$$

扩展 X ，生成后继 G ：

$$g(G) = g(X) + c(X, G) = 4 + 3 = 7, \quad h(G) = 0, \quad f(G) = 7.$$

此时

$$\text{fringe} = \{ P(6), G(7) \}, \quad \text{closed} = \{ S, Q, X \}.$$

5. 第 4 步：取出并扩展 P

取出 $P(6)$ 。 P 不是目标结点，将其状态加入 closed ：

$$\text{closed} = \{ S, Q, X, P \}.$$

扩展 P ，得到到达同一状态 X 的一条更优路径：

$$g'(X) = g(P) + c(P, X) = 2 + 1 = 3, \quad h(X) = 1, \quad f'(X) = 4.$$

将这个新的结点（记作 X' ，其状态仍为 X ，但 g 值更小）加入 fringe ：

$$\text{fringe} = \{ X'(4), G(7) \}, \quad \text{closed} = \{ S, Q, X, P \}.$$

6. 第 5 步：取出 X' ，但不再扩展

取出 $X'(4)$ 。此时它的状态为 X ，而 X 已经在 closed 中，说明该状态此前已经被扩展过。在这种图搜索策略下，遇到这种情况时不会再次扩展该状态，也不会用更小的 g 值去更新之前的搜索结果，相当于直接丢弃这条更优路径。因此，在这一轮之后：

$$\text{fringe} = \{ G(7) \}, \quad \text{closed} = \{ S, Q, X, P \}.$$

7. 第 6 步：取出 G 并终止

取出 $G(7)$ 。 G 为目标结点，搜索在此终止，返回路径

$$S \rightarrow Q \rightarrow X \rightarrow G,$$

其总代价为

$$g(G) = 7.$$

而真正的最优路径为 $S \rightarrow P \rightarrow X \rightarrow G$ ，总代价为 $2 + 1 + 3 = 6$ ，明显优于算法最终得到的代价 7。可见，在这种图搜索设置下，当启发式函数仅仅可采纳而不一致时，即便始终选择当前 f 值最小的结点扩展， A^* 也可能返回非最优解。

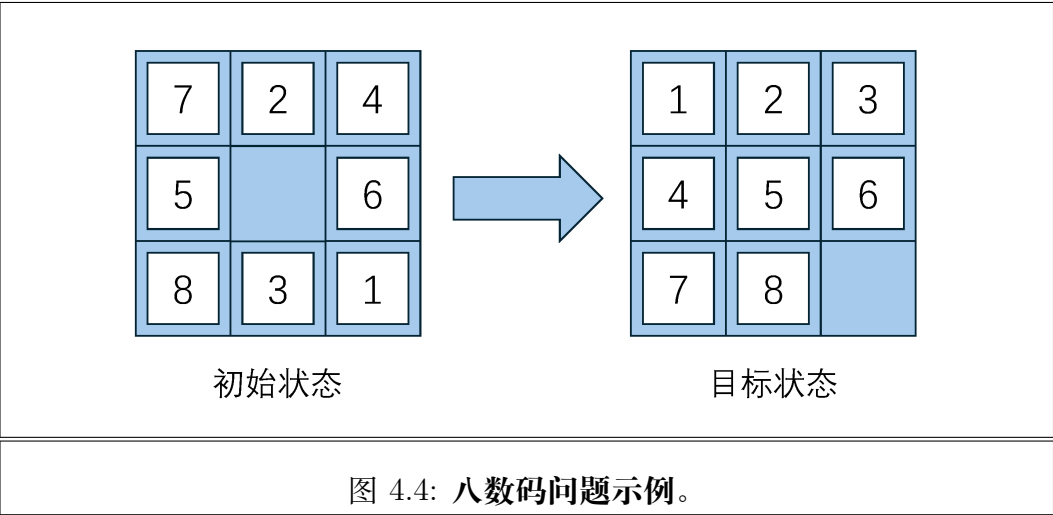
显然，对于任意的节点 n 都有 $h(n) = 0$ 的启发式函数是一致的。而之前讲述的一致代价搜索可以看作采用 $h(n) = 0$ 的启发式函数的 A^* 搜索。因此，不管使用树搜索还是图搜索，一致代价搜索都能找到最优解。

启发式函数的案例

我们在前面探讨了什么样的启发式函数能让 A^* 搜索具有完备性和最优性，但即使知道了可采纳性和一致性这两个很有效的约束条件，启发式函数也不会由计算机自动生成，需要人们根据搜索问题有针对性地进行设计。下面我们会通过一个案例——八数码问题 (8-Puzzle) 来展示启发式函数的构造方法。

八数码问题是人工智能领域中经典的状态搜索问题，属于滑动拼图类游戏的典型代表。它的规则是在一个九宫格中放置 8 个标有数字（通常是 1-8）的方块和 1 个空白格，初始状态下方块随机排列，目标是通过滑动相邻的方块（仅允许空白格与上下左右四个方向的方块交换位置），将数字按顺序排列为目标状态。（如图 4.4 所示）。

假设我们要解决的就是图 4.4 中的八数码问题，根据状态搜索问题的定义，我们已经可以确定搜索的初始状态和目标状态，而动作是空白格与上



下左右四个方向的数字交换位置，经过交换后的数字与空白格排列作为转移后的状态（空白格最多可以往 4 个方向移动，扩展出 4 个新状态，即 b 是 4），每一步的单步路径成本记为 1。

经过上述分析，我们可以发现这个问题存在重复状态的可能而且深度未知，用无信息搜索策略效率会比较低。因此可以考虑使用 A* 搜索来找出解决 8 数码问题的最优方案。

根据定理 4.3.1，我们需要设计一个可采纳的启发式函数。当前有两种设计思路：

- 1) h_1 : 计算当前状态下，1-8 这几个数字里面有多少个数字的位置和目标状态不同，此时初始状态的启发式函数值 $h_1(s_0) = 1+0+1+1+1+0+1+1 = 6$
- 2) h_2 : 计算当前状态下，1-8 这几个数字里面位置错误的数字距离目标位置的曼哈顿距离¹之和。即，当作每个数字都可以沿水平或垂直方向自由移动。此时初始状态的启发式函数值 $h_2(s_0) = 4+0+3+3+1+0+2+1 = 14$

¹曼哈顿距离是指平面上两个点 $((x_1, y_1), (x_2, y_2))$ 之间的水平距离 $(|x_2 - x_1|)$ 与垂直距离 $(|y_2 - y_1|)$ 之和，也称为折线距离。

那么这两个启发式函数是怎么设计出来的呢? h_1 和 h_2 都是对 8 数码问题剩余路径长度的估计, 但如果简化了八数码的动作规则, 让数字可以自由地移动到任何地方, 而不是只有空白格可以和相邻的数字交换位置, 那么 h_1 将给出这种情况下最优解的准确长度。同理, 如果动作规则变成一个数字可以和上下左右的任意数字交换, 那么 h_2 将给出最优解的准确长度。像这样对原问题减少了动作限制条件的问题被称为松弛问题 (Relaxed Problem)。我们可以把松弛问题的状态空间图看作是在原始状态空间上增加了一些新的可行路径。那么根据最优性的定义, 原问题的任意最优解肯定是松弛问题的一个解, 但是, 如果增加的边提供了捷径, 松弛问题可能有更好的解。因此, 松弛问题中最优解的路径成本可以作为原问题的一个可采纳的启发式函数。同时, 若启发式函数是松弛问题的最优解, 所以也会满足图 4.2 中的三角不等式, 因此该启发式函数也是一致的。

了解了这两个可采纳的启发式函数都可以使用, 那么我们应该优先选择哪一个呢? 首先, 我们可以定义两个启发式函数间的支配 (Dominate) 关系。具体讲, 对于任意状态 n , $h_2(n) \geq h_1(n)$ 恒成立, 此时 h_2 支配 h_1 。

随后, 我们可以有如下的性质: 使用树搜索的 A* 搜索算法, 当 h_1 和 h_2 都是可采纳的启发式函数, 且 h_2 支配 h_1 时, 使用 h_2 的 A* 搜索所扩展的节点集合 (除开终止状态对应的节点) 是使用 h_1 的 A* 搜索所扩展的节点集合的子集。这个性质的证明, 需要使用之前提及的一个性质: 使用树搜索和可采纳启发式函数的 A* 搜索, 一定会把任意满足 $f(n) < C^*$ 的节点 n 都扩展过, 其中 C^* 是最优路径的总成本。显然, 如果 $h_2(n) \geq h_1(n)$ 恒成立, 则 $f_2(n) \geq f_1(n)$ 恒成立。进而, 若节点 n 满足 $f_2(n) < C^*$, 则 $f_1(n) < C^*$, 即结论成立。

因此, 既然两个可采纳的启发式函数都能找到最优路径, 为了节省运算时间, 我们一般优先选择函数值高的启发式函数进行 A* 搜索。

小结：关于可采纳和一致的启发式函数的性质

在这里，我们把可采纳和一致的启发式函数的性质罗列出来，以方便读者比较和理解：

关于可采纳的启发式函数的性质：

- 使用树搜索和可采纳启发式函数的 A^* 搜索是最优的。
- 使用树搜索和可采纳启发式函数的 A^* 搜索，一定会把任意满足 $f(n) < C^*$ 的节点都扩展到，其中 C^* 是最优路径的总成本。
- 使用树搜索和可采纳启发式函数的 A^* 搜索，一定不会扩展到满足 $f(n) > C^*$ 的节点，其中 C^* 是最优路径的总成本。
- 使用树搜索的 A^* 搜索算法，当 h_1 和 h_2 都是可采纳的启发式函数，且 h_2 支配 h_1 时，使用 h_2 的 A^* 搜索所扩展的节点集合（除开终止状态对应的节点）是使用 h_1 的 A^* 搜索所扩展的节点集合的子集。

关于一致的启发式函数的性质：

- 启发式函数是一致的，则它一定是可采纳的。
- 使用图搜索和一致启发式函数的 A^* 搜索是最优的。
- 使用树搜索和一致的启发式函数的 A^* 搜索是最优的。
- 使用图搜索和可采纳的启发式函数的 A^* 搜索“不是”最优的。
- 使用图搜索和一致启发式函数的 A^* 搜索中， f 值沿路径单调不减
- 使用图搜索和一致启发式函数的 A^* 搜索中，对于任意被扩展的节点，都是初始状态到该状态的最优路径。

以上是关于 A^* 搜索的内容，可以看出， A^* 搜索综合了一致代价搜索和贪婪搜索的特点，同时考虑了路径的真实路径成本和有启发式函数得到的预估成本，能比较好地引导搜索方向，优先扩展最有潜力的节点，兼具完备性和最优性。

至此我们已经全面了解了状态搜索策略的基础理论，下面我们会进入一个经典的状态搜索问题情景——归结原理，来展示如何使用状态搜索算法解决实际问题。

4.4 状态搜索：归结原理的计算机实现

归结原理的初始状态是知识库和查询的否定，终止状态是包含空子句的知识库。我们希望得到从初始状态到终止状态的路径，同时希望它是最优的——也就是归结次数尽量少。因此，我们可以尝试用状态搜索来解决此问题。此时，状态可以定义为经过若干次（可以为 0 次）归结操作的知识库 KB。

很明显，每次归结操作的代价为 1。因此，宽度优先搜索一定能找到最优路径，只不过面对可归结子句较多的知识库时容易产生组合爆炸，可能需要消耗大量的时间和存储空间。

此外，我们也可以使用 A^* 搜索来解决这个问题。根据归结原理允许一个含有较多文字的子句可以反复和多个不同的子句归结的特点，我们可以运用树搜索框架来搜索最优解，因此需要设计一个可采纳的启发式函数。可以考虑以子句中文字的数量作为启发式函数。很明显，一个包含 n 个文字的子句，至少需要 n 次归结才能得到空子句。因此，我们可以将启发式函数定义为该知识库中的子句的文字数量的最小值，即：先统计每个子句的文字数量，再计算这些数字的最小值。显然，这个启发式函数是可采纳的。当然，启发式函数并不唯一，读者可以自行尝试设计适合归结原理的启发式函数。

4.5 状态搜索的总结和局限性

以上是关于状态搜索的全部内容。状态搜索是人工智能中解决状态空间问题的基础工具，其核心是通过系统遍历状态转移路径，在初始状态与目标状态间寻找可行解，特别适合处理目标明确、状态空间可离散化且需要得到详细的解决方案的问题情景。

下一章我们将聚焦最优化搜索——通过引入目标函数来量化解的优劣，能针对性地达成“找最优”的核心需求，从而解决很多现实中的问题。

第五章 最优化搜索

最优化搜索指的是在状态空间中寻找最优解的过程，即对于给定的目标函数 (Objective Function)，找到一个具体的状态，使这个目标函数最大或最小。典型的，深度学习的训练过程，都是在进行最优化搜索。因此，一般的最优化问题都可以表达成如下式子：

$$\max_{x \in X} \text{objective} - \text{function}(x) \quad (5.1)$$

最优化搜索和状态搜索不同的地方在于，最优化搜索没有明确的初始和终止状态，我们只知道要找到出现最值时的状态，但不知道最值具体是多少。同时，最优化搜索也不需要给出路径，只关注如何快速地找出最优解。

在这一章节中，我们会介绍两类最优化问题的搜索思路——贪婪算法和元启发式方法 (Meta-Heuristics)，并分别介绍不同的思路下的一些经典算法。

5.1 贪婪算法

最优化搜索中的贪婪算法的思路和前面提到的在状态搜索问题中贪婪算法类似，它们都会选择当前状态下的最优决策。在状态搜索中，贪婪算法是会在每个状态下优先选择预估离终点最近的下一个状态，而本节中提到的贪婪算法是比较当前状态和相邻状态的取值，如果当前状态的函数取值

比它的一个某个邻居状态差时，我们就可以把当前状态移动到这个邻居状态，以处于一个更好的状态。

相邻点的取值都大（或都小），那么这个状态就很可能是我们所需要的目标。

在具体的算法上，本节会介绍三种经典的贪婪算法——爬山算法 (Hill-climbing Search)、梯度法和束搜索 (Beam Search)。

5.1.1 爬山算法

爬山算法是比较典型的贪婪算法，它记录当前状态并在每次迭代中移动到更优的相邻状态；当它到达一个没有更优的邻居的状态时，算法终止。因为爬山算法不需要记录一个状态的改变过程，也就不会考虑去搜索当前状态的直接邻居之外的状态，所以实际的算法过程就像一个有健忘症的人在浓雾中试图找到喜马拉雅山的顶峰一样，只能看见眼前有没有相邻的更高山峰，但记不住自己爬过多高的山峰，也看不见更远处有没有更高的山峰。

爬山算法的具体算法伪码如下所示：

```
function Hill-Climbing(problem) returns a state that is a local
    maximum
inputs: problem, a problem
local variables: current, a node
                 neighbor, a node

current ← Make-Node(Initial-State[problem])
loop do
    neighbor ← a highest-valued successor of current
    if Value[neighbor] is NOT bigger than Value[current] then
        return State[current]
```

```
current ← neighbor  
end
```

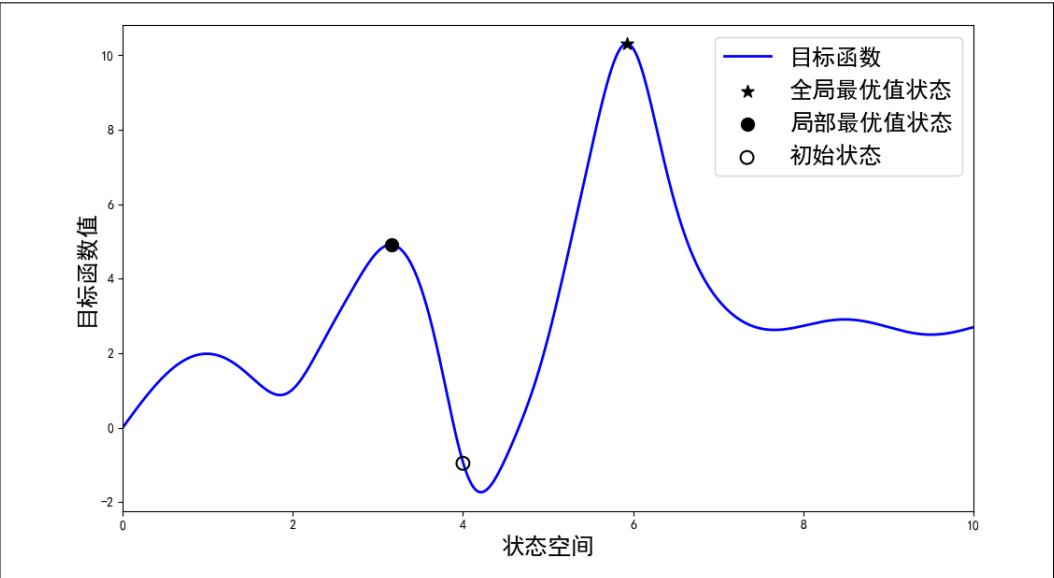


图 5.1: 局部最优值和全局最优值

具体讲，该算法从一个随机状态开始，随后选取更好的相邻状态作为当前状态，并不断重复该过程，这便是爬山算法的主要内容。不过值得注意的是，因为爬山算法只会比较直接相邻的状态，无法参考更远的状态，而且不会储存之前的状态，因此爬山算法容易可能只能找到局部最优解。

假设计算机使用爬山算法搜索如图5.1的目标函数的最大值状态（状态空间是连续的），并随机设置 4 为初始状态。根据爬山算法的原理，计算机会选择左侧的相邻状态进行搜索，然后不断向左侧移动，直到找到了一个比所有相邻状态的值都大的状态，即局部最优状态。但实际上，在初始状态的右侧存在更好的全局最优状态。这便是爬山算法的一大弊端。

5.1.2 梯度法

爬山法比较适合离散状态空间的最优化问题，虽然对于连续的状态空间，我们也可以选择把状态空间给切割成一个个离散的状态空间，然后再使用爬山算法。但有，没有更加适合状态空间连续的最优化问题的搜索方法呢？

爬山算法通过比较当前状态和相邻状态的大小来确定搜索方向，而在连续的状态空间中，我们可以使用梯度 (Gradient) 作为前进方向。梯度描述了一个多元函数在某一点变化最快的方向和速率，即：梯度方向是目标函数增大最快的方向，梯度的反方向是目标函数减小最快的方向。下面我们会用一个实际问题来介绍梯度法的具体应用。

假设图4.1的国家想要修建三个机场，需要选取三个地址坐标，使得每个城市到其最近机场的平方距离之和最小。根据问题情景可知，状态是由三个机场的坐标定义，设三个机场的坐标分别 (x_1, y_1) 、 (x_2, y_2) 、 (x_3, y_3) ，每个 x 和 y 都是连续的。因此，每个状态都是由 6 个连续变量定义，整个状态空间是连续的。设 C_i 是最近机场为机场 i 的城市的集合，那么这个最优化问题的目标函数可以表示为：

$$f(x) = f(x_1, y_1, x_2, y_2, x_3, y_3) = \sum_{i=1}^3 \sum_{c \in C_i} (x_i - x_c)^2 + (y_i - y_c)^2$$

在这个情景中，我们要找的寻找的是：使得目标函数最小的状态。因此，需要找到能让函数值不断变小的状态方向。基于这个思路，我们得出如下的梯度向量以及梯度法搜索过程：

$$\nabla f = \left(\frac{\partial f}{\partial x_1}, \frac{\partial f}{\partial y_1}, \frac{\partial f}{\partial x_2}, \frac{\partial f}{\partial y_2}, \frac{\partial f}{\partial x_3}, \frac{\partial f}{\partial y_3} \right)$$

$$\mathbf{x} \leftarrow \mathbf{x} - \alpha \nabla f(\mathbf{x})$$

其中， ∇f 代表目标函数在 $(x_1, y_1, x_2, y_2, x_3, y_3)$ 处的梯度， x 为参数向量 $(x_1, y_1, x_2, y_2, x_3, y_3)$ ，而 α 代表步长 (Step Size)，通常是一个很小的常数。

因为是最小化问题，我们每次迭代使得状态朝梯度的反方向移动一小步。而步长的具体大小也会对搜索过程产生影响：如果步长太小，可能会导致搜索到最优值的过程比较慢；如果步长过大，则可能导致算法不收敛，来回震荡。

通常，当 $\nabla f = 0$ 的时候，我们可以认定此时可能找到了极小值。然而，在实际使用梯度法的过程中， $\nabla f = 0$ 可能不存在解，因此也可以规定当 $|\nabla f|$ 收敛到某个阈值或者梯度法的迭代次数达到一定数值时停止搜索，则将停止时的状态视为输出。

当然，这是一阶偏导方法，也有二阶偏导方法。这种方法是根据牛顿-拉弗森法 (Newton-Raphson Method)¹来逼近 $\nabla f = 0$ 的解，因此我们可以把梯度法的状态转移过程写成如下形式：

$$x \leftarrow x - H_f^{-1}(x) \nabla f(x)$$

其中， $H_f(x)$ 为 $f(x)$ 二阶导数的海森矩阵 (Hessian Matrix)，其元素 H_{ij} 由元素 $\partial^2 f / \partial x_i \partial x_j$ 给出。不过当参数变量过多的时候，计算海森矩阵并对其求逆的算力开销非常大。因此，我们在实际应用中很少使用二阶的梯度法，在此我们也只做一下简单介绍。

以上是有关梯度法的全部内容，可以看出，梯度法很像是在连续状态空间中的爬山法，因此也比较容易受到初值影响，出现只能找到局部最优解的问题。

5.1.3 束搜索

爬山法和梯度法的搜索结果都会很大程度上受到初始值的影响，那么有没有缓解这种影响的方法呢？束搜索算法提供了一种解决思路，相比于

¹这是一种用于求解形如 $f(x) = 0$ 的方程的通用方法，通过 $x \leftarrow x - f(x)/f'(x)$ 的过程来找到方程的解。

前两个方法只保留一个最优状态，它会从 $k(k > 1)$ 个初始状态开始搜索，并在搜索过程中记录 k 个当前的最优状态。其具体过程如下所示：

- 从 k 个随机生成的状态开始，生成此 k 个状态的所有后续状态。
- 若其中有目标状态，则算法终止。
- 否则，从整个后续状态中选择 k 个最佳的后续，重复此过程。

值得注意的是，束搜索并不是简单地并行使用 k 次初始值爬山法或梯度法。在束搜索运行过程中，有用信息会在不同的搜索进程中传递。如果在选出的 k 个最佳状态中，有两个或以上的状态来自于同一个前驱状态，那就意味肯定至少已经有一个来自初始点不好的搜索进程被放弃了，从而将算力资源集中到了最有可能找到最优值的区域。

不过束搜索并不能完备地解决局部最优问题，如果 k 的取值过小或是初始的 k 个状态的取值不具有多样性，那么最后 k 个状态还是会聚焦在局部最优的区域。增大 k 值会更有机会避免这个问题，但也会因此增大算力消耗。

以上是关于贪婪算法下的几个主要搜索策略的介绍。可以看出，这三个算法都是基于一种“贪心”的思想，在每一步选择当前状态下最优的决策，不去记录多余的状态，通过逐步迭代改进候选解来逼近目标函数的最优值。这种思想可以大幅减少搜索的空间复杂度，并且总是朝着更优解的方向前进。不过也正是因为这个特点，这三个算法均可能陷入局部最优解，而无法找到全局最优解。

5.2 元启发式算法

元启发式算法有着区别于贪婪算法的另一种搜索思路，这类方法通常是从自然现象中获得灵感，通过模拟复杂系统行为解决最优化问题，从而尽可能避免陷入局部最优。同时，在一些实际问题中，难以计算目标函数的

梯度，甚至无法获得目标函数关于 x 的解析式。在这种情况下，可以使用元启发式方法进行优化。

在本节中，我们会介绍两种不同的元启发式算法——模拟退火算法和演化计算。

5.2.1 模拟退火算法

模拟退火算法 (Simulated Annealing) 是从冶金业中的金属退火过程得到启发，对贪婪算法进行了一定程度的调整。贪婪算法思路的最大问题就是从从不向“坏”的后继状态移动，从而容易陷入到局部最优之中，找不到比较远的最优解。但如果是在所有后继状态中随机选择一个后继状态移动，也许可以找到全局最优值，但效率却非常低。因此，如果能将爬山法和随机转移结合起来，那么这种算法很可能是既高效又完备的，而模拟退火就是这样一种算法。

金属退火的过程简单来说就是，温度高时形成形状，温度低时打磨细节，金属的变化幅度随着温度的降低而降低。与之类似，模拟退火算法通过允许一些“坏”的动作来逃脱局部最优，但是需要逐渐降低移动到“坏”状态的概率。其具体的算法伪码如下所示：

```
function Simulated-Annealing(problem, schedule) returns a solution
state
inputs: problem, a problem
        schedule, a mapping from time to "temperature"
local variables: current, a node
                 next, a node
                 T, a "temperature" controlling prob. of downward
                    steps
current  $\leftarrow$  Make-Node(Initial-State[problem])
for  $t \leftarrow 1$  to  $\infty$  do
```

```
T ← schedule[t]
if T = 0 then return current
next ← a randomly selected successor of current
ΔE ← Value[next] - Value[current]
if ΔE > 0 then
    current ← next
else
    current ← next only with probability  $\exp(\Delta E / T)$ 
```

其中, $\exp(\Delta E / T)$ 为 $e^{E/T}$ 。

总体来说, 模拟退火算法是一种随机转移策略, 只是会根据当前的温度 (伪码中的变量 T) 以及随机选取的后继状态与当前状态的函数值之差 (伪码中的变量 ΔE) 来决定转移发生的概率。如果选取的后继状态的函数值要比当前状态更好, 那么一定会发生转移; 而如果后继状态函数值比当前状态差, 那么转移就有概率不发生, 随着温度越低, 这种情况下不转移的概率越大, 直到温度为 0 的时候搜索结束。

在无限次迭代和冷却参数设置合理的理想条件下, 模拟退火算法在理论上一定能够收敛到最优解。然而, 在实际应用中, 模拟退火算法的搜索结果对于温度参数非常敏感, 如果初始温度太低或是温度下降的速度太快都可能会导致找不到最优解, 而初始温度过高或是温度下降太慢则会导致无效搜索增加, 浪费算力。

5.2.2 演化计算

除了模拟退火算法以外, 演化计算 (Evolutionary Computation, EC) 也是一种比较常用的元启发式算法。它是一种模拟自然界生物进化过程进行问题求解的自组织、自适应的随机搜索技术。这种算法的设计思路具有以下特点:

- 思想源于生物遗传学和适者生存的自然规律
- 在基因和种群层次上模拟自然界生物进化过程

具体而言，根据达尔文的自然选择学说，生物要生存下去，就必须进行生存斗争。在生存斗争中，具有有利变异的个体容易存活下来，并且有更多的机会将有利变异传给后代；具有不利变异的个体就容易被淘汰，产生后代的机会也少得多。其中，变异的产生和遗传规律能从孟德尔以及其他生物学家的理论中得到解释，感兴趣的读者可以自行查阅相关资料。

将这些生物学思想融入人工智能领域中，若将遗传和变异的过程抽象成数学算法，将具体问题抽象成环境，那么计算机或许也能像模拟生物进化一样找到问题的最优解。遗传算法是演化计算中最初形成的一种具有普遍影响的模拟进化优化的方法，因此我们主要讨论遗传算法。

遗传算法

遗传算法的基本思想是从初始种群出发，采用优胜劣汰、适者生存的自然法则选择个体，并通过交叉、变异来产生新一代种群。如此逐代进化，直到满足目标为止。遗传算法主要涉及如下基本概念。

种群 (Population) 多个备选解的集合。
个体 (Individual) 种群中的单个元素，代表一个可能的解，通常由一个用于描述其基本遗传结构的数据结构来表示。例如，长度为 L 的 0、1 串。
适应度 (Fitness) 函数

用来对种群中各个个体的环境适应性进行度量的函数，其函数值是遗传算法实现优胜劣汰的主要依据。

遗传操作 (Genetic Operator)

作用于种群而产生新的种群的操作，包括选择 (Selection)、交叉 (Cross-over)、变异 (Mutation) 等。

遗传算法主要由染色体编码、初始化种群设定、适应度函数设定、遗传操作设计等几大部分所组成。简单来说，染色体编码可以视为通过模仿生物学中染色体的特点来设计参数或状态空间的编码策略，例如二进制编码、格雷编码等。初始化种群设定类似于束搜索中随机选择几个初始解，遗传算法的求解过程是从一个问题的可能解集开始的。适应度函数可以基于待优化的目标函数来设计。但在某些实际问题中（如天线的设计、火车头外观设计等），目标函数并不能显式的获得，我们只能通过测量（如模拟测试等）获得个体的具体适应度值。遗传操作需要根据原问题的结构以及状态空间特点来设计如何选择、交叉和变异，以尽可能模仿在自然选择下的遗传过程。

遗传算法具体包括以下基本步骤：

1. 选择编码策略，将问题搜索空间中每个可能的点用相应的编码策略表示出来，即形成染色体；
2. 定义遗传策略，包括种群规模 N 、交叉、变异方法，以及选择概率 P_r 、交叉概率 P_c 、变异概率 P_m 等遗传参数；
3. 令 $t = 0$ ，随机选择 N 个染色体，放入初始化种群 $P(0)$ ；
4. 定义适应度函数 f ；
5. 计算 $P(t)$ 中每个染色体的适应值；

6. $t = t + 1$;

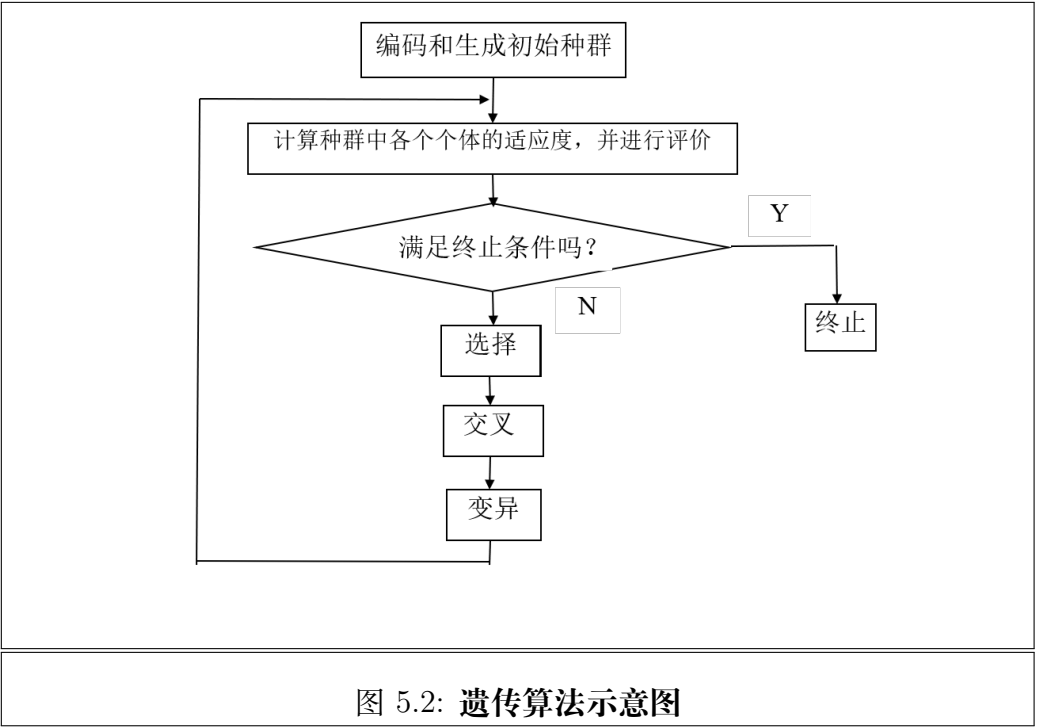
7. 运用选择算子, 从 $P(t - 1)$ 中得到 $P(t)$;

8. 对 $P(t)$ 中的每个染色体, 按概率 P_c 进行交叉;

9. 对染色体的基因, 以概率 P_m 进行变异;

10. 判断种群性能是否满足预设的终止标准, 若不满足返回 (5)。

遗传算法的运行过程如图5.2所示



实际上, 遗传算法在广泛的最优化方法中占有一席之地。因为遗传算法具有自组织、自适应和自学习性, 和模拟退火一样符合概率转移规则, 所以可能覆盖到全局最优解。在此基础上, 遗传算法本身适合大规模并行搜索, 各个个体独立进化, 同时搜索解空间的多个区域并相互间交流信息。不过, 遗传算法也并非完美, 它的收敛性很难被证明, 且不能解决那些“大海捞针”的问题。

5.3 最优化搜索的总结与局限性

以上就是关于最优化搜索的全部内容。这类搜索问题的目标很明确，就是求得使得目标函数最优时的状态。具体的搜索方法可以分为贪婪算法和元启发式算法两大类。贪婪算法的特点是搜索到解的速度快，但容易陷入局部最优之中；而元启发式算法可以缓解局部最优问题，但也容易面临收敛慢等问题，在实际应用中，我们需要根据实际问题的状态空间是否连续、目标函数的变量数量以及搜索时间限制等情况来有针对性地选取搜索方法。

状态搜索和最优化搜索并称为经典搜索，前者用于搜索从初始状态到终止状态的可行路径或最优路径，后者用于搜索使得目标函数最优的那个状态，这两个问题已覆盖现实中绝大多数决策场景。然而，还存在一类特殊的决策问题，如棋类对弈，棋手的每一步落子都需预判对手的反击；又如商业竞争，企业的市场策略需根据竞争对手的行动灵活调整。这类问题的关键特征是参与方需在多方博弈中根据对手策略动态调整自身决策。在人工智能领域，这类对抗性场景被系统性地抽象为对抗搜索问题。关于这类搜索问题的典型成果包括曾击败人类顶尖棋手的国际象棋 AI “深蓝” 和围棋 AI “阿尔法狗”，它们均基于对抗搜索算法，在对弈过程中动态预测对手可能的走法并制定反制策略，最终实现复杂博弈场景下的智能决策。在下一章中，我们会介绍对抗搜索的问题。

第六章 对抗搜索

这一章，我们考虑决策对抗的情景。当我们进入这类情景中时，会发现参与方不止自己，每个参与方的目标也各不相同。同时，除了自己以外，每一个参与者的决策都具有一定不可预测性，这一点会给我们的问题求解过程引入不确定性，也就导致了这类搜索问题要比经典搜索问题复杂得多。在本章中，我们研究的是这类充满竞争和交互的情景，也被称为游戏或博弈 (Game)，并探索使用计算机解决两人（或多人）博弈问题（例如下棋、打扑克牌）的方法，即对抗搜索 (Adversarial Search) 算法。

对抗搜索的核心是“在动态交互中预测对手行为并优化自身策略”。在这类问题中，多个参与方同时存在，且一方决策会直接触发对手的反制行为。此时，传统的只考虑自身最优的搜索方法不再适用，需通过预测对手策略、评估可能的后果，找到“最优应对策略”。接下来我们会介绍对抗搜索问题的详细定义，并介绍两种对抗搜索算法——极小极大搜索算法和 Alpha-Beta 剪枝算法。

6.1 对抗搜索的问题定义

在本章中，我们主要考虑两名玩家轮流采取行动的零和游戏，这类游戏比较简单，也是人工智能中最为常见的一种游戏。其中，零和这个概念源于经济学中的博弈论，代表了博弈中一方的总收益总是等于另一方的总损失。

同时，现实中常见的游戏可以分为确定性游戏和随机性游戏。确定性游戏是指游戏的结果完全由玩家的策略和行动决定，不受任何随机因素的影响，例如象棋、围棋，每一次落子的结果都是确定的；随机性游戏包含随机因素，游戏的结果部分或完全由随机事件决定，例如大富翁、赌骰子，游戏结果是无法根据玩家的行动来确定的。

另外，游戏也可以分为完全信息 (Perfect Information) 游戏和部分可观测信息 (Partially Observable Information) 游戏。完全信息游戏是指所有玩家在做出决策时都拥有关于游戏状态的完整信息。例如在下棋时，玩家能够看到整个棋局的所有信息，包括对手的所有棋子和对手的每一次落子，从而可以做出基于完整信息的决策。而部分可观测信息游戏是指玩家在决策时并不拥有关于游戏状态的完整信息，某些信息是不可见或未知的，例如打麻将或是打扑克牌，我们能看到对手打出了哪些牌，但是无法完全确定对手手里有哪些牌，只能基于自己的牌面和场上已经被对手打出的牌来做决策。相较而言，参与者更容易在确定性游戏和完全信息游戏中制定能稳定提高自己收益的决策。

因此总结来说，我们在本章中涉及到的游戏大部分都是**确定性的、两人轮流进行的、零和的完全信息游戏**，这类游戏主要有以下几个重要概念：

- **初始状态** (S_0)：指定游戏在开始时的设置方式。
- **玩家** ($\text{PLAYER}(s)$)：定义在某一状态下轮到哪个玩家行动 (s 表示当前状态)。
- **行动集合** ($\text{ACTIONS}(s)$)：返回在某一状态下的合法行动集合。
- **转移模型** ($\text{RESULT}(s, a)$)：定义一次行动后获得的新状态 (a 表示行动)。
- **终结测试** ($\text{TERMINAL - TEST}(s)$)：当游戏结束时为真，否则为假。游戏结束的状态称为终结状态。

- **效用函数** ($UTILITY(s, p)$): 也称为目标函数或收益函数, 反映了游戏结果对玩家的收益 (p 表示玩家)。

以井字棋游戏为例, 这个棋类游戏是非常典型的确定性的、两人轮流进行的、零和的完全信息游戏。游戏的初始状态就是一个 3×3 的空白棋盘, 玩家是参与下棋的两个智能体。名为 MAX 的人在下棋时填 “X”, 名为 MIN 的人填 “O”, 一个人填完后就换另一个人填, 棋盘的状态随着两个人的行动不断发生变化, 这就是井字棋的玩家的行动集合和转移模型。井字棋的终结测试有两个, 第一个是判断是否有三个 “X” 或三个 “O” 连成了一条竖线、横线或对角线, 第二个是判断 9 个格子有没有被填满。效用函数则是对输赢结果的抽象化表达, 在游戏结束时, 若 “X” 连成了一条竖线、横线或对角线, 则 MAX 获得的效用为 1, 而 MIN 获得的效用为 -1, 代表 MAX 获胜而 MIN 失败。若 “O” 连成了一条竖线、横线或对角线, 则 MAX 获得的效用为 -1, 而 MIN 获得的效用为 1, 代表 MAX 失败而 MIN 获胜。若没有一人将三个棋子连成一条竖线、横线或对角线, 则两人的效用都为 0, 代表和棋。图 6.1 展示了该游戏从初始状态到游戏结束所有可能的棋局序列。

以上是抗搜索问题的定义。在游戏中, 每个玩家都会想要让自己的效用最大化, 而由于零和游戏的特点, 一个人的效用增大, 另一个玩家效用必然减少, 所以玩家就必须考虑到对手可能会做出的阻挠, 制定一个行动策略 (即一系列有步骤的行动) 来确保自己的效用最大化, 这也就是对抗搜索算法所需要解决的问题。

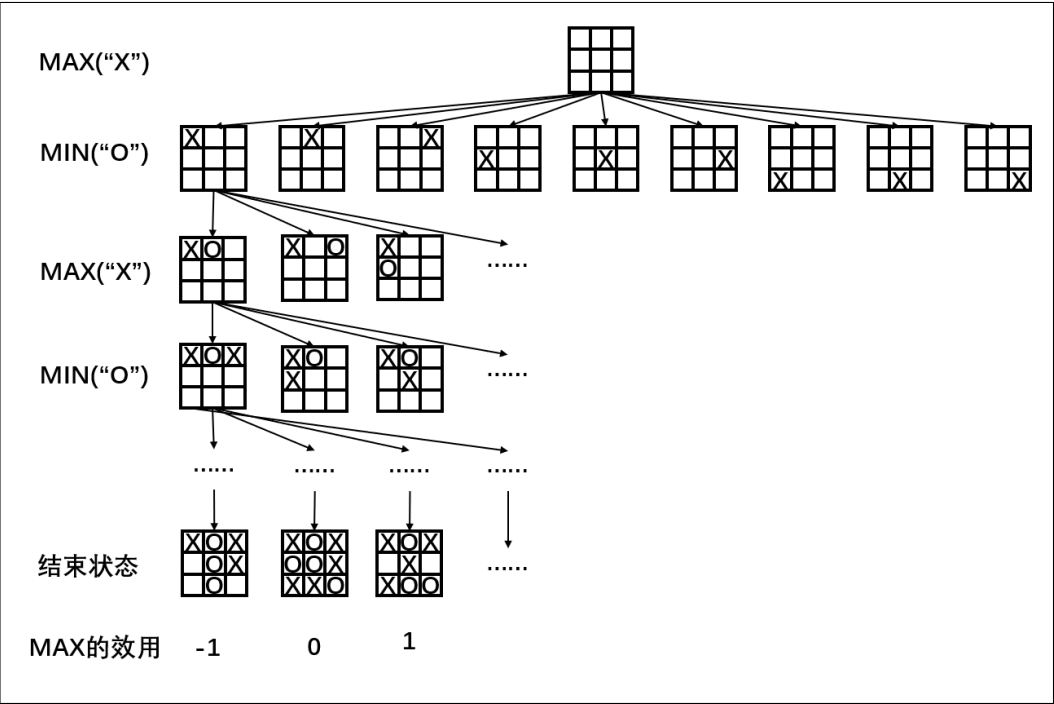


图 6.1: 井字棋游戏的过程，假设玩家分别叫作 MAX 和 MIN，MAX 使用“X”先下，MIN 使用“O”后下，游戏结束时，赢家效用为 1，输家效用为-1，平局效用为 0

6.2 极小极大搜索算法

极小极大搜索算法 (Minimax) 来源于冯诺依曼¹和经济学家摩根斯顿²合作的《博弈论和经济行为》一书中。它的核心思路是，假设对手非常聪明，并在这种情况下最大化自己的收益。

在一个确定性的、两人轮流行动的、零和的完全信息游戏中，参与游戏的两个玩家都会争取让自己的效用最大化，并且会根据对手的行动来调整自己的下一步行动。那么此时我们不妨假设自己是游戏的其中一位参与者，

¹约翰·冯·诺依曼 (John von Neumann)，匈牙利犹太裔美籍数学家、计算机科学家
²奥斯卡·摩根斯顿 (Oskar Morgenstern)，德国出生的美国经济学家，博弈论思想的最早提出者。

而我们的对手非常强悍，无论我们做出什么动作，他都会做出理性最优的动作，来让自己在游戏结束时的效用最大化（也就是让我们的效用最小化）。由于当前只考虑两人参加的游戏，我们可以只考虑一个效用函数，MAX 希望此效用函数越大越好，而 MIN 希望此效用函数越小越好。

具体讲，我们希望能获得这样一个效用函数，对于每一个棋局，该函数都能返回一个实数值，它能精确的评估该棋局对 MAX 的有利程度³。若这样的效用函数存在，MAX 将评估出当前棋局下的每个可能的走子获得的新状态的效用，然后选取效用最大的走子作为当前的动作；而 MIN 将选取效用最小的走子作为当前的动作⁴。这就是极小极大搜索算法的核心思想。

6.2.1 算法流程

那么，这样的效用函数究竟该怎么计算呢？我们可以通过递归的方式定义如下的 MINIMAX 函数，它就是我们所需要的效用函数。当此时的棋局是一个终止状态，此我们可以根据游戏规则判断输赢，若 MAX 获胜，则 MINIMAX 值设置为 1；若 MIN 获胜，则 MINIMAX 值设置为-1；若平棋，则 MINIMAX 值设置为 0。如当前的棋局不是一个终止状态，此时我们需要区分此时下棋的人是 MAX 还是 MIN。若此时轮到 MAX 下棋，MAX 希望 MINIMAX 越大越好，因此 MINIMAX 值应设置为所有走子的新状态的 MINIMAX 值的最大值；若此时轮到 MIN 下棋，MIN 希望 MINIMAX 越小越好，因此 MINIMAX 值应设置为所有走子的新状态的 MINIMAX 值的最小值。因此，MINIMAX 函数的详细定义如公式6.1所示。

³此效用函数实际就是第九章中定义的状态价值函数。

⁴此时选择的走子，就是第九章中定义的最佳策略。

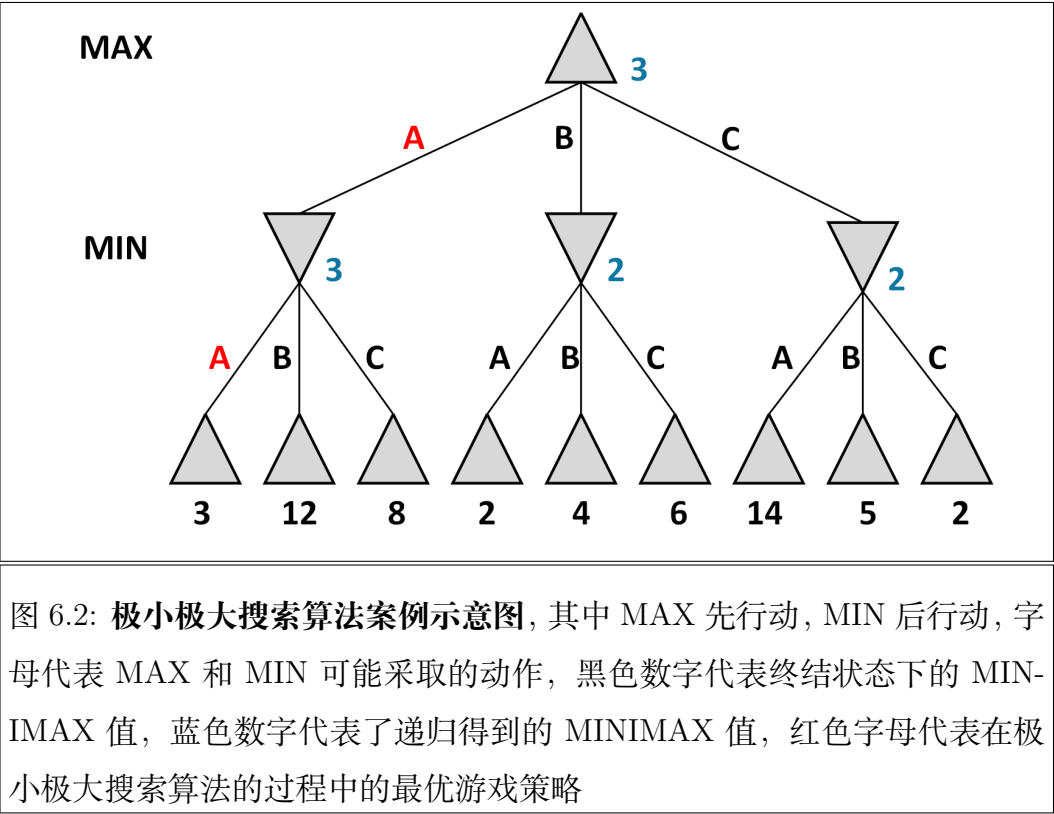
$$\text{MINIMAX}(s) = \begin{cases} \text{UTILITY}(s) & \text{if } \text{TERMINAL} - \text{TEST}(s) \\ \max_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MAX} \\ \min_{a \in \text{Actions}(s)} \text{MINIMAX}(\text{RESULT}(s, a)) & \text{if } \text{PLAYER}(s) = \text{MIN} \end{cases} \quad (6.1)$$

其中 $\text{MINIMAX}(s)$ 表示状态 s 下的极小极大值。

不难发现，计算不同状态下 MINIMAX 值的过程其实是一种递归算法，我们知道处于终结状态时确定的效用值，然后根据上一个状态的采取动作的玩家是谁，选取公式来倒推出上一个状态的极小极大值，然后以此类推，用这种方式把对抗搜索问题中每个状态的 MINIMAX 值都计算出来。因此，采取动作的玩家可以根据后继状态的 MINIMAX 值来决定当前的最优策略。

下面我们以一个虚构的零和游戏为例来演示极小极大搜索算法的搜索过程。假设有两个玩家——MAX 和 MIN，他们都会按照最优策略来参与行动。在游戏过程中他们只会有 A、B、C 三个动作选择，由 MAX 先行动，MIN 后行动。游戏过程中两个人都能看到对方的行动，也知道不同终结状态下他们的效用分别是多少。其过程和结果如图6.2所示：

当 MAX 开始采取行动之前，他会先去预演 MIN 会如何应对自己的行动。假如 MAX 选择了 A 动作，那么此时 MIN 会希望 MAX 的最终收益最小，那么它也会选择 A 动作，让 MAX 的最终效用只能为 3。同理，当 MAX 选择了 B 动作时，MIN 会选 A 动作；当 MAX 选择了 C 动作时，MIN 会选 C 动作。此时，MAX 能够确定自己的每个动作会带来的收益，选择 A，他的最终效用为 3；选择 B，他的最终效用为 2；选择 C，他的最终效用为 2。因此，MAX 最终会选择动作 A，而 MIN 会选择动作 A，此时 3 为 MAX 在初始状态的 MINIMAX 值。在此初始状态，MAX 选中动作 A



也就最佳的策略。

6.2.2 算法性质

在上一小节中我们已经了解了极小极大搜索算法的过程, 因此我们可以分析该算法的完备性、时间复杂度、空间复杂度和最优性, 其结果如下所示:

- 完备性: 只要搜索状态空间是有限的, 那么该算法完备。
- 时间复杂度: $O(b^m)$ 。
- 空间复杂度: $O(bm)$ (使用深度优先的扩展方式)。
- 最优性: 对抗一个最聪明的对手时能找到最优解。

很明显，该算法可以通过递归模拟双方的最优策略，找到当前玩家的最优解。不过它的缺陷在于时间复杂度实在过高，指数级的复杂度让极小极大搜索算法无法应用于复杂的博弈。假设国际象棋每步的平均可能走法有 35 种，每盘棋平均的走子总数为 80，则 35^{80} 也是一个天文数字。而围棋将更加复杂，在围棋中，每步的平均可能走法有 300 种，每盘棋平均的走子总数为 200，则 300^{200} 是一个更加巨大的数字。尽管如此，该算法的“极小极大”决策逻辑仍是理解对抗搜索问题的基础，后续的很多算法都是在极小极大搜索算法的基础上进行的改进。在后续关于强化学习的章节中，我们将学习如何利用深度强化学习和蒙特卡洛搜索树相结合的方法来解决国际象棋和围棋的问题。

6.2.3 算法拓展

极小极大算法是对抗搜索中非常经典算法，不仅能应用于确定性的、两人轮流行动的、零和的完全信息游戏，也能为其他博弈场景提供决策思路。下面我们将它拓展到多人轮流行动的非零和游戏场景。

首先，让我们假设如图6.3的一个三人博弈的游戏。在这个游戏中，共有 A、B、C 三个参与者轮流行动，每个状态下都仅有两个动作的选择。每个状态下都对应该状态的一个三维的向量，三个维度的值分别对应该状态对于 A、B、C 三人的效用。在这种情况下，B 和 C 都可以根据极小极大搜索算法来确定自己的最优策略，即每次选择使得第二维或第三维最大的动作；但是 A 却会陷入纠结。因为在初始状态，对 A 来说，无论选择哪个动作，他能得到的效用都是 1。也就是说，在这种情境下，A 存在**多个最优决策**。这就是多人非零和游戏中常常会出现的一种情况。因为在多人非零和游戏中，自己收益最大时，别人的收益不一定是最小的，所以原来的极小极大搜索算法需要进行一些调整。

接下来让我们假设如图6.4的一个多人博弈问题。可以看到，参与者在面对多种最优策略时的不同选择就会对最终博弈的结果产生显著影响了。以

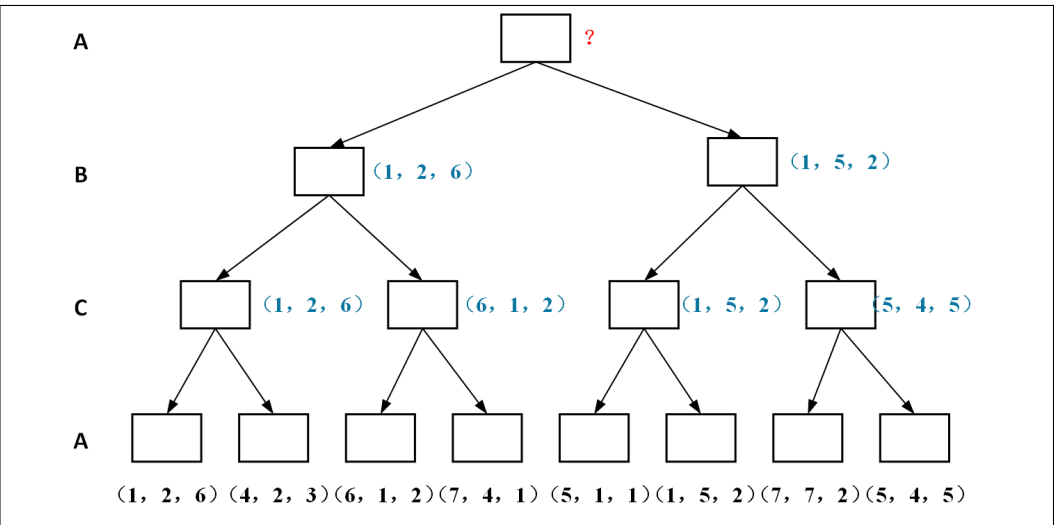


图 6.3: 多人博弈情景 1, 此时最后一行括号内数字代表 A、B、C 三人在终结状态的效用向量, 即 (A 的效用, B 的效用, C 的效用), 蓝色向量代表通过极大极小值算法推出的效用。

B 的效用为例, 在每个人都追求自己收益最大化的情况下, B 是有可能获得 5 的收益的 (如图6.4的蓝色向量构成的路径所示), 也可能获得 2 的收益 (如图6.4的红色向量构成的路径所示)。在这种情况下, 适用于两人零和游戏的极小极大算法搜索就无法找到一个确定性结果了, 那么该如何调整呢?

我们可以这样来思考: 当一个参与者面临多个最优选择的时候, 我们不知道这名参与者究竟选择哪一个最优动作, 但我们能推断出自己在这名参与者做出选择后获得的效用的最小值。因此, 可以把对这个参与者的最优效用和对其他参与者的最低效用组合起来构成一个新的效用向量。如图6.4中, A 在第 4 层的状态做决策时, 它的后续状态有两个, 分别的效用向量为 (2, 1, 5) 和 (2, 5, 1), 因此, 它无论采取哪个动作所获取的最佳效用都是 2; 对于其它两维的数值, 我们应该设置为在对应维度下的最小值。所以, 此状态的最终效用向量应该设置为 (2, 1, 1)。

这种做法也可以视为: 每个参与者都假设了其他参与者都会在保证各自收益最大化的前提下针对自己, 让自己永远处于一个收益最小的情况, 且

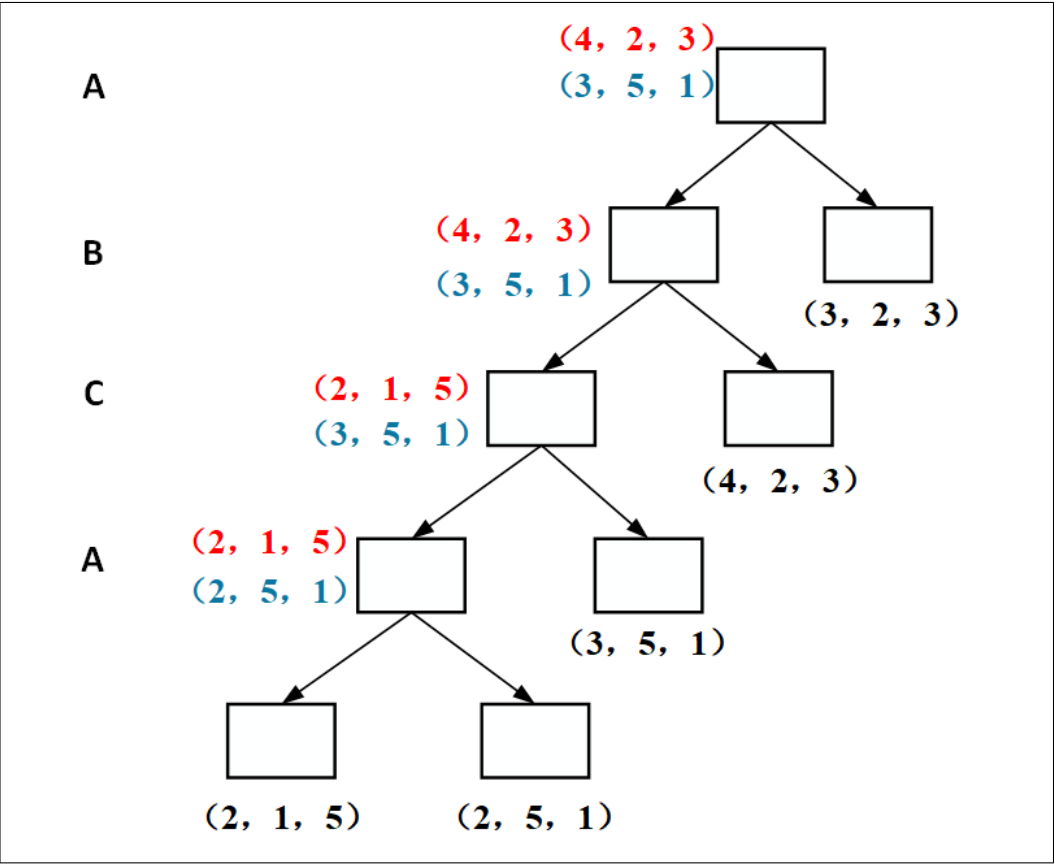


图 6.4: 多人博弈情景 2，此时的蓝色向量和红色向量分别代表在每个参与者都只考虑让自己收益最大时的两种可能的极小极大值

自己需要在这种不利情境下继续做最优决策。这其实也符合“极小极大”的博弈思路。基于这种思路下，图6.4中游戏的极小极大搜索算法的过程和结果如图6.5所示。最终的效用向量的每一维的数值，是对应参与者至少能获得的效用值。

6.3 Alpha-Beta 剪枝

上文提到了极小极大搜索算法通过递归遍历了对抗过程中所有可能的状态，为每个状态计算效用。遍历方式让其时间复杂度很高 ($O(b^m)$)，难

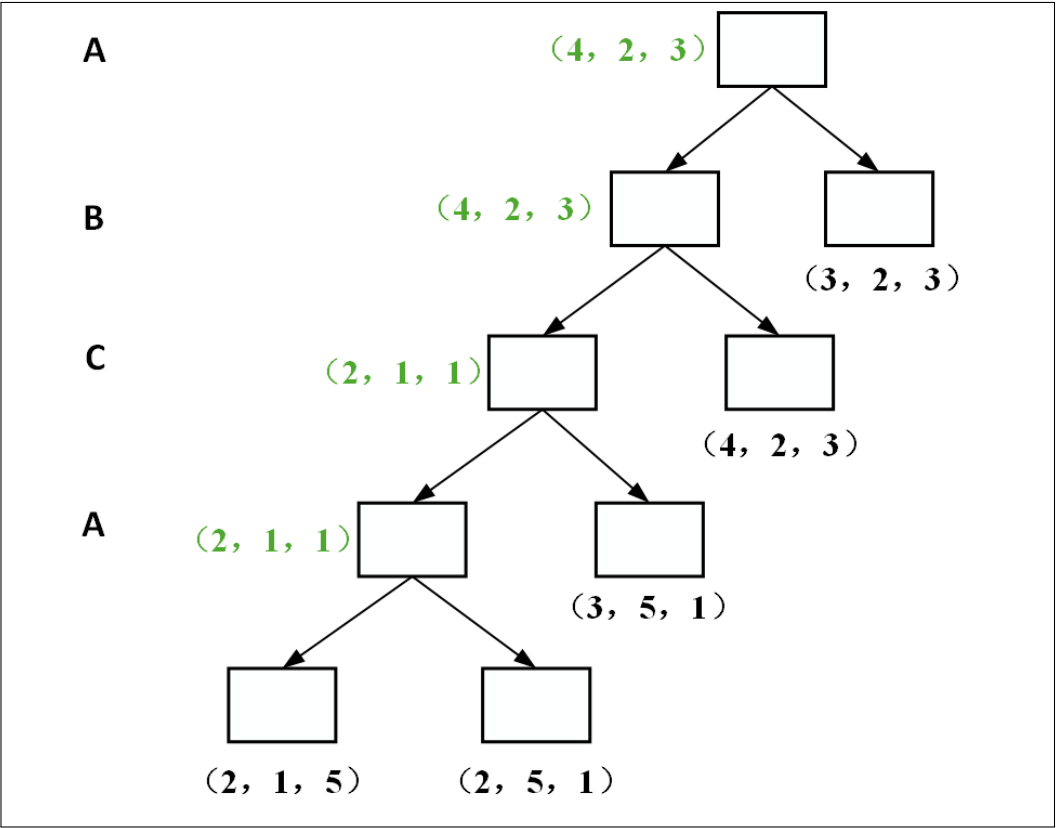


图 6.5: 改进后的极小极大算法思路下的多人博弈情景 2，绿色向量代表了在存在多个最优决策时，行动的参与者的最大收益和其他参与者的最小收益所构成的新的极小极大值效用向量

以用于复杂游戏。为了降低搜索的时间复杂度，以约翰·麦卡锡⁵为首的一些研究人员提出了 Alpha-Beta 剪枝算法。该算法通过动态维护两个参数 α (Alpha) 和 β (Beta)，可以在深度优先遍历所有状态空间的时候提前剪除不必要的分支，从而减少计算量，同时不影响计算结果的准确性。

⁵约翰·麦卡锡 (John McCarthy) 是美国的计算机科学家，人工智能领域的先驱之一，1971 年获得了图灵奖。

6.3.1 基本概念

Alpha-Beta 剪枝算法相对于极小极大搜索算法的改进之处，就在于找出了能在遍历状态空间过程中确定最优解范围的两个参数—— α (Alpha) 和 β (Beta)。

α (Alpha)

α 记录了最大值的下界，也就是 MAX 玩家探索到的已知最大值。这个值可以用来剪除效用更低的分支，因为 MAX 玩家不会允许在已知更好分支的情况下去选择它们。一般情况下， α 的初始值为 $-\infty$ ，在搜索过程中不断提高。

β (Beta)

β 记录了最小值的上界，也就是 MIN 玩家探索到的已知最小值。这个值可以用来剪除效用更高的分支，因为 MIN 玩家不会允许已知更差分支的情况下去选择它们。一般情况下， β 的初始值为 $+\infty$ ，在搜索过程中不断降低。

根据以上概念，我们可以看出 α 和 β 也是一种极小极大值，分别代表了 MAX 玩家和 MIN 玩家在搜索过程中对预期效用的一个底线，而他们也是根据这个底线来进行剪枝的。具体的搜索过程我们会在下一小节中详细介绍。

6.3.2 算法流程

Alpha-Beta 剪枝算法和极小极大搜索算法一样是以深度优先搜索的方式来遍历整个状态空间，只不过 Alpha-Beta 剪枝算法会在探索完一个节点后判断它和已知上下边界（即 $[\alpha, \beta]$ ）的关系，选择是否剪枝或者调整 α 或 β 的值。

对于 MAX 节点（玩家节点），如果当前的估值 $v > \alpha$ ，则将 α 值更新为 v ；如果 $v \geq \beta$ ，则剪除该分支。

对于 MIN 节点（对手节点），如果当前的估值 $v < \beta$ ，则将 β 值更新为 v ；如果 $v \leq \alpha$ ，则剪除该分支。

算法2、算法3和算法4展示了使用了 Alpha-Beta 剪枝的 Minimax 算法的伪代码。其中，算法3为 MAX 用户的搜索剪枝策略，算法4为 MIN 用户的搜索剪枝策略。

Algorithm 2 Alpha-Beta Search

```
function Alpha-Beta-Search(state) returns an action
     $v \leftarrow \text{Max-Value}(\text{state}, -\infty, +\infty)$ 
    return the action in ActionS(state) with value  $v$ 
```

Algorithm 3 Max-Value

```
function Max-Value(state,  $\alpha$ ,  $\beta$ ) returns a utility value
    if Terminal-Test(state) then return Utility(state)
     $v \leftarrow -\infty$ 
    for each  $a$  in ActionS(state) do
         $v \leftarrow \text{Max}(v, \text{Min-Value}(\text{result}(\text{state}, a), \alpha, \beta))$ 
        if  $v \geq \beta$  then return  $v$ 
         $\alpha \leftarrow \text{Max}(\alpha, v)$ 
    return  $v$ 
```

下面我们继续沿用6.2.1小节中虚构的游戏来演示 Alpha-Beta 剪枝算法的流程。为了方便表述，我们将游戏最开始的节点命名为 R 节点，将 MAX 采取 A、B、C 动作后的节点分别命名为 A 节点、B 节点和 C 节点。具体的算法执行过程如图6.6所示，一共分为六步。

开始 Alpha-Beta 剪枝算法后，第一步是 MAX 在根节点 (R 节点) 处运行算法3，先遍历到 A 节点。A 节点处 MIN 运行算法4，此时得到第三

Algorithm 4 Min-Value

```

function Min-Value(state,  $\alpha$ ,  $\beta$ ) returns a utility value
    if Terminal-Test(state) then return Utility(state)
     $v \leftarrow +\infty$ 
    for each  $a$  in ActionS(state) do
         $v \leftarrow \text{Min}(v, \text{Max-Value}(\text{result}(\text{state}, a), \alpha, \beta))$ 
        if  $v \leq \alpha$  then return  $v$ 
         $\beta \leftarrow \text{Min}(\beta, v)$ 
    return  $v$ 

```

层第一个节点的返回值 3，因为 3 要比此时的变量 $v(+\infty)$ 小，所以 v 被重新赋值为 3，然后 v 和 $\alpha(-\infty)$ 进行比较，得出 $v > \alpha$ ，所以暂时不用剪枝（在程序中表现为直接返回当前的 v 值而非继续遍历后续节点），最后因为 $v < \beta(+\infty)$ ，所以 β 也在这个节点也被赋值为 3。

在第二步中，MIN 在 A 节点继续遍历到第二个后继节点，得到返回值为 12，因为 12 比 v 大，所以不改变 v 和 β ，也不用剪枝。

来到第三步，因为 A 节点的第三个后继节点返回值是 8，仍然不会改变此时的 v 和 β ，至此 A 节点的所有后继状态都被遍历，返回 3。此时，对于根节点运行的算法 3 来说，A 节点的返回值 3 要大于 v （此时为 $-\infty$ ）和 $\alpha(-\infty)$ ，小于 $\beta(+\infty)$ ，因此根节点处的 v 和 α 被赋值为 3，然后开始用更改后的边界值去遍历节点 B。

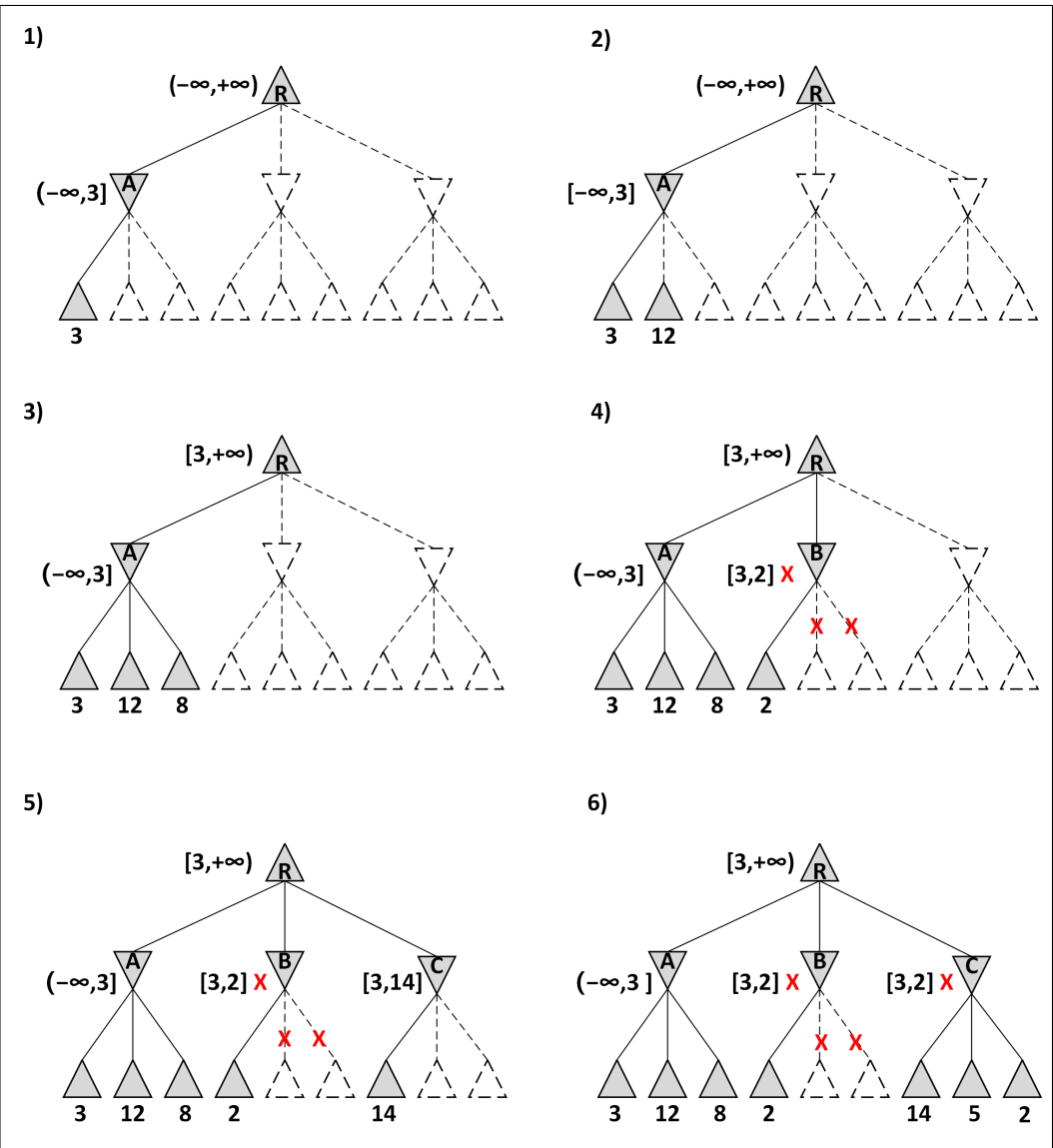


图 6.6: Alpha-Beta 剪枝算法案例示意图，其中第一层的节点为 MAX 节点，第二层节点均为 MIN 节点。节点旁的区间表示当前节点的 $[\alpha, \beta]$ ，虚线部分表示存在但尚未探索的节点和状态，红叉表明剪除该节点或路径

第四步是对节点 B 的剪枝过程，一开始和第一步类似，B 节点处运行算法 4 后首先得到了第一个后继节点的返回值 2，且小于此时的变量 $v(+\infty)$ ，所以 v 先被赋值为 2。但在 B 节点处，2 小于 α (此时为 3)，因此在程序运

行上 B 节点就直接向 R 节点返回 2，相当于 B 节点其他的几个后继节点都不用探索，也就是被剪除了。

对 B 节点剪枝后，MAX 在 R 节点处直接遍历 C 节点，也就是图 6.6 中的第五步。它的整体过程和第四步很像，只不过在扩展 C 节点的第一个后继节点后，发现算出的 $v(14)$ 要大于 $\alpha(3)$ 且小于 $\beta(+\infty)$ ，因此不用剪枝，此时的上下界变为了 $[3, 14]$ ，C 节点仍然值得继续扩展。

第六步是 MIN 继续遍历 C 节点的后继节点和向根节点返回值的過程。参考第四、第五步的过程，显然可知，虽然 C 节点的第二个后继节点仍不满足剪枝条件，但是第三个后继节点满足剪枝条件，虽然此时 C 节点已经不用被继续探索了，但依然可以认为 C 节点可以在此时被剪除。最后，MAX 得到了 C 节点的返回值 2，在 R 节点处和 $v(3)$ 比较，仍然选择不改变 v ，此时 R 节点处的算法也运行结束了，返回值 3，作为 Alpha-Beta 剪枝算法的搜索结果。

以上是对案例中 Alpha-Beta 剪枝算法运行过程的一个完整的梳理，重点阐释了 v 、 α 、 β 变化的过程以及剪枝发生的条件与效果。根据以上流程，我们可以看到只有极小极大值在 $[\alpha, \beta]$ 之间的节点有被扩展的可能，而且 MAX 只会去调整 α ，希望它越来越大；MIN 只会调整 β ，希望它越来越小。通过在搜索过程中不断缩小 α 和 β 的范围，Alpha-Beta 剪枝算法就更可能剪除无用的分支，并最终找到最优解。

6.3.3 算法性质

Alpha-Beta 剪枝算法的性质高度依赖于节点的顺序。如果 MAX 玩家很早就探索到了一个大于 β 的终止状态或是 MIN 玩家很早就探索到了一个小于 α 的终止状态，那么就可以尽早剪枝（就像图 6.6 中对于节点 B 的搜索一样）。相反，如果能触发剪枝条件的终结状态出现的都比较晚（像是图 6.6 中对于节点 C 的搜索一样），那么此时 Alpha-Beta 剪枝算法和极小极大搜索算法相比并无区别。

以下是对 Alpha-Beta 剪枝算法时间复杂度的总结：

- 最坏的情况：相当于没有剪枝，时间复杂度和极小极大搜索算法相同，为 $O(b^m)$ 。
- 最好的情况：总能够在第一步就找到“最好的”子节点，这样可以剪掉其所有的兄弟节点。这种情况下，时间复杂度为 $O(b^{\frac{m}{2}})$ ——这相当于把每个节点的度减少为 $\sqrt{b}$ ，或者将搜索深度增加一倍。
- 一般的情况：介于二者之间，时间复杂度为 $O(b^{\frac{3m}{4}})$ 。

对于 Alpha-Beta 算法来说，要想达到最好剪枝效果其实并不是特别困难，最常见的方法就是在搜索过程中使用迭代深度搜索还有启发式函数更快速地找到合适的上下界。例如，当我们先迭代到第一层的时候，通过启发式函数选择一个比较好的分支，然后对子节点重新排序，迭代到第二层时，优先探索比较好的节点，然后再对节点进行排序，以此类推，这样就能尽最大可能优先扩展减小 $[\alpha, \beta]$ 的节点，增大剪枝的可能性，从而接近最好的时间复杂度。感兴趣的读者可以自行探索如何在 Alpha-Beta 算法中应用启发式函数来减少时间复杂度。

在最好的时间复杂度 $O(b^{\frac{m}{2}})$ 的情况下，我们可以视每个节点的度为 $\sqrt{b}$ ，或者将搜索深度增加一倍。以在国际象棋中运用对抗搜索算法为例，平均来说，在任意棋盘状态下，每个棋手都有约 35 种不同的走法，也就是说国际象棋游戏中每一个状态的分支数平均约 35 个，而平均深度在 80 层左右，因此总状态大约有 $35^{80} \approx 10^{123}$ 个。如果使用极小极大搜索算法来搜索这么多状态显然不太现实。如果计算机每秒可以计算 10^4 个节点，每步棋最多允许 100 秒的思考时间，则每步棋最多可以访问 10^6 个状态。而 $10^6 \approx 35^4$ ，也就是说，深度优先搜索至多可以访问 4 层。如果访问了 4 层后的状态仍然不是终止状态，此时，我们可以通过机器学习的方法，以该棋局的特征（如：车的个数，马的个数等）为输入去估计效用值。以此再递归的估算其它状态的效用值。此时，如果使用 Alpha-Beta 剪枝算法，相对于

搜索的深度可以达到 8 层。此时的棋局更接近于终局，就更容易估算对应的效用值。因此，Alpha-Beta 剪枝算法在实际问题中的作用不可忽视。

6.4 对抗搜索的总结

以上是对抗搜索章节的全部内容。这类搜索问题因为引入了其他智能体的参与，让搜索的过程更加复杂。本章中提到的极小极大搜索算法和 Alpha-Beta 剪枝算法都是通过模拟双方的最优决策过程，在确定性、完全信息、两玩家轮流动作的零和游戏中寻找当前玩家的最优动作。虽然这两个算法无法对具有随机性和不完全信息的游戏使用（例如飞行棋、大富翁、桥牌等），而且容易出现状态空间爆炸的问题，不过他们都具有完备性和最优性，而且 Alpha-Beta 剪枝算法能够大幅降低时间复杂度。除此之外，这类算法的设计思路是理解智能体策略决策的基础，也是推动复杂游戏中对抗搜索算法发展的基石。

对搜索部分的总结

至此，我们对搜索部分的学习就告一段落。在状态搜索章节，我们学习了搜索的算法框架，并且了解了无信息搜索策略和有信息搜索策略，以及不同策略找到最优解的可行性与算法复杂度。在最优化搜索章节中，我们学习了贪婪算法和元启发式算法，探索了不同方法在搜索目标函数最优值时的优点与局限性。在对抗搜索章节中，我们将现实中的博弈问题抽象成了搜索问题的形式，并且介绍了如何用极小极大搜索算法和 Alpha-Beta 剪枝算法来模拟现实中人们在与别人竞争对抗的情境下找到最优决策的过程。

可以说，从状态搜索找到从初始状态到终止状态的最优路径，到最优化搜索追求目标函数的全局最优，再到对抗搜索破解多个参与者进行博弈的非经典搜索类问题，搜索技术始终是人工智能解决现实中问题的核心引擎。在实际应用中，我们需要先将现实中的问题抽象成其中一类搜索问题，然后选取适合该搜索问题情境的方法来选择具体方法。

对于人工智能领域而言，我们在状态搜索章节已经说明了状态搜索算法对于实现符号主义中的归结原理的意义，而最优化搜索算法更是在联结主义中训练模型参数的核心步骤，这一点我们会在下一部分联结主义的内容中为大家详细展开。

第三部分

学习：联结主义

联结主义与符号主义、行为主义并称为人工智能中的三大流派，认为人类智能可以通过大量简单的神经元之间的连接来模拟。这种理论强调整体性和分布式表征，即信息不是存储在单一的神经元中，而是分布在神经网络中。

在联结主义视角下，机器学习是一个核心概念，其本质是模拟人类学习过程——让计算机通过观测现实世界提升性能，以更高效地完成特定任务。其优势体现在两方面：其一，能处理传统方法难以应对的复杂或无明确算法的问题，通过数据分析自主探索解决方案，甚至反哺人类学习；其二，面对环境动态变化引发的突发问题，相较于频繁更新程序，期待机器学习算法能快速适应新情况。

本部分将介绍一些传统的机器学习方法和深度学习的内容，会涉及到多个机器学习的模型。对每个模型，我们都会讲述该模型的如下四个方面，俗称模型四要素：

1. 模型的基本假设是什么；
2. 基于此假设构造了什么目标函数；
3. 这个目标函数的优化问题怎么求解，即训练过程的算法是什么；
4. 模型参数确定后，预测的算法是什么。

首先，需要建模的输入和输出之间的关系往往比较复杂。每个机器学习模型，都会对这一复杂关系进行简化，即限定了模型从输入到输出的函数关系的基本形式。显然，模型的基本假设决定了模型的能力。如果基本假设过于简单，则并不能刻画复杂的数据关系；而基本假设过于复杂，则可能会造成在训练数据上的过拟合。因此，模型的基本假设是关键。

进而，基于模型的基本假设，我们可以引导出对应的目标函数。具体讲，模型的目标函数建立在已知的训练数据基础上，而模型的参数是未知

的变量。不管是有监督模型还是无监督模型，其实都有需要优化的目标函数。目标函数的具体形式，本质上也是数据建模的某种假设。

紧接着，我们需要求解使得目标函数最优的模型参数，即设计模型训练的算法。这个问题，实质就是第5章介绍的最优化搜索的问题。数学上有人专门研究最优化理论。但是，在工业应用中，训练数据非常大，则必须考虑方法的时间复杂度。因此，我们往往都采用非常简单的优化方法，即贪心算法。如果目标函数对模型参数不可导，则需要根据模型假设设计贪心算法；如果目标函数对模型参数是可导的，则可以直接使用梯度作为贪心的方向。总而言之，我们往往都能通过贪心算法找到该优化问题的局部最优解。

最后，需要谈谈模型的预测方法，即模型参数确定后，在给定的数据上怎么计算输出。通常，模型的预测方法都是非常简单的。但是，一些比较高级的机器学习模型的预测方法可能会比较复杂。比如，条件随机场的预测，实际需要用到维特比算法；而大模型的预测，涉及大量的计算机底层优化（包含计算机硬件和系统软件），使得推理更加高效（当前，大模型的预测推理已经成为一个庞大的产业）。这些内容，可能会在更高级的课程中学到。总而言之，在此统一的“机器学习模型四要素”框架下，读者更容易理解各个模型的核心和本质，并比较不同模型之间的差异。

在接下来的内容中，我们把机器学习模型分为两大类，分别是：经典机器学习模型和深度学习模型。经典的机器学习模型包括决策树、回归树、GBRT、逻辑回归以及集成学习方法。这些模型从上世纪八九十年代开始被陆续发明出来，而互联网和移动互联网的兴起给这些算法提供了广泛的实际应用场景。直到今天，这些算法仍支撑着数万亿的产业。

深度学习的研究起源于上世纪六十年代开始的人工神经网络的研究，但是这些研究经历了一些挫折和诟病（以后详谈）后，到上世纪九十年代已陷入了全面的低谷。但仍有一批神经网络的信徒在这个领域一直坚持，2012年，AlexNet 在 ImageNet 挑战赛（一个图片分类的任务）上以压倒性优势夺冠，错误率从传统方法的 26% 骤降到 15%。从此，神经网络的研究以一

个新的面孔——深度学习，点燃了全球范围的研究热潮。

第七章 经典机器学习方法

经典机器学习方法主要解决**单变量预测**的问题，即输出为一个值的任务。这样的任务在生活中十分常见，例如预测明天会不会下雨、一家公司的股票明天会不会涨等。简单来说，传统的机器学习就是从历史数据中找到目标变量和与之相关的一系列条件属性变量之间的关系；从而，在得到新的条件属性的数据后，能够精准预测对应的目标变量的值。

在本章中，我们会分别介绍决策树、回归树、GBRT、逻辑回归以及集成学习这五个模型，并从模型的基本假设、优化目标、训练方法和预测四个方面来讲述每个模型的原理；然后介绍特征工程方法，最后对经典机器学习方法进行总结。

7.1 决策树

决策树 (Decision Tree) 是一种非常简单，但却十分实用的机器学习方法。它主要用于分类任务，其输出为离散的标签值，俗称为类别标签；而模型的输入是包含了若干条件属性的一个样本数据。

7.1.1 决策树的基本假设

该模型的基本假设就是决策树的结构，具体来说，决策树通过执行一系列测试来实现其决策，它的节点可分为叶子节点和非叶子节点。非叶子节点上面是关于某一个属性的谓词（比如天气是否晴朗），叶子节点上面是

预测标签。在预测时，从根节点开始，根据样本是否满足非叶子节点上面的条件属性，来决定选择它的哪个孩子节点作为下一个检查的节点。当到达叶子节点时，选择上面的标签作为预测标签。

下面我们会用一个情景来展示决策树的具体结构。

情景案例

假设你是一个足球赛事分析师，目前拥有很多某球队过去的比赛记录，包括这些比赛的胜负情况、比赛日期、比赛当天的天气、该球队是否在主场比赛、他们的主力（郜林）是否首发等信息（如表 7.1 所示），现在你想根据这些记录，使用决策树模型来预测下周六的主场比赛是否能赢。

表 7.1: 某球队比赛的历史数据

比赛时间	是否在主场	比赛当天的天气	郜林是否首发	… …	比赛结果
周六	是	晴天	是	… …	胜
周日	是	阴天	是	… …	胜
周六	否	阴天	否	… …	负
周日	是	晴天	是	… …	负
周六	否	晴天	否	… …	负
… …	… …	… …	… …	… …	… …
周六	否	阴天	是	… …	胜

下面假设我们已经用计算机对该球队历史比赛数据进行学习，然后得到了一个如图 7.1 所示的决策树。假设，下周六该球队将在自己的主场比赛。我们首先检测此决策树的根节点“是否在主场”的谓词，结果为“是”。进而，我们检测根节点的左孩子节点的谓词“比赛时间”，结果为“周六”。因此，我们来到该节点的左孩子的叶节点，即预测下周的比赛结果为“是”，即“获胜”。

以上案例展示了决策树的结构，也就是该模型的基本假设。然而，决策树是如何形成的，即如何选择该树上的谓词节点的顺序，这就需要构建决

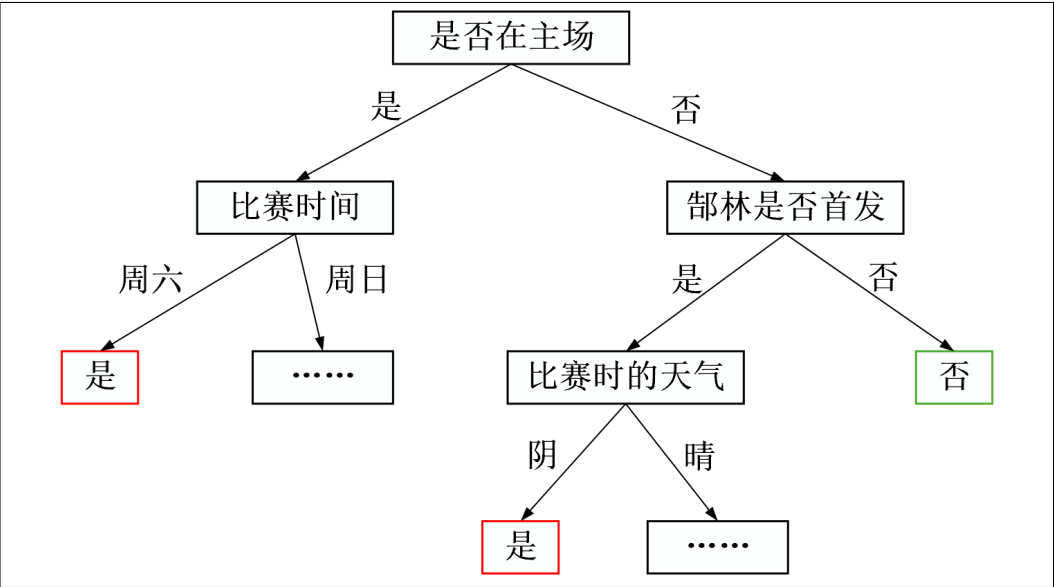


图 7.1: 预测某球队是否能赢的决策树，其中有省略号的节点代表了该节点下面还有许多子孙节点，黑色框的节点代表非叶子节点，红色框的节点代表预测标签为正的叶子节点，绿色框的节点代表预测标签为负的叶子节点

策树的优化目标。

7.1.2 决策树的优化目标

如何刻画一棵决策树在训练集上的好坏？为了回答这一问题，我们需要引入信息熵 (entropy) 的概念。

7.1.2.1 信息熵

熵本身是热力学中，用于度量分子混乱程度的单位。类比到人工智能领域，信息熵可以度量“预测随机变量 Y 的取值”这件事的难度，或者说度量随机变量 Y 的不确定性。

举一个简单的例子，随机变量 Y 是掷骰子游戏中的骰子值。显然，此时 Y 是一个离散型的随机变量，有 6 个可能的取值。当前，我们需要考虑

的是：如何去度量“预测骰子点数的问题”的难度。

我们可以考虑两个极端的情形。情形一：当 $P(y_1) = P(y_2) = P(y_3) = P(y_4) = P(y_5) = P(y_6) = \frac{1}{6}$ ，即 6 个取值出现的概率相等时，此预测问题的难度肯定是最大的；情形二：当 $P(y_1) = P(y_2) = P(y_3) = P(y_4) = P(y_5) = 0, P(y_6) = 1$ ，即某个取值（如 6 点）会确定的出现，而其它取值都不会出现时，此预测问题的难度肯定是最小的。

也就是说，对于离散随机变量 Y 和其分布 $P(Y)$ ，我们希望找到一个函数值 $H(Y)$ ，可以度量“预测随机变量 Y 的取值”这件事的难度。很明显，如果 Y 以均等的概率取所有可能的值，那么预测它的难度最高， $H(Y)$ 也最大；如果它只可能取一个值，那么预测它的难度最低， $H(Y)$ 也最小。换言之，如果 $H(Y)$ 越大，系统的不确定性也就越高，此时信息熵也就越大。不同形式的信息熵如表 7.2 所示。容易验证，它们都满足刚刚提到的性质。

表 7.2: 多种信息熵的形式

信息熵名称	信息熵表达式
Shannon Entropy	$\mathcal{H}_{sha}(Y) = \sum_{i=1}^m P(y_i) \log_2 \frac{1}{P(y_i)}$
Pal Entropy	$\mathcal{H}_{pal}(Y) = \sum_{i=1}^m P(y_i) e^{1-P(y_i)}$
Gini Index	$\mathcal{H}_{gin}(Y) = \sum_{i=1}^m P(y_i)(1 - P(y_i))$
Goodman-Kruskal	$\mathcal{H}_{goo}(Y) = 1 - \max_{i=1, \dots, m} P(y_i)$

其中， $P(y_i)$ 是随机变量 Y 取值为 y_i 的概率。

7.1.2.2 条件信息熵

信息熵描述了在没有任何前提信息的情况下，预测一个随机变量的难度。但是决策树的预测并不是凭空预测，而是基于一些已知的信息进行预测。因此，我们还需要另一个度量，用于描述已知相关信息的条件下，预测随机变量的难度。这个度量称为条件信息熵。

对于随机变量 X, Y 在已知 $P(Y|X)$, $P(X)$ 的情况下, 其条件信息熵定义为:

$$H(Y|X) = \sum_i P(x_i)H(Y|x_i) \quad (7.1)$$

其中, x_i 为随机变量 X 的某个具体取值。

举个例子: X 为代表“主客场比赛”的随机变量, 它有两个取值, x_1 代表主场比赛, x_2 代表客场比赛; Y 为代表“比赛结果”的随机变量, y_1, y_2, y_3 分别代表“胜”、“平”、“负”三个结果。根据历史比赛的统计, 我们可以获得如下的条件概率的估计:

$$P(y_1|x_1) = 0.4, P(y_2|x_1) = 0.6, P(y_3|x_1) = 0 \quad (7.2)$$

$$P(y_1|x_2) = 0.866, P(y_2|x_2) = 0.067, P(y_3|x_2) = 0.067 \quad (7.3)$$

根据这些信息, 我们可以分别计算出在主场和客场比赛的情况下, 比赛结果的信息熵, 即 $H(Y|x_1)$ 和 $H(Y|x_2)$ 。进而, 我们可以得到:

$$H(Y|X) = P(x_1)H(Y|x_1) + P(x_2)H(Y|x_2) = \frac{H(Y|x_1) + H(Y|x_2)}{2} \quad (7.4)$$

可以看出, 随机变量 X 为条件时 Y 的信息熵, 实际是在 X 的各个取值下 Y 的信息熵的算术平均。

有了信息熵和条件信息熵, 我们就可以定义信息增益, 即变量 X 对 Y 的预测能力:

$$\text{Gain}(X, Y) = H(Y) - H(Y|X) \quad (7.5)$$

根据信息增益的表达式, 我们可以看出, 若 $\text{Gain}(X, Y)$ 越大, 意味着: 在变量 X 已知的前提下, $H(Y|X)$ 越小, 此时预测 Y 的难度越低。因此, 我们可以将 $\text{Gain}(X, Y)$ 作为变量 X 对 Y 的预测能力的度量。

7.1.2.3 决策树的优化目标

给定一个包含 N 个样本的训练集 $\mathcal{D} = \{\mathbf{x}_i, y_i\}_{i=1}^N$, 假设某棵决策树有 K 个叶节点, 落入第 k 个叶节点的训练集样本为 $\mathcal{D}_k$ 。我们希望 $\mathcal{D}_k$ 中的

训练样本的标签都尽量一致，即 $H(\mathcal{D}_k)$ 要尽量的小；这里， $H(\mathcal{D}_k)$ 表示以 $\mathcal{D}_k$ 中样本的类别标签的分布计算的信息熵。则我们可以用如下的式子计算 $H(\mathcal{D}_k)$ 在所有叶节点上的均值：

$$\frac{1}{N} \sum_i^k H(\mathcal{D}_i) |\mathcal{D}_i| \quad (7.6)$$

上式越小，即表示该决策树对于训练集的分类效果越好。因此，它就是决策树的优化目标。

7.1.3 决策树的训练方法

在了解了决策树模型的优化目标后，我们就可以开始学习如何构造决策树了。首先，我们需要思考一个问题：决策树模型的参数是什么？它的参数实际是条件属性在决策树上的排列顺序，即：根节点应该选择什么条件属性，根节点下的各个子树的根节点又该安排什么条件属性等等。显然，目标函数对模型的参数是不可导的。因此，我们只能采用贪心算法来进行决策树的构建。

我们观察到，条件信息熵的定义（式7.1）和决策树的目标函数（式7.6）都是信息熵的算术平均，因此，条件信息熵可以用于决策树构建的贪心函数。具体讲，在构建决策树时，我们可以使用条件信息熵作为选择条件属性的度量，即每次贪心的选择条件信息熵最小的属性作为根节点；并依次递归，生成各子树的根节点。

以构建图7.1的决策树为例，我们首先会基于历史数据来分别计算每一个条件属性下目标变量的条件信息熵，然后我们会发现 $H(\text{比赛结果} | \text{是否} \text{ 在主场})$ 最小，就将“是否在主场”视为根节点。之后，我们把所有数据分为两个数据集：第一个是主场比赛的数据，第二个全是客场比赛的数据，然后在两个数据上分别重复刚才的过程，以确定左右两棵子树分别的根节点。依次递归的执行，计算出各个层次的子树的根节点。直到所有的叶子节点

的信息熵都足够小或是已经没有可用的属性条件时，整个决策树停止构建。

7.1.4 决策树的预测

决策树的预测过程比较直观，给定一棵决策树，对于一个没有标签的新样本，我们可以将该样本所落入叶节点对应的类别标签作为新样本的预测值。

让我们回到7.1小节的案例中，假设我们得到了如图7.1的决策树和如表7.3的用于预测的比赛数据。

表 7.3: 该球队下一场比赛的预测数据

比赛时间	是否在主场	比赛当天的天气	郜林是否首发	...
周六	是	阴天	是	...

我们根据属性值，先测试这场比赛是不是在主场，发现是主场比赛后，再测试它的比赛时间，然后发现周六比赛，此时我们得到预测比赛结果为“胜”的预测标签。

7.1.5 决策树的几何解释：在特征空间中划分超矩形

在了解了决策树的基本原理之后，我们可以从几何视角观察这一模型的行为。实际上，决策树的每一次特征划分，都可以看作是在原始特征空间中画出一条与坐标轴平行的切分线。随着划分不断进行，空间被分割成多个越来越小的超矩形区域，每个区域都对应树中的一个叶结点，并被赋予一个最终类别标签。

图7.2展示了一个二维数据集上决策树分类的可视化结果，其中横轴为特征 x_1 ，纵轴为特征 x_2 。红色实线代表决策树模型根据训练样本学习得到的划分边界，蓝色与红色的点分别为两类不同标签的样本，形状标记则代表模型对样本的预测类别。可以看到，决策树通过多次沿 x_1 或 x_2 方向进

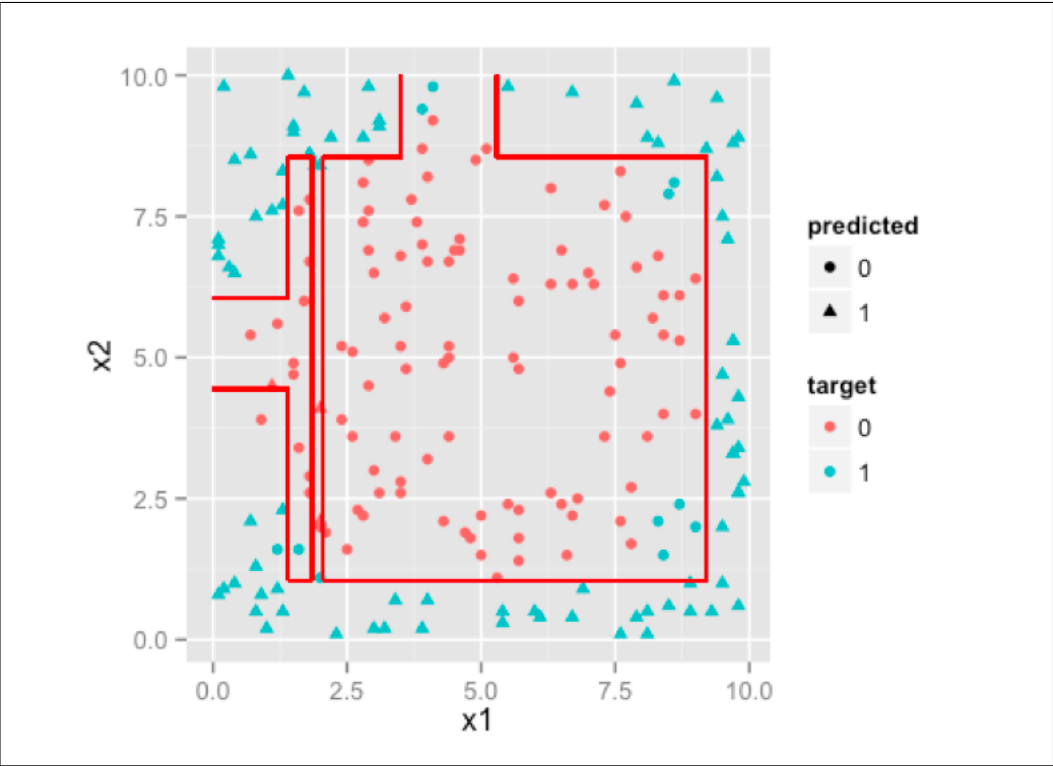


图 7.2: 二维空间中决策树逐步划分形成的矩形决策区域

行分裂，将整体空间划分成多个交错排列的矩形区域。落入相同区域的样本被映射到树的同一个叶节点，因此预测类别一致，而区域之间则形成清晰的分隔。

在二维中不同类别的区域表现为矩形，在高维特征空间中则会成为由多个上下界条件共同构成的超矩形区域。分类的过程，就是将新的样本判断属于哪个超矩形，并输出区域对应的类标。因此，决策树不仅是一种基于信息熵进行自上而下划分的学习模型，更可以理解为一种逐步切割空间、形成规则的超矩形分区的构造方式。

7.1.6 决策树的小结

以上就是决策树的全部内容，表7.4总结了决策树模型的四个要素。

表 7.4: 决策树的模型四要素

要素	内容
基本假设	构建树状结构：在非叶子节点排列待测试的条件属性，在叶子节点得到预测标签。
优化目标	最小化分类信息熵：落入叶节点的样本的信息熵均值最小。
训练过程	不可导的优化问题：以条件熵为准则的贪心算法。
预测	给定新样本，以落入的叶节点的类别为预测值。

总体来说，决策树是一个比较直观的机器学习算法，主要用于执行分类任务。这个模型易于理解和解释，易于使用，用途广泛且功能强大，也能作为其他复杂机器学习模型（如随机森林模型）的基础。

不过，决策树模型也存在一些局限性，因为决策树的训练方法是基于贪心算法，所以树的构建可能是次优的。除此之外，决策树也是不稳定的，它对于训练集比较敏感，即使仅添加一个新的样本，都有可能改变根节点选择到的条件属性，进而改变整棵树。同时，如果树深度太大，样本中的属性过多，也很容易出现过拟合问题。

7.2 回归树

决策树是分类模型，即待预测的变量是离散型的随机变量；但是现实中需要预测的事情不一定是离散变量，比如说预测明天的最高气温是多少度、预测一年后的房价。这时我们希望计算机能够给我们一个准确的数值作为答案。通常来说，我们把预测值为离散值的任务称为**分类任务**，而预测值为连续值的任务称为**回归任务**。

回归树 (Regression Tree) 适用于回归任务，其输出为连续值。其每一条训练数据都包含若干条件属性，和一个连续的输出值。整体上看，回归树

很像是用于预测一个连续数值类型输出的决策树，只不过回归树会在优化目标和训练方法上有所不同，这一点我们会在后面详细展开。

7.2.1 回归树的基本假设

回归树和决策树的结构基本相同，唯一区别是回归树的叶节点对应的并不是离散标签，而是所预测的连续数值。

除此之外，在回归树中，我们也会经常遇到条件属性值也为连续变量的情况，此时我们可以将其分割为若干个子段，将连续属性离散化，用于回归树的属性测试。例如，如果一个预测员工工资水平的回归任务的一个属性是工龄，我们可以用“3 年及以下”、“3 年以上”这样的分组方式来将这个连续变量进行切分。

7.2.2 回归树的优化目标

一个好的回归树应满足以下条件：

- 在训练数据上，预测尽量准确
- 树的结构简单，比如：树的叶节点数量小
- 叶节点对应的预测值的绝对值小

假设训练数据为 $\{\mathbf{x}_i, y_i\}_{i=1}^N$ ，第 j 个叶子上的预测值为 w_j ，树上有 T 个叶节点，则我们有如下目标函数：

$$\sum_{i=1}^n l(y_i, \hat{y}_i) + \gamma T + \frac{\lambda}{2} \sum_{j=1}^T \omega_j^2 \quad (7.7)$$

在7.7式中，第一项用于度量预测是否准确， $l(\cdot)$ 为误差 (loss) 函数，此处我们将其定为平方误差 (Square error) 函数， $\hat{y}_i$ 代表了第 i 个样本的预测值。第二项用于度量节点数量，其中 γ 是超参数，可以视为用于调整某个目标

重要程度的常数参数；在第二项中，如果我们认为叶节点数量小这个目标很重要，那么就把 γ 设置的大一点，反之就设置的小一点。第三项用于度量预测值的绝对值，其中 λ 也是一个超参数。回归树的优化目标是让整个目标函数尽可能小。

经过整理后，我们可以把目标函数写成如下形式：

$$\begin{aligned}
 & \sum_{i=1}^n l(y_i, \hat{y}_i) + \gamma T + \frac{\lambda}{2} \sum_{j=1}^T \omega_j^2 \\
 &= \sum_{j=1}^T \sum_{i \in I_j} l(y_i, \omega_j) + \frac{\lambda}{2} \sum_{j=1}^T \omega_j^2 + \gamma T \\
 &= \sum_{j=1}^T \sum_{i \in I_j} (\omega_j^2 + y_i^2 - 2y_i \omega_j) + \frac{\lambda}{2} \sum_{j=1}^T \omega_j^2 + \gamma T \\
 &= \sum_{j=1}^T \left[(A_j + \frac{\lambda}{2}) \omega_j^2 - 2B_j \omega_j \right] + \gamma T + \text{const} \quad (7.8)
 \end{aligned}$$

在这里， I_j 是所有落入第 j 个叶节点的样本的下标集合。 $A_j = |I_j|$ ，即落入第 j 个节点的样本数量， $B_j = \sum_{i \in I_j} y_i$ ，即落入第 j 个节点的样本预测值之和。

现在，我们回答如下问题：在如上的情况下，我们应该如何设定叶节点的预测值 ω_j ，才能使得目标函数最小。

根据韦达定理，当 $\omega_j^* = \frac{2B_j}{2A_j + \lambda}$ 时，7.8式取得最小值，为：

$$\sum_{j=1}^T \left(\frac{-2B_j^2}{2A_j + \lambda} \right) + \gamma T + \text{const} \quad (7.9)$$

这个分析告诉我们：如果我们确定了树的结构，每个叶节点对应的预测值为 ω_j^* 时，目标函数值最小。

因此，7.9式是评测一个回归树好坏的度量，也是回归树模型最终的优化目标。

7.2.3 回归树的训练方法

和决策树类似，回归树也使用贪心算法构建。对于当前树的某个节点，选择条件属性将它继续分裂为两个节点。因此，对于某个叶节点，它对应的目标函数应该是

$$\frac{-2B^2}{2A + \lambda} + \gamma \quad (7.10)$$

把它分裂为左右两个节点之后，产生的两个新叶节点对应的目标函数是

$$\frac{-2B_L^2}{2A_L + \lambda} + \frac{-2B_R^2}{2A_R + \lambda} + 2\gamma \quad (7.11)$$

这里，我们希望分裂之后目标函数的减小值尽量最大

$$\frac{2B_L^2}{2A_L + \lambda} + \frac{2B_R^2}{2A_R + \lambda} + \frac{-2B^2}{2A + \lambda} - \gamma \quad (7.12)$$

因此，7.12式为贪心算法选择条件属性的准则。

对于连续值的属性，可以将其切分为若干连续子段，使得目标函数最小。而如何划分连续值属性为若干段（划分的段数并未确定）才能使整个回归树的目标函数最小，其实也是一个最优化问题，可以通过动态规划的方法来求解，感兴趣的读者可以自行探索。

7.2.4 回归树的预测

回归树的预测方法和决策树一样，将预测样本落入的叶节点的对应的值作为预测值。

7.2.5 回归树的小结

以上是关于回归树的全部内容，它的模型四要素总结如表7.5所示。可以看出，回归树模型很像是决策树模型在输出值为连续变量时的延伸，因此也具有和决策树模型相似的优点和局限性。

表 7.5: 回归树的模型四要素

要素	内容
基本假设	和决策树模型相似，差别是回归树的叶节点对应的是所预测的连续数值。
优化目标	包含如下三个方面的综合目标：预测误差小；树的结构简单，即树的叶节点数量小；叶节点对应的预测值的绝对值小。
训练过程	不可导的优化问题：以最大化分裂前后的目标函数的减小值为准则的贪心算法。
预测	和决策树模型相同。

7.3 渐进梯度回归森林 GBRT

渐进梯度回归森林 (Gradient Boost Regression Trees,GBRT)，也被称为渐进梯度决策森林 (Gradient Boost Decision Trees,GBDT) 或多决策累加回归森林 (Multiple Additive Regression Trees,MART)。这个模型主要用于解决回归问题，其训练数据的形式也和回归树相同；不同的是，这个模型不是一棵树，而是很多棵树。在工业界，GBRT 被广泛用于处理向量化的高维大规模训练数据。同时，它也是各种预测类比赛的摘冠利器。

7.3.1 GBRT 的基本假设

GBRT 是一种加性模型 (Additive Model)。顾名思义，这种模型将多棵回归树的结果相加作为最终的预测值。假设我们有 K 棵树，则对于样本 $\mathbf{x}_i$ 的预测值为

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i),$$

(7.13)

其中， $f_k(\mathbf{x}_i)$ 为第 k 棵树在 $\mathbf{x}_i$ 上的预测值。

除此之外，GBRT 会基于不断逼近的思想构造新树：当发现前面几棵树的整体预测出现误差后，就再构造一棵树来对已生成的树的误差进行修正，也就是让 $\sum_{i=1}^n \text{loss}(y_i, \hat{y}_i)$ 在构建新树的迭代过程中不断减小。这种思路能让 GBRT 模型比单棵回归树的预测能力更强。具体的优化方法我们会在下面详细讲述。

7.3.2 GBRT 的优化目标

GBRT 的优化目标仍然遵循“让预测误差越小的同时尽可能减小模型的复杂程度”这一原则。此时，假设我们有 K 棵树，且每个样本的预测值 $\hat{y}_i$ 如7.13所示，那么此时的优化的目标函数为

$$\sum_{i=1}^n \text{loss}(y_i, \hat{y}_i) + \sum_{k=1}^K \Omega(f_k). \quad (7.14)$$

其中：

- $\sum_{i=1}^n \text{loss}(y_i, \hat{y}_i)$ 是训练误差。
- $\sum_{k=1}^K \Omega(f_k)$ 是模型的复杂度。

式7.14中的函数值越小，则模型的综合表现可能越好。

7.3.3 GBRT 的训练方法

和决策树与回归树类似，GBRT 的优化过程也需要使用贪心算法，不断添加新的树。每棵树的添加过程如下所示

$$\begin{aligned}
\hat{y}_i^{(0)} &= 0 \\
\hat{y}_i^{(1)} &= f_1(\mathbf{x}_i) = \hat{y}_i^{(0)} + f_1(\mathbf{x}_i) \\
\hat{y}_i^{(2)} &= f_1(\mathbf{x}_i) + f_2(\mathbf{x}_i) = \hat{y}_i^{(1)} + f_2(\mathbf{x}_i) \\
&\vdots \\
\hat{y}_i^{(t)} &= \sum_{k=1}^t f_k(\mathbf{x}_i) = \hat{y}_i^{(t-1)} + f_t(\mathbf{x}_i)
\end{aligned}$$

在 t 轮的训练中:

- $\hat{y}_i^{(t-1)}$ 是前 $t-1$ 轮的预测值。
- $f_t(x_i)$ 是需要新训练的函数。

也就是说,每一轮的训练目标实际上是前一轮的残差,即 $(y_i - \hat{y}_i^{(t-1)})$ 。具体讲,我们在训练第 t 棵树时,构造一个新的训练数据 $\{\mathbf{x}_i, (y_i - \hat{y}_i^{(t-1)})\}_{i=1}^N$,在此基础上使用回归树的训练方法进行训练。

7.3.4 GBRT 的预测

GBRT 的预测方法和决策树、回归树的预测方法类似:对于给定的新样本 $\mathbf{x}_i$,先确定在每棵树上落入的叶节点对应的预测值,然后将所有树上的预测值加起来作为整体模型的输出值,即:

$$\hat{y}_i = \sum_{k=1}^K f_k(\mathbf{x}_i) \quad (7.15)$$

7.3.5 GBRT 的小结

以上是关于 GBRT 的全部内容,该模型的四要素总结在表7.6中。

表 7.6: GBRT 的模型四要素

要素	内容
基本假设	基于加性模型，将多个回归树的预测值相加，作为最终的预测值。
优化目标	让预测误差越小的同时尽可能减小模型的复杂程度。
训练过程	使用贪心算法，每轮训练都聚焦前一轮预测结果的残差。
预测	给定样本，加总每棵树的预测结果为最终预测值。

可以看出，GBRT 每一轮的训练的目标都是逼近当前已有的 GBRT 预测结果的残差，所以随着训练轮次的增加，GBRT 预测结果的残差在理论上会越来越小。而在每一轮的训练中，仍然使用贪心算法来构建回归树。

至此，我们已经了解了决策树、回归树和 GBRT 三种机器学习模型，这三个模型在优化目标和预测方法上其实有非常多的共通之处。整体看来，他们的优化目标都要最小化某种表示预测误差的函数：对于分类问题是最小化条件信息熵，对于回归问题则是最小化均方误差，同时希望模型结构不要太复杂。它们的预测方法都比较直观，只需要将需要预测的样本输入模型，根据样本的属性值输出对应的叶节点上的预测值（GBRT 只是多了一个加总多棵树预测结果的过程而已）。三个模型都是基于分类回归的思想来建立的。

接下来，我们将介绍基于最大化数据似然的概率思想构建的机器学习模型，读者可以进一步比较不同方法在模型要素上的异同。

7.4 逻辑回归 (Logistic Regression)

7.1, 7.2和7.3这三节都是关于树的模型。接下来, 我们介绍基于概率建模的模型——逻辑回归¹。逻辑回归是一种分类模型, 将高维向量数据 (Tabular Data) 分为 0 和 1 两类。

本节内容会首先讲解逻辑回归模型的四个要素, 然后介绍逻辑回归中的正则化问题以及逻辑回归与线性回归的比较, 最后对整个模型进行总结。

7.4.1 逻辑回归的基本假设

作为一种基于概率建模的模型, 逻辑回归的工作原理就是为样本分配类别概率。因此, 我们需要先了解 sigmoid 函数, 其函数表达式和对应的一阶导数如下所示:

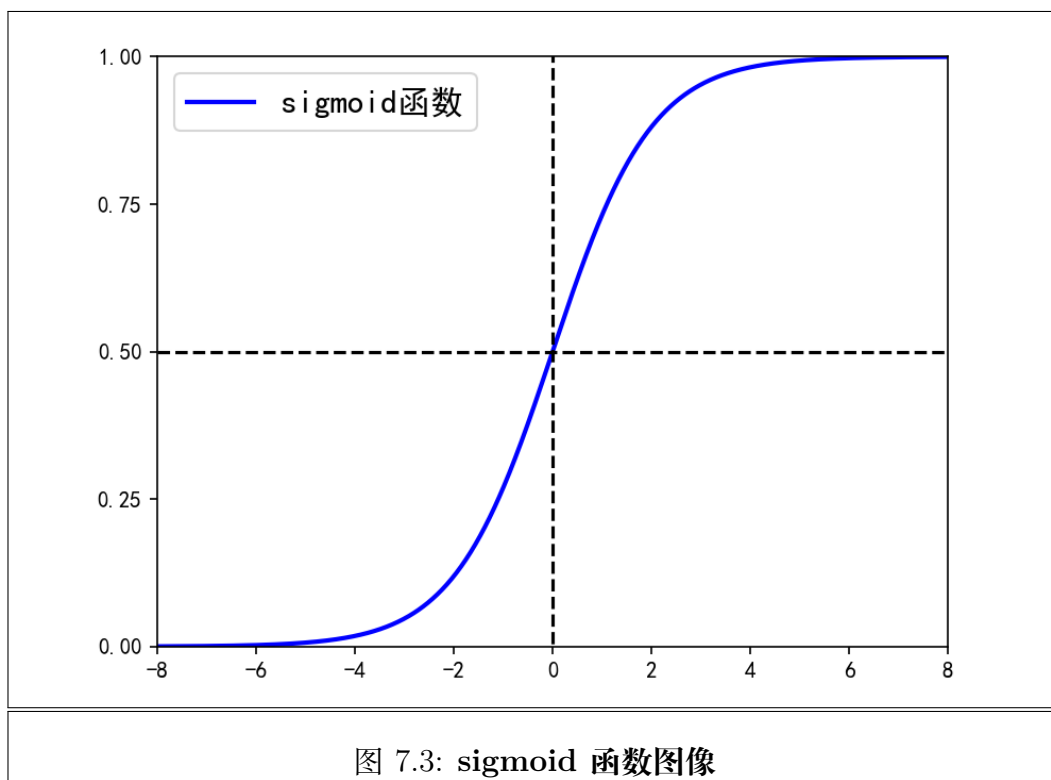
$$\sigma(t) = \frac{1}{1 + e^{-t}}, \quad (7.16)$$

$$\frac{d\sigma(t)}{dt} = \sigma(t)(1 - \sigma(t)), \quad (7.17)$$

其中, t 为函数的自变量。

可以看出, sigmoid 函数可以把 $(-\infty, +\infty)$ 映射到 $(0, 1)$, 其函数图像如7.3所示。当 $t > 0$ 时, 该函数的值大于 0.5, 且随着 t 的增大, 函数值也增大, 但增大的速度逐渐放缓。当 $t = 0$ 时, 该函数的值等于 0.5。当 $t < 0$ 时, 该函数的值小于 0.5, 且随着 t 的减小, 函数值也减小, 但减小的速度逐渐放缓。因为 sigmoid 函数的值域 $(0, 1)$ 中, 且具有良好的数学性质, 我们通常用它来表示概率。

¹这里, “logistic” 并非 “logic (逻辑)” 的同根词, 而是源自希腊语 *logistikē* (计算、算术)。因此, 把 “logistic regression” 译成 “逻辑回归” 属于历史沿用下来的约定俗成, 但并不准确。建议译为: “罗吉斯蒂回归”, 直接对应英文发音, 避免 “逻辑” 带来的歧义。



具体讲，若 $\mathbf{x}$ 为一个样本， θ 为模型参数， y 为样本标签。那么样本为正例的概率定义为：

$$P(y = 1 \mid \mathbf{x}, \theta) = \sigma(\theta^T \mathbf{x}), \quad (7.18)$$

其中， $\mathbf{x}$ 是表示该样本的实数向量， θ 是模型参数，它的维度与 $\mathbf{x}$ 的维度相同。

样本为负例的概率定义为：

$$P(y = 0 \mid \mathbf{x}, \theta) = 1 - \sigma(\theta^T \mathbf{x}). \quad (7.19)$$

若 y 只有 0 和 1 两个取值，所以我们可以综合以上两个式子得到：

$$P(y \mid \mathbf{x}, \theta) = \sigma(\theta^T \mathbf{x})^y (1 - \sigma(\theta^T \mathbf{x}))^{(1-y)}. \quad (7.20)$$

以上是有关逻辑回归模型的概率基础，也是该模型的基本假设。

7.4.2 逻辑回归的优化目标

逻辑回归的优化目标是：通过改变参数向量 θ ，让逻辑回归模型能够对训练样本中的数据都做出高概率估算。这涉及到了概率论中的最大似然估计 (Maximum Likelihood Estimation) 方法。

已知训练数据： $\{(\mathbf{x}_i, y_i)\}_{i=1}^N$ ，我们希望最大化的目标函数是数据似然，即

$$L(\theta) = \prod_{i=1}^n P(y_i | \mathbf{x}_i, \theta) \quad (7.21)$$

$$= \prod_{i=1}^n \sigma(\theta^T \mathbf{x}_i)^{y_i} (1 - \sigma(\theta^T \mathbf{x}_i))^{(1-y_i)}, \quad (7.22)$$

将式7.22取对数，再取负，则目标变为最小化7.23式：

$$- \sum_{i=1}^n y_i \ln(\sigma(\theta^T \mathbf{x}_i)) + (1 - y_i) \ln(1 - \sigma(\theta^T \mathbf{x}_i)). \quad (7.23)$$

7.23就是逻辑回归模型要最小化的目标函数。

7.4.3 逻辑回归的训练方法

在逻辑回归模型中， θ 是我们需要获得的模型参数。显然，目标函数对 θ 是可导的。我们可以使用梯度下降法求解。

$$l(\theta) = - \sum_{i=1}^n y_i \ln(\sigma(\theta^T \mathbf{x}_i)) + (1 - y_i) \ln(1 - \sigma(\theta^T \mathbf{x}_i))$$

$$\frac{\partial}{\partial \theta_j} l(\theta) = \sum_{i=1}^n (\sigma(\theta^T \mathbf{x}_i) - y_i) \mathbf{x}_i^j \quad (7.24)$$

其中 θ_j 是 θ 的第 j 维， $\mathbf{x}_i^j$ 是第 i 个样本的第 j 维。使用梯度下降方法，我们在迭代的每一轮有如下参数调整的方法：

$$\theta_j \leftarrow \theta_j - \alpha \frac{\partial}{\partial \theta_j} l(\theta), j = 1, \dots, |\theta| \quad (7.25)$$

其中 α 为每一步的步长，是一个需要自定义的正数。

不过在实际应用中，梯度下降法并不一定能找到全局最小值，可能陷入局部最小，这一点我们在最优化搜索章节就已经讨论过了。

除此之外，如果使用7.24式来计算梯度，每一次迭代都需要用到所有的样本，当数据量大时可能会造成巨大开销。因此，训练过程中，我们会选择随机梯度下降方法：即在单个样本或者小批量 (Mini-Batch) 数据上计算梯度。也就是说，该方法随机采样一小部分数据，而不是全部的数据，来计算梯度并更新参数。虽然随机梯度下降法不能保证每一次迭代都能让选取全部样本的目标函数降低，但只要迭代次数足够，还是能够达成逐渐优化的目标。

7.4.4 逻辑回归的预测

在预测时，先根据输入 $\mathbf{x}$ 和模型参数 θ 计算概率，

$$P(y = 1|\mathbf{x}, \theta) = \sigma(\theta^T \mathbf{x}) \quad (7.26)$$

$$P(y = 0|\mathbf{x}, \theta) = 1 - \sigma(\theta^T \mathbf{x}) \quad (7.27)$$

随后选择概率最大的标签作为预测值。

也就是说，逻辑回归通过计算输入特征向量与参数向量的点乘，即 $\theta^T \mathbf{x}$ ，来进行正负例的判定：

- 当 $\sigma(\theta^T \mathbf{x}) \geq 0.5$ ，即 $\theta^T \mathbf{x} \geq 0$ 时，预测结果为正例。
- 当 $\sigma(\theta^T \mathbf{x}) < 0.5$ ，即 $\theta^T \mathbf{x} < 0$ 时，预测结果为负例。

可以看出， $\theta^T \mathbf{x} = 0$ 是该模型的分界平面，位于平面一侧的数据都是正例，而另一侧的数据都是负例。因此，逻辑回归模型实际是一个线性分类器。

同时, $\theta^T \mathbf{x} = 0$ 是一个通过坐标原点的平面。为了增大模型的表达能力和灵活度, 我们可以再增加一个维度的模型参数 θ_0 , 并将数据隶属于正负例的概率定义为:

$$P(y = 1 | \mathbf{x}, \theta') = \sigma(\theta^T \mathbf{x} + \theta_0) \quad (7.28)$$

$$P(y = 0 | \mathbf{x}, \theta') = 1 - \sigma(\theta^T \mathbf{x} + \theta_0) \quad (7.29)$$

此时, 我们得到的分类平面为 $\theta^T \mathbf{x} + \theta_0 = 0$, 也就是一个带截距 θ_0 的平面。

至此, 我们介绍了逻辑回归模型的四个要素。总体来看, 这种机器学习方法是用一个基于概率学的回归模型来处理分类问题, 通过 sigmoid 函数将数据映射到概率空间中, 运用最大似然估计得到目标函数; 通过随机梯度下降法进行训练, 最后根据输入的新样本, 结合模型参数来计算概率, 输出类别标签。

不过, 和前三节中模型的优化目标相比, 我们好像并没有在逻辑回归模型中提到模型复杂度的问题。事实上, 任何模型都会面临过拟合的问题。如果想要增强模型的泛化能力, 除了提升训练集的质量和数量以外, 降低模型的复杂度也是非常重要的。对于同样用于分类任务的决策树模型, 我们可以用减少叶结点数量等方式来降低模型的复杂度, 而在逻辑回归模型中, 我们将约束模型的方法称为正则化 (Regularization)。它的核心思想是通过在损失函数中添加“惩罚项”, 对模型参数的取值范围施加约束, 具体的方法我们会在下一小节详细展开。

7.4.5 逻辑回归的正则化

逻辑回归模型中正则化是在代价函数中考虑权重 θ 自身的复杂程度, 避免权重太多太大, 造成过拟合。当前最常使用的正则化方法有以下两种:

- **L1 正则化:** $l(\theta) + \frac{\epsilon}{2} \sum |\theta_j|$
- **L2 正则化:** $l(\theta) + \frac{\epsilon}{2} \sum \theta_j^2$

其中 θ_j 是参数向量的第 j 维, c 为超参数。

L1 正则化更倾向于将一些参数变为 0, 从而达到特征选择的效果; L2 有可能保留更多维度, 但会让参数值变得更小。通过正则化, 我们能够得到更有泛化能力的参数。

7.4.6 逻辑回归与线性回归的比较

下面我们会比较逻辑回归与线性回归, 来探索逻辑回归在处理分类问题上的优势。

假设样本 $\mathbf{x}_i$ 的真实值为 y_i , 模型参数为 (θ, θ_0) , 模型的预测值为 $\hat{y}_i$ 。

线性回归模型采用平方误差的损失函数, 即在单个样本上的损失函数定义为:

$$l(y_i, \hat{y}_i) = (y_i - \hat{y}_i)^2, \quad (7.30)$$

其中, $\hat{y}_i = \theta^T \mathbf{x}_i + \theta_0$ 。

那么对于逻辑回归来说, 根据式7.23, 单个样本上的损失被定义为:

$$l(y_i, \hat{y}_i) = -[y_i \ln(\hat{y}_i) + (1 - y_i) \ln(1 - \hat{y}_i)], \quad (7.31)$$

其中, $\hat{y}_i = \sigma(\theta^T \mathbf{x}_i + \theta_0)$ 。这个损失函数被称为 Logistic Loss 函数。

进一步的, 线性回归的分类平面为

$$\theta^T \mathbf{x} + \theta_0 = 0.5, \quad (7.32)$$

而逻辑回归的分类平面为

$$\theta^T \mathbf{x} + \theta_0 = 0 \quad (7.33)$$

对 Logistic Loss 的讨论

逻辑回归对异常点的鲁棒性 (Robustness) 更好, 因为它们的损失值并不会因为偏离分类平面较远而变得过大。图7.4直观地展示了这一现象。图中红

点和蓝点是两类数据，绿色直线是逻辑回归的分类平面，紫色直线是线性回归的分类平面。容易发现，异常点的出现使线性回归的分类平面出现了严重的偏离，而逻辑回归的分类平面则几乎没有受到影响。

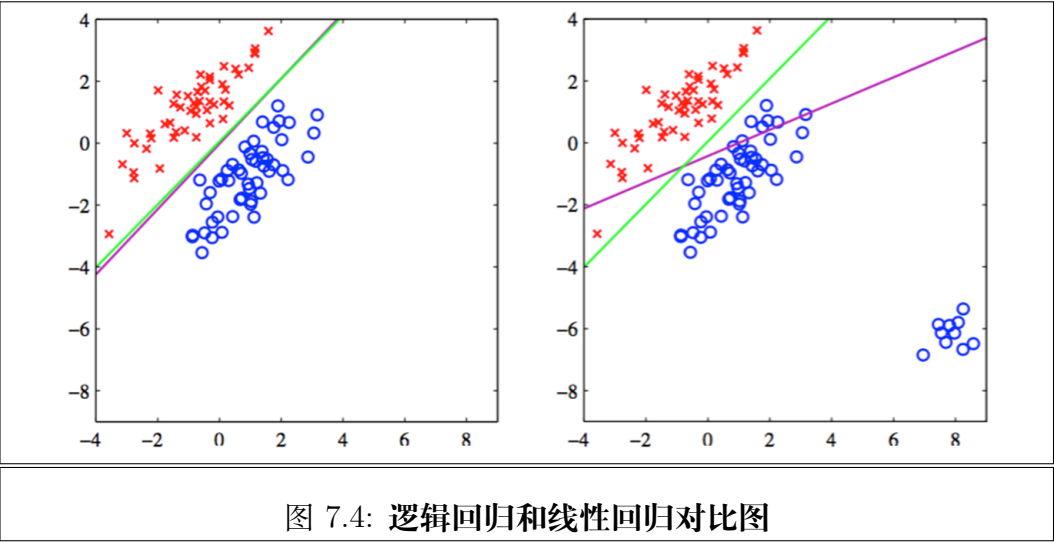


图 7.4: 逻辑回归和线性回归对比图

7.4.7 逻辑回归的小结

以上是关于逻辑回归的全部内容，它的模型四要素如表7.7所示：

表 7.7: 逻辑回归模型的模型四要素

要素	内容
基本假设	将数据用 sigmoid 函数映射到概率空间上，通过计算正负例出现的概率来输出结果。
优化目标	最大化数据似然：通过对数化，取负数转化为最小化 Logistic Loss 的问题；同时，可以加入正则化函数。
训练过程	使用随机梯度下降法。
预测	给定样本，取概率值最大的类别。

逻辑回归模型和前面提到的决策树模型都是常用于分类任务的机器学习

习模型。和决策树模型相比，尽管逻辑回归模型无法像决策树模型那样通过分支捕捉复杂的特征交互，但它随机梯度下降的训练方式能够提高机器学习的效率。

至此我们已经学习了很多种不同的机器学习模型，了解了它们的优点和不足。那如果多个模型对同一个结果进行预测，出现多个预测结果，我们应该如何处理呢？下一节里我们会介绍集成学习，它的核心思想就是通过综合多个预测器的结果来获得比单个预测器更好的预测效果。

7.5 集成学习 (Ensemble)

集成学习的灵感来源于众人的协作，就像听取多方意见的决策要比独断专行更不容易出错一样，多个模型共同的预测也可能比单个模型的预测效果更好。不同分类器集成的主要优势在于，所有分类器不太可能犯同样的错误。实际上，只要出现的错误是由少数分类器造成的，且多个分类器都能独立做出预测，那么使用少数服从多数的原则来选择多个分类器的结果，就很容易找到最正确的预测。

如何获得多个不同的分类模型呢？可以更换模型（算法），或者更换训练数据。不过遗憾的是，由于基本假设类似且使用数据相同，不同模型能够独立做决策的假设很难成立。因此，大部分情况下，我们都会通过改变训练数据来获得不同分类器。此时，我们需要一个对训练数据的微小变化很敏感的分类算法。

在本节中我们会介绍三种集成学习模型——自举汇聚法 (Bagging, Bootstrap Aggregating 的缩写)、随机森林 (Random Forest)，还有提升法 (Boosting)，并在最后对集成学习的各种方法进行总结。

7.5.1 自举汇聚法 (Bagging)

Bagging 法是指对数据进行可重复的抽样，以此来获得不同的训练数据，从而得到多个模型。具体来说，我们从训练集（样本数量为 N ）中选取 N 个样本，这些样本中的每一个都可以是我们之前选择过的；并基于选择到的 N 个样本训练模型。然后，我们重复这个过程 k 次，就会得到 k 个预测模型。当我们需要对新的输入进行预测时，我们可以汇总 k 个模型的预测结果。对于分类问题，Bagging 模型可以通过多数投票的方式选择被更多模型预测的分类标签；对于回归问题，Bagging 模型可以输出他们的平均值。

事实上，由于 $\lim_{N \rightarrow \infty} (1 - \frac{1}{N})^N = \frac{1}{e} \approx 0.3679$ ，因此当样本个数足够大时，大约有 37% 的数据永远无法被抽取到。这意味着，不同模型的训练集之间的差异还比较大，也使得不同模型并不会完全同质化。

7.5.2 随机森林 (Random Forest)

随机森林，顾名思义，是独立学习多个决策树，然后聚合多个决策树的预测结果进行投票。从形式上看有一点像是以决策树为分类器的 Bagging 方法，但是这里的每棵树都是随机生成的：即在构造一棵树的时候，随机选择一个条件属性作为子树的根节点；同时，如果遇到连续变量的属性，随机森林中的决策树会对该特征进行随机的分裂。这些处理相当于在构建每棵决策树的时候引入了更多的随机性，也就能让随机森林中的决策树具有更大的多样性。

随机森林模型看似随机性很强，但它的实际预测能力可能会不错，而且也不容易过拟合。一个直观的解释是：一些决策树会查询无用的特征，这些树实际上会做出随机预测。但有些树会恰好查询到有用的特征，从而做出准确的预测。如果树的数量足够多，随机的预测会作为噪声被抵消，最终只有那些好的决策树会对最终的分类结果产生影响。这个解释从直觉上讲

行得通，但并不是严谨的数学证明，因此无法严格说明随机森林模型的可靠性。

7.5.3 提升法 (Boosting)

Boosting 算法的总体思路是：循环训练多个分类器，每一轮的训练都对之前训练的模型进行不断改进，最后结合所有分类器作为整体预测的结果。我们前文提到的 GBRT 就用到了 Boosting 的思想。

而在本节中，我们主要介绍自适应提升法 (Adaptive Boosting, 缩写为 AdaBoost)。这个算法是由 Yoav Freund²和 Robert Schapire³于 1995 年提出的一种分类器算法，他们也因这项工作获得了 2003 年的哥德尔奖⁴。

AdaBoost 算法的提出可以说是机器学习发展史上一件非常了不起的事情。上世纪的 90 年代是统计机器学习方法发展的黄金时期，当时如支持向量机 (Support Vector Machines, 简称 SVM, 1995 年出现)、Bagging(1996 年出现)之类的分类算法理论不断出现，但这些都比不上 AdaBoost 算法的创新和影响。事实上，Boosting 算法的思想最早可以追溯到 1989 年，当时哈佛大学的 Leslie Valiant⁵和他的学生 Michael Kearns 提出了一个问题：“弱可学习性是否等价于强可学习性？”，也就是如果一个机器学习任务存在着比“随机猜测”略好一点的“弱学习算法”，那么是否就必然存在着准确率任意高（与该问题的理论上限任意接近）的“强学习算法”，而 Yoav Freund 和 Robert Schapire 在 1996 年给出了肯定的回答。

²Yoav Freund 是一名以色列裔美国计算机科学家，圣地亚哥加州大学教授，主要从事机器学习、概率论和相关领域及应用的研究。

³Robert Schapire 是一名美国计算机科学家，前普林斯顿大学计算机科学系教授，后转至微软研究院。因为发明 AdaBoost 算法对机器学习的巨大贡献，2014 年他获选为美国国家工程院院士，2016 年获选为美国国家科学院院士。

⁴哥德尔奖是一个颁发给理论计算机科学领域杰出论文的年度奖项，为纪念库尔特·哥德尔而设立（哥德尔的具体贡献在本书 3.6 节有所介绍）。

⁵Leslie Valiant 是计算学习理论奠基人、2010 年 ACM 图灵奖得主

在 AdaBoost 算法中，前一个分类器分错的训练样本会被用来训练下一个分类器。单独一个分类器可能很弱，但只要它的分类效果比随机猜测好一点（比如两类问题分类错误率略小于 0.5），而且每次都能改正一些前一个分类器的错误，最后集合所有分类器的预测就能得到一个预测准确率很高的预测模型。

具体来说，任何一个分类器都会出现分类准确的训练样本和分类不准确的训练样本。而 AdaBoost 的设计思路的巧妙之处就在于：它会增加每一次训练后分类不准的样本的权重，从而在下一次训练中，让分类器多去优化还无法被准确分类的训练样本。随着迭代次数增加，分类不准的样本会越来越少，也意味着整体模型的预测能力越来越强。AdaBoost 模型开创性地展示了通过串行、自适应地组合大量“弱”且简单的模型，可以构建出极其强大的预测模型。这种思想简单而深刻，直接启发了后续更强大的集成方法。下面我们会详细讲述这个方法。

Algorithm 5 AdaBoost(W, D, K)

```

1:  $d^{(0)} \leftarrow \langle \frac{1}{N}, \frac{1}{N}, \dots, \frac{1}{N} \rangle$  {Initialize uniform importance to each example}
2: for  $k = 1$  to  $K$  do
3:    $f^{(k)} \leftarrow W(D, d^{(k-1)})$  {Train  $k^{th}$  classifier on weighted data}
4:    $\hat{y}_i \leftarrow f^{(k)}(x_i), \forall i$  {Make predictions on training data}
5:    $\hat{\epsilon}^{(k)} \leftarrow \sum_i d_i^{(k-1)} [y_i \neq \hat{y}_i]$  {Compute weighted training error}
6:    $\alpha^{(k)} \leftarrow \frac{1}{2} \log \left( \frac{1 - \hat{\epsilon}^{(k)}}{\hat{\epsilon}^{(k)}} \right)$  {Compute “adaptive” parameter}
7:    $d_i^{(k)} \leftarrow \frac{1}{Z} d_i^{(k-1)} \exp[-\alpha^{(k)} y_i \hat{y}_i], \forall i$  {Re-weight examples and normalize}
8: end for
9: Return  $f(\mathbf{x}) = \text{sgn} \left[ \sum_k \alpha^{(k)} f^{(k)}(\mathbf{x}) \right]$  {Return weighted voted classifier}
```

AdaBoost 算法的伪代码如算法5所示，这个算法适用于分类任务，将正例赋值为 1，负例赋值为-1。从伪码中可以看出，AdaBoost 的每一轮迭代可大致分为训练模型、计算误差、更新数据权重三个部分。首先，该算法

的第 3 行，是在加权的数据上训练模型。具体讲，以逻辑回归为例，加权的数据对应的优化函数为：

$$\sum_{i=1}^n d_i (y_i \ln(1 + e^{-\theta^T \mathbf{x}_i}) + (1 - y_i) \ln(1 + e^{\theta^T \mathbf{x}_i})). \quad (7.34)$$

其中 d_i 代表了第 i 个样本的权重。所有样本的初始权重都是平均值。在 AdaBoost 模型训练过程中，每个样本的权重会随着训练次数 k 的增加而不断更新。

当得到了第 k 次训练的分类器 $f^{(k)}$ 后，下一个环节就是计算分类器的预测误差。对于任意训练样本 x_i ，AdaBoost 算法会先计算出当前分类器下的每个样本的预测值（即伪码第 4 行的内容）。之后的一步是计算加权训练误差：具体讲，伪码中的 $[y_i \neq \hat{y}_i]$ 是分类器对第 i 个样本预测值的误差计算结果，对于分类任务来说，当 $y_i \neq \hat{y}_i$ 时返回 1，否则返回 0。接着，让该样本误差乘以该样本的当前权重 $d_i^{(k-1)}$ 得到加权误差，再将所有样本的加权误差汇总得到加权训练误差 $\hat{\epsilon}^{(k)}$ 。通常， $\hat{\epsilon}^{(k)} < 0.5$ 。

计算完训练误差后，接下来就要更新数据权重的部分，这也需要分两步完成。首先，计算当前分类器的适应性参数 $\alpha^{(k)}$ ，若加权训练误差越小， $\alpha^{(k)}$ 越大，则说明这个分类器的分类效果越优秀，反之则说明该分类器分类效果不够好。整个迭代的最后一步就是更新样本权重，也就是伪码第 7 行的内容：其中， $\exp[-\alpha^{(k)} y_i \hat{y}_i]$ 用于计算判断第 k 次训练后的第 i 个样本的权重 $d_i^{(k)}$ 的调整方向以及幅度；若 $y_i \neq \hat{y}_i$ ，则 $\exp[-\alpha^{(k)} y_i \hat{y}_i]$ 大于 1， $d_i^{(k)}$ 一定会增大，反之则 $d_i^{(k)}$ 减小；增大或减小的幅度由适应性因子决定。最后经过归一化系数 Z 的处理使得 $\sum_i^N d_i^{(k)} = 1$ 。之后，AdaBoost 模型开始进入下一轮迭代。

当模型训练完成以后，如伪码第 9 行所示，对于给定输入，AdaBoost 算法会以模型中每个分类器的适应性参数为权重，计算所有分类器的结果的加权之和，并由符号函数将预测值转化为 -1 或 1 输出。这就是 AdaBoost 模型的完整学习和预测过程。

AdaBoost 算法的训练误差上界

与随机森林模型有所不同的是, AdaBoost 算法具有理论上的训练误差上界:

$$\frac{1}{N} \sum_{i=1}^N I(f(\mathbf{x}_i) \neq y_i) \leq \exp \left(-2 \sum_{k=1}^K \gamma_k^2 \right) \quad (7.35)$$

其中 $\gamma_k = \frac{1}{2} - \epsilon(k)$ (通常, $\gamma_k > 0$ 成立)。 $I()$ 是判断 $f(x_i)$ 是否与 y_i 相等的函数, $f(\mathbf{x}_i) \neq y_i$ 时返回 1, 反之返回 0。

这是一个相当优雅的不等式。不等式的左侧是 AdaBoost 算法模型的训练误差, 右侧是该误差的上界。显然, 随着训练轮次 K 的增大, 该上界会逐渐减小。意味着, 随着训练轮次的增加, 该算法的训练误差可能在逐渐减小。

不等式 7.35 的证明.

对于这个不等式, 我们可以采取放缩的方法, 通过一些中间引理来证明不等式成立。以下是具体的放缩过程:

引理一:

$$\frac{1}{N} \sum_{i=1}^N I(f(x_i) \neq y_i) \leq \frac{1}{N} \sum_{i=1}^N e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} \quad (7.36)$$

证明:

当 $f(x_i) \neq y_i$ 时:

$$y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i) \leq 0 \quad (7.37)$$

$$e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} \geq 1 \quad (7.38)$$

由 (7.38) 得:

$$\frac{1}{N} \sum_{i=1}^N I(f(x_i) \neq y_i) \leq \frac{1}{N} \sum_{i=1}^N e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)}$$

证毕。

引理二：

$$\frac{1}{N} \sum_{i=1}^N e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} = \prod_{k=1}^K Z^{(k)} \quad (7.39)$$

其中 $Z^{(k)}$ 为第 k 次训练过程中的标准化参数。

证明：

$$\begin{aligned} \frac{1}{N} e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} &= d_i^{(0)} e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} \\ &= d_i^{(0)} \prod_{k=1}^K e^{-y_i \alpha^{(k)} f^{(k)}(x_i)} \\ &= d_i^{(0)} e^{-y_i \alpha^{(1)} f^{(1)}(x_i)} \prod_{k=2}^K e^{-y_i \alpha^{(k)} f^{(k)}(x_i)} \\ &= d_i^{(1)} Z^{(1)} \prod_{k=2}^K e^{-y_i \alpha^{(k)} f^{(k)}(x_i)} \end{aligned} \quad (7.40)$$

...

$$\begin{aligned} &= d_i^{(k)} \prod_{k=1}^K Z^{(k)} \\ \sum_{i=1}^N d_i^{(k)} &= 1 \end{aligned} \quad (7.41)$$

由 (7.40)、(7.41) 可得：

$$\frac{1}{N} \sum_{i=1}^N e^{-y_i \sum_{k=1}^K \alpha^{(k)} f^{(k)}(x_i)} = \prod_{k=1}^K Z^{(k)}$$

证毕。

引理三：

$$Z^{(k)} = \sqrt{1 - 4\gamma_k^2} \quad (7.42)$$

证明：

$$\begin{aligned} Z^{(k)} &= \sum_i d_i^{(k)} e^{-y_i \alpha^{(k)} f^{(k)}(x_i)} \\ &= \sum_{y_i = f^{(k)}(x_i)} d_i^{(k)} e^{-\alpha^{(k)}} + \sum_{y_i \neq f^{(k)}(x_i)} d_i^{(k)} e^{\alpha^{(k)}} \\ &= (1 - \epsilon^{(k)}) e^{-\alpha^{(k)}} + \epsilon^{(k)} e^{\alpha^{(k)}} \end{aligned} \quad (7.43)$$

我们有：

$$\alpha^{(k)} = \frac{1}{2} \ln \frac{1 + 2\gamma_k}{1 - 2\gamma_k} \quad (7.44)$$

代入 (7.43) 可得：

$$Z^{(k)} = \sqrt{1 - 4\gamma_k^2}$$

证毕。

引理四：

$$\prod_{k=1}^K \sqrt{1 - 4\gamma_k^2} \leq e^{-2 \sum_{k=1}^K \gamma_k^2} \quad (7.45)$$

证明：

对于任意的 x ，令 $F(x) = e^{-2x^2} - \sqrt{1 - 4x^2}$ ， $x \in [0, \frac{1}{2}]$ ，则：

$$F'(x) = \frac{dF(x)}{dx} = -4xe^{-2x^2} + 4x(1 - 4x^2)^{-\frac{1}{2}} \quad (7.46)$$

令 $G(x) = -e^{-2x^2} + (1 - 4x^2)^{-\frac{1}{2}}$ ，则：

$$G'(x) = \frac{dG(x)}{dx} = 4xe^{-2x^2} + 4x(1 - 4x^2)^{-\frac{3}{2}} > 0 \quad (7.47)$$

由 $G(0) = 0$ 和 (7.47) 可知 $F'(x) \geq 0$, 因此:

$$F(x) \geq F(0) = 0 \quad (7.48)$$

由 (7.48) 可知, 对任意的 x :

$$\sqrt{1 - 4x^2} \leq e^{-2x^2}, \text{ 即:}$$

$$\prod_{k=1}^K \sqrt{1 - 4\gamma_k^2} \leq e^{-2\sum_{k=1}^K \gamma_k^2}$$

证毕。

综上所述:

$$\frac{1}{N} \sum_{i=1}^n I(f(x_i) \neq y_i) \leq e^{-2\sum_{k=1}^K \gamma_k^2}$$

证毕

□

这个性质给出了 AdaBoost 模型的训练效果的理论保障: 随着训练轮数越来越多, 训练误差的上界会越来越小。

以上就是关于 Boosting 的主要内容, 我们在这一节中主要介绍的是 AdaBoost 模型, 它的算法过程非常简单 (伪码仅有十行), 但却具有非常优雅的性质, 这让它有着很强的实用性。同时, 作为最早被提出来的 Boosting 算法, AdaBoost 模型对整个集成学习的发展都有着很深刻的影响。

7.5.4 集成学习的小结

以上是关于集成学习的全部内容。可以看出, 无论是 Bagging、随机森林还是 Boosting, 它们的基本假设都是类似的, 即根据多个预测器的预测结果汇总出一个最有可能的预测结果。在优化目标和训练过程上, 集成学习并没有自己单独的一个目标函数, 只要让被集成的预测器达到最优就行了, 而集成学习模型的训练过程其实就是每个被集成的预测器的训练过程,

只不过集成学习模型在每一轮训练时可能调整训练样本，以期得到不同的预测器。

因此，不同集成学习方法的重点内容主要集中在对样本的调整方法和集成学习模型最后的预测方法上，对 Bagging、随机森林和 AdaBoost 的总结如表7.8所示：

表 7.8: 不同集成学习方法的总结

模型	调整样本的方法	预测方法
Bagging	可放回重复采样	分类问题：投票； 回归问题：均值
随机森林	随机选取条件属性作为子树的根节点，随机划分连续属性的类别	投票
AdaBoost	根据前一个预测器的预测结果调整样本权重	加权投票

至此我们已经学习了很多的机器学习模型的理论知识，在下一小节中，我们会探讨机器学习在解决实际问题的过程中需要注意哪些问题。

7.6 如何用机器学习方法实际问题

如何用机器学习解决实际问题呢？我们使用机器学习模型是希望计算机通过学习给定的训练示例，在它从未见过的样本上进行预测。而这项任务能否成功，一方面取决于模型的算法本身，一方面也取决于训练数据的质量和数量；而往往训练数据才是一切的基础。

大规模高质量的训练数据是关键

用机器学习解决实际问题的项目中，我们往往需要花费 70% 的时间在标注、收集和整理数据，这是一切成败的关键。这里需要强调两点：训练数据的质

量和数量。

首先是训练数据的质量。这里的质量包含两个层面的含义：1) 样本特征的质量。我们得保证在每个样本上提取的特征信息是准确的，没有出错且尽量减少误差。但有些实际问题中，如果采集数据需要和现实场景的物理世界交互，这一点是很难保障的。2) 标注信息的质量。标注信息，即每个样本上的类别标签或者真实的预测值。因此，标注信息往往需要借助应用领域的专家来提供。而标注专家都可能倦怠，因而，为了提高标注质量，我们不得不采用在同一个样本上多人标注的方法，只有多个人的标注结果都相同时，我们才能使用这条数据。

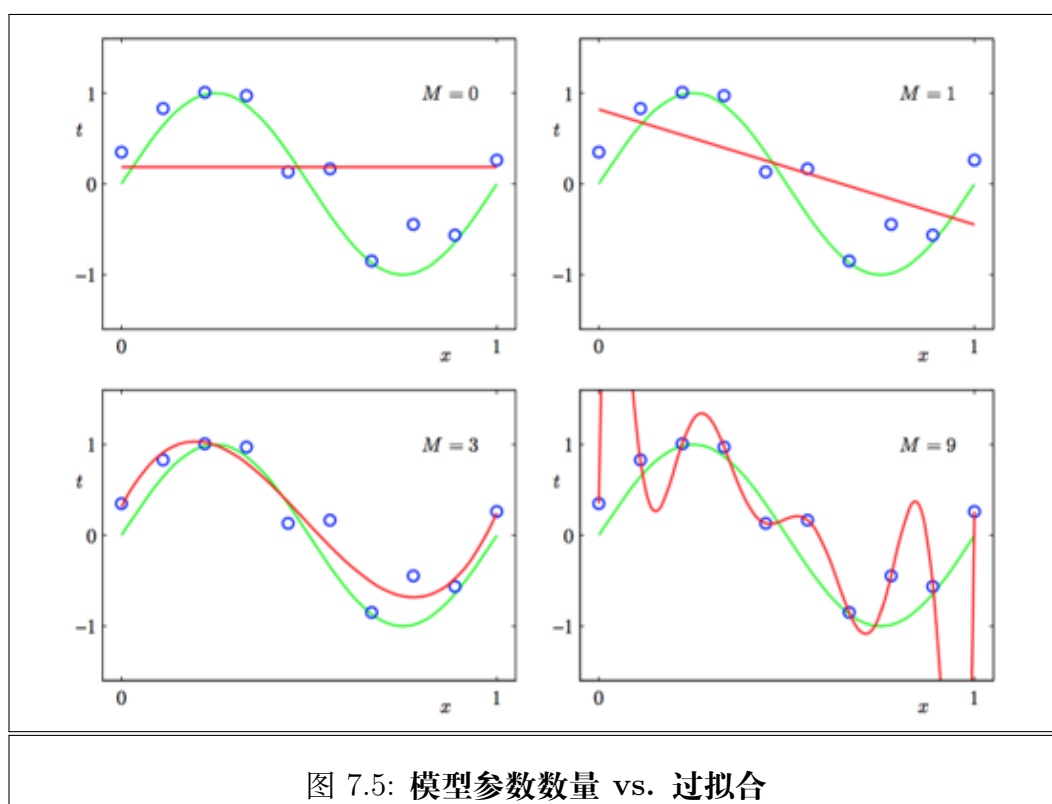
其次是训练数据的数量。大家肯定会问，针对一个特定的实际问题，我们究竟需要多少训练问题呢？对于人工智能的研究人员，这绝对是顶级的“灵魂拷问”的问题。但遗憾的是，我们并不能提供一个具备理论保障的数字，只能凭实际经验估计需要的训练数据的量级。显然，这个数字也是跟实际问题的场景和难度密切相关的。

我们发现，大型互联网公司提供的机器学习应用，如短视频推荐、广告推荐等，都能取得巨大的成功；其背后的真实奥义就在于训练数据的质量和数量。因为这些系统天然就在网络的电子空间中运行，用户使用这些软件产品的过程实际就是数据标注的过程；在这些系统中的数据采集成本极低。然而，一旦机器学习模型需要应用于与物理世界交互的场景，训练数据的采集变得相当困难。因此，如何打通物理世界和电子世界的通道，是普及机器学习系统广泛应用的首要核心问题。

模型训练是有迹可循的技术活

现在，如果已经具备了大规模高质量的训练数据，我们应该如何训练和优化模型呢？大家可能首先会想到“过拟合”的问题，那么什么是过拟合呢？

请看图7.5给出的直观的例子。在这个例子中，蓝色的点是已经提供的训练数据，我们可以用一个一元 M 次函数（有 $(M + 1)$ 个参数）来拟合。



你会看到，随着 M 的增加（即模型参数的增加），拟合成的曲线（即每个子图中的红色曲线）更能拟合训练数据。具体的，当 $M = 9$ 时，红色的曲线已经能完美的穿越每个蓝色的点；但此时，在除了训练数据之外的其它位置，红色曲线与真实的绿色曲线相差很大。而当 $M = 3$ 时，虽然红色的曲线并不能完美的穿越蓝色的点，但它应该是更好的选择。

当然这是一个由研究人员构造的极端的例子，让读者能充分感受什么是“过拟合”。但是，我们在实际的机器学习项目中，并不要惧怕“过拟合”，而追求在训练数据上的完美效果应该成为我们的首要目标。为什么这么说呢？因为，现实问题往往比较复杂，如果训练数据是大规模且高质量的，我们首先需要一个具备足够能力的模型来应对数据本身的复杂性。在此基础上，我们再逐渐减小模型的参数，看看参数小的模型是否会带来效果的变化，然后在模型参数和预测效果之间探求一个相对的平衡点。

具体讲，我们通常会把大规模且高质量的标注数据按 (8 : 1 : 1) 的比例随机分成三份，形成训练集、验证集和测试集。通常，我们首先在训练集上训练模型，并根据模型在验证集上的效果调整模型的超参，获得在验证集上效果最优的模型；最后，我们把此模型在测试集上进行测试。由于测试集中的数据完全没有在模型训练和调优中使用，在测试集上的效果指标应该就是模型在实际场景上线后实际效果的合理估计。

因此，我们需要一个能力足够强大的模型，能在大规模且高质量的训练数据上获得完美的效果。然而，回忆一下，我们之前讲授的机器学习模型的能力都是很有限的。比如，决策树实际是在特征空间中划分超矩形，而逻辑回归是在高维空间中的分类平面。这些单个的模型都不足以应对实际场景的复杂数据。

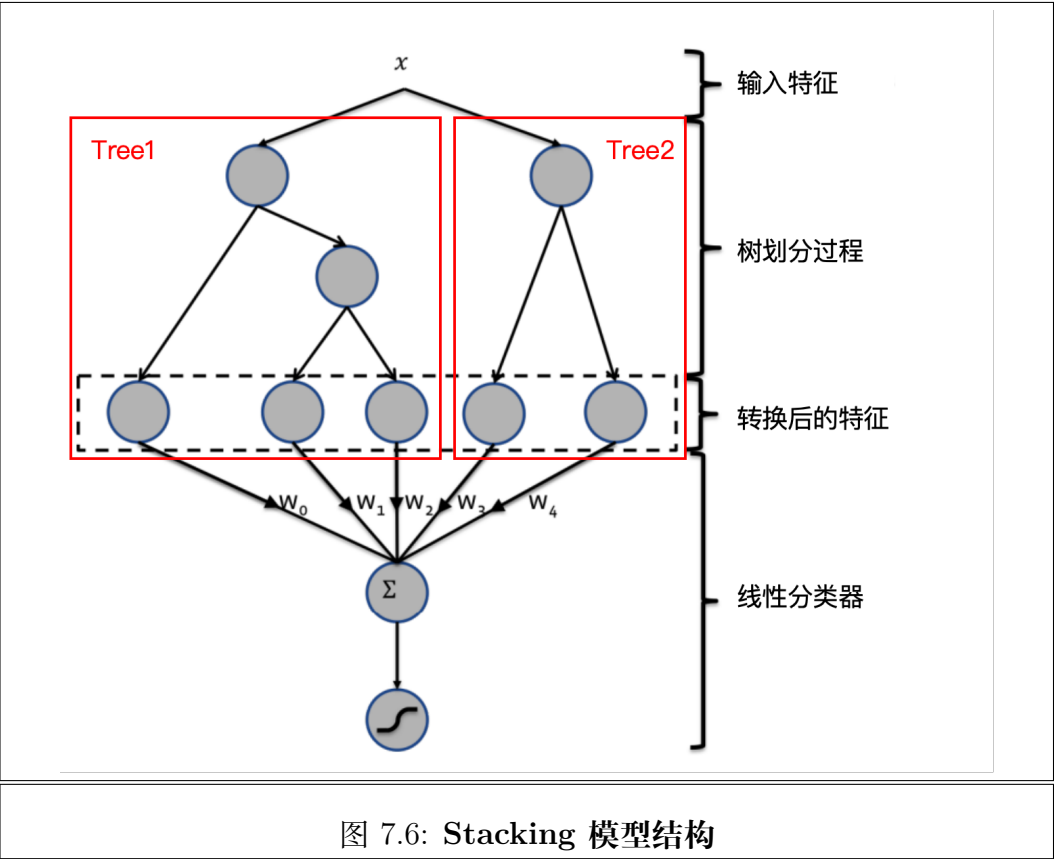


图 7.6: Stacking 模型结构

在工程实践中，常用一种如图7.6所示的“GBRT 作为特征变换器 + 线性模型作为最终预测器”的 Stacking 模型方案：先用 GBRT 在原始特征上学习一组分裂规则，再把每个样本在每棵树中最终落到的叶节点编号视为该树的一个类别输出，并对其进行 one-hot 编码。将所有树的 one-hot 向量拼接后，可得到一个高维但极其稀疏的二值特征向量，该向量的每一维对应“一条由根到叶的规则”，线性模型在其上学习权重进行最终的目标预测，等价于学习“哪些规则对目标更有贡献”。Stacking 模型方案可以把原本复杂、难以线性拟合的原始空间映射到一个更“规则化”的新空间，使得线性模型更容易拟合，其于 2014 年在论文 Practical Lessons from Predicting Clicks on Ads at Facebook 中被提出，并已在 Facebook 的广告点击率预测、在线广告投放排序等工业实践中取得巨大成功，感兴趣的同学可以自行阅读该论文。

应用机器学习本质上是特征工程

那么，如果大规模高质量的标注数据已经提供，能力足够强大的机器学习模型已经具备，机器学习工程师每天都在忙什么呢？答案就是“特征工程”。

机器学习工业界那个众所周知的秘密：数据决定了模型的上限，而特征工程决定了模型能多大程度上逼近这个上限。这句看似简单的回答，实际上揭示了机器学习从理论走向实践时最关键的鸿沟。在学术界的理想实验中，我们往往假设数据是完美的、分布均匀的；但在现实世界里，原始数据往往是粗糙、嘈杂且非结构化的。机器学习模型，无论其数学原理多么高深，本质上仍然是对数字敏感的统计工具，它们并不具备人类的“常识”或“直觉”。因此，特征工程的本质，就是利用工程师的领域知识，将原始数据“翻译”成模型能够高效理解和利用的数学表达形式。这不仅是数据的预处理，更是一场将人类对业务逻辑的理解注入算法的认知重构过程。

从数学的角度来看，如果我们将原始数据看作输入空间 X ，将最终的预测结果看作输出 Y ，那么特征工程的任务就是寻找一个变换函数 f ，将

X 映射到一个新的特征空间 X' 。在这个新的空间里，数据的内在规律应当变得更加线性可分，或者其流形结构变得更加平滑。例如，在房价预测任务中，原始数据可能只包含房屋的“长”和“宽”。对于一个简单的线性模型来说，它很难直接理解“面积”这个概念，因为面积是长与宽的乘积。此时，如果工程师通过特征工程构建了一个名为“面积”的新特征，即 $Area = Length \times Width$ ，就等于帮模型完成了一次关键的非线性变换，极大地降低了模型的学习难度。这就是特征工程的核心价值：通过人为引入先验知识，降低模型的假设空间复杂度。

即便在深度学习大行其道的今天，有人可能会认为神经网络的“自动特征提取”能力让传统特征工程变得过时，但这其实是一种误解。虽然深度神经网络在图像和文本等非结构化数据上表现出了惊人的表征学习能力，但在处理金融、风控、推荐系统等领域的结构化表格数据 (Tabular Data) 时，精心设计的人工特征依然是提升模型性能的“杀手锏”。

因此，如果我们将机器学习系统比作一辆赛车，算法模型是强劲引擎，那么特征工程就是对燃油的提炼工艺。无论引擎多么先进，如果注入的是含有杂质的劣质原油，赛车也无法跑出高性能。

7.7 经典机器学习方法的总结

这一章中，我们介绍了一些经典的机器学习方法，包括如下的三类：

- 关于树的机器学习模型：决策树、回归树、GBRT；
- 逻辑回归；
- 集成学习方法：Bagging，随机森林，AdaBoosting。

对于前两类方法，每个独立模型都包含了模型的四要素，即模型的基本假设、目标函数、训练方法、预测方法。

模型的基本假设其实决定了模型的能力。例如：逻辑回归模型，用 sigmoid 函数建模样本归属于某个类别的概率，模型的参数是一个向量，这就决定了这个方法实质是一个线性分类器，并不能处理线性不可分的数据。而决策树模型，假设了以树的形式来进行分类，实质是在高维空间中切分高维的立方体；落入某个立方体的数据都属于同一个类别；因此它其实可以处理线性不可分的数据。所以，对模型的基本假设是关键。

给出模型的目标函数后，模型的训练过程实质就是求解第5章介绍的最优化搜索的问题。数学上有人专门研究最优化理论。但是，在工业应用上，训练数据非常大，必须考虑方法的时间复杂度。因此，我们往往都采用非常简单的优化方法，即贪心算法。如果目标函数对模型参数不可导，则需要根据模型的目标函数设计贪心算法，如回归树的贪心算法。如果目标函数对模型参数是可导的，则可以直接使用梯度作为贪心的方向。进一步的，如果数据量非常大，在全量数据上计算梯度也至少需要扫描所有数据，那么我们也可以随机采样一小部分数据（一个 mini batch），在这一小部分数据上计算梯度后进行参数更新，这就是随机梯度算法。

最后，需要谈谈模型的预测方法。到目前为止，介绍的模型的预测方法都是非常简单的。

在现实中，机器学习算法有非常广泛的应用。它能够通过学习历史样本的数据特征，来对新的样本进行精准的分类或预测。凭借其可解释性强、计算成本低等优势，传统机器学习模型仍在互联网广告、推荐系统，精准营销、金融风控等场景快速落地，支撑着数万亿的产业。

不过传统机器学习算法也存在缺陷，也就是高度依赖人工特征工程。例如在比较复杂的图像识别、自然语言处理领域，研究者需要手动设计特征（如图像的边缘、纹理，文本的词频），这一过程耗时且易遗漏关键模式。而我们接下来会介绍深度学习，它通过多层神经网络的非线性变换，突破了传统机器学习在特征提取、模式建模上的限制，在更多输入模态复杂的任务上有着更出色的表现。

第八章 深度学习基础

深度学习 (Deep Learning) 是目前应用最广泛的机器学习方法，在计算机视觉、自然语言处理、语音合成等方面都发挥着重要的作用。其中“深度”的含义是指模型中计算路径的长度，即原始输入数据进行非线性特征转换的次数。举例来说，逻辑回归模型从输入到输出的路径非常短，只是将输入样本中的变量进行一次函数变换就得到输出。而在深度学习中，类似上述的转换过程在模型中会进行很多次。

我们之前介绍过的联结主义主张：只要让计算机模仿大脑里神经元彼此连接的方式，就能逐步“学”出人类水平的智能。今天的深度学习正是把这一理念付诸实践——用一层层“人工神经元”搭成网络，把生物神经网络加以抽象、简化和数学化，最终形成一套能够完成复杂学习任务的模型。

在本章中，我们会先讲述前馈神经网络以及在神经网络模型中比较重要的反向传播算法，让大家先学习神经网络模型的基本知识。然后依次介绍卷积神经网络、循环神经网络，以增加大家对深度学习中几个比较基础的神经网络模型的工作原理以及应用场景的了解。

8.1 前馈神经网络

前馈神经网络 (Feedforward Neural Network, FNN) 是最早被发明的简单人工神经网络，同时也是理解更复杂的神经网络的基础。作为深度学习章节的第一节，本节的内容也会涉及到神经网络模型的一些基础知识，以

帮助大家更快掌握神经网络模型的原理。

8.1.1 前馈神经网络模型基本假设

8.1.1.1 神经元

神经元是构成神经网络的基本单位，大家可能对生物学中的神经元并不陌生，它是大脑中的神经细胞，与其他神经元相互连接，按照固定方向传递信息。类比到人工智能，神经元接收高维的输入，并给出一个一维的输出。例如下式展示的就是以 sigmoid 函数来将高维输入映射到一维输出的过程。

$$h_{\mathbf{w},b} = f(\mathbf{w}^T \mathbf{x} + b) \quad (8.1)$$

$$f(z) = \frac{1}{1 + e^{-z}} \quad (8.2)$$

其中 w 代表权重向量， b 为常数项，代表偏差 (Bias)。而 $f(z)$ 被称为激活函数。

这个神经元的示意图如图8.1所示。

单个神经元的功能看似简单，可就像人脑中数以亿计的神经元相互联结才涌现出智能一样，人工神经网络也是由大量这样的“微小组件”协同而成。每个单元都与若干同伴相连，通过约定的连接拓扑和信息传递规则共同完成复杂任务；这种彼此交织的群体便被形象地称为“神经网络”。

8.1.1.2 全连接神经网络

在前馈神经网络中，各个神经元按接收到信息的先后分为不同的层，每一层的神经元接收前一层神经元的输入，并经过自身函数转换后输出到下一层神经元，整个网络的信息朝一个方向传播。

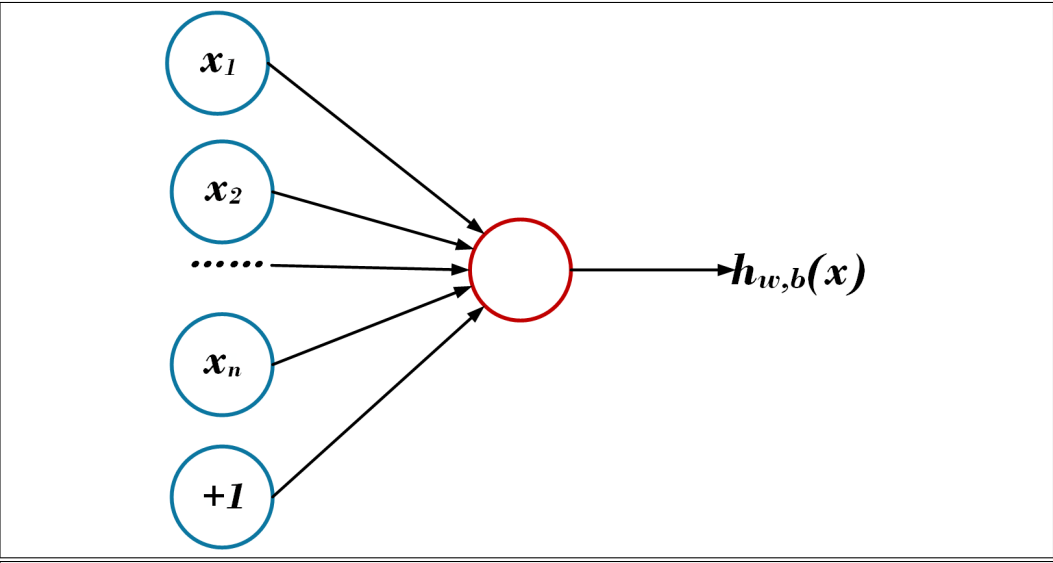


图 8.1: **神经元示意图**，每个圈都代表一个节点，箭头代表信息流动的方向，蓝色节点为输入的向量，红色节点为神经元

把前馈神经网络按信息流向拆开，可以清晰地看到三层结构：输入层、若干隐藏层和输出层。数据从输入层起步，先被第一层隐藏节点各自加工，再依次向下传递；隐藏层之间层层接力，直至最后一层隐藏节点把结果送到输出层，完成一次完整的前向计算。

如果神经网络中的每个节点都和其相邻层的所有节点连接，我们就称之为全连接神经网络 (Fully Connected Neural Network, FCNN)。这也是最为常见的一种前馈神经网络模型。

图8.2展示了全连接神经网络的一个示意图。下面我们会使用全连接神经网络模型来讲解神经网络的前向计算过程。

符号定义

在正式讲解从输入到输出的详细计算过程之前，我们需要进行一些符号定义，然后将这个计算过程表示为矩阵计算的形式。

我们将定义如下的符号：

- 输入层有 n 个神经元 (n 维)，第 l 层有 q^l 个神经元。

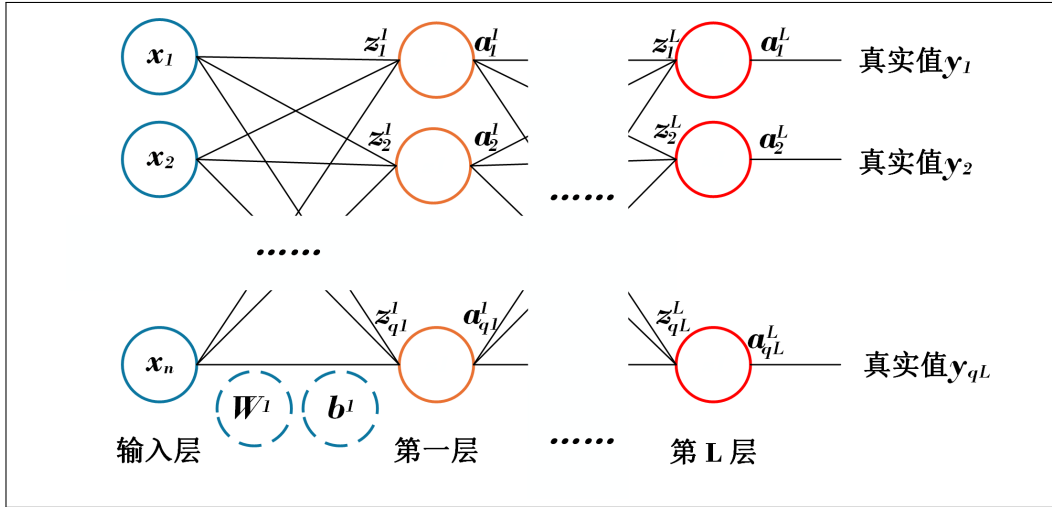
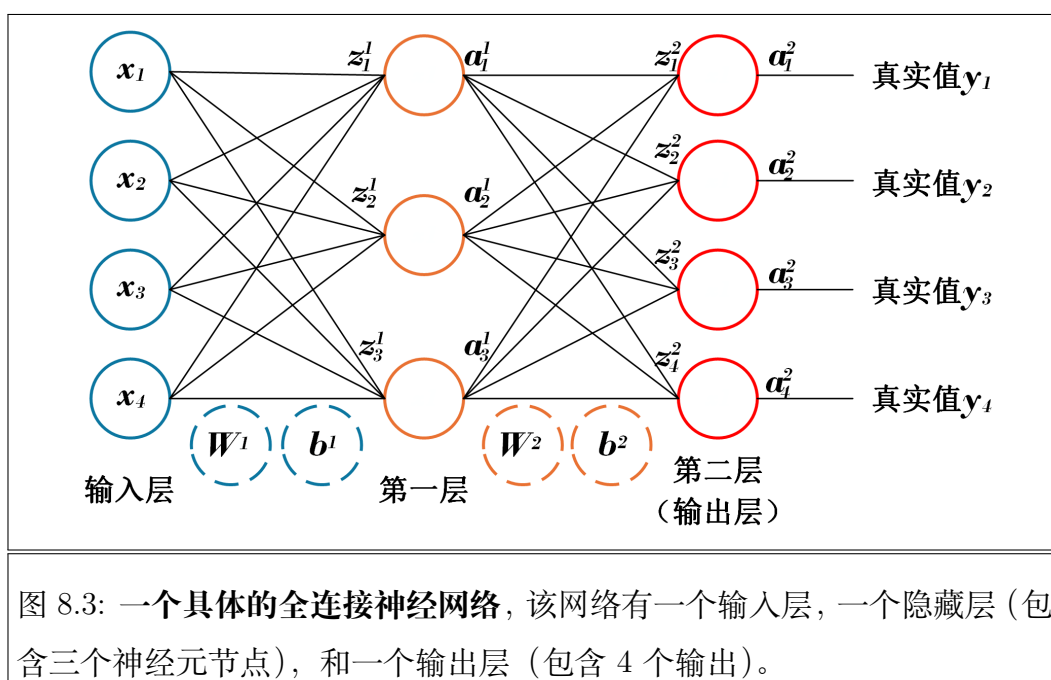


图 8.2: 全连接神经网络示意图, 每一层的每一个节点都会接收前一层所有节点的输入, 自己的输出也会流向下一层所有的节点。

- 除了输入层, 网络共有 L 层。
- W^1 是 $q^1 \times n$ 的矩阵, 代表输入层与第一层之间的权重矩阵, 其第 j 行的 n 个数值是输入层第 j 个神经元与 n 个输入之间的连接权重。
- 截距向量 $b^1 = (b_1^1, b_2^1, \dots, b_{q^1}^1)$ 。
- 第 l 层输入向量 $\mathbf{z}^l = (z_1^l, z_2^l, \dots, z_{q^l}^l)$ 。
- 第 l 层输出向量 $\mathbf{a}^l = (a_1^l, a_2^l, \dots, a_{q^l}^l)$ 。
- $W^{(l+1)}$ 是 $q^{(l+1)} \times q^l$ 的矩阵, 代表第 l 层与第 $l+1$ 层之间的权重矩阵, 其第 j 行的 q^l 个数值是第 $l+1$ 层的第 j 个神经元与 q^l 个第 l 层的输入之间的连接权重。
- 截距向量 $b^{l+1} = (b_1^{l+1}, b_2^{l+1}, \dots, b_{q^{l+1}}^{l+1})$
- 真实值向量 $\mathbf{y} = (y_1, y_2, \dots, y_{q^L})$ 。



下面我们会用一个具体的全连接神经网络模型来展示这些符号。对于一个如图8.3的神经网络, $n = 4$, $L = 2$, 输入向量 $\mathbf{x} = (x_1, x_2, \dots, x_4)$, 真实向量 $\mathbf{y} = (y_1, y_2, y_3, y_4)$ 。神经网络中各层的数据如下所示:

- **输入层到第一层之间:** 权重矩阵 W^1 , W^1 是 3×4 的矩阵, 它的第 1 行的 4 个数值是第一层第 1 个神经元与 4 个输入之间的连接权重, 其他行也以此类推; 截距向量 $b^1 = (b_1^1, b_2^1, b_3^1)$ 。
- **第一层:** $q^1 = 3$; 输入向量 $\mathbf{z}^1 = (z_1^1, z_2^1, z_3^1)$; 输出向量 $\mathbf{a}^1 = (a_1^1, a_2^1, a_3^1)$ 。
- **第一层到第二层之间:** 权重矩阵 W^2 , W^2 是 4×3 的矩阵, 它的第 1 行的 3 个数值是第二层 (也是输出层) 第 1 个神经元与 3 个第一层的输入之间的连接权重, 其他行也以此类推; 截距向量 $b^2 = (b_1^2, b_2^2, b_3^2, b_4^2)$ 。
- **第二层:** $q^2 = 4$; 输入向量 $\mathbf{z}^2 = (z_1^2, z_2^2, z_3^2, z_4^2)$; 输出向量 $\mathbf{a}^2 = (a_1^2, a_2^2, a_3^2, a_4^2)$ 。

从输入到输出的计算过程

基于以上的符号定义，我们可以用如下的矩阵运算表示全连接神经网络从输入到输出的前向计算过程：

- 首先， $W^1 \cdot \mathbf{x} + b^1 = \mathbf{z}^1$ 。
- 然后， $\mathbf{a}^l = f(\mathbf{z}^l)$ ，这里 f 是神经元的激活函数，如 sigmoid 函数，它将输入向量每一维的值转化为对应的 sigmoid 值，形成输出函数。
- 再次， $W^{(l+1)} \cdot \mathbf{a}^l + b^{(l+1)} = \mathbf{z}^{l+1}$ 。

通过以上三个式子，我们可以从输入 $\mathbf{x}$ 迭代的计算出最终的输出 $\mathbf{a}^L$ 。下面我们继续用图8.3的例子来展示计算过程

1. 计算第一层输出： $a^1 = f(z^1) = f(W^1 \cdot \mathbf{x} + b^1)$
2. 计算第二层输出： $a^2 = f(z^2) = f(W^2 \cdot \mathbf{a}^1 + b^2)$

以上就是全连接神经网络的基础假设。我们可以把它看成一个多层嵌套的函数，通过非线性函数（即采用的激活函数）的多次复合，实现输入空间到输出空间的复杂映射。

8.1.2 神经网络的优化目标

8.1.2.1 分类任务

神经网络在最后可以连接具有不同的功能的网络层，产生的具有不同物理意义的输出。在分类任务中， K 分类的 softmax 函数表示为：

$$p_k = \frac{e^{\mathbf{w}_k \mathbf{a}}}{\sum_{k'=1}^K e^{\mathbf{w}_{k'} \mathbf{a}}} \quad (8.3)$$

其中 $\{\mathbf{w}_k\}_{k=1}^K$ 是模型参数， $\mathbf{h}$ 可以是全连接神经网络最后一个隐层输出的向量（即 $\mathbf{a} = \mathbf{a}^L(\mathbf{x})$ ）， p_k 是模型估计的样本 $\mathbf{x}$ 在第 k 类上的概率。

事实上, softmax 函数可以视作 sigmoid 函数在多分类 ($K > 2$) 任务上的扩展。效仿逻辑回归中的最大似然估计, 我们可以得到 softmax 的目标函数:

$$J(W) = -\frac{1}{N} \left[\sum_{i=1}^N \sum_{k=1}^K \text{judge}\{y_i = k\} \log \frac{e^{\mathbf{w}_k^T \mathbf{a}_i}}{\sum_{k'=1}^K e^{\mathbf{w}_{k'}^T \mathbf{a}_i}} \right] \quad (8.4)$$

其中, i 为样本的标号, N 为样本总数, K 为类别个数,

$$\text{judge}\{S\} = \begin{cases} 1, & \text{谓词 } S \text{ 为真} \\ 0, & \text{谓词 } S \text{ 为假} \end{cases} \quad (8.5)$$

8.1.2.2 回归任务

我们还可以训练神经网络以回归某个函数。给定一个数据集 $\mathcal{D} = \{(\mathbf{x}, y)\}$, $\mathbf{x}$ 为样本的输入值, y 为样本的真实输出值。若神经网络对数据 $\mathbf{x}$ 的输出值为 $a^L(\mathbf{x})$, 则我们最小化损失函数可以为

$$C = \frac{1}{2|\mathcal{D}|} \sum_{(\mathbf{x}, y) \in \mathcal{D}} (y - a^L(\mathbf{x}))^2 \quad (8.6)$$

也就是最小化整体的平方误差。

图8.3的全连接神经网络模型就属于回归任务的神经网络。其中该神经网络只有一个隐藏层, 输出层输出 a_i^2 , 代表输出层第 i 个神经元的输出。若 a_i^2 越接近 y_i , 则回归效果越好 ($i = 1, 2, 3, 4$)。

以上是神经网络模型的目标函数内容, 无论对于分类任务还是回归任务, 神经网络模型设置目标函数的原则都是让模型输出值更加接近真实值。下面我们会以全连接神经网络为例来探讨如何去调整神经网络中模型的参数, 以优化训练目标。

8.1.3 全连接神经网络的训练方法

对于全连接神经网络，每个隐层对应的 $W^l, b^l (l = 1, \dots, L)$ 是我们需要优化的模型参数。如前所述，我们可以采用梯度下降法进行参数优化，因此，我们必须计算出目标函数对这些模型参数的导数。

8.1.3.1 反向传播 (BP, Back Propagation) 算法

这个计算梯度的方法是反向传播算法 (Back Propagation)，简称为 BP 算法。

假设我们要完成一个回归任务，训练需要的数据集、激活函数和误差函数如下所示：

- 数据集： $\mathcal{D}$
- 激活函数： $f(\cdot)$
- 误差函数： $\mathcal{C} = \frac{1}{2|\mathcal{D}|} \sum_{(x,y) \in \mathcal{D}} \|y - a^L(x)\|^2$

使用 BP 算法的神经网络模型训练过程可分为以下步骤：

1. 网络初始化

- 给模型参数分别赋一个随机数，随机初始化方法有多种，这里我们用较简单的 $(-0.01, 0.01)$ 内的均匀随机数。
- 给定计算精度值 ϵ 和最大学习次数。

2. 从训练集中抽取一个样本 (x, y) ，计算隐含层、输出层各神经元的输入和输出。

- 各层的输入和输出数据的流动方向可以参考图8.3；第 l 层的输入和输出有如下关系：

$$z^l = W^l a^{l-1} + b^l$$

$$a^l = f(z^l)$$

3. 利用网络输出值 a^L 和样本的真实值 y ，计算目标函数对参数的导数。

我们定义：第 l 层的第 j 个神经元的“误差”为 $\delta_j^l \equiv \frac{\partial C}{\partial z_j^l}$ ，即目标函数对神经元的输入的导数。

由 $C = \frac{1}{2} \sum (y - a^L)^2$,

最末层的 $\delta_j^L = \frac{\partial C}{\partial z_j^L} = \frac{\partial C}{\partial a_j^L} \cdot \frac{\partial a_j^L}{\partial z_j^L} = (a_j^L - y_j) f'(z_j^L)$,

记 $\delta^l = (\delta_1^l, \delta_2^l, \dots, \delta_q^l)$ ，则：

$$\delta^L = (a^L - y) \odot f'(z^L) \quad (8.7)$$

其中 $\odot$ 代表向量的按位乘积，也就是同维向量对应位置元素相乘。

同时，根据神经网络模型的结构和多元函数的复合求导和链式法则，我们能得到任意神经元的损失函数误差：

$$\delta_j^l = \frac{\partial C}{\partial z_j^l} = \sum_k \frac{\partial C}{\partial z_k^{l+1}} \frac{\partial z_k^{l+1}}{\partial z_j^l} = \sum_k \delta_k^{l+1} \frac{\partial z_k^{l+1}}{\partial z_j^l} \quad (8.8)$$

公式8.8的推导如图8.4的图像解释所示。

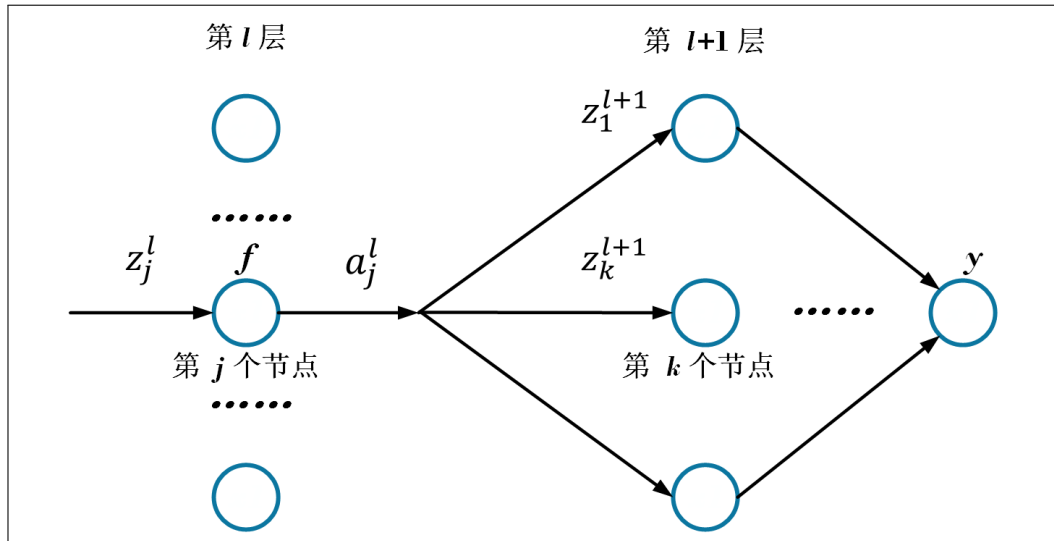


图 8.4: 公式8.8的图像解释

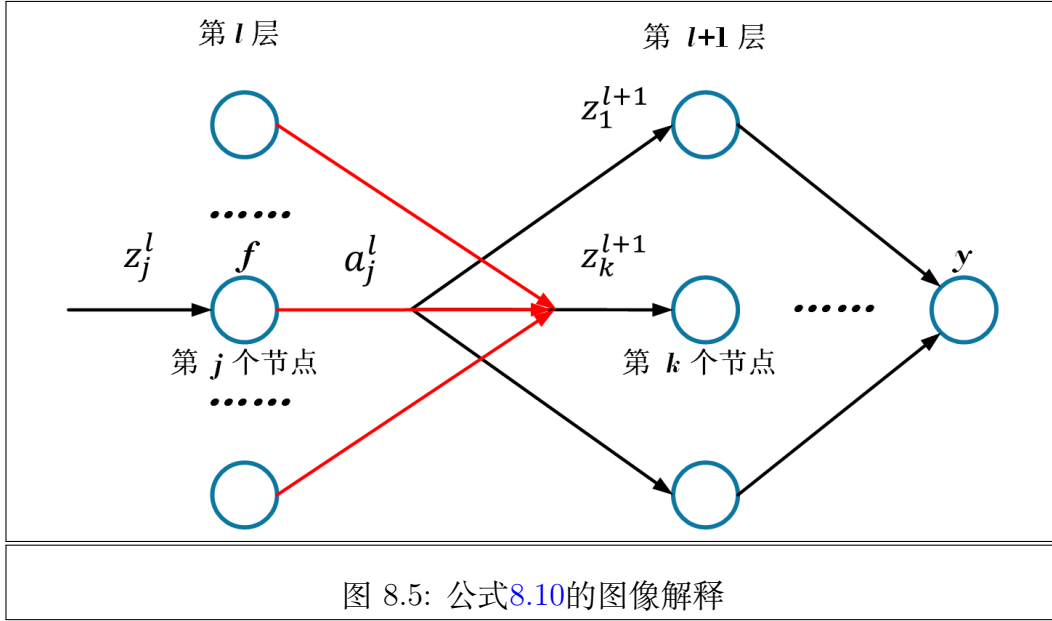
接着，我们计算 $\frac{\partial z_k^{l+1}}{\partial z_j^l}$ 。因为，我们有：

$$z_k^{l+1} = \sum_j w_{kj}^{l+1} a_j^l + b_k^{l+1} = \sum_j w_{kj}^{l+1} f(z_j^l) + b_k^{l+1} \quad (8.9)$$

则：

$$\frac{\partial z_k^{l+1}}{\partial z_j^l} = w_{kj}^{l+1} f'(z_j^l) \quad (8.10)$$

公式8.10的推导如图8.5的图像解释所示。



综合公式8.8和8.10，我们有：

$$\delta_j^l = \left(\sum_k \delta_k^{l+1} w_{kj}^{l+1} \right) f'(z_j^l) \quad (8.11)$$

把上式表示成矩阵的形式，我们有：

$$\delta^l = ((W^{l+1})^T \cdot \delta^{l+1}) \odot f'(z^l) \quad (8.12)$$

其中， $\cdot$ 为一般的矩阵乘法。

综上，根据公式8.7和8.12，我们可以计算出所有隐层的 $\delta^l (l = 1, \dots, L)$ 。具体讲，首先根据公式8.7，计算出 δ^L ；然后根据公式8.12和 δ^L ，计算出 $\delta^{(L-1)}$ ；以此类推，计算出 $\delta^{(L-1)}, \dots, \delta^1$ 。从计算方向看，这个误差的计算是从输出层反向计算到输入层，因此叫反向传播算法。

4. 利用每一层计算出的误差 δ ，计算目标函数对模型参数的导数。

$$\frac{\partial C}{\partial b_j^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial b_j^l} = \delta_j^l \quad (8.13)$$

$$\frac{\partial C}{\partial W_{jk}^l} = \frac{\partial C}{\partial z_j^l} \frac{\partial z_j^l}{\partial W_{jk}^l} = \delta_j^l a_k^{l-1} \quad (8.14)$$

5. 更新模型参数。

$$b_j^{l \text{ new}} = b_j - \eta \frac{\partial C}{\partial b_j^l} \quad (8.15)$$

$$W_{jk}^{l \text{ new}} = W_{jk}^l - \eta \frac{\partial C}{\partial W_{jk}^l} \quad (8.16)$$

其中 η 是学习率，一般较小，如 0.1。

6. 判断训练误差是否满足要求。

- 当误差达到预设精度或学习次数大于设定的最大次数，则结束算法。
- 否则，再进行下一轮训练，即跳到步骤 2 执行。

以上是使用神经网络模型训练回归问题的方法，如果要解决的是分类问题，只需改变目标函数即可。

8.1.3.2 神经网络的训练技巧

数据块的选择

在每一轮的梯度计算中，我们有如下三种方式选择训练数据：

- 批量训练 (Batch training)
 - 选择所有数据；
 - 但是每次要遍历整个数据集，计算效率低。

- **随机梯度下降法 (Stochastic Gradient Descent, SGD)**

- 随机选择单个样本，并在此样本上计算梯度；
- 更随机，可能会跳出局部最优点。

- **小批量随机梯度下降 (miniBatch)**

- 折中方案，随机选择小部分数据来计算梯度。
- 兼顾了随机跳跃和对梯度估计稳定的双方优势。

正则化

正则化是通过限制模型复杂度，从而避免过拟合，提高模型泛化能力的方法。以下展示两种比较常见的正则化方法：

- **增加惩罚项**：在原始的损失函数后边加上 $\sum_w \|w\|_F + \sum_b \|b\|_F$ 。
- **丢弃法 (Dropout)**：通过随机丢弃部分神经元来减少过拟合。具体讲，首先选定一定的丢弃比例 β ；然后，在前向计算中，对比例为 β 的神经元，激活值为 0；接着，在后向计算中，这些激活值为 0 的神经元反传误差也清零，即误差不会经过这些神经元。

学习率调节

学习率是神经网络优化时的重要超参数，关系到梯度下降法能否收敛和收敛速度。常见的学习率设置方式有以下几种：

- **固定学习率**：预设一个学习率，在训练过程中保持不变。
- **参考验证集调节**：通过监测验证集的性能来动态调整学习率。

- **学习率指数衰减**：在迭代过程中让学习率逐渐减小：一开始让学习率比较大，来保证收敛速度；在收敛到最优点附近时，让学习率小一些避免来回震荡。具体讲，设初始学习率 α_0 ，第 t 次迭代时学习率 $\alpha_t = \alpha_0 \beta^t$ ，其中衰减率 $\beta < 1$ 。
- **自适应调整学习率方法（如 AdaDELTA, RMSProp 等）**：自适应的调整单个参数的学习率，梯度大时让学习率减小，梯度小时让学习率增大，通过梯度变化的相对幅度动态调整，适应不同优化阶段。

8.1.4 全连接神经网络的预测

虽然前馈神经网络模型的训练方法要比我们前面学过的传统机器学习模型都要复杂，但如果我们将模型训练好以后，神经网络模型的预测过程仍然比较简单。首先，将待预测的样本输入神经网络模型中，然后神经网络根据其前向计算过程来处理样本，最后得到预测结果。

至此，我们已经介绍了神经网络模型的基本假设、目标函数、预测方法以及全连接神经网络的训练过程。可以看出，神经网络是人工智能领域的一个重大发明，但它的问世却并非一帆风顺，下面我们带大家简单了解神经网络的发展历程。

8.1.5 纪念 Frank Rosenblatt：神经网络的曲折发展史

Frank Rosenblatt 是神经网络领域的开创者。1957 年，这位康奈尔大学实验心理学家提出了 Perceptron（感知器）模型——一种旨在模仿人类感知功能的神经网络雏形。他在计算机上实现了这一模型，并证明：面对线性可分的模式识别任务，单层网络能够稳定收敛；一系列学习实验随之展开，轰动一时。这些工作，首次用实验向世界证明：机器可以像大脑一样通过简单单元协同完成感知与学习。之后几年里，神经网络以其独特的结构和处理信息的方式，在许多实际应用领域（自动控制、模式识别等）中崭露

头角。

然而,神经网络的发展很快遇上了低谷期,原因恰巧来自于 Frank Rosenblatt 在 Bronx High School of Science (全美知名高中) 的校友——Marvin Minsky。1963 年, Marvin Minsky 在《感知机》一书中指出: Perceptron 无法解决 XOR (异或) 的问题。异或是一个基本的逻辑问题, 如果连这个问题都解决不了, 那么说明神经网络的计算能力实在有限。这个缺点被 Marvin Minsky 以一种敌意的方式呈现出来, 当时对 Frank Rosenblatt 是一个很严重的打击, 也导致政府资助机构逐渐停止了对神经网络研究的支持, 使得此后近 20 年时间神经网络的研究处于长年停滞的低潮状态, 这段时间被加州理工学院的集成电路大拿 Carver Mead 称为“二十年大饥荒”。

1971 年, Frank Rosenblatt 在 43 岁生日那天, 划船淹死; 很多人认为是自杀。当 1988 年 Marvin Minsky 在《感知机》一书再版时, 他删除了原版中对罗森布拉特的个人攻击的句子, 并手写了“纪念罗森布拉特”(In memory of Frank Rosenblatt)。但其他在“大饥荒”时期受到压迫的科学家认为明斯基不可原谅。后来神经网络得势后, 这些人纷纷对 Minsky 口诛笔伐。最后, 美国电气电子工程师协会 (IEEE) 于 2004 年设立了 Rosenblatt 奖, 以奖励在神经网络领域的杰出研究。

以上是有关 Frank Rosenblatt 和他所创造的感知器的故事。虽然他发明的感知器存在缺陷, 但在 1974 年, 哈佛大学的 Paul Werbos 的博士论文提出了将链式法则用于训练多层网络的误差反传思想, 这重新激起了人们对神经网络的兴趣。因为反向传播算法在计算梯度上的便利性, 随后许多科学家又提出了多种神经网络模型, 例如 20 世纪 80 年代, David Rumelhart 和 James McClelland 等人提出了分布式并行处理模型 (Parallel Distributed Processing, PDP), 该模型主张将知识存储于大量简单处理单元 (类似于神经元) 的连接权重中, 然后用反向传播算法让权重在训练过程中自动调整。该模型的提出奠定了神经网络模型“输入层 → 隐藏层 → 输出层”的结构基础, 进一步引领了神经网络的复兴。随后, Yann LeCun 和 Yoshua Bengio

将反向传播算法引入卷积神经网络中，并在手写体数字识别上获得了巨大的成功。

然而，此时的神经网络又面临了新的问题，当神经网络层数增加、神经元数量增多的时候，虽然能在理论上构造出可以解决更复杂问题的模型，但伴随而来的就是计算复杂性的增长，当时的计算机性能和数据规模都不足以支持训练太大规模的神经网络。因此，在 20 世纪 90 年代中期，经典的机器学习方法开始兴起。相比之下，神经网络的理论基础不清晰、可解释性差的问题也被凸显出来，这让神经网络的研究又一次陷入低潮。

直到 2006 年，英伟达推出 CUDA(Compute Unified Device Architecture)，使得允许 GPU（即图形处理器）执行通用计算任务更加方便。随后，Geoffrey Hinton 团队运用 GPU 来训练卷积神经网络，大幅提高了神经网络运算效率。这在当时是一个非常创新的尝试，因为 GPU 虽然在 20 世纪 90 年代就被发明出来，但它一直都只用于在图形渲染方面给计算机提速，而 GPU 支持矩阵运算和并行任务，在理论上也能帮助计算机处理神经网络的计算过程。2012 年，Hinton 团队在 ImageNet 的图片分类比赛中取得了巨大的成果（这段故事我们会放在 8.3.3 小节中详细讲述），这一实践证实了用 GPU 来支撑大规模神经网络模型计算的可行性。同时，伴随着互联网的不断发展，其产生的海量数据也增加了机器学习的数据规模。在强大的计算能力和海量数据的支持下，计算机已经可以端到端地训练一个大规模神经网络，这让各大科技公司都投入巨资研究深度学习。神经网络迎来第三次高潮，并持续至今。

简单回顾深度学习的历史，我们可以看出有两个神经网络发展历程的重大突破，一个是反向传播算法，另一个是在 GPU 上计算神经网络（更多涉及到硬件的运用，我们在本书中不做深入学习）。前者是目前深度学习领域的核心训练算法，虽然我们在本节中介绍了全连接神经网络中梯度反向传递公式（即公式 8.12），但这个公式只适用于全连接神经网络。当神经元连接方式改变，模型结构更加复杂后，推导梯度传播的公式也就更加困难。

而反向传播算法的伟大之处就在于，它能够将各种神经网络模型的梯度用一个统一形式来传递。如果我们掌握该算法的原理，那么面对其他类型的神经网络，只要已知模型是如何将输入数据转化为输出数据的前向计算过程，就能自动计算出它在当前状态下的梯度。因此，我们在下一节就会重点讲述能够适用于任意神经网络的一般反向传播算法。

8.2 一般的反向传播算法

反向传播 (Backward Propagation) 其实是与前向计算或前向传播 (Forward Propagation) 相对应的概念。前向计算是数据从输入层流向输出层的过程，通过逐层计算神经元的激活值，最终生成预测结果；反向传播是从输出层向输入层反向计算损失函数对每一层参数的梯度，逐层传递误差信号。

在任何神经网络模型的训练过程中，一次完整的迭代都包括前向计算、损失计算、反向传播、参数更新四步，前向计算可以看作是模型的推理过程，而损失计算、反向传播和参数更新可以看作是模型的学习过程。在学习过程中，反向传播起到了将损失计算结果和参数更新连接起来的作用，也是整个深度学习过程中最为重要，计算最复杂的一步。

在本节中，我们会先介绍计算图，这是一种将神经网络的前向计算过程进行图形化表示的方法，我们可以借由它来直观理解神经网络的前向计算路径；任何的前向计算过程，都可以用计算图来表示。然后，基于导数计算的链式求导法则和多元复合函数求导法则，我们进一步讲解计算图上的梯度计算过程，即反向传播算法。最后，我们通过全连接神经网络的计算图的实例，展示反向传播算法的计算过程。

8.2.1 计算图

计算图由边和节点组成。节点代表运算数，边代表计算操作，通常由父节点指向子节点。计算图的作用就是将复杂的计算表示成一系列简单操作，

并以图的方式将输入与输出的关系直观呈现出来。

图8.6的右侧展示了一个计算图，它对应的函数为 $y = \ln(x_1) + x_1 \times x_2 - \sin(x_2)$ 。在该计算图上，我们只要输入 x_1, x_2 的具体值，就能计算出对应的 y 。

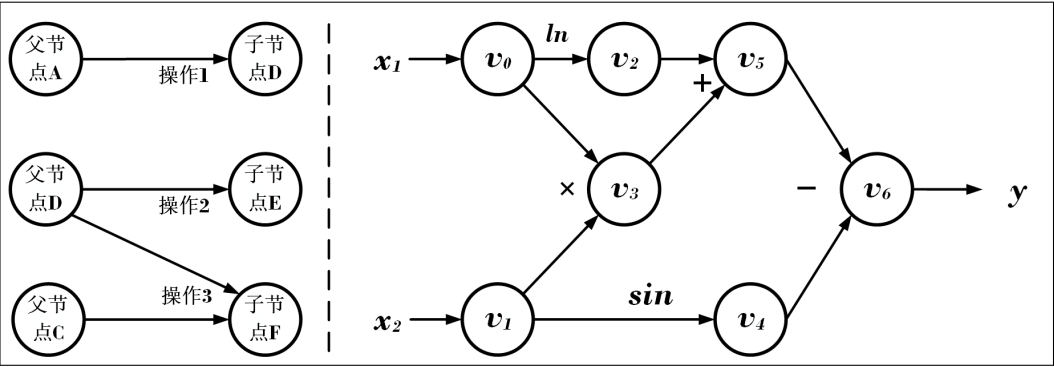


图 8.6: 计算图的表示方法及具体示例

8.2.2 计算图上的梯度计算

我们在介绍全连接神经网络的训练过程的时候，提到了使用多元函数的链式求导法则和复合函数求导法则来计算梯度的方法。而若想将这种反向传播算法拓展到更一般的神经网络模型中，我们就需要认识此两种法则在传递梯度时的具体作用。

8.2.2.1 链状路径上的梯度

在有向图中，路径是从一个起点节点到一个终止节点的有向边序列，求一条路径的梯度时，可以使用链式求导法则。其公式如下所示：

$$z = f(y), y = g(x), \frac{\partial z}{\partial x} = \frac{\partial z}{\partial y} \frac{\partial y}{\partial x} \quad (8.17)$$

图8.7展示了链式求导法则的一个示意图。

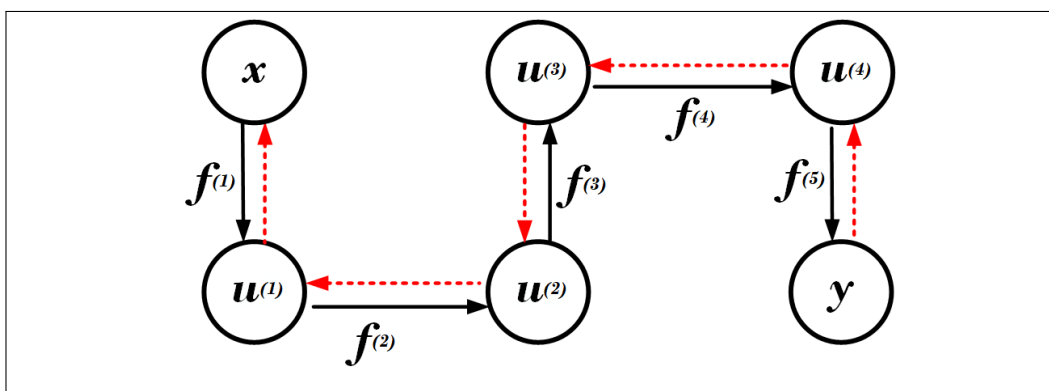


图 8.7: 链式法则示意图, 其中 $u^{(i)}$ 为不同节点的数据, $f^{(i)}$ 代表节点之间的操作 ($i=1,2,3,4$), x 代表原始输入节点, y 代表最终输出节点。黑色实线为从输入到输出的计算方向, 红色虚线为链式法则求导的方向, 操作为相邻两个节点求偏导

从图像中我们可以看出, 原始输入 x 经过五次函数操作后得到了 y , 该计算图对应的函数为 $y = f^{(5)}(f^{(4)}(f^{(3)}(f^{(2)}(f^{(1)}(x))))$ 。如果想要求出 y 对 x 的导数, 我们可以逐层应用链式求导法则, 过程如下所示:

$$\frac{\partial y}{\partial x} = \frac{\partial y}{\partial u^{(4)}} \times \frac{\partial u^{(4)}}{\partial u^{(3)}} \times \frac{\partial u^{(3)}}{\partial u^{(2)}} \times \frac{\partial u^{(2)}}{\partial u^{(1)}} \times \frac{\partial u^{(1)}}{\partial x} \quad (8.18)$$

此时的计算图是一个链状图, 即前向计算的每个节点都只有一个子节点。因此, 在这条路径上梯度的反向传播, 只需要从节点 y 出发, 沿反向路径将每个节点处的梯度相乘就可以了。

不过, 在有些复杂的计算图的前向计算中, 一个节点可能有多个子节点, 此时, 我们需要用到多元复合函数的求导法则。

8.2.2.2 具有多个孩子的节点的计算图上的梯度

多元复合函数求导法则, 其公式如下所示:

已知 $z = f(y_1, y_2, y_3, \dots, y_n)$, $y_1 = g_1(x)$, $y_2 = g_2(x)$, $\dots$, $y_n = g_n(x)$, 我们有

$$\frac{\partial z}{\partial x} = \sum_{i=1}^n \frac{\partial z}{\partial y_i} \frac{\partial y_i}{\partial x} \quad (8.19)$$

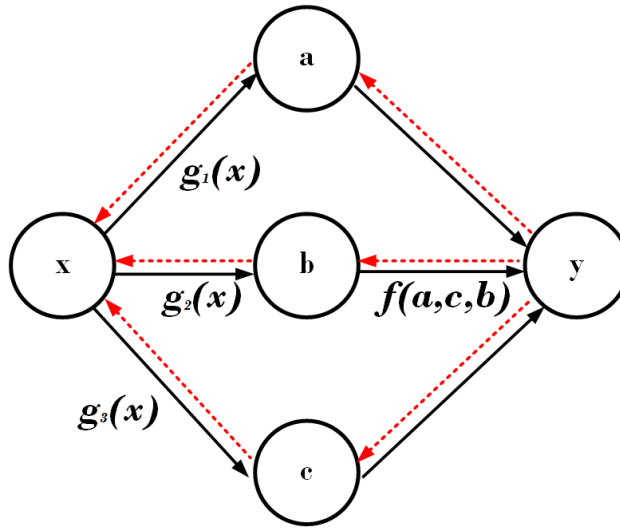


图 8.8: 多元复合函数求导法则示意图, 其中 f, g 代表节点之间的操作。黑色实线为从输入到输出的计算方向, 红色虚线为多元复合函数求导的方向, 操作为相邻两个节点求偏导

图8.8展示了一个多元复合函数的计算图。以 x 为变量的 y 的函数表示为 $y = f(g_1(x), g_2(x), g_3(x))$ 。显然, 在前向的计算图中, x 有 3 个子节点 a, b, c 。我们可以应用多元复合函数求导公式来计算 y 对 x 的导数。

$$\frac{\partial y}{\partial x} = \sum_{i \in \{i | u_i \text{ is } x's \text{ child which leads } x \text{ to } y\}} \frac{\partial y}{\partial u_i} \times \frac{\partial u_i}{\partial x} \quad (8.20)$$

$$= \frac{\partial y}{\partial a} \times \frac{\partial a}{\partial x} + \frac{\partial y}{\partial b} \times \frac{\partial b}{\partial x} + \frac{\partial y}{\partial c} \times \frac{\partial c}{\partial x} \quad (8.21)$$

以上是对链式求导法则和多元复合函数求导法则的进一步介绍。只要两个节点之间存在路径，那么就可以通过链式求导法则来传递梯度，如果存在多条路径，则可以通过多元复合函数求导法则来汇总通过不同路径传递的梯度。

8.2.2.3 任意计算图上的梯度计算

将以上两种方法结合起来，即可以对计算图中任意节点求梯度。首先找出该节点和最终节点（目标函数节点）之间的所有路径，通过前向传播得到每个路径上的各个节点的值。然后用多元复合函数求导法则和链式求导法则来计算每个节点的梯度的具体数值，其运算过程如下所示：

$$\frac{\partial y}{\partial x} = \sum_{j \in \{j | u_j \text{ is } X's \text{ child which leads } x \text{ to } y\}} \frac{\partial y}{\partial u_j} \times \frac{\partial u_j}{\partial x} \quad (8.22)$$

$$= \sum_{j \in \{j | u_j \text{ is } X's \text{ child which leads } x \text{ to } y\}} \prod_{i \in \{i | u_i \in \text{path}_j\}} \frac{\partial u^{(i)}}{\partial u^{(i-1)}} \quad (8.23)$$

其中， path_j 是以 x 节点为起点，以 y 为终点的一条路径。

下面我们用一个简单的例子来展示在任意计算图上求解梯度的计算过程。假设目标函数为 $y = f(x_1, x_2) = \ln(x_1) + x_1 * x_2 - \sin(x_2)$ ，我们希望得到 x_1 、 x_2 关于 y 的梯度。图8.9的左图和右图分别表示了前向计算和后向传播的过程。

此时，我们计算当 $x_1 = 2, x_2 = 5$ 时，目标函数 y 对 x_1 、 x_2 的导数。具体计算过程如下所示：

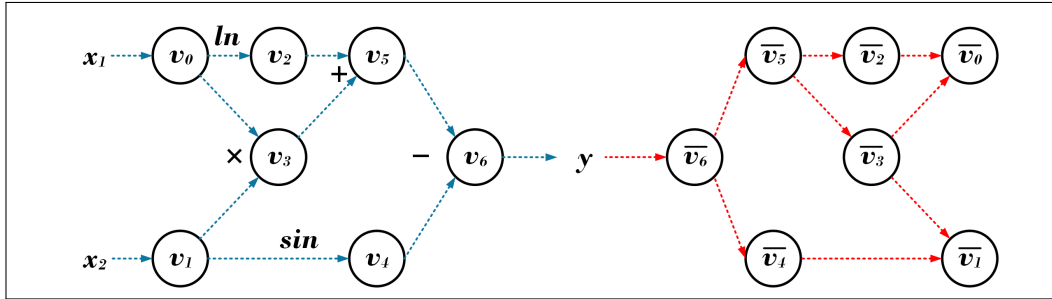


图 8.9: 前向计算和反向传播的过程, 其中蓝色边代表前向计算过程, 红色边代表反向传播过程。 $\bar{v}_i, i = 0, 1, 2, 3, 4, 5, 6$ 代表对应下标节点的梯度, 即 $\bar{v}_i = \frac{\partial y}{\partial v_i}$

前向计算过程:

$$v_0 = x_1 = 2$$

$$v_1 = x_2 = 5$$

$$v_2 = \ln(v_0) = \ln 2 \approx 0.693$$

$$v_3 = v_0 * v_1 = 10$$

$$v_4 = \sin(v_1) = \sin(5) \approx -0.959$$

$$v_5 = v_2 + v_3 = 10.693$$

$$v_6 = v_5 - v_4 = 11.652$$

后向传播过程:

$$\bar{v}_6 = \frac{\partial y}{\partial v_6} = 1$$

$$\bar{v}_5 = \bar{v}_6 \frac{\partial v_6}{\partial v_5} = 1$$

$$\bar{v}_4 = \bar{v}_6 \frac{\partial v_6}{\partial v_4} = -1$$

$$\bar{v}_3 = \bar{v}_5 \frac{\partial v_5}{\partial v_3} = 1$$

$$\bar{v}_2 = \bar{v}_5 \frac{\partial v_5}{\partial v_2} = 1$$

$$\bar{v}_1 = \bar{v}_4 \frac{\partial v_4}{\partial v_1} + \bar{v}_3 \frac{\partial v_3}{\partial v_1} = 1.716$$

$$\bar{v}_0 = \bar{v}_2 \frac{\partial v_2}{\partial v_0} + \bar{v}_3 \frac{\partial v_3}{\partial v_0} = 5.5$$

其中, $\bar{v}_0$ 和 $\bar{v}_1$ 就是 x_1 、 x_2 关于 y 的梯度。从这个例子中可以看出, 只要确定了神经网络的前向传播过程, 那么, 目标函数对任意节点的梯度都可以自动获得。下面我们会用一个比较简单的神经网络来演示如何梯度下降法的运用过程。

8.2.3 全连接神经网络的计算图以及梯度计算

我们前文中介绍的全连接神经网络可以看作是一个包含多层嵌套的复合函数，其每个隐藏层都有对应的模型参数。在8.1.3节中，我们根据全连接神经网络的结构，推导了梯度计算的反向传播迭代计算公式。在这里，我们以全连接神经网络为例，展示出其平方差损失函数的计算图，并根据计算图上梯度计算的一般算法，计算出目标函数对所有模型参数的导数。

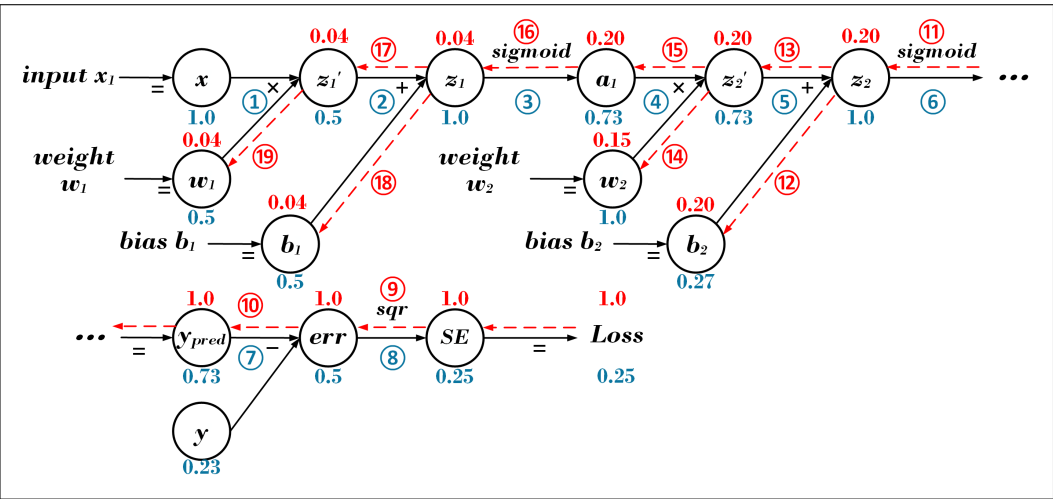


图 8.10: 全连接神经网络的损失函数的计算图，其中每个节点下方的蓝色数字代表该节点的前向计算结果，红色数字代表该节点关于损失函数的梯度，序号代表了运算顺序，蓝色为前向计算，红色为反向传播

为了方便计算，我们以一个输入和隐藏层都是 1 维的全连接神经网络为例，图8.10展示了该神经网络的平方差损失函数。当模型参数的当前值设置为 $w_1 = 0.5, b_1 = 0.5, w_2 = 1, b_2 = 0.27$ 时，其前向计算和反向传播的运算过程如下所示，其运算过程中的序号和图8.10中的序号一一对应，结果最多保留两位小数)：

前向传播

$$\textcircled{1} \quad z'_1 = x \times w_1 = 1.0 \times 0.5 = 0.5$$

$$\textcircled{2} \quad z_1 = z'_1 + b_1 = 0.5 + 0.5 = 1.0$$

$$\textcircled{3} \quad a_1 = \text{sigmoid}(z_1) = \frac{1}{1+e^{-1}} \approx 0.73$$

$$\textcircled{4} \quad z'_2 = a_1 \times w_2 = 0.73 \times 1.0 = 0.73$$

$$\textcircled{5} \quad z_2 = z'_2 + b_2 = 0.73 + 0.27 = 1.0$$

$$\textcircled{6} \quad y_{pred} = \text{sigmoid}(z_2) = \frac{1}{1+e^{-1}} \approx 0.73$$

$$\textcircled{7} \quad err = y_{pred} - y = 0.73 - 0.23 = 0.5$$

$$\textcircled{8} \quad SE = (err)^2 = 0.25$$

反向传播 (节点 v 的梯度由 $\bar{v}$ 来表示)

$$\textcircled{9} \quad \overline{err} = \overline{sqrr_e} \times \frac{\partial SE}{\partial err} = \frac{\partial Loss}{\partial SE} \times (2 \times err) = 1.0 \times 2 \times 0.5 = 1.0$$

$$\textcircled{10} \quad \overline{y_{pred}} = \overline{err} \times \frac{\partial err}{\partial y_{pred}} = 1.0 \times 1 = 1.0$$

$$\textcircled{11} \quad \overline{z_2} = \overline{y_{pred}} \times \frac{\partial y_{pred}}{\partial z_2} = 1.0 \times \text{sigmoid}(z_2) \times (1 - \text{sigmoid}(z_2)) = 1.0 \times 0.73 \times 0.27 \approx 0.20$$

$$\textcircled{12} \quad \overline{b_2} = \overline{z_2} \times \frac{\partial z_2}{\partial b_2} = 0.20 \times 1 = 0.20$$

$$\textcircled{13} \quad \overline{z'_2} = \overline{z_2} \times \frac{\partial z_2}{\partial z'_2} = 0.20 \times 1 = 0.20$$

$$\textcircled{14} \quad \overline{w_2} = \overline{z'_2} \times \frac{\partial z'_2}{\partial w_2} = 0.20 \times a_1 = 0.20 \times 0.73 \approx 0.15$$

$$\textcircled{15} \quad \overline{a_1} = \overline{z'_2} \times \frac{\partial z'_2}{\partial a_1} = 0.20 \times w_2 = 0.20 \times 1.0 = 0.20$$

$$\textcircled{16} \quad \overline{z_1} = \overline{a_1} \times \frac{\partial a_1}{\partial z_1} = 0.20 \times \text{sigmoid}(z_1) \times (1 - \text{sigmoid}(z_1)) = 0.20 \times 0.73 \times 0.27 \approx 0.04$$

$$\textcircled{17} \quad \overline{b_1} = \overline{z_1} \times \frac{\partial z_1}{\partial b_1} = 0.04 \times 1 = 0.04$$

$$\textcircled{18} \quad \overline{z_1'} = \overline{z_1} \times \frac{\partial z_1}{\partial z_1'} = 0.04 \times 1.0 = 0.04$$

$$\textcircled{19} \quad \overline{w_1} = \overline{z_1'} \times \frac{\partial z_1'}{\partial w_1} = 0.04 \times x = 0.04 \times 1.0 = 0.04$$

可以看出，只要是确定了神经网络目标函数的前向计算过程，并给定当前的模型参数，我们就可以计算此时目标函数对模型参数的梯度。也就是说，对于任意的神经网络，其梯度的计算并不需要手动的进行梯度公式的推导，其梯度的计算是一个机械的计算过程。这使得神经网络的训练优化，完全可以自动化的完成；这极大的推进了大规模复杂神经网络在工业界的应用。

8.2.4 反向传播算法的历史回顾和深度学习编程框架

追溯起来，学界普遍认可现代反向传播算法在 1970 年由芬兰硕士生 Seppo Linnainmaa 首次发表，Paul Werbos 在 1974 年提出用它来训练神经网络。而在 1986 年，Geoffrey Hinton 等人在《Nature》上发表论文《Learning representations by back-propagating errors》，详细描述了前向计算和反向传播的过程，并成功演示了反向传播算法在多层级神经网络中的实用价值。尽管 Geoffrey Hinton 并不是第一个提出这种方法的人，但他对推广反向传播起到了很重要的作用。

伴随着反向传播算法日益成熟，任意的神经网络模型都可以用一种公式化的方法来进行训练。在这一基础上，很多学者开始在计算机上搭建神经网络模型的通用训练框架，以便于展开有关深度学习的研究。最早的开源深度学习框架当属 Theano，它是由蒙特利尔大学 LISA 实验室（现 Mila 实验室）主导开发，旨在为机器学习研究提供高效的数值计算工具。Theano

首次实现了自动运行反向传播算法的自动微分功能以及支持 GPU 加速，虽然在 2017 年停止更新，但它的设计理念仍深刻影响了其他的深度学习编程框架的设计。

当前有两个比较常用的开源的深度学习编程框架——Tensorflow 和 Pytorch。Tensorflow 由 Google Brain 团队在 2011 年开发，最初用于服务他们的内部研究和生产，为 Google Search、Google Photos 等大规模服务提供了支持。于 2015 年正式开源，迅速成为了最受欢迎的深度学习库，特别适合大规模的机器学习。Pytorch 由 Facebook 的 AI 研究团队研发，其旨在提供一个快速，灵活且动态的深度学习框架。Pytorch 在设计的一大特点就是选用了动态计算图作为核心（Tensorflow 是静态计算图），它允许我们在运行时改变计算图。这使得 PyTorch 在处理复杂模型时更具灵活性，并且对于研究人员来说，更易于理解和调试。

除此之外，深度学习领域还有其他开源的深度学习编程框架，感兴趣的读者可以自行探索各个框架的区别。通常而言，当我们使用一个深度学习框架时，只需定义其损失函数的前向计算过程，并明确哪些变量是模型参数，然后该框架就可以根据前向计算的过程自动得出反向传播的过程，并计算梯度。同时，很多机器学习框架也支持 GPU 加速，显著提升大规模计算任务的效率。

以上是本节关于反向传播算法的内容。至此，我们已经介绍了前馈神经网络模型的学习过程，包括神经网络模型的基本假设、函数的选择、训练的过程和预测过程。然后拓展到了如何在一般的神经网络模型运用梯度反向传播来进行训练，下面我们会对神经网络模型的基础内容进行总结。

8.2.5 神经网络模型的四要素

神经网络其实也是一种机器学习模型，当然也有自己的四要素，如表8.1所示。

在学习了一般反向传播算法之后，对于任何神经网络，我们只需要建

表 8.1: 神经网络的模型四要素

要素	内容
基本假设	神经网络从输入到输出的前向计算过程。
优化目标	根据实际问题建立的优化目标，如回归误差、最大数据似然等。
训练过程	通常来说，目标函数对模型参数是可导的，因此，可使用梯度下降法进行优化；同时，梯度的计算是完全自动化的，所以，在目标函数确立之后，模型的训练就完全自动化了。
预测	进行一次从输入到输出的前向计算。

立从输入到输出的计算过程和确立目标函数，后续的梯度计算和参数更新都可以完全自动化了。因此，在下面的卷积神经网络和循环神经网络的介绍中，我们将讲述重点放在模型的前向计算过程上，并且介绍各种模型的主要应用场景。

8.3 卷积神经网络

卷积神经网络 (Convolutional Neural Network,CNN) 主要应用于图像分类、人脸识别、物体识别等图像和视频分析等任务上。在计算机视觉任务中，传统方法需要手工设计特征，将图片转换为向量；随后，使用经典的机器学习模型或者神经网络模型对向量进行分类。这种方法严重依赖设计者在生成图像特征数据时的经验和专业问题领域知识；而且，特征和分类器的优化目标是割裂的。因此，与其设计特征，不如设计特征学习器，并和分类任务进行联合训练。

卷积神经网络是一类特殊的多层感知器，能够很好的实现特征表达学习。它通过局部连接、权值共享和空间降采样 (down sampling) 的方法，实

现了稀疏结构和减小参数的设计理念。目前，卷积神经网络在各类计算机视觉任务上取得了巨大成功，已经成为了深度学习领域中最常用、最有效的模型之一。

本节先介绍卷积神经网络的基本结构，带大家了解卷积神经网络的前向计算过程，然后会介绍它在各类计算机视觉问题上的应用。

8.3.1 卷积神经网络的前向计算

卷积神经网络的出现是受到了生物学中的视觉机制的启发。在 1959 年，加拿大神经科学家 David Hubel 和瑞典神经科学家 Torsten Nils Wiesel 在研究猫的视觉神经时发现，视觉皮层细胞有着局部敏感的感受野（感受的范围），即相邻细胞的感受野也彼此相邻。同时，他们又发现视觉皮层存在简单、复杂两类细胞：简单细胞感受野小，响应特定方向和位置的边缘刺激；复杂细胞感受野大，同样响应特定方向边缘刺激，而对刺激的位置并无严格要求。他们以此为依据，猜想：视觉皮层对信息的处理是一种层级结构，最简单的信息在下层提取，然后不断地处理简单信息，逐渐得到了高层次的抽象信息。

而在计算机中，卷积神经网络的神经元一般可分为卷积层、池化层和全连接层三种。接下来，我们将介绍他们的具体计算过程。

8.3.1.1 卷积层

卷积层（Convolutional Layer）是卷积神经网络的重要组件，其作用是从输入数据中高效提取局部特征。卷积层的主要运算操作就是卷积，实际上这就是一种加权求和操作，其核心思想是让一个权重矩阵在输入数据上滑动，逐区域地对对应元素相乘，进行加权求和；其中参与卷积运算的权重被称为卷积核（Convolution Kernel）。当卷积核滑过输入矩阵后的所有位置后，每一次加权求和的结果会组成一个新的输出矩阵，被称为特征映射或特征图（Feature Map）。

具体来说，对于卷积核 W 和输入数据 X ，计算特征图 C 中坐标为 (k_1, k_2) 的元素的公式为：

$$C(k_1, k_2) = (W \star X)(k_1, k_2) = \sum_i \sum_j W(i, j) X(k_1 + i, k_2 + j) \quad (8.24)$$

其中卷积核矩阵为 W ， $W(i, j)$ 为卷积核内坐标为 (i, j) 的元素¹。

为了覆盖全部的输入，卷积核需要移动多次，进行多次卷积，每次移动的距离称为步长 (stride)。图8.11展示了卷积操作的一个示意图，它展示了两步卷积的计算过程，获得了特征图矩阵在 $(1, 2)$ 和 $(1, 3)$ 两个位置的值。有时，为了能够得到输入矩阵边缘的特征，还会先在输入矩阵的四周补零 (zero padding)，之后再行卷积。具体操作可以参考图8.12。

通过调整卷积核的滑动步长和补零，我们可以得到不同尺度的特征图。假设输入的尺寸为 $H_1 \times W_1$ ，步长为 S ，竖向每侧补零行数为 P_1 ，横向每侧补零列数 P_2 ，卷积核尺寸为 $F_1 \times F_2$ 。那么经过卷积操作后，输出的尺寸为 $H_2 = (H_1 - F_1 + 2 * P_1) / S + 1$ ， $W_2 = (W_1 - F_2 + 2 * P_2) / S + 1$ （需做一些细微调整，以保证输出的尺寸为整数）。

卷积神经网络中的卷积层在层与层之间用卷积操作，代替了全连接神经网络中对前一层所有神经元的输出都要加权求和才能得到本层一个神经元输入的计算，这也让卷积层有了两个比较重要的性质：

- **局部连接**：在卷积层中的每一个神经元都只和下一层中的某个局部窗口内的神经元相连接，构成一个局部连接的网络²。
- **权重共享**：同一层的卷积核的参数对所有神经元都是相同的。

¹该公式在数学上被称为**互相关** (Cross Correlation)，而非严格的卷积。

²有兴趣的读者可以试着画出卷积操作的计算图。

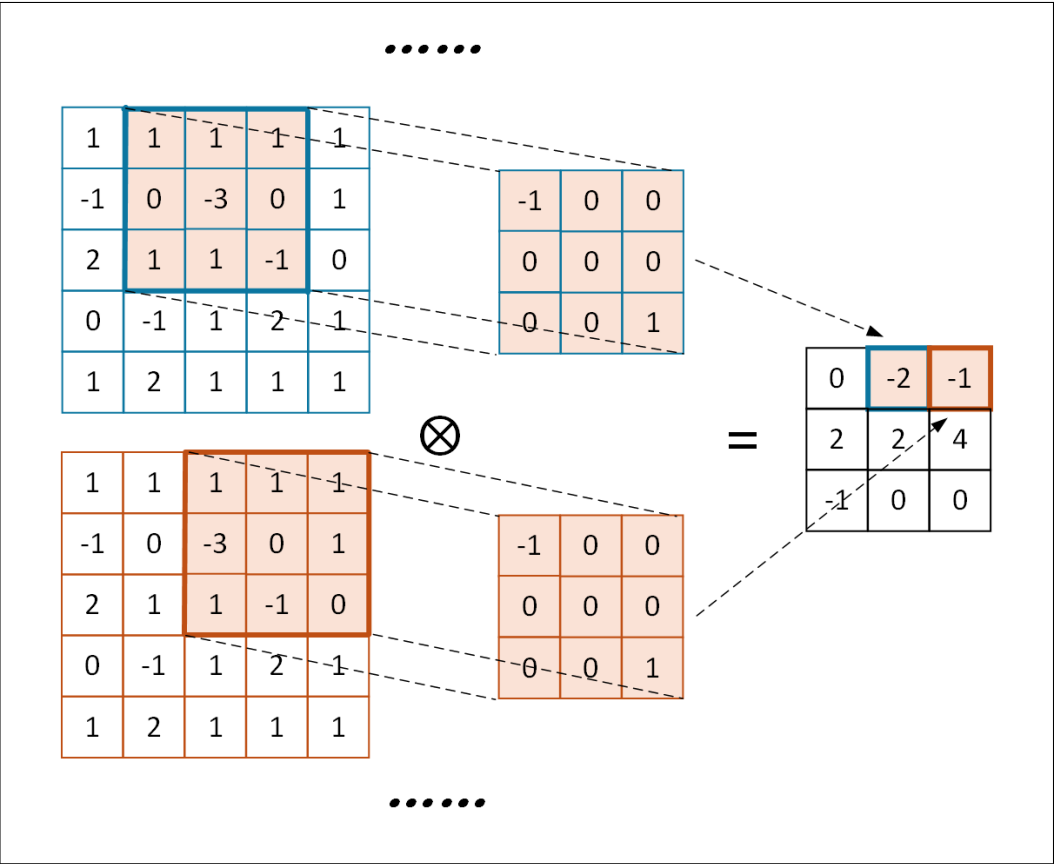


图 8.11: **卷积示意图**，展示了两点卷积的计算过程，其中左侧矩阵为输入数据矩阵，中间的矩阵是卷积核，右侧矩阵为卷积计算结果，卷积核向右移动的步长为 1。

多通道卷积

在实际应用中，输入数据并不一定只有两个维度，它可能有第三个维度：通道 (C)。也就是说，它的形状实际上是 $H \times W \times C$ 的三维矩阵。例如，我们平常接触最多的 RGB 彩色图像就有三个通道（一个像素点有 RGB 三个数据值），而黑白图像只有一个通道。针对多通道输入计算卷积时，卷积核也需要有多个通道。将所有通道上的卷积结果相加，即可得到最终的结果。

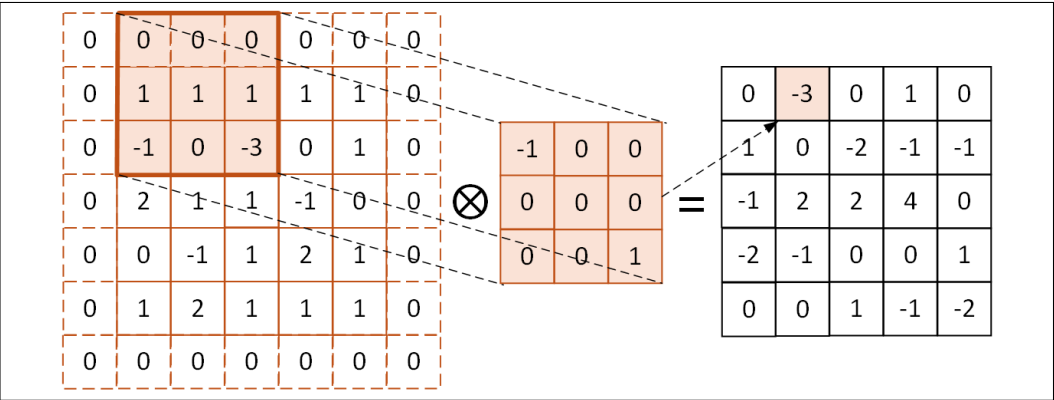


图 8.12: 输入矩阵补零后的卷积计算示意图, 其中输入矩阵和卷积核仍然与图8.11相同, 竖向和横向的补零层数都为 1 层

与此同时, 实际应用中的卷积核也不一定只有一个, 而是有很多个。将每个卷积核计算出的结果视为输出特征图中的一个通道, 这样我们就可以得到一个多通道的输出。图8.13展示了一个多通道卷积的示意图。

从图中我们可以看出, 卷积核的通道数量是由输入数据的通道数量决定的, 但特征图上的通道数量是由卷积核的个数决定的。在图像识别任务中, 设置多个卷积核的作用是从同一组数据上提取不同特性的特征, 以此增强图像识别能力。

8.3.1.2 池化层

池化层 (Pooling Layer) 也被称为子采样层, 其作用是将一个较大的矩阵变为一个较小的矩阵。通过对卷积层输出的特征图进行降维或抽样, 池化操作可以减少数据量, 降低数据复杂度, 并且使模型获得对局部微小平移、缩放的不变性。

常见的池化方法包括最大池化 (Max Pooling) 和平均池化 (Mean Pooling)。顾名思义, 前者是抽取特定区域内的最大值, 后者则是计算特定区域内的平均值。

图8.14展示了池化操作的一个示意图。容易发现, 经过池化操作后, 特

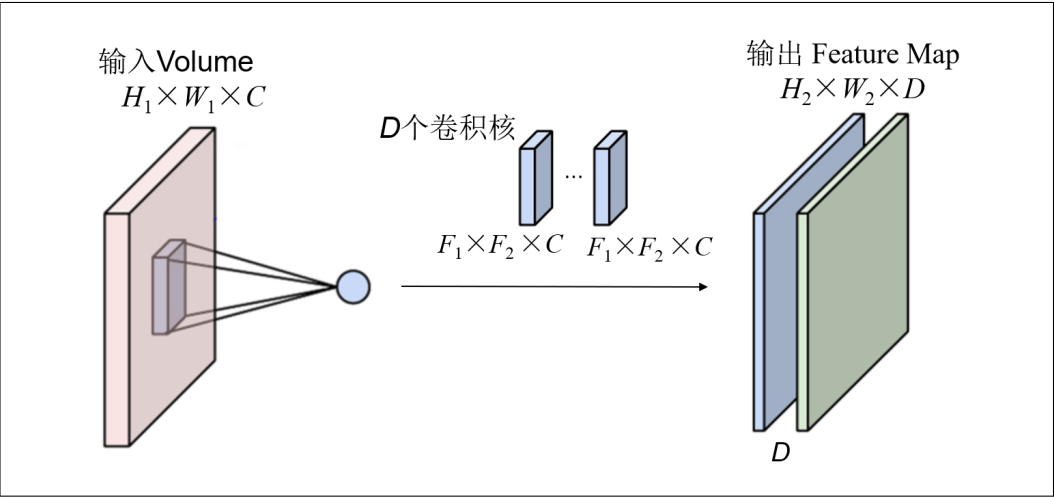


图 8.13: 多通道卷积示意图, 其中输入和卷积核都有 C 个通道, 经过一个卷积核的计算后会生成只有一个通道的特征图; 而与 D 个卷积核进行卷积, 最后生成 D 个通道的特征图

征图从 4×4 变为了 2×2 , 有效减少了数据量。

同时我们也可以看出, 如果输入特征中有一些数据的位置发生了变化, 只要它仍在池化操作的区域内, 那么就不会影响池化操作的结果, 也就是说: 池化计算的结果对一些小的局部形态改变不敏感。我们可以把池化的过程理解成对于特征图的概括, 它可以把特征图一部分区域内最显著的特点给提炼出来, 推动模型从识别细节到抽象出高级语义。

8.3.1.3 激活函数

在池化计算后, 我们仍然需要引入一些非线性的变换, 即激活函数的操作, 使得神经网络能够学习复杂的非线性关系。常用的激活函数包括 ReLU (Rectified Linear Unit)、sigmoid 和 Tanh 等。ReLU 函数能够有效地缓解梯度消失问题, 并加速网络的收敛; sigmoid 函数和 Tanh 函数则用于二分类和多分类问题, 将输出值映射到 $[0, 1]$ 或 $[-1, 1]$ 之间, 实现输出的归一化和概率化处理。

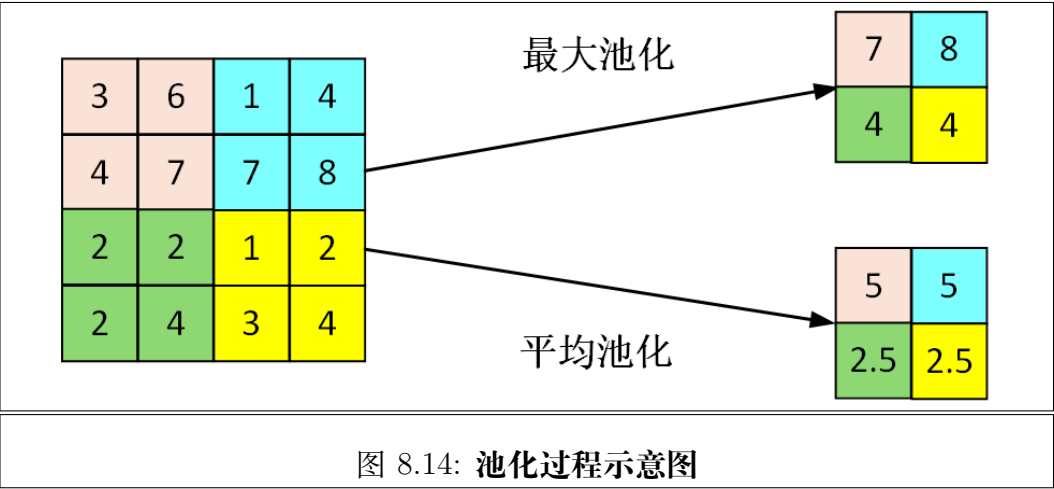


图 8.14: 池化过程示意图

具体讲，以经过池化计算的特征矩阵为输入，将该矩阵每个位置的数据 z 转化为 $a = f(z)$ ，这里 $f()$ 为激活函数，以获得新的特征矩阵。

综上，我们介绍了卷积、池化、激活三个步骤的计算过程，这三个步骤组合起来就完成了一次局部特征抽取的非线性变换；同时，我们可以不断的叠加这样的组合变换，形成多层的卷积神经网络（这里的一层指的是一次卷积、池化、激活的组合计算）。每层的计算是对输入数据的抽象，随着层数的增加，即完成了从细节特征到抽象特征的转换。

至此，我们已经了解了卷积神经网络的基本结构以及各部分的作用，下面我们会介绍一些卷积神经网络的实际应用。

8.3.2 卷积神经网络在计算机视觉中的应用

我们提到过卷积神经网络主要应用在计算机视觉任务上，而计算机视觉的经典任务包含如下四种——图像分类 (Image Classification)、目标定位 (Object Localization)、语义分割 (Semantic Segmentation)、实例分割 (Instance Segmentation)。图8.15展示了这四种任务的功能特点

具体来说，这四个任务有着从图像整体理解到细节理解的递进关系。图像分类是判断一张图像整体的“主要类别”，即回答“这张图里有什么？”的问题，例如判断出图8.15的图片中有水瓶、大口杯、小方块。目标检测要在

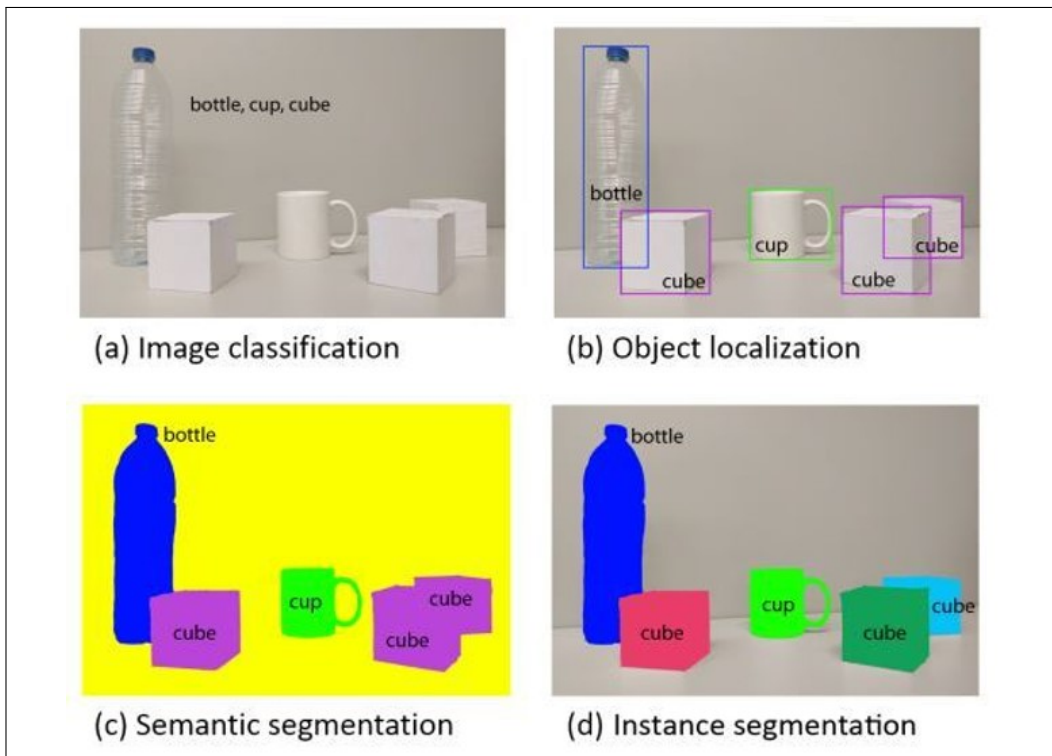


图 8.15: 四种计算机视觉核心任务示意图，其中 a 图为图像分类任务，b 图为目标定位任务，c 图为语义分割任务，d 图为实例分割任务。

图像分类的基础上，进一步定位目标的具体位置，回答“这个目标在哪里？”的问题，例如判断出图8.15的图片中大口杯在中间，而水瓶靠近图片的最左侧，并画出各个物品位置的矩形框。语义分割是为图像中的每个像素分配类别标签，回答“图像中每个位置的像素属于什么类别？”的问题，例如在图8.15的图片中，计算机判断出某一部分像素都属于水瓶，就标为蓝色；某一部分像素都属于小方块，就标为紫色；而有些部分的像素没有识别出来属于哪个物品，可能确实什么都没有，就标记为黄色。实例分割是在语义分割的基础上，进一步区分同一类别中的不同个体实例，回答“图像中每个目标实例的像素属于哪个具体对象？”的问题，例如在图8.15的图片中，计算机判断出组成小方块图像的像素包括了三个小方块实体，然后就用不同的颜色将实体区分出来。

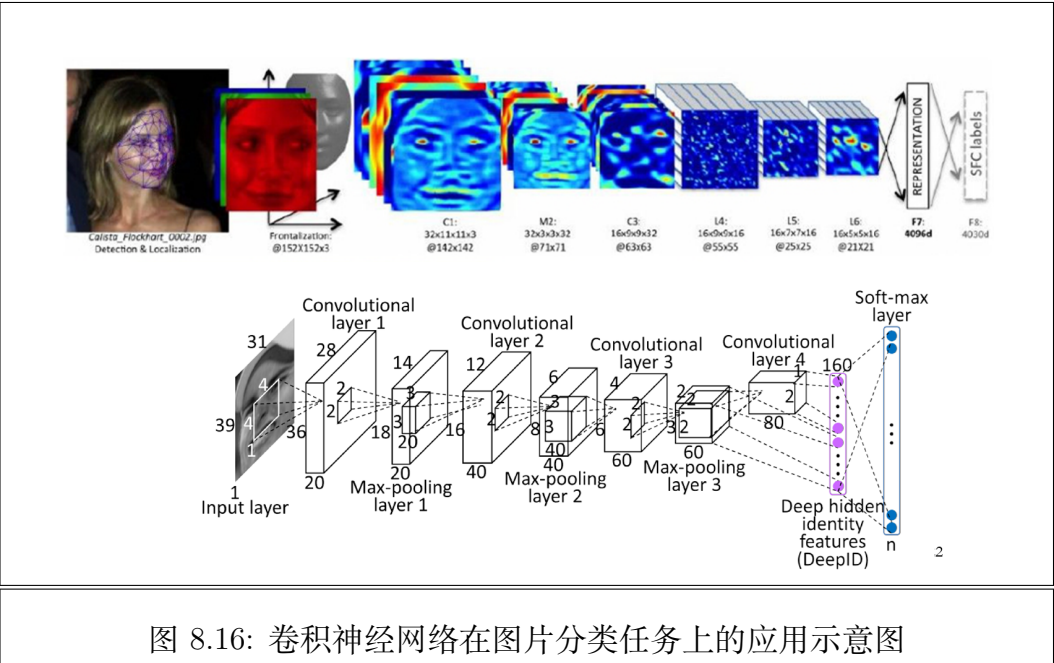


图8.16展示了一个使用卷积神经网络进行图片分类的例子。该网络共有 4 个卷积层和 3 个池化层，输入为 31×39 的单通道图像，经过若干次卷积、池化和激活的组合操作后得到了一个 160 维的向量，随后经过一个 softmax 层得到最终的分类结果。

对于其它的计算机视觉的任务，如目标检测、实例分割等，都是使用卷积神经网络作为基座模型 (Backbone)；不同的是，在输出部分引入跟问题相关的模块，如在目标检测中需要回归目标矩形的位置坐标。如果读者对这部分内容感兴趣，可以继续深入学习计算机视觉领域的书籍或课程。总的来说，卷积神经网络在提取图像特征上效果显著，广泛应用于各种计算机视觉的任务。

8.3.3 卷积神经网络研究工作的回顾

首先，我们需要提一个对人工智能和计算机视觉研究都有非常重大贡献的一个里程碑事件——ImageNet 的 ILSVRC 大赛（全称为 ImageNet

Large Scale Visual Recognition Challenge)。ImageNet 是斯坦福大学李飞飞教授领导的团队于 2009 年创建并发布的一个计算机视觉数据库，包含约两百万两千个类别的一千四百多万张带数据标签的图片。在此之前，大部分图像数据集规模相对较小，通常只有几千到几万张图片，限制了复杂模型（尤其是深度学习模型）的训练和评估。ImageNet 的团队解决了当时计算机视觉研究过程中缺乏高质量、大规模标注数据的问题，并且每年都举办围绕 ImageNet 数据集的 ILSVRC 大赛，用于评估大规模目标检测和图像分类算法的性能。

ILSVRC 大赛从 2010 年办到了 2017 年，要求参赛的算法或模型对所提供的图片进行分类。这个比赛可以说是计算机视觉领域的顶尖挑战赛事，汇集全球学术界和工业界的顶尖研究人员参与，推动无数先进算法模型的出现。而就在这样一个全球顶尖的计算机视觉赛事中，有两个基于卷积神经网络的图像分类模型在不同年份分别以远超往届冠军的成绩夺得魁首，对后续的计算机领域研究都有着非常深远的影响。

第一个是 2012 年由 Geoffrey Hinton 和他的学生 Alex Krizhevsky 设计的 AlexNet，在比赛中的图片分类误差仅为 15.4%（这也是 2011 年的 ILSVRC 大赛冠军的成绩），远远超过第二位的 26%。该卷积神经网络模型有 5 个卷积层、3 个池化层和 3 个全连接层，包含六千万左右的参数。作为第一个现代深度卷积网络模型，AlexNet 模型使用了几个在当时非常创新的技术方法，比如使用 GPU 进行并行训练、采用了 ReLU 作为非线性激活函数、使用 Dropout 防止过拟合等，给后来的研究者很多的启发。

第二个是 2014 年 GoogLeNet 模型，它的图片分类误差为 6.7%，显著低于上一年冠军的 12%，而且第一次将误差降至 10% 以内。在结构设计上，GoogLeNet 网络使用级联的 Inception 模块（包含多个不同大小的卷积核的卷积层），而且完全去除全连接层，并在中间层加入辅助损失函数，改善深度增长带来的梯度消失问题。除了优异的图像识别能力外，GoogLeNet 模型还有一个优点，就是只有五百万左右个参数（AlexNet 的 $\frac{1}{12}$ ），不会占据

太多显存空间。

在卷积神经网络的发展史上还有不少优异的模型，本书限于篇幅，不再一一举例介绍。现在很多的卷积神经网络模型的框架和用于图像识别的训练数据都是开源的，感兴趣的读者可以在理解了卷积神经网络原理的基础上，自行寻找典型的卷积神经网络框架并实践计算机视觉的有关任务。

8.3.4 卷积神经网络的小结

卷积神经网络将特征提取和模型输出（分类器、位置回归等）这两个部分结合起来，进行联合优化，从而得到了一种端到端的模型。相比于传统方法，二者之间不存在任何割裂，因此在效率和性能上都得到了显著提升。

整体来看，卷积神经网络是一种特殊的前馈神经网络，而且卷积层和池化层的连接方式也可以看作是一种函数运算方式，因此也可以使用反向传播算法，计算目标函数对每个参数的梯度，从而用梯度下降法来进行训练。目前，卷积神经网络已经成为了计算机视觉领域的主流模型，并且在自然语言处理、推荐系统等领域也有一定程度的应用。

至此，我们所学习的神经网络模型都是单向传递信息的，数据只会从输入层开始，一路往下一层传递，而且每一次的输入都是独立的，这让网络的输出结果只会依赖于当前的输入。但在很多现实任务中，模型的输出不仅和当前时刻的输入相关，也会和过去一段时间的输入和输出相关。例如在自然语言处理中，一句话的每个词前后都有关联；模型在输出宾语前，必须参考前面输出的主语和谓语是什么，而这是前馈神经网络模型很难做到的。因此，我们需要更加强化的模型来建模序列数据。

8.4 循环神经网络

人工智能领域普遍认为，在神经网络模型中的网络的连接参数，蕴含的是“知识”。类似的数据重新输入后，经过这些参数的转换，可以复现类

似的结论。那么试想，如果输入的数据也包含了来自过去的信息，那是不是就代表了这个神经网络拥有了记住过去，然后结合当前输入，得出新的结论的能力呢？以现实中的情形来类比，例如我们已经说过了“我今天去了海滩……”，那后面可能会继续说赶海、游泳之类和海滩相关的活动，而大概率不会突然说出滑雪、攀岩等前言不搭后语的事情。

而在计算机领域，我们将诸如自然语言这样需要按照一定顺序先后排列的数据，称为序列数据。序列数据的最大特点就是它的排列顺序本身也会蕴含信息。在前面提到的神经网络中，输入的样本需要彼此是独立的，不同时刻的输出是没有任何联系的，因此它们很难处理序列数据。而循环神经网络 (Recurrent Neural Network, RNN) 则弥补了这个问题。它可以学习序列数据，并根据上下文复现学习到的序列。

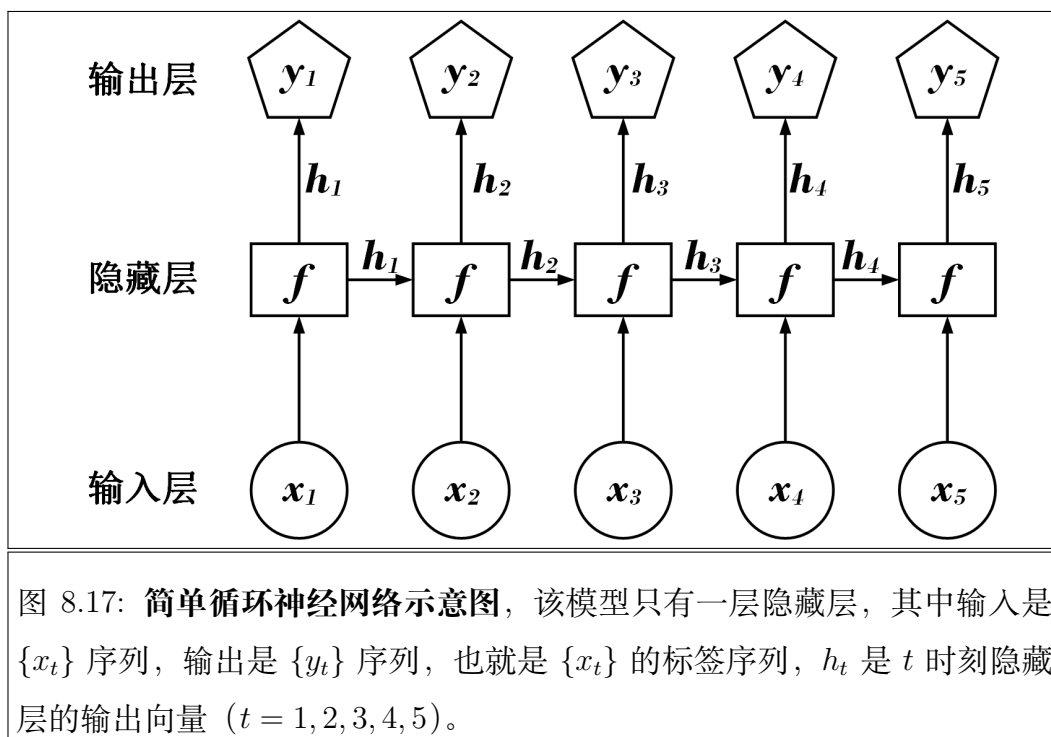
循环神经网络是一类具有“记忆”的网络，这里的记忆指的是它的神元不仅能够接收其他神经元的输入的数据，还能接收该网络在以前时刻的输入和输出。当前循环神经网络主要用于序列标注，从而解决语言模型、词性标注、分词等任务。

本节会先介绍循环神经网络的前向计算过程，带大家了解它处理序列数据的原理，然后会介绍循环神经网络的一些现实应用。

8.4.1 循环神经网络的前向计算

一个简单的循环神经网络如图8.17所示，其中的 x 、 h 、 y 的下标代表时刻。假设开始时刻为 1，此时，隐藏层会先接收 x_1 输入，经过隐藏层的转化得到 h_1 ；此刻的 h_1 有两个去处，一个是流入输出层，作为获得 y_1 的输入，另一个是流入到下一个时刻。当 y_1 输出后，模型进入时刻 2， x_2 被输入到隐藏层中；与此同时，时刻 1 的 h_1 也输入到该隐藏层，它们经过隐藏层的转化获得 h_2 。以此类推，每个时刻都获得对应的输出，直到序列结束。

对比前馈神经网络，我们不难发现，除了最开始的时刻之外，每个时刻循环神经网络的输出结果都要结合上一时刻的输出和当前时刻的输入。从



预测概率的视角出发, 循环神经网络实际是建模了序列的条件概率, 以过去的信息和此刻的输入作为预测此刻输出的条件。即, 通过一个输入序列来预测输出序列, 其输出序列的生成概率可以有公式8.25表示:

$$p(y_1, y_2, \dots, y_T | x_1, x_2, \dots, x_T) = \prod_t p(y_t | Y_{<t}, X_{\leq t}) \quad (8.25)$$

式8.25等式假设了生成一个序列的条件概率是序列上每个时刻条件概率的乘积, 而每个时刻的概率的条件包括当前和之前所有时刻的输入 $x_t (t = 1, \dots, t)$, 以及 t 时刻之前所有的输出 $y_t (t = 1, \dots, t - 1)$ 。

事实上, 式8.25等式并不是恒成立的。若严格依照贝叶斯条件概率公式展开, 我们有

$$p(y_1, y_2, \dots, y_T | x_1, x_2, \dots, x_T) = \prod_t p(y_t | Y_{<t}, X) \quad (8.26)$$

比较式8.25和8.26, 我们可以看到: 式8.25实际上是假设了: t 时刻的输

出 y_t 与该时刻之后输入（即 $x_i(i > t)$ ）条件独立³。

进一步的，我们假设：

$$p(y_t|Y_{<t}, X_{\leq t}) = \text{softmax}(h_t) \quad h_0 = 0 \text{ 作为初始输入} \quad (8.27)$$

其中， y_t 是 t 时刻的输出标签，

$$h_t = f(x_t, h_{t-1}) \quad (8.28)$$

即隐藏层函数 f 以当前时刻的输入 x_t 和上一时刻的输出隐向量 h_{t-1} 为两个输入，以输出当前时刻的隐向量 h_t 。

式8.27实际是做了进一步的简化：假设向量 h_t 能够包含 $(Y_{<t}, X_{\leq t})$ 中的所有信息。综上，循环神经网络的建模思路就是让每个时刻的输入数据，与上一时刻输出向量，合并生成当前时刻的输出向量，然后一步一步循环下去。

进一步的，我们需要探讨 t 时刻的输入 x_t 的表示。首先，我们比较一下图片与自然语言输入的不同。在计算机视觉任务中，图片用 RGB 等编码方式来表示，所以输入天然就是实数矩阵；而自然语言的基本单位就是文字或单词，无论采用什么编码策略，它的输入都是离散的。而在神经网络内部，不管是前向计算，还是梯度的反向传播，我们都在进行各种矩阵的实数运算。因此，我们需要将离散的自然语言的输入，转化为实数向量的形式；这就是所谓的词嵌入 (Word Embedding) 技术。

词嵌入，即是把每个单词表示为实数向量的形式，即把每个离散的单词映射到高维实数空间的一个点上。我们希望，对应的实数向量能够包含该单词的语义信息，且语义相近的两个单词在高维空间中的距离也相近。引入了词向量后，循环神经网络的输入从离散数据转化为连续数据，方便了后续神经网络中的实数运算。另外，需要指出的是，每个单词的词嵌入也可以作为神经网络的模型参数，使用反向传播算法，进行有监督的学习。

³感兴趣的读者可自行查阅什么是条件独立。

到目前为止，我们还没有介绍隐层计算的 f 函数的具体形式，后续(8.4.1.1节, 8.4.1.2节)将分别介绍两个这样的 f 函数。

8.4.1.1 朴素的 f 函数

我们可以设计一个简单的 f 函数，是由两个输入的线性组合和一个激活函数组成。设第 t 个时刻输入的词向量为 x_t ，上一时刻隐向量 h_{t-1} 。则有：

$$f(x_t, h_{t-1}) = \sigma(Uh_{t-1} + Wx_t + b) \quad (8.29)$$

其中， U 、 W 、 b 是模型要学习的参数， σ 是激活函数，如 sigmoid, tanh 等。

从上式中也可以看出， U 矩阵提取了前一时刻隐向量中的有用信息， W 矩阵提取了当前时刻输入的有用信息，将二者结合后再引入偏差 b 的影响，最后经过激活函数的处理，便得到了当前时刻的隐向量。

同时，需要指出的是，三部分参数 U 、 W 、 b 是跟时刻 t 无关的模型参数，也就是说，每个时刻都共用这三部分参数。为了进一步说明这一点，图8.18展示了使用朴素的 f 函数的循环神经网络的计算图，根据计算图，我们可以推导出循环神经网络的梯度传递过程，以此来训练模型。

以上是有关简单的循环神经网络模型的全部内容，这部分的循环网络模型的参数传递过程比较直观，易于理解。然而，在朴素的循环神经网络模型之中，由于模型参数简单，当前时刻的输出很难有效利用时刻较远的隐状态进行预测。也就是说，简单的循环神经网络很难建立长序列的依赖关系。

如果某个隐状态的时刻和当前时刻差距太远，梯度的传递路径会非常长，这样可能会导致隐状态梯度爆炸或梯度消失的问题。此时，时刻比较靠前的参数很难利用上当前时刻的损失函数的梯度进行调整，进而使得当前

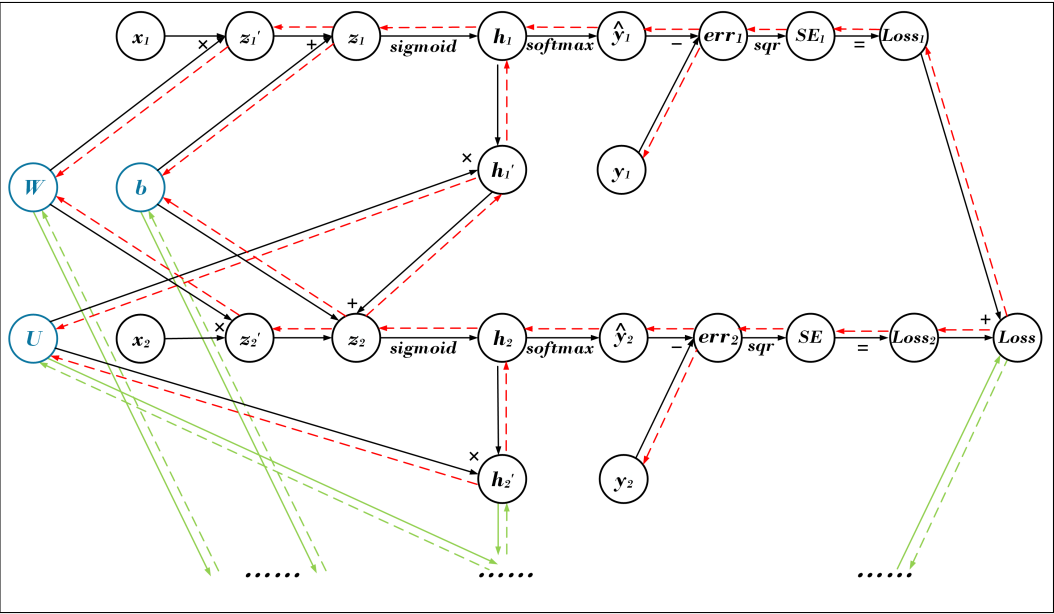


图 8.18: 简单循环神经网络的计算图示例, 其中假设激活函数为 sigmoid, 分类用函数为 softmax, 损失函数为平方差函数 (为了表示方便), 黑色边为前向传播, 红色边为反向传播。蓝色节点表示该参数为多个时刻共享的参数, 绿色的实线和虚线边对应时刻 2 以后未在图中画出的更多时刻的前向计算和梯度反向传播过程

时刻的神经网络模型也很难有效使用时刻较远的隐状态进行预测。为了改善各种情形, 我们需要引入其他的 f 函数。

8.4.1.2 f 函数: 长短期记忆单元

长短期记忆 (Long Short-Term Memory, LSTM) 单元是最常用的一种循环神经网络的 f 函数, 使用这种函数的循环神经网络模型被称为长短期记忆网络模型。

从结构上看, 长短期记忆单元包含输入门 i 、遗忘门 f 、输出门 o 和记忆单元 c , 其具体定义如下所示:

$$\begin{aligned}
i_t &= \sigma(W_i x_t + U_i h_{t-1} + b_i) \\
f_t &= \sigma(W_f x_t + U_f h_{t-1} + b_f) \\
o_t &= \sigma(W_o x_t + U_o h_{t-1} + b_o) \\
\hat{c}_t &= \tanh(W_c x_t + U_c h_{t-1} + b_c) \\
c_t &= f_t \odot c_{t-1} + i_t \odot \hat{c}_t \\
h_t &= o_t \tanh(c_t)
\end{aligned}$$

其中 x_t 为 t 时刻的输入向量, h_{t-1} 是前一个时刻的输出隐向量, $\odot$ 为向量元素点乘, $\sigma()$ 为 sigmoid 函数; 模型的参数包括 $W_i, U_i, b_i, W_f, U_f, b_f, W_o, U_o, b_o, W_c, U_c, b_c$ 。

通过引入更多的模型参数, 长短期神经单元试图记忆来自更早期时刻的有用信息。

8.4.2 循环神经网络的应用

循环神经网络的典型应用是序列标注任务, 这类任务的输入是一个序列, 输出是序列上每个位置的标签。这个标签可以是下一个位置的内容, 即预测当前位置的下一个词, 这就是语言模型; 也可以是当前词的词性, 从而用于词性标注任务; 或者是词的切分边界, 用于中文分词。

8.4.2.1 词性标注和中文分词

词性标注和中文分词都是序列标注的任务。词性标注是分析一句话中的每个词的作用, 为句子中的每个词语标注其词性, 包括: 动词、名词、代词等类别。它是一个经典的自然语言处理任务。而中文分词是将连续的无空格的汉字序列分割成有独立语义的词语, 使计算机能够理解文本的结构和语义边界, 它是中文文本处理的基础步骤。两种任务的示意图如8.19所示

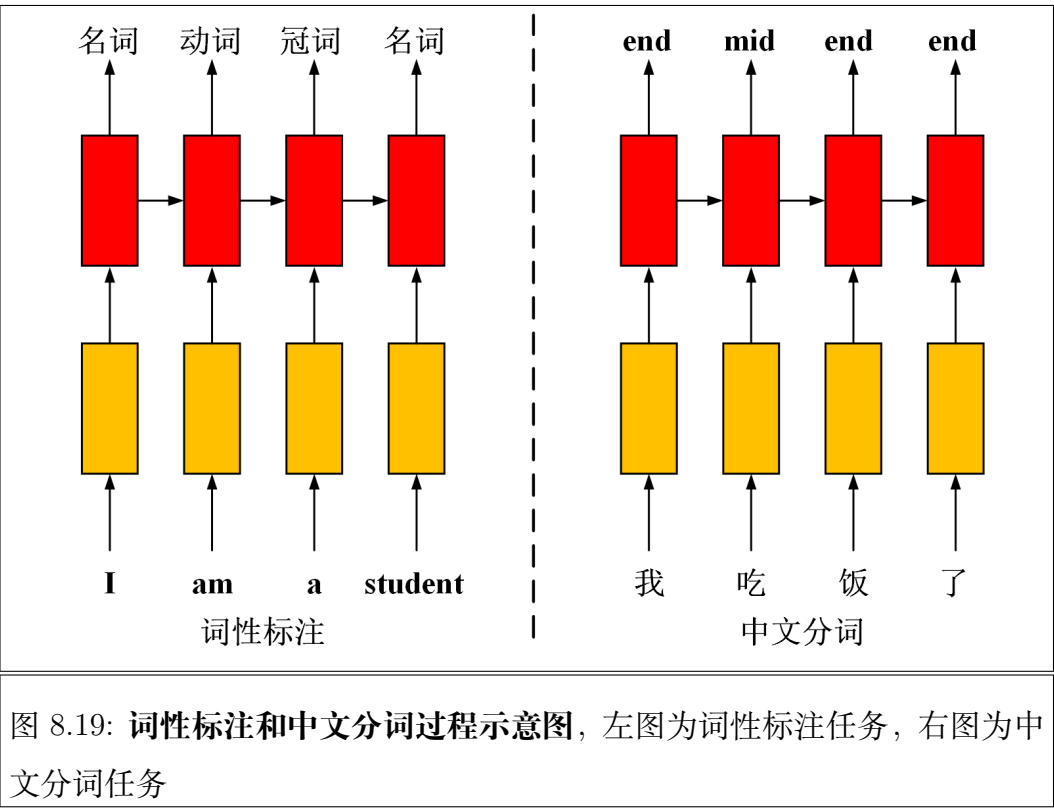


图 8.19: 词性标注和中文分词过程示意图，左图为词性标注任务，右图为中文分词任务

在词性标注任务中，每一个时刻的神经元会接收句中当前时刻单词的词向量输入，然后接收上个时刻传来的包含句子序列前面信息的隐向量，经过 f 函数处理后向下一个时刻传递隐向量；同时，连接分类函数 softmax，选择最符合当前时刻的单词的词性类别作为输出。

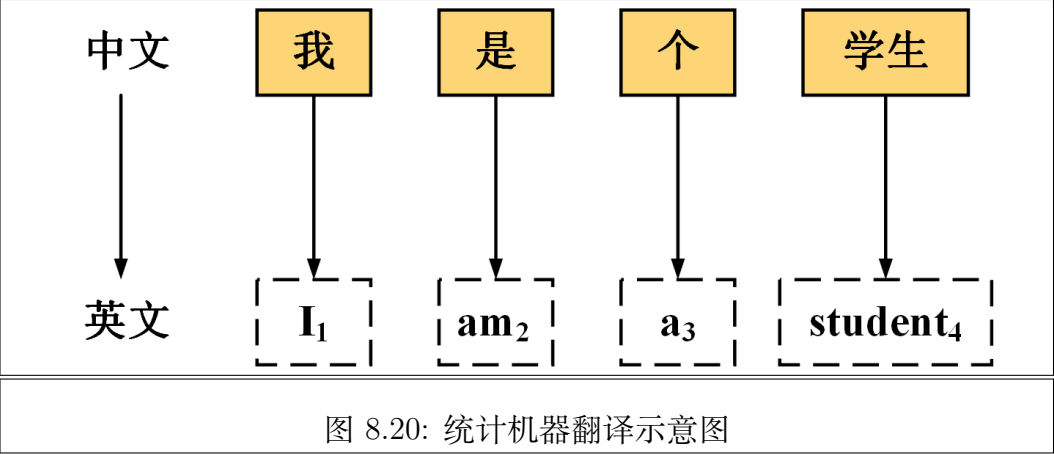
在中文分词任务中，每个时刻输入是中文的一个字的词向量，而输出包含两个可能的标签——“end”和“mid”。其中，“end”代表当前文字是一个词的结尾，而“mid”代表当前文字并不是一个词的结尾。在循环神经网络预测完成后，我们可以在标注为“end”的文字位置后进行分词，连续两个“end”标签中间的文字即构成一个中文的词。例如，图8.19中的分词结果为 { “我”，“吃饭”，“了” }。

在本小节接下来的内容中，我们会以自然语言中的翻译任务为例，对比深度学习方法相较于传统计算机翻译方法的优势，并介绍循环神经网络

是如何生成新的语言序列的。

8.4.2.2 机器翻译

在深度学习时代以前，最常用的机器翻译方法是统计机器翻译。该方法的核心技术是短语对齐，将一种语言中的短语转换为另一种语言的短语，随后把对齐后的短语串成一句话。图8.20展示了基于短语对齐技术的机器翻译方法。



可以看出，这种翻译方法的优点是非常直观，但是难以处理短语间的重排序，并且短语之间的衔接可能不顺畅。这主要是因为，不同语言体系之间会存在语法差异，有些信息的呈现方式可能不同。例如，如果用短语对齐的方式来翻译“成为一个完美的人是件困难的事”，可能会排序成“Being a perfect person is a difficult thing”，不过如果让一个英语母语者来翻译这句话，他更可能会把这句话翻译成“It is difficult to be a perfect person”。因此，相比于将句子中的短语一一翻译后组合成一个句子，我们更倾向于能够把句子完整的翻译出来，而基于循环神经网络的翻译模型则可以达到这个目的。

循环神经网络翻译模型的结构

循环神经网络可以直接建模 $P(\text{目标语句}|\text{源语句})$ ，从而在翻译任务中使用，而且可以端到端训练。应用于翻译任务的循环神经网络通常使用编码器-解码器 (Encoder-Decoder) 结构，其中编码器和解码器是两个不同的循环神经网络。具体讲，编码器将输入编码成一个隐向量，而解码器则接收编码器的隐向量作为输入，逐步解码出最终的输出。

图8.21展示了编码器-解码器模型的一个示意图。可以看出，编码器-解码器的结构很像是人们在现实中翻译语句的方法。当我们看到一个句子后，我们会先理解整个句子的含义，再去根据目标语言的语法特点来决定该如何翻译句子。这类模型也可以使用多层的循环神经网络，这样会更有利于提取更多语义特征，处理复杂的长文本翻译任务。一个两层的编码器-解码器循环神经网络结构如图8.22所示，其中每一层的 f 函数采用不同的模型参数。

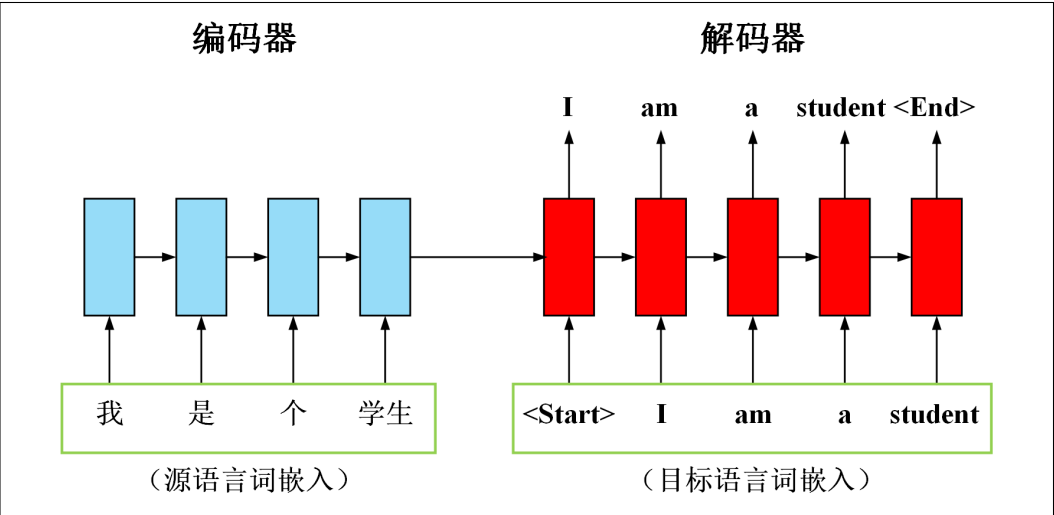


图 8.21: **Encoder-Decoder** 示意图，其中蓝色矩形为编码器的循环神经网络，红色矩形为解码器的循环神经网络。

在了解了循环神经网络翻译模型的基本结构之后，我们会以图8.22继续

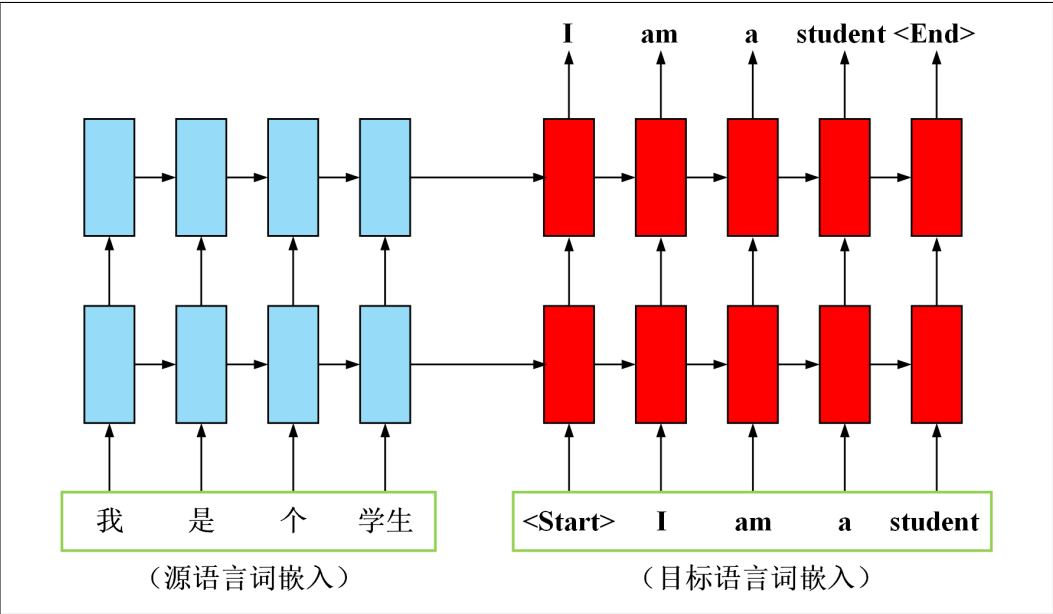


图 8.22: 二层的 **Encoder-Decoder** 示意图，此时的编码器和解码器都是两层的循环神经网络

深入地讲解该模型的前向计算过程。

循环神经网络翻译模型的前向计算过程

用于机器翻译的编码器-解码器 (Encoder-Decoder) 结构包含编码器和解码器两个不同的循环神经网络。如图8.22所示，考虑一个简单的翻译任务，讲中文的一句话“我是个学生”，翻译成英文。首先，将给定的这句中文输入到编码器中，在该编码器的最末位置获得对应的隐向量（因为是双层的循环神经网络，所以得到了两个隐向量）。此时，编码器的计算工作结束，它实际是把输入的句子编码成实数向量，得到句子级别的嵌入 (Sentence Embedding)。

接着，我们开始解码，也就是开始生成翻译的句子。在解码的第一个位置，输入的隐向量就是编码器已生成的句子嵌入，输入字符是一个特殊字符 <START> 以代表句子的起始位置。结合这些输入，在解码器的第一个位

置,计算出此时所有英文单词出现的概率,即 $p(\text{word}|\text{我是个学生}, \langle \text{START} \rangle)$ (这里,每个单词代表一个类别,是一个多分类的问题),并选择出现概率最高的单词作为当前位置的输出。假设,此时出现概率最高的单词是“I”,那么第一个位置生成的单词就是“I”。然后,生成第二个位置的单词。此时,因为第一个位置的单词已经生成,我们就将“I”作为第二个位置的输入,并结合第一个位置输出的隐向量,我们就可以计算所有英文单词在第二个位置出现的概率,即 $p(\text{word}|\text{我是个学生}, \langle \text{START} \rangle \text{ “I”})$ 。此时,假设出现概率最高的单词是“am”,那么第二个位置生成的单词就是“am”。以此往复,我们生成到一个特殊的符号 $\langle \text{END} \rangle$,代表生成的句子结束。此时,整个翻译的解码过程结束。这实际就是一个一边计算概率分布,一边采样的过程。

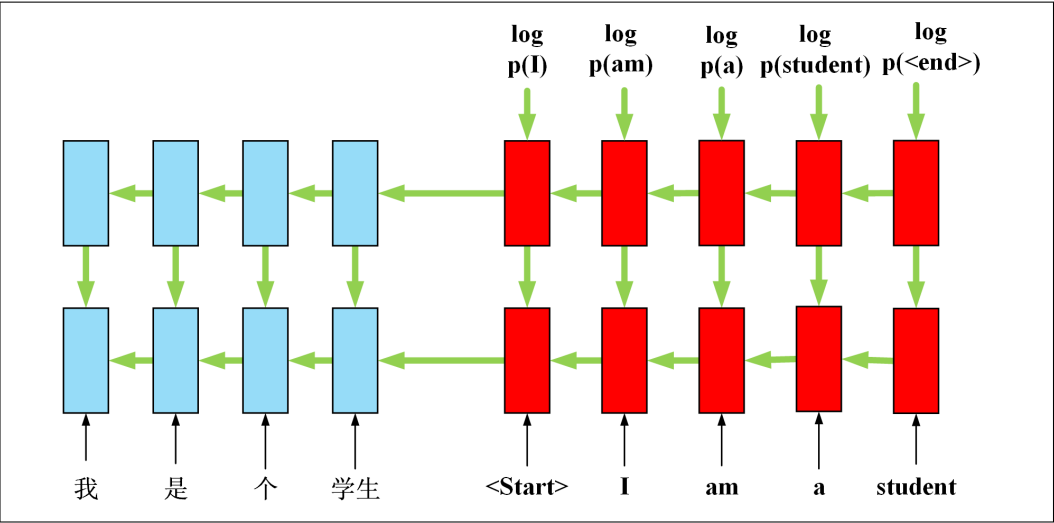


图 8.23: 多层 Encoder-Decoder 反传示意图, 其中绿色箭头代表了梯度反向传播的过程

图8.23展示了误差梯度反传的过程。在解码器的输出中,模型在每个时刻最后生成的是序列中该位置的词的分布概率,整个输出序列的生成概率为 $p(Y|X) = \prod_{y_t \in Y} p(y_t|X, Y_{<t})$ 。因此,循环神经网络翻译模型可以使用最大似然估计法来计算损失,即解码器的每个时刻都对应一个损失函数,即

$-\log(y_t|X, Y_{<t})$ 。然后，我们就可以用梯度下降法来优化模型参数。

注意力机制 (Attention)

在如前所述的机器翻译的编码器-解码器结构中，只有编码器最后位置的隐向量输入到解码器中，而这个末位置的隐向量可能会遗忘掉输入句子前期位置的信息。例如，该模型可能会将“你是不是不爱我了”翻译成“Don't love me”，而应该翻译成“Don't you love me or not”。因此，研究人员引入了注意力机制 (Attention Mechanism)。

注意力机制最早在 Dzmitry Bahdanau 等人发表的论文《Neural Machine Translation by Jointly Learning to Align and Translate》出现，目的是提升序列到序列的机器翻译任务的准确率。注意力是指人可以在关注一些信息的同时忽略另一些信息的选择能力，这让大脑能够从大量输入信息（比如视觉、听觉、味觉等方面的外部刺激）中选择小部分有用信息来处理。

受到人类注意力机制的启发，我们希望将这种能力引入到编码器-解码器结构中。我们首先定义如下的符号： h_j 是编码器中位置 j 输出的隐向量， s_t 是解码器中位置 t 输出的隐向量， y_{t-1} 是解码器在 $(t-1)$ 时刻的输出单词。理论上，编码器中每个位置的隐向量都对解码器中的隐向量 s_t 有影响，但是这些隐向量并不是都具有相同的重要性。因此，我们设计一种注意力机制，给编码器中的每个隐向量赋予一个权重，和当前解码时刻有更高相关性的隐向量具有更高的权重。下面介绍注意力机制的具体计算过程，所使用的公式如下所示。

$$\beta_{tj} = h_j^T W s_{t-1} \quad (8.30)$$

$$a_{tj} = \frac{\exp \beta_{tj}}{\sum_i \exp \beta_{ti}} \quad (8.31)$$

$$c_t = \sum_{j=1}^J a_{tj} h_j \quad (8.32)$$

$$s_t = f(y_{t-1}, s_{t-1}, c_t) \quad (8.33)$$

其中 W 为一个需要学习的参数矩阵, β_{tj} 为解码器的位置 t 对编码器的位置 j 的注意力分数, α_{tj} 为使用 softmax 函数归一化后的注意力分数; c_t 为编码器隐向量的加权求和结果, 其中 α 为注意力权重, 我们希望 c_t 能包含对当前解码位置 t 重要的来自于编码器所有位置的重要信息; 最后, 以 y_{t-1}, s_{t-1}, c_t 为输入, 利用循环神经网络的处理单元 f 函数, 获得最终的 s_t 。

与公式8.29中的 f 函数不同, 这里的 f 函数有三个输入。实际上, 我们可以引入额外的参数, 将 s_{t-1}, c_t 合并为一个向量, 从而直接使用原来的 f 函数; 在此不再赘述。

图8.24给出了一个注意力机制的示意图。可以看出, 注意力机制的计算过程: 首先是计算当前时刻对于编码器中每个位置的注意力权重, 此时, 编码器中和这一时刻比较相关的第一、二时刻的权重分数 β_{21} 、 β_{22} 会较高; 然后对编码器的隐向量进行加权求和获得 c_2 ; 最后, 将这个 c_2 、前一时刻隐向量 s_1 和当前时刻输入 (单词 “I” 的词向量) 一起输入模型, 计算当前时刻的输出向量。

注意力机制使循环神经网络模型能够更好地使用已有的信息, 通过“动态分配权重”的思想, 解决了长距离依赖、序列对齐等核心问题。使用注意力机制的模型在机器翻译任务中表现优秀, 尤其是对长句子的翻译, 取得了前所未有的效果。不过, 虽然注意力机制最早来源于机器翻译任务, 但它的应用领域早已不限于此。现在热门的大语言模型所使用的 Transformer 构架就运用了注意力机制, 而这部分内容我们会在大语言模型章节详细介绍。

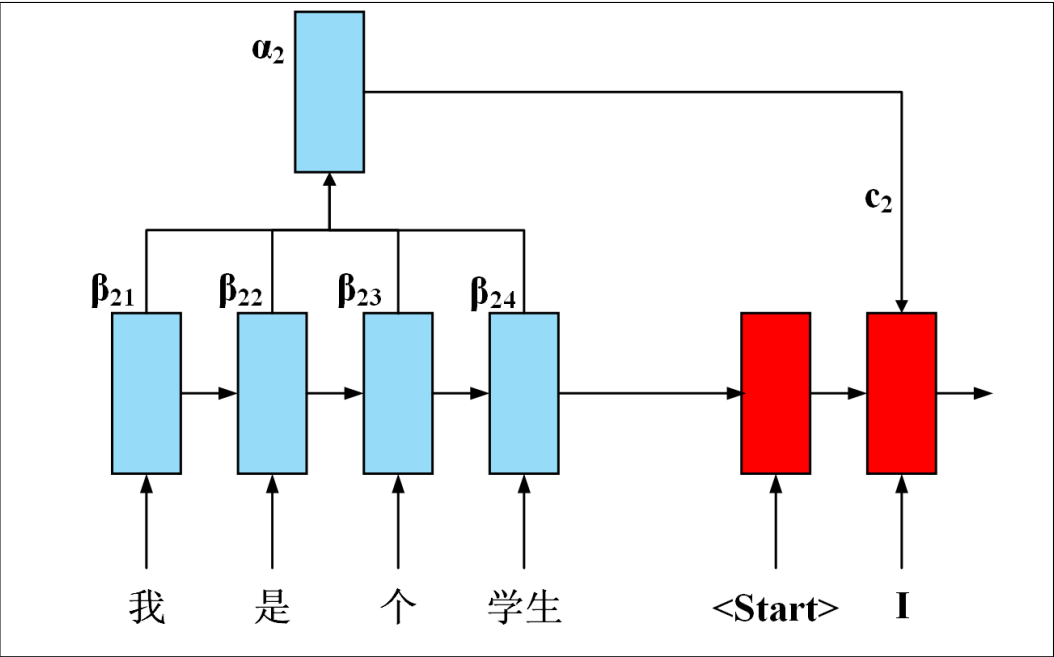


图 8.24: 注意力机制示意图, 此时要求解的是解码器第二个时刻的输出隐向量 s_2

8.4.3 循环神经网络小结

循环神经网络通过将隐向量在不同时刻之间传递, 为序列建模提供了基础框架, 推动了机器翻译、语音识别、对话系统等领域的发展。我们在这一小节中介绍了简单的循环神经网络、长短期记忆单元网络, 随后以机器翻译任务为例, 展示了编码器-解码器结构以及注意力机制, 以处理从序列到序列的学习问题。在下一节中, 我们会继续介绍自然语言处理领域中的大语言模型, 让大家能对当下比较火热的人工智能产品有更为深刻的认识。

8.5 大语言模型

大语言模型 (Large Language Model, LLM) 是一类基于深度学习的人工智能模型, 具有海量参数 (通常数十亿到上千亿), 能够理解、生成和推

理自然语言。它们通过在大规模文本数据上进行预训练，学习语言的结构、语义和上下文关系，从而完成多种语言任务，如问答、翻译、摘要、对话、代码生成等。

本节中，我们会先简单介绍大语言模型的基本原理以及常用的 Transformer 网络结构，并讨论大语言模型中的幻觉问题和缓解方法。

8.5.1 大语言模型的基本原理

大语言模型实际是一个根据上文来预测下一个词的模型。形式化讲，我们希望大语言模型能模拟下面的条件概率：

$$P(T_{N+1} | T_1, T_2, \dots, T_N) \quad (8.34)$$

其中 $T_1, T_2, \dots, T_N$ 是给定的上文的 N 个单词， T_{N+1} 是下一个位置的单词。

拥有了这样一个函数，我们就能根据上文估算出每个单词在下一个位置出现的概率。例如，现在的上文是“After the game we went”，那么，下一个位置出现“home”的概率应该很高，出现“eating”的概率应该适中，而出现“hiking”的概率应该比较低。在此函数的支持下，我们就可以依据此概率进行随机采样，获得下一位置单词。此时，新生成的单词被扩充进上文，然后以此类推生成新的下一个位置的单词。直到生成出代表“停止”的特殊字符，才结束这个过程。在使用像 ChatGPT 的大语言模型产品的时候，用户提交一个问题后，大语言模型即以用户的提问为上文，计算下一个词出现的概率，并以此进行概率采样。你会看到这些产品总是一个词一个词的给出回答，实际就是在执行这一过程。

大语言模型使用的“世界语言”

在大语言模型出现之前，自然语言处理模型通常是分语言的，例如英文的词性标注模型无法直接应用于中文。然而，大语言模型却展现出了跨语言、跨任务的“万能”特性，能够进行多语言翻译、代码编写等多种任务。ChatGPT 发布时，一个模型就能支持如此多样的功能，令人惊叹。其核心原因在于训练语料的规模与多样性。大模型本质上是基于上文预测下一个词的概率模型，因此互联网上的所有文本——无论语言或格式——都可作为训练数据。例如，将任意文本的前 100 词作为输入，第 101 词作为预测目标，这种学习方式使得可用语料规模极其庞大。更重要的是，模型无需区分人类语言或机器语言：中文、英文、Python 代码、LaTeX、HTML 等，在计算机看来都是字符串序列。这种涵盖“世界语言”的混合语料，使模型在训练过程中自然内化了多语言知识、编程逻辑乃至跨领域专业知识，从而呈现出“多才多艺”的能力。

大语言模型使用的“单词”

大语言模型处理的是世界语言，包含来自不同地区国家的人类语言，还包含互联网上的各种机器语言。不同语言的基本元素不同，中文的基本元素是中文字，而英文的基本元素是英文单词。但不管是什么语言，在计算机内都使用统一的编码，比如 UTF-8，将文字存储成数字的字符串⁴。

为了让大语言模型处理的世界语言更加高效，我们有必要重新定义什么是这个语言的基本单元，即所谓的 Token。Token 的设计，需要考虑如下的目标：

- Token 的种类少。Token，即语言的基本单元，它的个数不能太多。正如，我们不希望中文的字太多。

⁴有兴趣的读者可以自行了解什么是 UTF-8 编码

- 一个句子，我们希望尽量用更少的 token 生成出来。比如，一个人类语言中常用的词组，我们是否可以把它当作一个 token。

其实，这两个目标是相互冲突的：把常用的词组组合起来，变成了新的 token，必定会增加 token 的种类。我们需要取得两者的一个折中，大致需要 5-6 万种 token。具体讲，我们在计算机对字符统一编码的数字序列上，计算频繁出现的子序列作为 token，具体的算法包括：Byte Pair Encoding (BPE) 和 Byte-Level Byte Pair Encoding (BBPE) 等。

所以，在大语言模型的世界里，token 才是它能直接理解的“单词”。在大模型与人的交互中，在输入端，用户的输入都会转化为 token 的序列输入给大模型；而在输出端，我们会通过计算机中字符的统一编码把 token 解码为对应的原本字符，以方便用户阅读。

但是，需要指出的是，token 与人类的“词”或“字”只是统计相关，并无固定对应关系。比如，一个 token 可以是下划线和单词“the”的组合，即“_the”。所以，我们并不能通过数汉字的个数，来计算一段中文中的 token 数量。同时，不同的大语言模型可能采用不同的切分 token 的方法 (Tokenizer)，导致哪怕是同一段话，不同方法转化的 token 数量可能有 1-2 倍的差异。总的来说，大模型中计费、限速的基本单位是 token，而非人类理解的字符。在下面的介绍中，我们不再严格区分字、词、token 的说法，通常与大模型相关的输入都是 token。

8.5.2 大语言模型常用的 Transformer 网络结构

大语言模型的框架最早来源于 2017 年由 Google Brain 的 Ashish Vaswani 等人发表的文章《Attention Is All You Need》。这篇文章介绍了完全基于自注意力 (Self-Attention) 机制的 Transformer 架构，该机制能克服循环神经网络的长程依赖问题和其顺序处理导致的速度问题。Transformer 模型架构如图 8.25 所示，这里我们仅大致描述过程，具体细节建议参考原文。

设输入序列中包含 N 个词，通过 Input Embedding 模块转化为 N 个

向量, 即 $X' = [x'_1, x'_2, \dots, x'_N]$ 。 $P = [p_1, \dots, p_m]$ 是一个预定义的位置向量矩阵 (图中的 Positional Encoding), 其中 p_i 表示在第 i 个位置上的位置向量。 P 的特点是如果 i, j 两个位置相近, 那么 p_i 和 p_j 就相似, 也就是说 $p_i^T p_j$ 更大⁵。将词嵌入和位置嵌入相加作为输入序列带位置的最终嵌入表示, 即 $X = X' + P_{1:N}$, 其中 $P_{1:N}$ 表示 P 的前 N 个向量。

接下来我们介绍自注意力机制的计算。首先, 对于每个 x_i , 通过 3 个矩阵 W_q, W_k, W_v , 将每个词的表示映射成查询向量、键向量和值向量:

- 查询向量 q_i : $q_i = W_q x_i$;
- 键向量 k_i : $k_i = W_k x_i$;
- 值向量 v_i : $v_i = W_v x_i$ 。

之后对于第 i 个词, 计算对应输出向量 h_i , 方法如下: 1. 计算第 i 个词对所有词的注意力分数 $s_i = [q_i^T k_1, q_i^T k_2, \dots, q_i^T k_N]$ 。2. 对分数进行归一化 $a_i = \text{Softmax}(s_i)$ 。3. 使用这个注意力分布对所有词的值向量进行加权平均, 获得这个词的新的表示 $h_i = a_{i1}v_1 + a_{i2}v_2 + \dots + a_{iN}v_N$ 。

以上过程对所有词的计算可以使用矩阵运算进行描述。将所有词的查询向量、键向量、值向量分别拼成 Q, K, V 矩阵, 例如 $Q = [q_1, \dots, q_N]$, 所有词的自注意力计算可表示为:

$$\text{Attention}(Q, K, V) = \text{Softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V, \quad (8.35)$$

其中 $\sqrt{d_k}$ 是一个缩放系数, 用来缩放点乘的结果使得训练更稳定。

下面简要介绍均方根归一化方法 (Root Mean Square Layer Normalization, RMSNorm)。RMSNorm 是一种归一化方法, 对应图中的 Norm 模块。尽管原始 Transformer 论文中使用的是 LayerNorm, 但是在 Qwen2.5/LLaMA 等当前主流模型中普遍使用的是 RMSNorm。RMSNorm 的计算方法是对每

⁵关于为什么这样设计, 读者可以结合注意力的计算过程进行思考。

个词的隐向量 h_i 进行缩放：

$$\bar{h}_i = \frac{h_i}{\text{RMS}(h_i)} \cdot \gamma, \quad (8.36)$$

其中 $\text{RMS}(h_i) = \sqrt{\frac{1}{d} \sum_{j=1}^d h_{ij}^2}$ 表示向量 h_i 的均方根值， d 是隐向量维度， γ 是一个可学习的缩放参数。归一化方法通过重新缩放来控制中间层向量的数值范围，防止梯度爆炸或消失，从而维持训练过程的稳定性。

以上是自注意力机制的主要介绍，Transformer 的模型还有很多其他模块。例如自注意力机制模型需要提取多种信息，因此会有多个不同的自注意力机制模型并行运算，形成多头注意力模型 (Multi-Head Attention)。每个注意力关注输入信息的不同部分，最后将这些部分拼接在一起，再进入前馈神经网络中处理，得到一个输出词向量。此外，注意力机制之后还有残差连接和前述的归一化 (Add & Norm)、前向神经网络 (Feed Forward) 等组件。这些组件共同组成了 Transformer 的一层。由于每一层的输出的形式和输入的形式是相同的，Transformer 是由 N 层这种结构堆叠形成的。

Transformer 的一大特点是计算的并行性。即输入一个序列 X ，我们可以并行计算所有词的输出向量。与之相对的，前述 RNN 的方法只能从 x_1 到 x_N 逐个计算每个词的输出向量。例如输入一个长度为 1000 个词的序列，一个 64 层的 Transformer 进行 64 层的串行前向计算，而层内所有词的计算是并行的，而一个 RNN 却必须进行 1000 次串行前向计算。因此 Transformer 更适合充分利用 GPU 等设备的并行计算能力，也因此引发了大规模预训练的浪潮。

8.5.3 大语言模型中的幻觉问题和缓解方法

虽然大语言模型在多种任务上取得了突破性的进展，但是依然面临一个很大的问题，即幻觉问题 (hallucination)。比如事实错误、数学计算问题、缺乏专业知识等。比方说，当我们问大模型一家上市公司某年的年收入，在没有开启“联结互联网”的模式下，大模型有非常大的概率都是答不对的。

检索增强的生成 (Retrieval Augmented Generation)

如果说模型微调是闭卷考试，那么检索增强就是通过“开卷考试”的方式解决大语言模型的幻觉问题。它的基本步骤如下：针对用户给出的问题，从文档库中检索跟问题相关的文档片段；随后，将检索到的文档片段和问题一起输入提示词，让大模型根据检索到的内容回答所提的问题。

可以看出，在检索增强的生成中，大模型是基于检索到的内容进行答案生成的；因此，检索的质量决定了最终答案的质量。当前有大量的工作从如下几个方面提升从文档中检索相关内容的质量：

- 文档解析。检索的前提是从文档中获取文本内容，但这个任务并不是容易的事情。从不同文档格式的文档中提取内容的方法也不同。我们可以把文档大致分为两大类，带标签文档和无标签文档。带标签文档，如 HTML 格式的网页，可以通过文档中的标签，获取文档中的段落、表格等内容；但是，这里的挑战是如何去除掉跟网页主要内容无关的广告、导航等信息。无标签文档，如 PDF 文档，只提供了文档页面的视觉信息，而并不包含文档的结构信息；也就是说，我们并不能直接获取文档中的段落、表格以及表格的行列信息；当前有大量工作研究高质量的 PDF 文档解析，即通过识别文档页面的视觉信息，以还原文档结构。因此，高质量的文档解析，才能为检索提供优质的内容基础。
- 上下文相关的检索 (Contextual Retrieval)。检索的目标，是搜索到在众多的文档中与查询相关的内容片段。因此，需要在构建文档内容索引时，对文档切片。然而，文档切片会造成内容的上下文语义缺失。例如，切片中的一些代词的具体指代（人名、公司名等）可能并不在当前片段，而出现在之前的片段中。又例如，文档经常采用“总述段-分述段”的形式表述内容，总述段指在语篇中承担“提纲”功能的段落或语句块，常以“如下几步”、“主要包括以下几个方面”等话语为标

记，分述段指紧随总述段之后，依次展开并具体化该框架的若干段落或语句块。当原始文档被切分为若干独立片段后，上述“总—分”结构常被机械截断：总述段与部分或全部分述段被分配到不同片段，其结果是，分述段一旦脱离原上位语境，便失去了来自总述段的显性指涉，形成上下文的语义缺口。最终的结果是，很多分述段并不能被检索到。因此，如何考虑文档切片的上下文信息，提升检索质量，也是当前研究的热点。

8.5.4 大语言模型小结

以上是有关于大语言模型的一些基础内容，可以看出大语言模型是世界语言的集大成者，可以根据用户的提问给出对应的回答。在这些内容中，我们还重点介绍了什么是大语言模型能理解的 token，以及常用的大模型的前向计算构架 Transformer；并讨论了大模型的幻觉问题和用于缓解它的检索增强的生成方法。

这些内容是大语言模型的入门的知识，一些深入的内容，比如大模型的训练方法（预训练、指令微调等），可以在该领域的专门课程中获得。

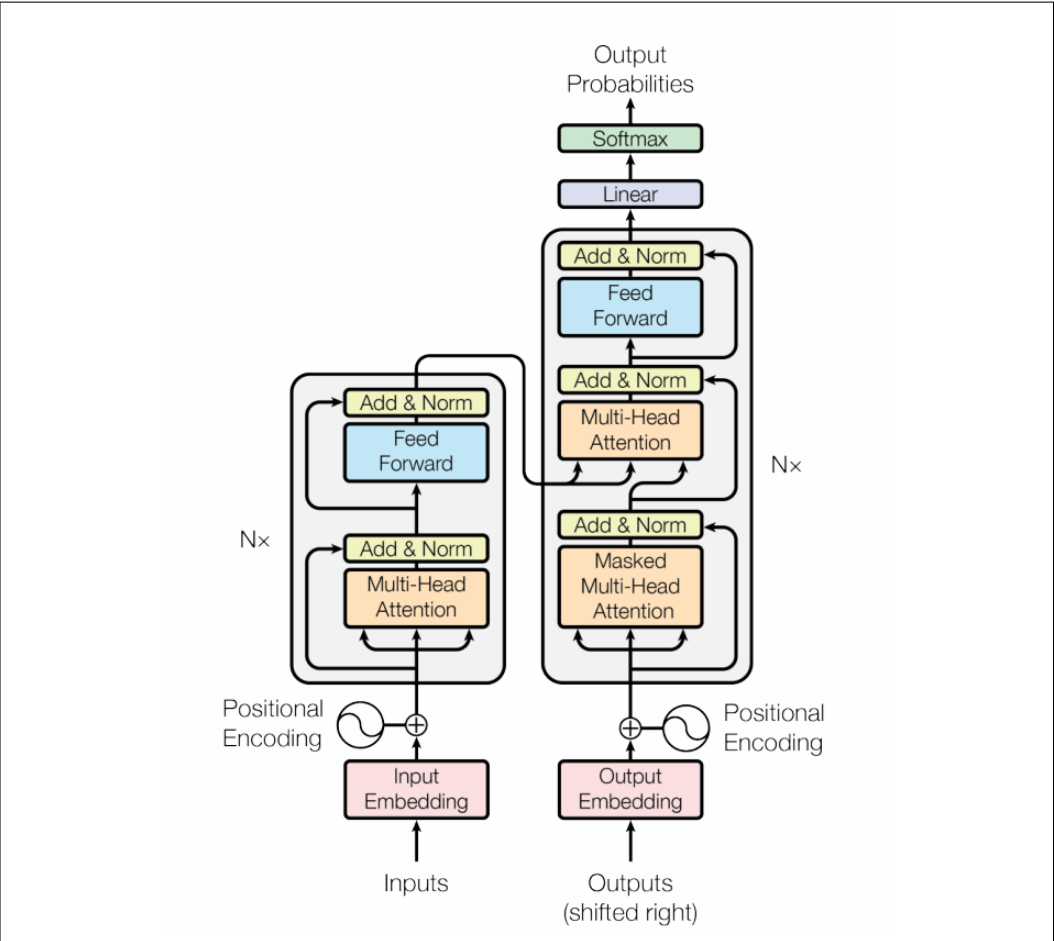


图 8.25: **Transformer** 框架示意图, 其中左侧为编码器, 右侧为解码器, “Positional Encoding” 是指将序列位置信息编码入词向量中, “Add & Norm” 代表了残差连接和层归一化, “Linear” 为线性处理, “ $N \times$ ” 代表重复了 N 次

对联结主义部分的总结

到此，我们关于联结主义的学习就告一段落。在这一部分中，我们系统梳理了联结主义框架下的两大核心分支——经典机器学习方法与深度学习方法，既回顾了经典模型的建模逻辑与训练机制，也探讨了在当前技术场景中广泛应用的深度学习的演进与应用。

经典机器学习方法是联结主义的早期实践，其核心是通过数学建模将数据中的规律转化成一套可以计算的规则。我们在这一章中提到了机器学习的两类任务——分类任务和回归任务，三种建模思想——分类回归思想、概率思想和集成思想，四个联结主义模型要素——基本假设、目标函数、训练方法和预测方法。这些内容彼此是相互关联的，我们在此以模型要素为线索再梳理一遍。

当我们去理解模型的基本假设的时候，往往需要先从它的建模思想出发，例如与树相关的模型会以属性为标准对训练数据中的标签进行分类；逻辑回归模型则是根据属性来设计一个平面来对样本空间进行分类；基于集成思想的 Bagging 和随机森林则是以多个弱分类器的投票结果作为预测的方法。模型的基本假设对模型的能力有决定性的作用，也会影响后续的元素。

模型的优化目标一般包括让模型预测的更准确和简化模型两部分，其具体形式会与模型的基本假设有关，也会和模型的任务类型有关。对于分类任务，我们可以以最小化条件信息熵（例如决策树模型）或是最大数据似然（例如逻辑回归模型）作为目标函数；对于回归任务，我们可以用最小化

平方误差（例如回归树模型）或是自定义损失函数作为目标函数。除此之外，我们还可以通过增加惩罚项和正则化的方式来设计目标函数。

模型的训练过程一般会基于最优化搜索中的各种贪心算法（如当目标函数对模型参数可导时，采用梯度下降的方法），基于集成方法的模型（如 AdaBoost 模型，随机森林模型）会在训练中调整样本的权重。模型的预测一般会来源于它的基本假设。

经典机器学习在十几年前非常受欢迎，当时基于神经网络的深度学习算法因为模型运行速度慢的问题难以大规模使用，所以很多的科技公司都是以使用经典机器学习算法为主。例如当时的网络搜索引擎、广告推荐算法等产品，其底层逻辑都是收集用户在使用过程中的行为作为属性，然后使用机器学习算法进行分类任务学习，这样才会给客户推荐他想看的页面或产品。即使现在深度学习在处理复杂的任务（如计算机视觉、自然语言处理）中有非常显著的优势，但经典机器学习算法可解释性强、运行速度快的特点仍在许多关键应用中有不可替代的作用。

在深度学习章节，我们主要介绍了前馈神经网络模型、卷积神经网络模型、循环神经网络模型以及大语言模型。神经网络模型的基本假设是模拟人脑的信息处理方式，建模神经元作为处理数据的单位，然后通过特定连接方式让数据在不同神经元之间传递转化。而对于整个神经网络模型的理论来说，最为重要的就是反向传播算法。借由这种算法，我们可以对任意已知前向计算过程的神经网络进行梯度训练，所以在介绍卷积神经网络和循环神经网络时，我们更多地把讲述重点放到了模型的基本结构、前向计算过程以及应用上。

卷积神经网络由卷积层、池化层和全连接层构成，虽然它依然属于前馈神经网络，但它在卷积层的神经元连接方式为卷积操作，这使得卷积神经网络拥有了局部连接和权重共享的特点。卷积层、池化层和全连接层的主要作用可以概括为提取特征、特征选择和将特征转化为输出。在介绍了卷积神经网络的结构后，我们也展示了卷积神经网络的主要应用场景，包括图

像分类、目标检测、语义分割和实例分割这几个计算机视觉中的核心任务，并在之后展示了在卷积神经网络中比较经典的 AlexNet 模型和 GoogLeNet 模型以及它们在图像识别研究历史上曾有过的重大突破。

循环神经网络在结构上和前馈神经网络有着很大的区别，它的隐藏层神经元不仅可以和下一层连接，也可以和自己本层的神经元连接，这也就能让数据在不同时刻以隐向量的方式进行传递，也因此让循环神经网络拥有了处理像自然语言一样的序列数据。而在激活函数上，比较简单的选择就是对输入的数据（包括前一时刻隐向量和当前时刻输入向量）都用加权的方式处理，而这种方式无法处理长程依赖问题，因此我们也可以选择使用长短期记忆单元。在讲完循环神经网络的基本假设后，我们进一步介绍了如何运用循环神经网络来完成自然语言中的序列标注任务，并以机器翻译任务为例介绍了编码器-解码器框架和注意力机制。

在理解了注意力机制的基础上，我们又拓展了和大语言模型相关的知识，它利用注意力机制建立端到端的自然语言序列生成模型。它的特点就是能够分析处理以人类语言和机器语言构成的世界语言。在语言处理任务上，大语言模型使用 token 作为最小的处理单位；除此之外，大语言模型运用 Transformer 框架，能有效解决循环神经网络模型的串行计算的时间效率问题。

在现实应用中，凭借其强大的文本理解、生成和推理能力，各式各样的大语言模型产品正在深刻改变众多行业和工作方式。例如很多程序员会运用大语言模型来帮助写代码，一些作家会通过大语言模型来查找资料、修改文章的语法错误甚至找灵感。虽然作为概率生成模型，大语言模型仍然难以避免幻觉问题，但我们可以检索增强生成的方法提升其回答准确性、减少幻觉的出现。大语言模型的能力和应用场景仍在快速发展和拓展中，未来潜力巨大。

第四部分

强化：行为主义

在这部分的内容中，我们考虑一类新的问题，序列决策 (Sequential Decision Process) 问题。在这类问题中，我们需要在一连串前后有关联的时间点上，依次做出选择，前一选择的结果会影响后一选择的结果；而我们的目标不是当下这一步获得的奖励最大，而是整个过程结束后获得的奖励的总和最大。

其实，现实中有大量的序列决策的问题，我们在这里举一些例子：

- 多臂老虎机问题 (Multi-Armed Bandit)。你面前有 K 台老虎机 (臂)，每轮只能选择拉动其中的一台，而每台老虎机每轮吐出的奖金数量服从一个未知的概率分布。我们需要得到选择拉动哪台老虎机的策略，使得在 T 轮后获得的累计奖金尽可能高。
- 超级玛丽游戏。超级玛丽 (Super Mario Bros.) 是任天堂在 1985 年推出的横版动作游戏。在通关过程中，玩家需要控制意大利水管工马里奥前进、后退、跳跃等动作，以躲避外在环境的各种袭击，并以成功营救碧琪公主为最终目标。
- 围棋。游戏黑白两方，在一块画满横竖线的 19×19 方板 (共 361 个交叉点) 上，轮流摆石子。游戏的最终目标是：看谁围的空格子多，谁就赢。
- 大语言模型求解数学问题。以前的大模型是“高级自动填充”，你问一句，它靠统计概率接下一句，看起来懂，其实没“真动脑”。有推理能力的模型，会把复杂问题拆成几步，自己“心里”先算一遍，再给你答案。比如解数学题，它不是背答案，而是一步步列式、验算、检查，在输出最终的答案。我们希望大模型先输出思考的过程，再由此得到最终的答案。
- 星际争霸游戏。星际争霸是一个多人协同游戏，游戏中玩家分成若干组。每组内共同修建，完善家园；组与组之间相互攻击。谁先打爆对

方所有建筑谁就赢。

你可以看到，这些问题，都需要在一连串时间点上依次做出动作 (Action) 的选择。在每个时间点，多臂老虎机问题的选择是拉哪台老虎机，超级玛丽游戏的选择是控制马里奥的动作（前进、后退、跳跃等），围棋玩家的选择是在哪个位置落子，大语言模型的选择是当前应该生成哪一个 token，而星际争霸玩家的选择是适合当前状态的任意动作。

在做出某个动作后，系统的状态 (State) 会发生改变。显然，在各种游戏中，玩家选择执行某个动作后，游戏的局面就发生了变化，例如：围棋的棋局多了一个落子。而在大语言模型中，选择生成某个 token 后，整个序列就多了一个 token，那么整个系统状态自然也发生了变化。只有多臂老虎机问题是特殊的情况，玩家的任何动作，并不改变系统的状态；也就是说，它是一个无状态的简化版的序列决策问题（第十一节中将会详细说明）。

同时，在这些问题中，在每做出一个动作后，环境 (Environment) 也会给出当前的奖励 (Reward)。多臂老虎机问题的当前奖励是最近拉动的这台老虎机的奖励。超级玛丽游戏的当前奖励或者是吃到金币，或者是被击毙，或者没有任何奖励。围棋的当前奖励是占领或丢失了一些新地盘，或者没有任何奖励。在大语言模型求解数学问题中，只有在生成完最后的 token 后，我们才可以判断生成的最终答案是否正确（与标准答案进行比较），因此给出或正或负的奖励；而在其它时刻，环境不会提供任何奖励。而在星际争霸游戏中，当前的奖励可能是消灭了对方某个建筑、消灭了对方某个士兵等，只有在游戏结束才能获得最终的胜负奖励。总结起来，在不同的问题中，当前的奖励各有不同：在有些问题中，如多臂老虎机问题，环境在每个时刻都能反馈当前奖励，因此当前奖励是稠密的；而在另一些问题中，如围棋和大语言模型求解数学问题，环境在绝大多数时刻都不能提供当前奖励，只有在终止状态才能获得最终的奖励，因此它们是稀疏奖励 (Sparse Rewards) 和延迟奖励 (Delayed Rewards)。同时，需要指出的是：我们的目标是整个过程结束后获得的奖励的总和最大，而不用计较过程中的小的得

失。

因此，序列决策就是强化学习解决的核心问题：在与环境的大量交互中，我们获取了大量关于 (状态，动作，奖励) 的元组数据 (记录了在什么状态下，采用了什么动作，获得了什么当前奖励)；基于这样的数据，强化学习的训练目标是获得一个策略函数，即在某个状态下应该采取什么动作的函数，以实现奖励总和 (即最终奖励) 的最大化。

而在强化学习的过程中，我们通常有在线学习 (Online Learning) 和离线学习 (Offline Learning) 两种模型，它们的区别是：获取的交互数据，即关于 (状态，动作，奖励) 的元组数据，的时间不同。在在线学习中，一边生成这样的交互数据，一边学习策略函数；而在离线学习中，先生成大量交互数据，再基于这些离线数据学习策略函数。但不管是哪种情况，我们都是基于与环境的行为交互数据，学习最优的动作策略。所以，我们通常将强化学习归属于行为主义人工智能的研究内容。

其实，我们可以把人生当成“在线强化学习”：我们天生带着一个初始策略，婴儿版的“我”，在真实世界里实时决策。每一次跌倒、每一句表扬、每一道挫折，都是环境给的即时奖励。大脑用这些奖励调整下一版的策略函数。升学、跳槽、创业、恋爱，都是一边试错一边攒经验，没有外挂可回档，只能现场调策略。所谓成长，就是在这台不能暂停的“人生服务器”里，靠一条接一条的实时反馈，把自己从“菜鸟号”练成“满级号”。

因此，强化学习无处不在，与我们休戚相关。

第九章 强化学习：基本问题和性质

在本章中，我们给出马尔可夫决策过程的形式化定义，并探讨该问题的一些重要性质。

9.1 强化学习问题中的专业术语

强化学习的序列决策过程可被形式化的定义为马尔可夫决策过程 (Markov Decision Processes, MDPs)。一个 MDP 通常由状态空间、动作空间、状态转移矩阵、奖励函数以及折扣因子等组成。简单地说，强化学习试图找到一个策略规则使得系统获得最大的累积奖励值，即获得最大价值。为了方便读者理解和记忆，我们用一些例子来解释该问题中的专业术语。

状态 (State) ，记为 S

状态是对当前环境的一个概括。在超级玛丽的例子中，可以把屏幕当前的画面看作状态。玩家只需要知道当前画面（或者最近几帧画面）就能够做出正确的决策，决定下一步是让超级玛丽向左、向右，或是向上。也就是说，状态是做决策的唯一依据。

在有些游戏中，玩家只能获取环境状态的部分信息，例如：麻将游戏看不到对方手中的牌；我们称这样问题叫部分观测 (Partial Observation) 问

题。反之，我们称它为完美观测 (Perfect Observation) 问题。

另外，**状态空间** (State Space) 是指所有可能存在的状态的集合，记作花体字母 $\mathfrak{S}$ 。状态空间可能是有限集合，也可能是无限集合。在超级玛丽、星际争霸、无人驾驶这些例子中，状态空间是无限集合，存在无穷多种可能的状态。围棋、五子棋、麻将、德州扑克这些游戏中，状态空间是有限集合，可以枚举出所有可能的状态。

动作 (Action)，记为 A

动作是指在某个状态下做出的决策。在超级玛丽的例子中，假设玛丽奥只能向左走、向右走、向上跳。那么动作就是左、右、上三者中的一种。

动作空间 (Action Space) 是指所有可能动作的集合，记作花体字母 $\mathfrak{A}$ 。在超级玛丽例子中，动作空间是 $\mathfrak{A} = \{, , \}$ 。而在围棋例子中，动作空间是 $\mathfrak{A} = \{1, 2, 3, \dots, 361\}$ ，即所有可能落子的位置。

智能体 (Agent)

智能体是指做动作的主体：由谁做动作，谁就是智能体。在各种游戏中，每个玩家就是一个智能体。在自动驾驶的应用中，无人车就是智能体。

环境 (Environment) 和环境模型 (Environment Model)

智能体在执行完一个动作之后，环境返回给智能体的一个奖励的数值，称作**奖励** (Reward)。奖励往往由我们自己来定义；奖励定义得好坏非常影响强化学习的结果。比如，在超级玛丽游戏中，可以这样定义：玛丽奥吃到一个金币，获得奖励 +1；如果玛丽奥通过一局关卡，奖励是 +1000；如果玛丽奥碰到敌人，游戏结束，奖励是 -1000；如果这一步什么都没发生，奖励就是 0。怎么定义奖励就见仁见智了。我们应该把最终打赢游戏的奖励定义得大一些，这样才能鼓励玛丽奥通过关卡，而不是一味地收集金币。

同时,需要指出的是,在某个状态下执行某一动作所获得的奖励,可以是随机的。虽然在超级玛丽的游戏中,这个奖励是确定性的;然而,在其它问题中往往并不是这样。例如,在多臂老虎机游戏中,玩家在拉下某个老虎机后获得的奖励就是随机的。

而且,在不同的问题中,当前的奖励各有不同:在有些问题中,如多臂老虎机问题,环境在每个时刻都能反馈当前奖励,因此当前奖励是稠密的;而在另一些问题中,如围棋和大语言模型求解数学问题,环境在绝大多数时刻都不能提供即时奖励,只有在终止状态才能获得最终的奖励,因此它们是稀疏奖励 (Sparse Rewards) 和延迟奖励 (Delayed Rewards)。稀疏奖励会给强化学习带来巨大的挑战。

另外,智能体在执行完一个动作之后,系统的状态也会发生变化,我们称之为**状态转移**。例如,在围棋中,每步落子后,游戏状态就从一个新的棋局转变成一个新的棋局。同样的,除了确定性的状态转移外,状态的转移也可能是随机的。

那么, **环境**又是什么呢? 在各种游戏中, 游戏程序就是环境。在围棋、象棋的例子中, 游戏规则就是环境。在无人驾驶的应用中, 真实的物理世界就是环境。谁能生成新的状态, 谁产生奖励, 谁就是环境。

综上, 我们可以用如下的**环境模型**来刻画在某个状态下执行某一动作后, 所获得的奖励和发生的状态转移:

$$p(s', r | s, a) \doteq \Pr\{S_t = s', R_t = r \mid S_{t-1} = s, A_{t-1} = a\} \quad (9.1)$$

该式子展示了, 在 $(t-1)$ 时刻, 环境的状态为 s , 并执行了动作 a 后, 系统状态转变为 s' , 并获得 r 的奖励的概率为 $p(s', r | s, a)$ 。显然, 我们使用了一个条件概率函数来描述整个环境模型。需要指出的是, 在环境模型中, 系统的下一状态只与当前状态有关, 这就是所谓的“马尔可夫假设”。当然, 我们也可以打破这个假设, 考虑当前状态之前的若干状态对下一状态的影响。具体讲, 我们只需要在以上的环境模型中引入更多以前时刻的状态, 就可以获得更复杂的环境模型。

前面我们提到，在有些问题中，如多臂老虎机问题，动态的执行并不改变环境的状态，因此，我们也没必要引入状态的变量。因此，此时的环境模型表示为：

$$p(r|a) \doteq \Pr\{R_t=r \mid A_{t-1}=a\}, \quad (9.2)$$

该式子展示了，在 $(t-1)$ 时刻，执行动作 a 后，系统获得 r 的奖励的概率为 $p(r|a)$ 。

显然，已知环境模型后，我们可以获得如下的**状态转移函数**，即：

$$p(s'|s, a) \doteq \Pr\{S_t=s' \mid S_{t-1}=s, A_{t-1}=a\} = \sum_{r \in \mathcal{R}} p(s', r \mid s, a) \quad (9.3)$$

另外，我们也需要探讨一下**白盒环境模型**与**灰盒环境模型**的区别。白盒环境模型，也称为环境模型是已知的，指的是环境模型的条件转移概率有闭式方程表达，或查表即可得。而灰盒环境模型，也称为环境模型是未知的，指的是写不出环境模型的解析式，但我们拥有一台环境模型的“模拟器”，对给定的任意 (s, a) ，就能返回一条样本 (s', r) 。

在绝大多数实际问题中，环境模型都是灰盒的。白盒环境模型适用于在简化的问题下，对马尔可夫决策过程做一些理论推导，详细的内容请见第十章。

策略函数 (Policy Function) ，记为 π

策略函数是根据观测到的当前状态做出的决策，以控制智能体动作。比如，假设你在玩超级玛丽游戏，当前屏幕上的画面有个敌人已经非常接近超级玛丽了，请问你该做什么决策呢？有很大概率你会决定向上跳，这样可以避开敌人，还能吃到金币。“向上跳”这个动作就是你的策略。

策略函数可以由不同的方式定义。这里介绍一种最常用的定义。把状态记作 S 或 s ，动作记作 A 或 a 。策略函数 $\pi: \mathcal{S} \times \mathcal{A} \mapsto [0, 1]$ 是一个条件概率函数：

$$\pi(a|s) \doteq \Pr(A=a \mid S=s) \quad (9.4)$$

也就是说，策略函数的输入是状态 s 和动作 a ，输出是一个 0 到 1 之间的概率值。举个例子，把超级玛丽游戏中的某个画面作为状态 s 输入策略函数，策略函数输出动作的概率值为：

$$\pi(\text{左}|s) = 0.1, \pi(\text{右}|s) = 0.05, \pi(\text{上}|s) = 0.85 \quad (9.5)$$

如果你让这个策略函数 π 来自动操作玛丽奥打游戏，它就会做一个随机抽样：以 0.1 的概率向左走，0.05 的概率向右走，0.85 的概率向上跳。三种动作都有可能发生，但是向上的概率最大，向左概率较小，向右概率最小。

强化学习学什么？就是学这个策略函数。只要有了策略函数，就可以让它自动控制玛丽奥打赢游戏。稍后，我们会定义什么是最优策略函数，进而给出强化学习问题的形式化定义。

9.2 强化学习问题的形式化定义和基本性质

9.2.1 更多强化学习的基本概念和性质

需要强调的是，强化学习的目标并不是最大化当前的奖励，而是最大化一系列动作后的最终**回报**。打个比方，下棋的时候，你的目标是赢得一局比赛（回报），而非吃掉对方一个棋子（奖励）；超级玛丽游戏中，你的目标是最终通关，而不是不断的吃掉金币。因此，我们首先得定义什么是**回报**。

回报 (Return)

回报是从当前时刻开始到一回合结束的所有奖励的总和，所以回报也叫作累计奖励 (Cumulative Future Reward)。把 t 时刻的回报记作随机变量 U_t ，那么回报的定义是这样的：

$$U_t \doteq R_t + R_{t+1} + R_{t+2} + R_{t+3} + \cdots \quad (9.6)$$

显然，回报是未来获得的奖励总和，所以智能体的目标就是让回报越大越好。强化学习的目标就是寻找一个策略，使得回报的期望最大化。

折扣回报 (Discounted Return)

在强化学习中，通常使用**折扣回报**给未来的奖励做折扣。这是折扣回报的定义：

$$U_t \doteq R_t + \gamma \cdot R_{t+1} + \gamma^2 \cdot R_{t+2} + \gamma^3 \cdot R_{t+3} + \cdots \quad (9.7)$$

这里的 $\gamma \in [0, 1]$ 叫作折扣率。对待越久远的未来，给奖励打的折扣越大。折扣率是个超参数，需要手动调，折扣率的设置会影响强化学习的结果。显然，我们有如下关于折扣回报的性质：

$$U_t = R_t + \gamma \cdot U_{t+1} \quad (9.8)$$

在后续的内容，如果没有特殊说明，我们讨论的回报都指的是折扣回报。

下面，我们分析一下**回报的随机性**。假设一回合游戏一共有 n 步。当完成这一回合之后，我们观测到所有 n 个奖励： $r_1, r_2, \cdots, r_n$ 。这些奖励不是随机变量，而是实际观测到的数值。此时，我们可以实际计算出折扣回报，即：

$$u_t = r_t + \gamma \cdot r_{t+1} + \gamma^2 \cdot r_{t+2} + \cdots + \gamma^{n-t} \cdot r_n, \quad \forall t = 1, \cdots, n \quad (9.9)$$

此时的折扣回报 u_t 是实际观测到的数值，不具有随机性。

假设我们此时在第 t 时刻，我们只观测到 s_t 及其之前的状态、动作和奖励：

$$s_1, a_1, r_1, \quad s_2, a_2, r_2, \quad \cdots, \quad s_{t-1}, a_{t-1}, r_{t-1}, \quad s_t, \quad (9.10)$$

我们称以上这个“状态、动作、奖励”的序列叫一条**轨迹** (Trajectory)。而 t 时间后的情况都是未知的，可以用如下的随机变量表示：

$$A_t, R_t, S_{t+1}, A_{t+1}, R_{t+1}, \cdots S_n, A_n, R_n. \quad (9.11)$$

显然，回报 U_t 依赖于奖励 $R_t, R_{t+1}, \dots, R_n$ ，而这些奖励全都是未知的随机变量，所以 U_t 也是未知的随机变量。

也就是说，在生成一条轨迹的过程中，动作的随机性来自于策略函数，状态和奖励的随机性来自于环境模型。

动作价值函数 (Action-Value Function) 和状态价值函数 (State-Value Function)

前面介绍了（折扣）回报 U_t ，它是 t 时刻之后所有奖励的（加权）和。在 t 时刻，假如我们知道 U_t 的值，我们就知道游戏是快赢了还是快输了。然而在 t 时刻我们并不知道 U_t 的值，此时 U_t 仍然是个随机变量。然而，我们又想预判 U_t 的值从而知道局势的好坏。该怎么办呢？解决方案就是对 U_t 求期望，消除其中的随机性。

假设我们已经观测到状态 s_t ，而且做完决策，选中动作 a_t 。那么 U_t 中的随机性来自于 $t+1$ 时刻之后用某个策略 π 决定的状态和动作：

$$S_{t+1}, A_{t+1}, S_{t+2}, A_{t+2}, \dots, S_n, A_n \quad (9.12)$$

对 U_t 关于变量 $S_{t+1}, A_{t+1}, \dots, S_n, A_n$ 求条件期望，得到

$$Q_\pi(s_t, a_t) = \mathbb{E}_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} [U_t \mid S_t = s_t, A_t = a_t]. \quad (9.13)$$

期望中的 $S_t = s_t$ 和 $A_t = a_t$ 是条件，意思是已经观测到 S_t 和 A_t 值。后续我们采用策略函数 π 来选择动作，获得的折扣回报的期望就是 $Q_\pi(s_t, a_t)$ 。这个函数被称为**动作价值函数**。

同样的，我们在仅观测到状态 s_t 后，且并没有选中下一个动作时，也可以直接计算 U_t 的期望，即得到了**状态价值函数**：

$$V_\pi(s_t) = \mathbb{E}_{S_{t+1}, A_{t+1}, \dots, S_n, A_n} [U_t \mid S_t = s_t]. \quad (9.14)$$

状态价值 $V_\pi(s_t)$ 越大，就意味着回报 U_t 的期望越大。用状态价值可以衡量策略 π 与状态 s_t 的好坏。

如果状态和奖励都是离散的，动作价值函数和状态价值函数之间显然有如下的关系：

$$Q_{\pi}(s_t, a_t) = \sum_{s', r} p(s', r | s_t, a_t) [r + \gamma \cdot V_{\pi}(s')], \quad (9.15)$$

$$V_{\pi}(s_t) = \sum_a \pi(a | s) Q_{\pi}(s_t, a). \quad (9.16)$$

有兴趣的同学可以自行证明。

最优动作价值函数 (Optimal Action-Value Function) 和最优状态价值函数 (Optimal State-Value Function)

怎样才能排除掉策略 π 的影响，只评价当前状态和动作的好坏呢？解决方案就是**最优动作价值函数**：

$$Q_{\star}(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t), \quad \forall s_t \in \mathcal{S}, \quad a_t \in \mathcal{A}. \quad (9.17)$$

该公式的意思是有很多种策略函数 π 可供选择，而 $Q_{\star}(s_t, a_t)$ 是可能获得的期望回报的最大值。

最优动作价值函数 $Q_{\star}$ 非常有用：它就像是一个先知，能指引智能体做出正确的最优决策。比如玩超级玛丽，给定当前状态 s_t ，智能体该执行动作空间 $\mathcal{A} = \{\text{左}, \text{右}, \text{上}\}$ 中的哪个动作呢？假设我们已知 $Q_{\star}$ 函数，那么我们就让它给三个动作打分，比如：

$$Q_{\star}(s_t, \text{左}) = 125, Q_{\star}(s_t, \text{右}) = -50, Q_{\star}(s_t, \text{上}) = 300. \quad (9.18)$$

这三个值是什么意思呢？ $Q_{\star}(s_t, \text{左}) = 125$ 的意思是：如果现在智能体选择向左走，那么不管以后智能体用什么策略函数，回报 U_t 的期望最多不会超过 125。同理，如果现在向右走，则回报的期望最多不超过 -50。如果现在向上跳，则回报的期望最多不超过 300。智能体应该执行哪个动作呢？毫无疑问，智能体当然应该向上跳，这样才能有希望获得尽量高的回报。

因此, 根据 $Q_*(s_t, a)$ 函数, 我们可以获得如下的确定性的**最优策略**, 即:

$$\pi_*(s_t) = \arg \max_{a \in \mathcal{A}} Q_*(s_t, a). \quad (9.19)$$

类似的, 我们也可以定义**最优状态价值函数**:

$$V_*(s_t) = \max_{\pi} V_{\pi}(s_t), \quad \forall s_t \in \mathcal{S}. \quad (9.20)$$

同样, 根据 $V_*(s_t)$, 我们也可以构造出对应的确定性的**最优策略**。

9.2.2 强化学习问题的形式化定义

经过以上所有专业术语和概念的铺垫之后, 我们可以给出强化学习问题的形式化定义。根据所提供的环境模型的不同, 我们给出两个强化学习问题的变体:

白盒环境模型下的强化学习问题:

已知一个白盒的环境模型, 计算出如9.19式定义的最优策略。

这个问题也称为基于模型 (Model-based) 的强化学习问题。在第十章中, 我们将对这个问题进行理论推导, 提出具体的算法。

灰盒环境模型下的强化学习问题:

已知一个灰盒的环境模型, 计算出如9.19式定义的最优策略。

这个问题也称为无模型 (Model-free) 的强化学习问题。在第十三章中, 我们将对这个问题进行分析, 提出具体的算法。

在这个问题中, 我们需要利用灰盒的环境模型来采样出轨迹数据, 即“状态、动作、奖励”的序列。根据获取轨迹数据的时间的不同, 我们分为在线学习和离线学习。在在线学习中, 我们一边生成这样的交互数据, 一边学习最优策略函数; 而在离线学习中, 我们先生成大量的交互数据, 在基于这些离线数据学习最优策略函数。

我们之前提到的大多数强化学习的问题，如多臂老虎机问题、超级玛丽游戏、围棋、星际争霸游戏等，我们往往会采用在线学习的模式。而用大语言模型求解数学问题，则可能使用离线学习模式。后续，我们也将比较这两种不同的学习模式，他们面临什么不同的挑战和应对的方法。

第十章 白盒环境模型下的强化学习

在本章中，我们讨论在白盒环境模型下求解强化学习问题的情形。我们假设环境模型 $p(s', r|s, a)$ 是完全已知的，即其形式要么具有闭式表达式，要么可以通过查表精确获得。在这种条件下，我们能够精确地推导出最优策略。尽管在实际问题中，环境模型往往难以完全已知，但这些理论分析为解决更一般的强化学习问题提供了重要的启示。

为了便于叙述，10.1、10.2 与 10.3 各节的推导均假设状态与奖励为离散随机变量，分别介绍贝尔曼方程及其在白盒强化学习求解中的应用。最后，在第 10.4 节中，我们将进一步探讨连续状态空间下，如何利用已知的闭式环境模型推导对应的最优策略。

10.1 贝尔曼方程及其用途

我们首要的目标是计算动作价值函数 $Q_\pi(s, a)$ 和状态价值函数 $V_\pi(s)$ 在每个状态和动作下的取值。此时，我们假设已知：一个给定的策略函数 π 和一个白盒的环境模型 $p(s', r|s, a)$ 。由于状态估值函数 $V_\pi(s)$ 的输入只有一个，即状态 s ，而动作价值函数有两个输入，我们先计算相对简单函数 $V_\pi(s)$ 在每个状态 s 下的取值。

回忆之前的公式9.15和9.16，动作价值函数和状态价值函数之间存在如

下的等式关系：

$$Q_{\pi}(s_t, a_t) = \sum_{s', r} p(s', r | s_t, a_t) [r + \gamma \cdot V_{\pi}(s')], \quad (10.1)$$

$$V_{\pi}(s_t) = \sum_a \pi(a | s) Q_{\pi}(s_t, a). \quad (10.2)$$

将10.1式中的 $Q_{\pi}(s_t, a)$ 代入10.2式中，我们就得到如下的等式：

状态价值函数的贝尔曼方程^a：

$$V_{\pi}(s_t) = \sum_a \pi(a | s) \sum_{s', r} p(s', r | s_t, a) [r + \gamma \cdot V_{\pi}(s')]. \quad (10.3)$$

^a以 Richard E. Bellman 是一位杰出的美国应用数学家，被誉为动态规划 (Dynamic Programming) 的创始人。以他的名字命名的贝尔曼方程，是强化学习的先驱性工作。

需要指出的是：根据问题的假设，我们已知策略函数和环境模型。也就是说，在上式中， $\pi(a | s)$ 和 $p(s', r | s_t, a)$ 都是常数。因此，此公式实际是表达了某个状态 s_t 的价值与其所有的后继状态 s' 的价值之间的等式关系。

那么，如果状态的取值一共有 n 个。我们将上式左侧的 s_t 遍历到所有可能的状态，进而就得到了一个具有 n 个方程和 n 个变量的线性方程组。因此，求解该方程组即可得到该策略下每个状态的估值。

类似的，我们也可以推导出动作价值函数的贝尔曼公式：

动作价值函数的贝尔曼方程：

$$Q_{\pi}(s_t, a_t) = \sum_{s', r} p(s', r | s_t, a_t) [r + \gamma \cdot \sum_a \pi(a | s) Q_{\pi}(s', a)]. \quad (10.4)$$

同样的，如果存在 n 个不同的状态和 m 个不同的动作，我们也可以得到一个具有 $n \times m$ 个方程和 $n \times m$ 个变量的线性方程组，进而求解出每个组合 (s, a) 下的动作价值 $Q_{\pi}(s, a)$ 。

当然，求解 $Q_{\pi}(s, a)$ 的一种更简便的方法可能是：先计算 $V_{\pi}(s)$ ，再根据10.1式计算出 $Q_{\pi}(s, a)$ 。

下面，我们举一个例子，展示上面的计算状态价值函数的过程。

格子游戏

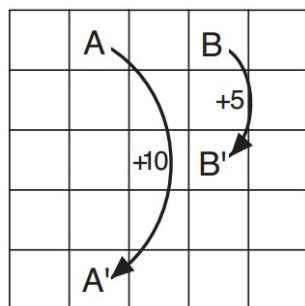


图 10.1: 格子游戏的示意图

格子游戏的规则是：我们有如图 10.1 的 5×5 的格子。在格子中，正常移动一次的奖励为 0；出界时回到当前位置，奖励为 -1；A 位置选择任意方向都到达 A' ，奖励为 +10；B 位置选择任意方向都到达 B' ，奖励为 +5；折扣率 $\gamma = 0.9$ 。**当前的策略 π 为：**从每个格子等概率选择四个移动方向。

问题分析：格子游戏的规则实际上是告知了此强化学习问题的环境模型。在这个问题中，在某个位置 s 做出某个动作 a （上移，下移，左移，右移这些动作之一），位置会确定的移动到另一个位置，记为 $f(s, a)$ ；同时，也会确定性的获得某个奖励，记为 $g(s, a)$ 。因此，如 10.3 式的状态价值函数的贝尔曼公式被简化为：

$$V_{\pi}(s) = \sum_a \pi(a|s)(g(s, a) + \gamma \cdot V_{\pi}(f(s, a))). \quad (10.5)$$

我们用 $V_{x,y}$ 表示在位置 (x, y) 时的状态估值，那么根据游戏规则，我们可

以列出如下包含 25 个方程和 25 个未知量的方程组：

$$\begin{aligned}
V_{0,0} &= -\frac{1}{2} + \frac{\gamma}{4}(V_{0,0} + V_{0,0} + V_{1,0} + V_{0,1}), \\
V_{0,1} &= 10 + \gamma V_{4,1}, \\
V_{0,2} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{0,2} + V_{0,1} + V_{1,2} + V_{0,3}), \\
V_{0,3} &= 5 + \gamma V_{2,3}, \\
V_{0,4} &= -\frac{1}{2} + \frac{\gamma}{4}(V_{0,4} + V_{0,3} + V_{1,4} + V_{0,4}), \\
V_{1,0} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{0,0} + V_{2,0} + V_{1,0} + V_{1,1}), \\
V_{1,1} &= 0 + \frac{\gamma}{4}(V_{0,1} + V_{2,1} + V_{1,0} + V_{1,2}), \\
V_{1,2} &= 0 + \frac{\gamma}{4}(V_{0,2} + V_{2,2} + V_{1,1} + V_{1,3}), \\
V_{1,3} &= 0 + \frac{\gamma}{4}(V_{0,3} + V_{2,3} + V_{1,2} + V_{1,4}), \\
V_{1,4} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{0,4} + V_{2,4} + V_{1,3} + V_{1,4}), \\
V_{2,0} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{1,0} + V_{3,0} + V_{2,0} + V_{2,1}), \\
V_{2,1} &= 0 + \frac{\gamma}{4}(V_{1,1} + V_{3,1} + V_{2,0} + V_{2,2}), \\
V_{2,2} &= 0 + \frac{\gamma}{4}(V_{1,2} + V_{3,2} + V_{2,1} + V_{2,3}), \\
V_{2,3} &= 0 + \frac{\gamma}{4}(V_{1,3} + V_{3,3} + V_{2,2} + V_{2,4}), \\
V_{2,4} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{1,4} + V_{3,4} + V_{2,3} + V_{2,4}), \\
V_{3,0} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{2,0} + V_{4,0} + V_{3,0} + V_{3,1}), \\
V_{3,1} &= 0 + \frac{\gamma}{4}(V_{2,1} + V_{4,1} + V_{3,0} + V_{3,2}), \\
V_{3,2} &= 0 + \frac{\gamma}{4}(V_{2,2} + V_{4,2} + V_{3,1} + V_{3,3}), \\
V_{3,3} &= 0 + \frac{\gamma}{4}(V_{2,3} + V_{4,3} + V_{3,2} + V_{3,4}), \\
V_{3,4} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{2,4} + V_{4,4} + V_{3,3} + V_{3,4}), \\
V_{4,0} &= -\frac{1}{2} + \frac{\gamma}{4}(V_{3,0} + V_{4,0} + V_{4,0} + V_{4,1}), \\
V_{4,1} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{3,1} + V_{4,1} + V_{4,0} + V_{4,2}), \\
V_{4,2} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{3,2} + V_{4,2} + V_{4,1} + V_{4,3}), \\
V_{4,3} &= -\frac{1}{4} + \frac{\gamma}{4}(V_{3,3} + V_{4,3} + V_{4,2} + V_{4,4}), \\
V_{4,4} &= -\frac{1}{2} + \frac{\gamma}{4}(V_{3,4} + V_{4,4} + V_{4,3} + V_{4,4}).
\end{aligned}$$

求解这个方程组，我们就可以获得在此策略下的状态价值，答案如图10.2所示。

3.3	8.8	4.4	5.3	1.5
1.5	3.0	2.3	1.9	0.5
0.1	0.7	0.7	0.4	-0.4
-1.0	-0.4	-0.4	-0.6	-1.2
-1.9	-1.3	-1.2	-1.4	-2.0

图 10.2: 格子游戏：在当前策略 π 下的状态价值（保留小数点一位的有效数字）

然而，强化学习的最终目标是获得最优策略，我们这里的方法只能计算出在给定策略 π 下的状态价值。因此，我们仍然需要继续探索。

10.2 白盒环境模型下获取最优策略的算法

虽然状态价值函数的贝尔曼方程只能求解在给定策略 π 下的状态价值，但是我们可以设计如下**策略提升**的迭代算法获得最终的最优策略：

1) **初始化：**

初始化一个策略 π 。

2) **计算策略 π 下的状态估值：**

对于策略 π ，基于状态估值函数的贝尔曼方程，求解在该策略下的状态价值函数。

3) **策略提升：**

根据上一步求得的状态价值函数，将 π 更新为当前状态价值函数下的贪心策略，即 $\pi(s) = \arg \max_{a \in \mathcal{A}} Q(s, a)$ 。

4) 迭代:

迭代以上步骤 2 和 3，直到收敛。

根据以上算法，我们可以计算出10.1节中的格子游戏的最优状态价值函数 V_* 和对应的最优策略 π_{star} ，最终结果如图10.3所示。

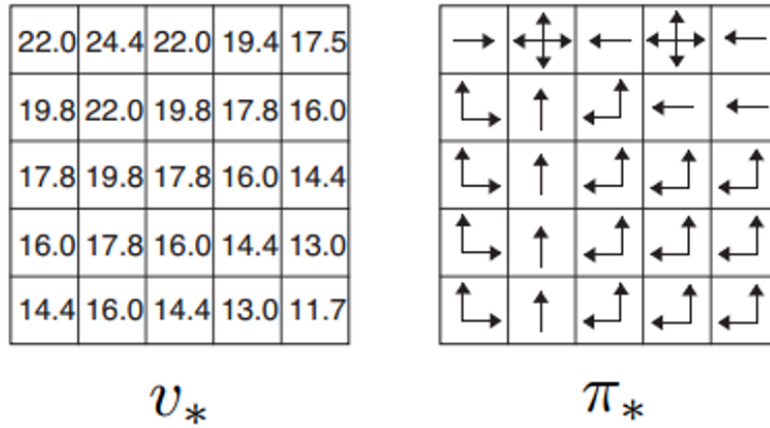


图 10.3: 格子游戏：最优状态价值函数 V_* （保留小数点一位的有效数字）和对应的最优策略 π_*

10.3 贝尔曼最优性方程及其用途

假设 π_* 是最优策略， $V_*(s)$ 是最优策略下的状态价值函数， $Q_*(s, a)$ 是最优策略下的动作价值函数。不难证明，我们有如下的等式关系：

状态价值函数的贝尔曼最优性方程：

$$V_*(s) = \max_{a \in \mathcal{A}} Q_*(s, a) \quad (10.6)$$

$$= \max_{a \in \mathcal{A}} \sum_{s', r} p(s', r | s, a) [r + \gamma V_*(s')]. \quad (10.7)$$

显然，此公式实际是表达了在最优策略下，某个状态 s 的价值 $V_*(s)$ 与其所有的后继状态 s' 的价值 $V_*(s')$ 之间的等式关系。

类似的，我们也可以获得如下的动作价值函数的贝尔曼最优性方程：

动作价值函数的贝尔曼最优性方程：

$$Q_{\star}(s, a) = \sum_{s', r} p(s', r | s, a) [r + \gamma \max_{a' \in \mathcal{A}} Q_{\star}(s', a')]. \quad (10.8)$$

当然，我们也试图通过求解状态（或动作）价值函数的贝尔曼最优性方程组，来获得 $V_{\star}(s)$ （或 $Q_{\star}(s, a)$ ），进而获得最优策略。然而，这里的方程组中都包含最大值的运算，并不能直接求解。最优性方程会在后续第十三章关于灰盒环境模型下的强化学习算法中被使用。

10.4 连续状态的贝尔曼方程及其求解

以上小节中我们把状态与奖励视为离散随机变量，利用线性方程组和策略提升算法求解最优策略。在许多实际问题中，状态空间是连续的，此时无法再列出线性方程组（状态的取值有无穷多个）。然而，在白盒环境模型 $p(s', r | s, a)$ 已知的前提下，我们仍可基于贝尔曼方程构造一个稳定可实现的近似算法，例如拟合值迭代（Fitted Value Iteration, FVI），来获取最优策略。

连续状态下的贝尔曼方程

如果状态空间是连续而非离散的，给定策略 π ，状态价值函数满足如下积分形式的贝尔曼方程（与式 (10.3) 对应的连续形式）：

$$V_{\pi}(s) = \sum_a \pi(a | s) \iint [r + \gamma V_{\pi}(s')] p(s', r | s, a) dr ds'. \quad (10.9)$$

当折扣因子 $\gamma < 1$ 时，这个方程在理论上可以通过不断迭代稳定地收敛到一个唯一解 V_{π} 。

由于连续状态空间无法像离散状态一样逐点存储或计算 $V(s)$ ，通常需要选取一个具有足够表达能力的函数族 $\{V_\theta\}$ 来近似真实的价值函数。例如，可以采用神经网络形式：

$$V_\theta(s) = f_\theta(s), \quad (10.10)$$

其中 θ 表示网络的参数。输入任意状态 s ，模型即可输出该状态的价值。接下来，我们将介绍一种名为“拟合值迭代”的方法，用于优化 $V_\theta(s)$ 使其逐步逼近真实的价值函数。

拟合值迭代 (Fitted Value Iteration, FVI)

拟合值迭代 (FVI) 是一种在连续状态空间下对给定策略进行价值评估的实用方法。FVI 的核心思想非常直观：先在一批具有代表性的状态上，根据贝尔曼方程计算出右侧的“目标值”，然后使用一个可参数化的函数（例如神经网络）去拟合这些目标值。重复这一“计算目标—回归拟合”的过程，直到函数的输出趋于稳定，即完成了价值函数的近似求解。

拟合值迭代 (FVI) 算法步骤

- 1) **采样代表性状态集**：从任务的状态空间中选取一组具有代表性的状态样本

$$\mathcal{S} = \{s_i\}_{i=1}^N,$$

这些样本可以从任务初始分布、轨迹数据或随机采样得到。

- 2) **初始化近似函数**：选择一个用于表示价值函数的函数族（例如神经网络 V_θ ），并随机初始化参数 $\theta^{(0)}$ 。
- 3) **计算目标值**：在第 k 次迭代时 ($k \geq 1$)，利用当前的价值函数近似 $V_{\theta^{(k)}}$ ，对每个样本点 s_i 计算贝尔曼方程右端的目标值：

$$y_i = \sum_a \pi(a | s_i) \iint [r + \gamma V_{\theta^{(k)}}(s')] p(s', r | s_i, a) dr ds'. \quad (10.11)$$

如果上式中的积分无法精确计算，可以用蒙特卡洛采样等方法进行近似。

- 4) **拟合更新**：将步骤 3) 得到的 $\{(s_i, y_i)\}$ 看作训练数据，通过最小化平方误差来更新参数：

$$\theta^{(k+1)} = \arg \min_{\theta} \frac{1}{N} \sum_{i=1}^N (V_{\theta}(s_i) - y_i)^2 \quad (10.12)$$

- 5) **重复迭代**：令 $k \leftarrow k+1$ ，重复步骤 3)–4)，直到贝尔曼残差 ($|V_{\theta^{(k+1)}} - V_{\theta^{(k)}}|$) 收敛到足够小的范围。

FVI 本质上就是“计算—拟合”的循环：每次先根据当前的价值估计计算出新的目标值，再用函数近似器去拟合这些目标。经过多次迭代后， V_{θ} 会逐渐逼近真实的价值函数，从而在连续状态空间中得到接近最优的解。

如果将连续状态空间离散化为足够细的网格，并令 V_{θ} 在每个网格单元内取常数值（并在迭代中不断更新这些表项），那么 FVI 就退化为第 10.1 节中通过值迭代求解线性方程组的做法。

在此基础上，若进一步重复执行“策略评估 $\rightarrow$ 策略提升”的循环，则可以进行策略优化，并找到最优策略。这与第 10.2 节中的策略提升过程是完全对应的。

10.5 小结：白盒环境模型下的强化学习

在本章的前三节内容中，我们假设环境模型是白盒的，且状态空间是离散而有限的。我们可以看到，在此假设下的强化学习问题能够精确的计

算出最优策略。在最后一节，我们分析了连续状态的贝尔曼方程及其求解。但是，除了一些人工构造的简单游戏以外，很少有现实的强化学习问题满足这些假设，特别是：现实场景的环境模型通常都是灰盒的。因此，本章的内容只是帮助读者熟悉强化学习的基本概念和性质。

在下一章中，我们将学习灰盒环境模型下的强化学习，这些内容将帮助我们解决实际场景的强化学习问题。

第十一章 灰盒环境模型下的强化学习问题及其各种变体

我们首先得回顾一下灰盒环境模型下的强化学习问题的形式化描述。我们首先定义了强化学习问题中的基本概念，包括：**状态** (State)、**动作** (Action)、**奖励** (Reward)、**智能体** (Agent) 和**环境** (Environment)。在这些问题上，每个智能体都拥有自己的初始**策略函数** (Policy Function)。在任意时刻 t ，智能体会使用此策略函数决定在当前状态 s_t 下采取的动作 a_t 。随后，（灰盒的）环境模型会根据输入的“状态、动作”对，即 (s_t, a_t) ，反馈出即时奖励 r_t 和系统转移到的新状态 s_{t+1} 。需要注意的是，由于环境模型可能是一个概率模型，我们每次获得的 r_t 和 s_{t+1} 是在该模型上的一次概率采样；因此，即便是相同的 (s_t, a_t) 输入，也可能获得不同的奖励和新状态。接着，根据新状态 s_{t+1} ，智能再次使用策略函数决定采用的新动作 a_{t+1} ；而后，环境模型有采样出 r_{t+1} 和 s_{t+2} 。因此类推下去，直到我们处于某个终止状态，则我们可以获得如下的一条**轨迹** (Trajectory) 数据：

$$s_1, a_1, r_1, \dots, s_t, a_t, r_t, s_{t+1}, a_{t+1}, r_{t+1}, \dots, s_{n-1}, a_{n-1}, r_{n-1}, s_n, r_n. \quad (11.1)$$

其中， s_n 为终止状态。此时，我们也可以计算出这条轨迹数据的总的**折扣回报** (Discounted Return)：

$$u = r_1 + \gamma \cdot r_2 + \gamma^2 \cdot r_3 + \dots + \gamma^{n-1} \cdot r_n \quad (\gamma \in [0 \ 1]) \quad (11.2)$$

如果我们能收集到很多条不同的轨迹数据，则我们希望能够学习到一个新的策略函数，即**最优策略函数**，使得在给定的灰盒环境模型下，获得的折扣回报能最大化。这就是强化学习问题的一般标准定义。

然而，现实场景的强化学习问题往往是以上标准定义的变体。接下来，我们讲介绍一些具体的强化学习的问题，在这些问题中，状态、动作、奖励、环境模型、策略函数分别是什么，有哪些新的改变和挑战。

11.1 多臂老虎机问题

我们先回忆一下这个问题：你面前有 K 台老虎机（臂），每轮只能选择拉动其中的一台，而每台老虎机每轮吐出的奖金数量服从一个参数未知的高斯函数分布，且每台老虎机的参数不同。我们需要得到选择拉动哪台老虎机的策略，使得在 n 轮后获得的累计奖金尽可能高。

在此问题中，每轮所选择的那台老虎机就是该轮的动作，因此动作空间为 $\mathcal{A} = \{1, 2, \dots, K\}$ 。某台老虎机被选中后，就用它对应的高斯函数做一次概率采样，获得当前的奖励。然而，我们每轮选择老虎机的动作都是独立的，并不会改变系统所处的状态。因此，我们可以把这个问题简化为没有状态的强化学习问题。那么， n 轮后获得的轨迹数据为：

$$a_1, r_1, \dots, a_t, r_t, a_{t+1}, r_{t+1}, \dots, a_n, r_n. \quad (11.3)$$

那么， n 轮结束后获得的回报的总和为 $u = \sum_{t=1}^n r_t$ 。

当然，在获取了足够多的轨迹数据之后，我们可以估算出每台老虎机上获得的奖励的均值，即它对应的高斯函数的均值。因此，最优策略就是每轮都选择这个均值最大的那台老虎机。显然，以上的方法是先生成数据再学习策略的**离线学习**的过程。

然而，在现实的多臂老虎机问题中，每次摇臂是需要付出一定成本的。我们不希望仅在完成 n 次摇臂之后才改变策略，我们希望的是：一边从单

次摇臂中获取奖励，一边进行策略改变，使得在 n 次摇臂之后获得的奖励最大。这显然是一个一边生成数据一边进行策略学习的**在线学习**的过程。

因此，多臂老虎机问题是一个**无状态的在线强化学习**问题。

11.2 围棋

在围棋中，黑白两方在一块画满横竖线的 19×19 方板（共 361 个交叉点）上，轮流摆石子。游戏的最终目标是：看谁围的空格子多，谁就赢。

显然，在此游戏中，状态是当前棋局的局面，可以用一张该棋局的图片来表示；那么，一个动作就是在棋盘的某个位置放上黑或白的棋子。一个动作完成后，棋局会确定性的进入另一个状态，即：增加该动作添加的棋子到指定位置，并根据游戏规则“吃”掉对手的一些区域。同时，我们可以设定如下简单的奖励规则，只有在能分出胜负的状态才给出 +1 或 -1 的奖励，在其它状态的奖励都为 0。我们的目标是学习一个策略函数，即学习一个从棋局（状态）到走子（动作）的映射，使得总能获得游戏胜利（折扣回报最大化）。

然而，围棋的强化学习问题有如下的挑战：首先，状态空间巨大。现代围棋对局通常持续约 200（双方合计），则可能的棋局的数量大致是 $361^{200} \approx 10^{500}$ 这个数量级。因此，我们肯定无法生成包含所有棋局的轨迹数据，进行线下学习；我们可能需要一边生成一些高质量对弈棋局，一边进行在线学习。

其次，延迟奖励问题。如前所述，只有在终局状态才能获得 +1 或 -1 的奖励，在其它状态的奖励都为 0。也就是说，所有的奖励都被延迟到了终局。这将导致在在线学习的大部分时间，都无法获得任何奖励反馈；这将大大增加在线学习的难度。

因此，围棋是一个**状态空间巨大且延迟奖励的在线强化学习**问题。

11.3 训练具有思考能力的大语言模型

我们知道，大语言模型的思维链 (CoT, Chain of Thought) 技术是一种通过逐步推理来解决复杂问题的方法，旨在提升大语言模型在逻辑推理、数学计算、常识判断等任务上的表现。具有推理能力的大模型会“像人一样一步步思考”，在输出整个思考过程之后，在输出最后的答案，而不是直接给出答案。

为什么这个问题可能被视作强化学习问题呢，它的状态、动作、奖励分别是什么？具体讲，在这个问题中，**状态**是当前已经生成的所有 token，**动作空间**是所有的 token 的集合，**策略函数**就是从当前状态到 token 的一个概率分布函数（显然，这里的策略函数就是大语言模型）。在生成了一个 token（选择了这个动作）之后，我们将新的 token 添加到当前状态的尾部，获得了一个新状态。然而，在思考过程中生成的 token 并不能获得即时奖励；只有在整个输出结束后，我们才能将输出的最终答案与标准答案进行对比（例如：数学题目的最终结果），获得 +1 或 -1 的最终奖励。

训练这样的大语言模型的主要问题是：获取包含思考过程的标注数据成本巨大。以数学题目为例，我们往往能获得包含“题目-最终答案”的数据，但中间的推理过程是缺失的。因此，在推理过程数据缺失的情况下，我们的训练目标是：学到一个策略函数，既能写出清晰、有效的思考过程，又能给出正确的最终答案，并在未见过的新题上表现稳定。

那么，这个强化学习问题有哪些新的挑战呢？首先，**动作空间与状态空间极其庞大**。以大语言模型为例，其动作空间即词表大小，往往可达 10^5 级别；而状态是“已生成的全部 Token 序列”，假设序列长度允许达到 10^4 ，则状态空间规模约为 $(10^5)^{10^4} = 10^{50,000}$ ，这是一个远远超越围棋（约 10^{500} ）的巨大规模，人类直觉完全无法企及，普通强化学习方法也难以在如此巨大的空间中高效探索。其次，**奖励极度稀疏且高度延迟**。一条推理轨迹可能包含数百到数千步，而只有在最后一步评估答案是否正确时才会给出奖

励，其余步骤的即时奖励均为 0。这意味着策略梯度信号几乎全部来自轨迹末端，导致学习极不稳定。最后，与围棋不同，推理行为的“专家过程”极难获取。人类可以留下棋谱，但无法轻易提供逐步的思考链条。因此，**离线模仿学习并不足以复现强推理能力**，更需要依赖规模化的在线强化学习，通过大规模采样—评估—更新的方式不断改进模型的思考策略。

因此，训练具有思考能力的大语言模型的问题是一个**状态空间极大、奖励延迟严重的在线强化学习问题**。

11.4 小结

可以看到，现实中的强化学习问题都仅提供灰盒的环境模型，且往往要求进行在线学习。同时，这些问题都具有状态空间巨大和奖励延迟的挑战。接下来，我们将对这类问题提出学习算法。

第十二章 灰盒环境模型下的无状态的在线强化学习问题

12.1 多臂老虎机在线学习算法

如前所述，多臂老虎机问题是一个无状态的在线强化学习问题。我们希望：一边从单次摇臂中获取奖励，一边进行策略改变，使得在 n 次摇臂之后获得的奖励最大。还需要指出的是，此问题中我们的折扣率 $\gamma = 1$ 。

由于该问题无状态，那么每次拉动老虎机获取奖励的事件都是独立的。根据动作价值函数的定义（公式10.4），则动作价值函数可简化为只有一个动作 a 的输入：

$$Q_{\pi}(a) = \sum_r p(r|a) \cdot r + \sum_{a'} \pi(a') \cdot Q_{\pi}(a') \quad (12.1)$$

$$= \mathbb{E}[R|A = a] + \mathbb{E}_{\pi}[Q_{\pi}] \quad (12.2)$$

上式的第一项实际是在采取动作 a 后，获得的即时奖励的均值；而第二项在策略 π 下获得的长期价值的均值，是一个与动作 a 无关的常量。因此，选择使得 $\mathbb{E}[R|A = a]$ 最大的动作，也就是说，选择获得即时奖励的均值最大的动作，就是最优的策略。

那么，问题的关键就是如何去估算这个均值 $\mathbb{E}[R|A = a]$ 。当然，我们可以用当前的策略 π 生成一批轨迹数据，然后更新对 $\mathbb{E}[R|A = a]$ 的估计，接着再根据最新的均值估计去获得更新后的策略。如果把这个策略更新的

频率增加到最大，即在每一次获得即时奖励之后都更新，我们就获得了如下的在线学习的算法：

Algorithm 6 多臂老虎机贪心在线学习算法

Require: K 台多臂老虎机

Initialize, for $a = 1$ to K :

$Q(a) \leftarrow$ a big enough constant

$N(a) \leftarrow 0$

Repeat n rounds:

$A \leftarrow \arg \max_a Q(a)$

$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

可以看到，这个算法每轮用贪心算法获取一个动作 A ，然后拉动 A 这台老虎机获取了即时奖励 R ；随后， A 这台老虎机被选中的次数增加 1；同时，用新获得的奖励 R 去更新执行动作 A 后获得的奖励的均值 $Q(A)$ ，实际就是所有执行动作 A 后获得的奖励的均值。这就是多臂老虎机贪心在线学习算法。需要注意的是，我们一开始将每个 $Q(a)$ 设置为一个足够大的常数，是为了使得在初始阶段每个动作都会被选择到；这是一种对奖励的乐观估计。

即便是使用了对 $Q(a)$ 的乐观初始化，仍然可能存在如下的问题：有些动作被选中的次数较少，而它对应的奖励的均值被低估了。因此，我们需要引入一定的随机性，这就是 ϵ 贪心在线学习算法（算法7）。在此算法中的每轮动作选择中，我们会以 ϵ 的概率随机选择一个动作，这是对可能的更好的动作的**探索** (Exploration)；以 $(1 - \epsilon)$ 的概率贪心的选择当前最好的动作，这是当前策略的一次**应用** (Exploitation)。参数 ϵ 的设置是对“探索与应用”的平衡。这是强化学习中的重要思想，特别是在在线的强化学习算法

中，我们都需要一边生成数据，一边更新策略；而在生成数据时，我们可以利用当前的策略来生成，或者去探索更多动作的可能。因此，我们需要做好“探索与应用”的权衡。

Algorithm 7 多臂老虎机 ϵ 贪心在线学习算法

Require: K 台多臂老虎机

Initialize, for $a = 1$ to K :

$Q(a) \leftarrow$ a big enough constant

$N(a) \leftarrow 0$

Repeat n rounds:

$A \leftarrow \begin{cases} \arg \max_a Q(a) & \text{with probability } 1 - \epsilon \quad (\text{breaking ties randomly}) \\ \text{a random action} & \text{with probability } \epsilon \end{cases}$

$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

进一步的，我们可以把 ϵ 贪心算法改进为置信上界 (UCB, Upper Confidence Bound) 贪心算法 (算法8)。在此算法中， c 是一个需要设置的参数；那些选中次数少的状态会有更大的概率被选中。一般来说，置信上界贪心算法优于 ϵ 贪心算法。

12.2 多臂老虎机问题的应用

我们现实中的一些实际问题都可以建模成多臂老虎机问题，例如：新闻的投放问题。假如一定时间内新产生了 K 条新闻，我需要投放给 n 个用户。那么，每条新闻分别该投放多少次，才能使得新闻的点击次数最高？

这里，可以把 K 条新闻看成 K 个老虎机，每向用户投放一次，相当于拉了一次这台老虎机。此时，若用户点击了这个新闻，则即时的奖励为 1，

Algorithm 8 多臂老虎机置信上界 (UCB, Upper Confidence Bound) 贪心
在线学习算法

Require: K 台多臂老虎机

Initialize, for $a = 1$ to K :

$Q(a) \leftarrow$ a big enough constant

$N(a) \leftarrow 0$

$t \leftarrow 0$

Repeat n rounds:

$t \leftarrow t + 1$

$A \leftarrow \arg \max_a [Q(a) + c\sqrt{\frac{\ln t}{N(a)}}]$

$R \leftarrow \text{bandit}(A)$

$N(A) \leftarrow N(A) + 1$

$Q(A) \leftarrow Q(A) + \frac{1}{N(A)}[R - Q(A)]$

否则奖励为 0。当然，我们可以使用之前介绍的在线学习算法，如 ϵ 贪心算法和置信上界贪心算法，来解决这个问题。显然，这些算法都是在评估相对于大众的点击概率，它并不考虑新闻的内容，也并不是个性化的推荐算法。

类似的，当我们面临 K 种选择，需要尽快的从中选择出收益最大的选项，这样的问题都可以建模为多臂老虎机问题。更多的例子包括：1) 产品 A/B 测试：测试多个界面设计（臂）的点击率（收益），快速找到最优设计（避免长时间测试低收益设计）；2) 推荐系统：向用户推荐多个商品（臂），根据点击/购买数据（收益）动态调整推荐策略，平衡“推荐热门商品（利用）”和“尝试新商品（探索）”；3) 广告投放：为多个广告素材（臂）分配曝光量，根据转化率（收益）优化投放比例，提升总转化；4) 临床试验：测试多种治疗方案（臂）的疗效（收益），在保证伦理的前提下，尽快将资源倾斜到更有效的方案。

第十三章 灰盒环境模型下有状态的在线强化学习问题

在上一节中，我们探讨了灰盒环境模型下无状态的在线强化学习问题，环境可以由 $p(r|a)$ 建模。而更一般的强化学习问题中，状态往往随行为改变，环境需要建模为 $p(s', r|s, a)$ 。在灰盒场景中，我们不再像白盒模型那样完全掌握环境的转移规律 $p(s', r|s, a)$ ，智能体只能通过与环境的交互逐步积累经验。

在这种情况下，一个自然的想法是：能否先学习环境模型本身？——也就是近似出 $p(s', r|s, a)$ ，然后和前面处理白盒环境的方式一样进行求解。然而，实践中人们往往更倾向于另一条更直接的路径：不再试图重建整个环境，而是直接去学习“怎样做才好”——即直接学习动作价值或策略。这样做通常更高效、更稳健，主要有以下两点原因：

- 近似整个环境模型往往比直接学习价值或策略复杂得多，对计算资源和数据量的要求也更高；
- 我们真正关心的目标是找到高回报的行为，而不是重现环境的全部细节。直接学习价值或策略能让模型更专注于最终目标。

因此，在灰盒环境下，强化学习通常沿着以下两条主要方向展开：

1. **价值学习 (Value-Based Learning)**: 通过近似动作价值函数 $Q(s, a)$ 来指导决策。智能体在不断的交互中，根据实际获得的经验更新 $Q(s, a)$

的估计值,使其逐步接近真实的期望回报。若已知 $Q(s, a)$, 每一步决策时, 动作通常按照 $a^* = \arg \max_a Q(s, a)$ 选取, 即选择价值最高的动作。

2. **策略学习 (Policy-Based Learning)**: 直接对策略函数 $\pi(a|s)$ 建模。智能体不再依赖显式的价值函数, 而是通过与环境交互获得的奖励反馈, 不断调整策略参数, 使策略能够产出更高的长期回报。

接下来的两节内容将分别介绍这两类方法的核心思想与典型算法。

13.1 价值学习

所谓价值学习, 是指通过估计在特定状态-动作对下的期望回报, 从而指导智能体选择最优动作。本章将依次介绍动作价值函数 (Q 函数) 的基本定义、如何建模与近似 Q 函数、基于时序差分的优化思想, 以及两种经典算法 (SARSA 与 Q-learning) 的原理与联系。

首先我们来简单回顾动作价值函数并介绍价值学习的主要思想。如式(11.1)所示的轨迹数据, 从起始到结束的奖励序列可表示为:

$$R_1, R_2, \dots, R_t, \dots, R_n, \quad (13.1)$$

其中 R_t 表示在第 t 步获得的奖励。定义折扣率 $\gamma \in [0, 1]$, 则折扣回报为:

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \dots + \gamma^{n-t} R_n. \quad (13.2)$$

在游戏尚未结束的时刻 t , U_t 是一个随机变量, 其不确定性来自于未来的状态、动作与奖励。动作价值函数定义为:

$$Q_\pi(s_t, a_t) = \mathbb{E}[U_t \mid S_t = s_t, A_t = a_t], \quad (13.3)$$

即在给定状态 s_t 和动作 a_t 的条件下, 按照策略 π 进行后续的动作选择所能获得的期望折扣回报。

最优动作价值函数通过最大化策略消去 π 的影响：

$$Q_{\star}(s_t, a_t) = \max_{\pi} Q_{\pi}(s_t, a_t), \quad \forall s_t \in \mathcal{S}, a_t \in \mathcal{A}. \quad (13.4)$$

它表示在状态 s_t 执行动作 a_t 后，所能达到的最大期望回报。

如果我们已知 $Q_{\star}$ ，就能依据它选择最优动作。例如，在超级玛丽游戏中，给定当前状态 s_t ，智能体可计算每个可选动作（{左, 右, 上}）的 $Q_{\star}(s_t, a)$ ，并选择价值最大的动作。直观地说， $Q_{\star}$ 就像“未来回报的预言者”：它告诉智能体在当前时刻做出不同选择所能获得的长期期望回报。价值学习的目标，正是通过与环境交互，不断近似这个“预言者”。接下来，我们介绍如何建模和近似动作价值函数。

如何建模与近似 Q

在离散状态与动作空间中，可用表格法直接存储 $Q(s, a)$ ：为每个状态—动作对设置一个值，并在交互后更新这个值，这种方法直观简单。但当状态空间巨大或连续时，表格法将不再可行。

为应对这种情况，现实任务通常采用神经网络近似 Q 函数，形成**深度 Q 网络 (Deep Q-Network, DQN)**，记作 $Q_{\theta}(s, a)$ 。其中 θ 表示神经网络的参数。当动作空间是离散集合（如 {左, 右, 上}）时，DQN 以当前状态 s 作为输入，输出所有动作对应的 Q 值向量：

$$[Q_{\theta}(s, a_1), Q_{\theta}(s, a_2), \dots, Q_{\theta}(s, a_{|\mathcal{A}|})]. \quad (13.5)$$

智能体据此选择动作 $a = \arg \max_a Q_{\theta}(s, a)$ 。这相当于用函数替代查表机制，使模型能处理连续状态并在相似状态间迁移经验。训练目标是优化参数 θ 使得网络预测的 $Q_{\theta}(s, a)$ 尽量接近真实的最优动作价值 $Q_{\star}(s, a)$ 。在下一节我们介绍优化 Q 网络的经典思想“时序差分”。

基于时序差分的优化方法

无论是表格形式还是神经网络近似，时序差分（Temporal Difference, TD）的核心思想都是利用“当前奖励 + 未来回报的估计”来更新当前的价值预测。下面以 Q-Learning 为例，推导其核心更新过程。更多利用 TD 思想的算法将在下节进行介绍和对比。

回顾折扣回报的定义：

$$U_t = \sum_{k=t}^n \gamma^{k-t} R_k, \quad U_{t+1} = \sum_{k=t+1}^n \gamma^{k-t-1} R_k, \quad (13.6)$$

因此可以得到递推关系：

$$U_t = R_t + \gamma U_{t+1}. \quad (13.7)$$

最优动作价值函数定义为：

$$Q_{\star}(s_t, a_t) = \max_{\pi} \mathbb{E}[U_t \mid S_t = s_t, A_t = a_t]. \quad (13.8)$$

结合上面的递推关系，我们可以写出：

$$Q_{\star}(s_t, a_t) = \max_{\pi} \mathbb{E}[R_t + \gamma U_{t+1} \mid S_t = s_t, A_t = a_t] \approx r_t + \gamma \max_{a \in \mathcal{A}} Q_{\star}(s_{t+1}, a). \quad (13.9)$$

这个式子表示：在时刻 t 执行动作 a_t 后，当前的最优价值近似等于“即时奖励”加上“下一时刻最优动作的折扣价值”。这是 TD 学习的核心思想——用当前奖励加上对未来的预测来更新当前的估计。

如前所述，在实际中我们通常用一个带参数的神经网络去近似 $Q_{\star}(s, a)$ 。于是我们令：

$$Q_{\theta}(s, a) \approx Q_{\star}(s, a), \quad (13.10)$$

又根据式 (13.9)，我们得到以下近似关系：

$$\underbrace{Q_{\theta}(s_t, a_t)}_{\hat{q}_t} \approx r_t + \underbrace{\gamma \max_{a \in \mathcal{A}} Q_{\theta}(s_{t+1}, a)}_{\hat{y}_t}. \quad (13.11)$$

上式中, 左边的 $\hat{q}_t \triangleq Q_\theta(s_t, a_t)$ 是神经网络在 t 时刻做出的预测, 其中没有任何事实成分; 右边的 TD 目标 $\hat{y}_t$ 是部分基于真实观测到的奖励 r_t 。 $\hat{q}_t$ 和 $\hat{y}_t$ 两者都是对最优动作价值 $Q_*(s_t, a_t)$ 的估计, 但是 $\hat{y}_t$ 部分基于事实, 因此比 $\hat{q}_t$ 更可信。应当鼓励 $\hat{q}_t$ 接近 $\hat{y}_t$ 。于是我们定义损失函数:

$$L(\theta) = \frac{1}{2} [Q_\theta(s_t, a_t) - \hat{y}_t]^2, \quad (13.12)$$

并通过最小化该损失 (使用梯度下降) 来更新网络参数 θ 。

对损失函数 $L(\theta)$ 求梯度, 可以得到参数更新方向。为了便于讨论, 假设将 $\hat{y}_t$ 视为常数, 有

$$\nabla_\theta L(\theta) = (Q_\theta(s_t, a_t) - \hat{y}_t) \nabla_\theta Q_\theta(s_t, a_t). \quad (13.13)$$

令

$$\delta_t \triangleq Q_\theta(s_t, a_t) - \hat{y}_t, \quad (13.14)$$

称为 TD 误差。它反映了当前预测与 TD 目标之间的差距: 若 $\delta_t > 0$, 说明网络的预测偏大; 若 $\delta_t < 0$, 说明预测偏小。

采用梯度下降更新参数 θ (使预测值更接近 TD 目标) 得到:

$$\theta \leftarrow \theta - \alpha \delta_t \nabla_\theta Q_\theta(s_t, a_t), \quad (13.15)$$

其中 α 是学习率。以上更新过程, 就是时序差分学习的基本形式。

时序差分学习的一般流程

在本节中我们给出基于时序差分思想的价值学习方法的一般流程。它们都遵循相似的思路: 利用行为策略与环境交互得到数据样本, 构造 TD 目标, 并让当前价值估计向该目标靠拢。需要注意的是, 我们关注的两个经典算法 Q-learning 与 SARSA 在采样与目标构造上略有不同。

时序差分的一般流程

1) **采样数据**: 智能体按照某个行为策略与环境交互:

- 对于 Q-learning, 得到四元组

$$(s_t, a_t, r_t, s_{t+1}); \quad (13.16)$$

- 对于 SARSA, 还需在 s_{t+1} 下根据同一行为策略采样下一动作 a_{t+1} , 得到五元组

$$(s_t, a_t, r_t, s_{t+1}, a_{t+1}). \quad (13.17)$$

2) **计算当前预测**: 利用当前的价值函数近似器 (表格或神经网络) 计算

$$\hat{q}_t = Q_\theta(s_t, a_t). \quad (13.18)$$

3) **构造 TD 目标**: 不同算法采用不同的 TD 目标:

$$\hat{y}_t = \begin{cases} r_t + \gamma \max_{a'} Q_\theta(s_{t+1}, a'), & \text{Q-learning,} \\ r_t + \gamma Q_\theta(s_{t+1}, a_{t+1}), & \text{SARSA.} \end{cases} \quad (13.19)$$

4) **计算 TD 误差**:

$$\delta_t = \hat{q}_t - \hat{y}_t. \quad (13.20)$$

5) **更新价值函数**: 利用梯度下降 (或表格更新)

$$\theta \leftarrow \theta - \alpha \delta_t \nabla_\theta Q_\theta(s_t, a_t), \quad (13.21)$$

其中 α 为学习率。

6) **重复上述步骤**：智能体不断与环境交互并更新价值函数，从而逐渐逼近真实的动作价值。

以上流程概括了所有基于 TD 的方法的共同结构：它们都通过“当前奖励 + 未来价值估计”构造更新目标，并使用该目标不断调整动作价值函数。

需要强调的是，该流程中的第一步，采用怎样的行为策略来采样数据是整个算法的关键。常见的行为策略包括：

- **ϵ -greedy 策略**：以概率 $1 - \epsilon$ 选择当前价值函数预测的最优动作，以概率 ϵ 随机选择任意动作，从而在“利用”与“探索”之间取得平衡；
- **Softmax (Boltzmann) 策略**：按照

$$\pi(a|s) \propto \exp(Q(s, a)/\tau) \quad (13.22)$$

对动作进行加权随机采样，其中温度参数 τ 控制探索程度；

- **确定性策略 + 噪声**（在连续动作中常用）：先选择确定性动作

$$a = \arg \max_a Q(s, a), \quad (13.23)$$

再加入小幅随机噪声以保持探索。

这些行为策略决定了智能体与环境交互时所收集的数据分布。

同时，值得注意的是，各类 TD 算法之间最根本的区别体现在它们如何构造 TD 目标 $\hat{y}_t$ 。不同的 TD 目标会直接影响算法的偏差、方差以及学习过程的稳定性。在下一节中，我们将重点比较 Q-learning 与 SARSA 算法在 TD 目标上的差异，并说明这些差异如何进一步影响它们的行为特性与收敛表现。

Q-learning 与 SARSA

在时序差分 (TD) 思想的基础上, 衍生出了两种经典的价值学习算法: **Q-learning** 和 **SARSA**。它们的核心思路相同, 但在更新目标上存在关键区别。

- **Q-learning (off-policy)**: 更新目标基于下一状态中最优可能动作的价值估计:

$$y_t = r_{t+1} + \gamma \max_{a'} Q_{\theta}(s_{t+1}, a'). \quad (13.24)$$

Q-learning 假设智能体在下一步总是会选择最优动作, 因此学习过程与当前执行的策略无关 (off-policy)。它直接逼近最优动作价值函数 $Q_{\star}(s, a)$, 收敛速度较快, 但由于假设较为理想化, 在高噪声 (如 reward 的获取存在噪声) 或高随机的环境中容易出现震荡甚至发散。

- **SARSA (on-policy)**: 更新目标依赖于智能体实际执行的下一个动作:

$$y_t = r_{t+1} + \gamma Q_{\theta}(s_{t+1}, a_{t+1}). \quad (13.25)$$

SARSA 学习的是当前策略下的期望回报。由于学习和执行使用同一策略 (on-policy), 更新过程更加平滑、稳定, 尤其在存在探索行为 (如 ϵ -greedy 策略) 时表现更可靠。

在选择算法时, 如果环境具有较强的随机性或采集数据噪声较大时, **SARSA** 通常更加稳健; 而当希望快速逼近最优策略, 或环境较为确定且采样成本较低时, **Q-learning** 通常效率更高。实际应用中, 人们也常在两者之间做折中, 例如在 Q-learning 中采用 ϵ -greedy 策略保持探索, 或使用 Double Q-learning 等改进算法, 以平衡学习的稳定性与收敛速度。

小结

本节介绍了价值学习的核心思想及其实现方法。我们首先讨论了如何用深度 Q 网络 (DQN) 近似动作价值函数, 并从时序差分 (TD) 思想出发, 介绍了 Q-learning 与 SARSA 两种经典算法。Q-learning 直接逼近最优动作价值函数, 而 SARSA 强调行为与目标策略的一致性。以上内容共同构成了强化学习中“价值学习”方法的基础。

13.2 策略学习

所谓策略学习, 是指直接学习最优策略函数或其近似形式 (如策略网络), 而不是像价值学习那样先估计动作价值函数 $Q(s, a)$ 再间接导出策略。与前文介绍的价值学习相比, 策略学习直接利用策略网络建模智能体在状态 s 下选择动作 a 的概率分布 $\pi(a|s; \theta)$, 并通过优化参数 θ 使长期回报最大化。本章将介绍策略网络、策略优化目标的定义、策略梯度的推导, 以及两种典型的策略学习算法——REINFORCE 与 Actor-Critic。

策略函数与策略网络

首先我们对策略进行建模。考虑一个离散动作空间, 例如在超级玛丽游戏中 $\mathcal{A} = \{\text{左}, \text{右}, \text{上}\}$ 。策略函数 π 定义为:

$$\pi(a|s) \triangleq \mathbb{P}(A = a | S = s), \quad (13.26)$$

即在状态 s 下选择动作 a 的概率。换句话说, 策略函数为每个状态 s 输出一个在动作空间上的概率分布。以超级玛丽为例, 若当前画面为状态 s , 策略函数会输出每个动作 (左、右、上) 的概率分布, 智能体据此进行采样, 得到要执行的动作 a 。

类似于价值学习中使用 DQN 来近似 Q 函数, 我们常使用参数化的神经网络来近似策略函数, 记作 $\pi(a|s; \theta)$, 称为**策略网络**。其中 θ 为网络参

数，初始时随机设定，随后利用与环境交互得到的状态、动作、奖励等数据不断更新，从而优化策略的表现。接下来我们介绍策略学习的学习目标。

策略学习的目标函数

在推导目标函数前，先回顾回报与价值函数的定义。折扣回报 U_t 为从时刻 t 开始获得的所有奖励的加权和：

$$U_t = R_t + \gamma R_{t+1} + \gamma^2 R_{t+2} + \cdots \quad (13.27)$$

动作价值函数定义为：

$$Q_\pi(s_t, a_t) = \mathbb{E}[U_t \mid S_t = s_t, A_t = a_t], \quad (13.28)$$

表示在状态 s_t 采取动作 a_t 后，按照策略 π 能获得的期望折扣回报。

进一步，状态价值函数定义为：

$$V_\pi(s_t) = \mathbb{E}_{A_t \sim \pi(\cdot \mid s_t; \theta)}[Q_\pi(s_t, A_t)], \quad (13.29)$$

即在状态 s_t 下，遵循策略 π 的期望回报。显然， $V_\pi(s_t)$ 既依赖当前状态，也依赖于策略参数 θ 。若状态 s_t 越有利，或策略越优，则 $V_\pi(s_t)$ 越大。

若一个策略整体较好，那么对于所有状态，其状态价值的平均应当较高。由此可定义策略学习的目标函数：

$$J(\theta) = \mathbb{E}_S[V_\pi(S)], \quad (13.30)$$

即在所有可能状态下，策略所能获得的平均价值。该目标仅依赖于策略参数 θ 。策略学习的任务，便是通过优化 θ 使 $J(\theta)$ 最大化：

$$\max_{\theta} J(\theta). \quad (13.31)$$

通过梯度上升可更新策略参数：

$$\theta_{\text{new}} \leftarrow \theta_{\text{now}} + \beta \nabla_{\theta} J(\theta_{\text{now}}), \quad (13.32)$$

其中 β 为学习率。梯度

$$\nabla_{\theta} J(\theta_{\text{now}}) \triangleq \left. \frac{\partial J(\theta)}{\partial \theta} \right|_{\theta=\theta_{\text{now}}} \quad (13.33)$$

称为**策略梯度**。接下来我们推导策略梯度的具体形式。

策略梯度的简化推导

首先我们给出结论，策略梯度可以表示为如下的期望形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_S \left[\mathbb{E}_{A \sim \pi(\cdot|S;\theta)} \left[\nabla_{\theta} \ln \pi(A|S;\theta) \cdot Q_{\pi}(S, A) \right] \right]. \quad (13.34)$$

完整的推导较为复杂，本节将给出该表达式的简化推导，帮助读者直观理解。

将策略函数 $\pi(a|s;\theta)$ 视为动作的概率质量函数。状态价值函数可以写为：

$$V_{\pi}(s) = \mathbb{E}_{A \sim \pi(\cdot|s;\theta)} [Q_{\pi}(s, A)] = \sum_{a \in \mathcal{A}} \pi(a|s;\theta) Q_{\pi}(s, a). \quad (13.35)$$

对策略参数 θ 求导，可得：

$$\frac{\partial V_{\pi}(s)}{\partial \theta} = \sum_{a \in \mathcal{A}} \frac{\partial \pi(a|s;\theta)}{\partial \theta} Q_{\pi}(s, a) + \sum_{a \in \mathcal{A}} \pi(a|s;\theta) \frac{\partial Q_{\pi}(s, a)}{\partial \theta}. \quad (13.36)$$

上式第二项较难处理，因为 $Q_{\pi}(s, a)$ 还依赖于策略本身。在常见的近似推导中忽略该项，从而得到简化形式：

$$\frac{\partial V_{\pi}(s)}{\partial \theta} \approx \sum_{a \in \mathcal{A}} \frac{\partial \pi(a|s;\theta)}{\partial \theta} Q_{\pi}(s, a). \quad (13.37)$$

注意到

$$\frac{\partial \pi(a|s;\theta)}{\partial \theta} = \pi(a|s;\theta) \frac{\partial \ln \pi(a|s;\theta)}{\partial \theta}, \quad (13.38)$$

上式可改写为：

$$\frac{\partial V_{\pi}(s)}{\partial \theta} \approx \sum_{a \in \mathcal{A}} \pi(a|s;\theta) \frac{\partial \ln \pi(a|s;\theta)}{\partial \theta} Q_{\pi}(s, a), \quad (13.39)$$

即

$$\frac{\partial V_{\pi}(s)}{\partial \theta} \approx \mathbb{E}_{A \sim \pi(\cdot|s; \theta)} \left[\nabla_{\theta} \ln \pi(A|s; \theta) Q_{\pi}(s, A) \right]. \quad (13.40)$$

最后，对所有状态取期望，便得到策略梯度的一般形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E}_S \left[\mathbb{E}_{A \sim \pi(\cdot|S; \theta)} \left[\nabla_{\theta} \ln \pi(A|S; \theta) Q_{\pi}(S, A) \right] \right]. \quad (13.41)$$

然而，这个期望无法解析计算，因为我们通常并不知道状态 S 的概率密度函数；即便已知，直接通过积分或求和计算其期望在高维空间中也极为昂贵。因此，实际训练中我们采用采样近似的方法。

每当智能体与环境交互时，观测到的一个状态 s 可以视作随机变量 S 的一次采样。然后，依据当前策略网络（其参数为最新的 θ ）从策略分布中随机采样一个动作：

$$a \sim \pi(a|s; \theta). \quad (13.42)$$

由此，我们定义一个随机梯度估计量：

$$g(s, a; \theta) \triangleq Q_{\pi}(s, a) \cdot \nabla_{\theta} \ln \pi(a|s; \theta). \quad (13.43)$$

显然， $g(s, a; \theta)$ 是策略梯度 $\nabla_{\theta} J(\theta)$ 的无偏估计，即它在期望意义上与真实梯度一致。

但在实际操作中，我们仍面临一个关键问题：动作价值函数 $Q_{\pi}(s, a)$ 的真实值未知。为此，我们介绍两种经典的近似思路：

1. **REINFORCE 方法**：直接使用一次完整轨迹中观测到的实际回报 U_t 来近似 $Q_{\pi}(s, a)$ ；
2. **Actor-Critic 方法**：使用另一个神经网络 $Q(s, a; w)$ （称为评论家 *Critic*）来近似 $Q_{\pi}(s, a)$ 。

接下来我们首先给出策略学习的一般流程，然后分别介绍这两种方法及其在策略优化中的实现方式。

策略学习的一般流程

在前文中，我们推导了策略梯度的基本形式：

$$\nabla_{\theta} J(\theta) = \mathbb{E} \left[\nabla_{\theta} \ln \pi(A|S; \theta) Q_{\pi}(S, A) \right], \quad (13.44)$$

但由于真实的 $Q_{\pi}(s, a)$ 难以获得，我们必须通过采样与近似来构建可计算的更新方式。无论采用 REINFORCE 还是 Actor-Critic，它们都遵循一套共同的策略学习流程，其核心思想是：通过策略网络采样动作，与环境交互收集回报，构造策略梯度估计，进而更新策略参数。这一流程可以概括如下：

策略学习的一般流程

- 1) **从策略中采样动作：**给定当前状态 s_t ，根据策略网络采样动作：

$$a_t \sim \pi(\cdot | s_t; \theta). \quad (13.45)$$

- 2) **与环境交互并收集反馈：**执行动作获得奖励 r_t 和下一状态 s_{t+1} 。这一步提供了策略优化所需的全部经验信息。

- 3) **构造策略梯度估计：**对 $Q_{\pi}(s_t, a_t)$ 进行近似，计算策略梯度估计：

$$g_t \approx Q_{\pi}(s_t, a_t) \cdot \nabla_{\theta} \ln \pi(a_t | s_t; \theta). \quad (13.46)$$

不同算法的区别，就在于如何近似 $Q_{\pi}(s_t, a_t)$ ：REINFORCE 使用完整轨迹的回报 U_t ；Actor-Critic 使用 Critic 网络 $Q(s, a; \mathbf{w})$ 的估计值。

- 4) **更新策略参数：**使用随机梯度上升更新策略网络：

$$\theta \leftarrow \theta + \beta g_t. \quad (13.47)$$

5) **重复上述步骤**：智能体不断与环境交互并更新策略，从而逐渐逼近最优策略。

可以看到，策略学习的核心在于：**通过采样近似策略梯度，不断更新策略网络，使其倾向于产生高回报的动作**。REINFORCE 与 Actor-Critic 的区别，正是体现在步骤 3) 的不同近似方式上，下面我们将分别介绍这两种方法的具体实现。

近似策略梯度

REINFORCE 与 Actor-Critic 本质上都在解决同一个问题：如何在不知道真实动作价值 $Q_\pi(s, a)$ 的情况下，近似地计算并优化策略梯度。二者的区别在于所采用的近似方式不同：REINFORCE 直接使用经验回报进行估计，而 Actor-Critic 则利用一个可学习的“评论家”网络提供更平滑、更稳定的价值近似。

1) REINFORCE REINFORCE 是最直接的策略梯度算法，它利用**经验回报**来近似动作价值函数，即用从时刻 t 开始直到回合结束的折扣累计奖励代替 $Q_\pi(s_t, a_t)$ ：

$$Q_\pi(s_t, a_t) \approx u_t = r_t + \gamma r_{t+1} + \gamma^2 r_{t+2} + \dots \quad (13.48)$$

对于每一次完整轨迹中的每个决策点 (s_t, a_t) ，我们使用经验回报 u_t 构造策略梯度估计：

$$g_t = u_t \nabla_{\theta} \ln \pi(a_t | s_t; \theta). \quad (13.49)$$

由于 u_t 涉及到从 t 到轨迹结束的所有随机奖励，它本身是一个随机变量，其方差

$$\text{Var}(u_t | s_t, a_t) = \mathbb{E}[(u_t - \mathbb{E}[u_t])^2 | s_t, a_t] \quad (13.50)$$

通常非常大，原因包括：

- 回报累积了多个时间步的随机性，时间越长，不确定性越大；
- 后续状态、动作、奖励均带有噪声，使得 u_t 波动明显；
- 当 γ 接近 1 时，远期奖励对 u_t 的影响很大，使波动进一步放大。

因此式 (13.49) 中的梯度 g_t 也呈现高方差，更新过程常常不稳定，导致学习效率较低。为减小梯度估计的方差，REINFORCE 引入一个 **baseline** b ，并用

$$g_t = (u_t - b) \nabla_{\theta} \ln \pi(a_t | s_t; \theta) \quad (13.51)$$

替代式 (13.49)。这种引入方式的关键性质是：只要 b 不依赖动作 a_t ，则

$$\mathbb{E}_{a_t \sim \pi(\cdot | s_t)} [b \nabla_{\theta} \ln \pi(a_t | s_t; \theta)] = b \nabla_{\theta} \sum_a \pi(a | s_t; \theta) = b \nabla_{\theta} 1 = 0, \quad (13.52)$$

因此 baseline 不改变梯度的期望值。选取恰当的 baseline 能显著降低方差。我们在此介绍最直观、最容易实现的 baseline：

- **常数 baseline：**

$$b = c, \quad (13.53)$$

其中 c 为一个固定常数（如基于统计的回报均值或或其他经验值）。这种方式简单且能降低一部分方差。

- **批平均 baseline：**在一次训练迭代中，假设我们从当前策略中采样了 N 条轨迹。第 i 条轨迹的长度为 $T^{(i)}$ ，在其中的每一个决策时刻 t 都可以计算出对应的经验回报 $u_t^{(i)}$ 。批平均 baseline 定义为本批次中所有轨迹、所有决策点经验回报的平均值：

$$b = \frac{1}{M} \sum_{i=1}^N \sum_{t=1}^{T^{(i)}} u_t^{(i)}, \quad (13.54)$$

其中

$$M = \sum_{i=1}^N T^{(i)} \quad (13.55)$$

表示本批次中所有决策点的总数。这样得到的 b 可以看作是“在当前策略下，一个典型决策点的平均回报”的估计。实际更新时，将经验回报 $u_t^{(i)}$ 替换为

$$u_t^{(i)} - b,$$

使得用于更新的信号在整个批次上的平均值接近 0，从而显著降低方差。直观地说，此时梯度更新依赖的是“当前动作比批次平均水平好多少或差多少”（相对优势），而不是回报的绝对大小。由于 b 与具体选择的动作无关，这一处理不会改变策略梯度的期望值，却能有效减小其波动，使 REINFORCE 的训练更加稳定。

REINFORCE 使用最直接的蒙特卡洛方式近似 $Q_\pi(s, a)$ ，概念清晰，实现简单；然而经验回报的高方差导致学习效率低下。通过引入 baseline，REINFORCE 能够显著降低方差而保持无偏性，使训练更加稳定。

2) Actor–Critic 与 REINFORCE 类似，Actor–Critic 同样基于策略梯度思想，但不同的是，它不再直接使用整条轨迹的经验回报 u_t 来近似动作价值函数，而是引入一个**评论家网络 (Critic)** 来对 $Q_\pi(s, a)$ 进行估计，从而使训练更高效、更稳定。

该方法包含两个可学习的网络：

- **Actor (行为者)**：策略网络 $\pi(a|s; \theta)$ ，根据当前状态 s_t 输出各个动作的概率分布；
- **Critic (评论家)**：价值网络 $Q(s, a; w)$ ，用于估计在状态 s 下执行动作 a 所能获得的期望回报。

在训练过程中，智能体通过策略网络采样动作

$$a_t \sim \pi(\cdot | s_t; \theta), \quad (13.56)$$

并与环境交互获得奖励 r_t 和下一状态 s_{t+1} 。Critic 使用时序差分思想更新参数，使其预测值更贴近实际回报：

$$\delta_t = r_t + \gamma Q(s_{t+1}, a_{t+1}; \mathbf{w}) - Q(s_t, a_t; \mathbf{w}), \quad (13.57)$$

$$\mathbf{w} \leftarrow \mathbf{w} + \beta_c \delta_t \nabla_{\mathbf{w}} Q(s_t, a_t; \mathbf{w}), \quad (13.58)$$

其中 δ_t 是 TD 误差，反映了 Critic 当前估计与经验之间的差距。

Actor 则利用 Critic 提供的当前动作价值 $Q(s_t, a_t; \mathbf{w})$ 来更新策略：

$$\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} + \beta_a Q(s_t, a_t; \mathbf{w}) \nabla_{\boldsymbol{\theta}} \ln \pi(a_t | s_t; \boldsymbol{\theta}). \quad (13.59)$$

这样，若 Critic 认为当前动作的价值高，策略网络会增加其在相似状态下被选择的概率；反之则减少。

与 REINFORCE 相比，Actor-Critic 不再依赖整条轨迹的累计回报，而是利用 Critic 的局部估计来进行更频繁、更稳定的更新，从而显著降低了方差并加快收敛。它也为后续的深度强化学习算法奠定了基础。

小结

本节介绍了策略学习的核心思想及其主要实现方法。我们首先介绍了如何通过策略网络直接表示策略函数，并从策略学习的目标出发，推导了策略梯度的基本形式。随后介绍了两种典型的策略学习算法：REINFORCE 和 Actor-Critic。前者利用实际回报直接估计策略梯度，方法简单但存在高方差的问题；后者引入评论家网络对动作价值进行近似，实现更稳定、高效的更新。以上内容共同构成了强化学习中“策略学习”方法的基础。

13.3 强化学习在现实中的应用：围棋

在前两节中，我们系统介绍了强化学习中的两大核心思想：价值学习与策略学习。价值学习通过时序差分近似最优动作价值函数，并以此指导

动作选择；策略学习则借助策略梯度直接优化策略网络，使其倾向于选择高回报的动作。本节以一个兼具复杂性与现实意义的典型案例——**围棋程序 AlphaGo**——展示如何将这些理论真正落地到实际系统之中。AlphaGo 正是凭借强化学习与搜索技术的结合，在 2016 年战胜世界冠军李世石，首次让人工智能在围棋这一顶级复杂博弈中超越人类，成为人工智能发展史上的里程碑。

我们将依次说明：如何将围棋建模为强化学习问题、AlphaGo 的整体结构，以及其中策略网络、价值网络与蒙特卡洛树搜索（Monte Carlo Tree Search, MCTS）是如何协同工作的。

围棋如何建模为强化学习问题

尽管在第11.2节我们从强化学习视角对围棋进行了简单的建模，在这里我们给出更具体和深入的分析。要让强化学习算法处理现实任务，第一步总是将任务形式化为一个马尔可夫决策过程（Markov Decision Process, MDP）。围棋作为一个回合制双人零和博弈，可以自然地对应为以下 MDP 结构：

- **状态 s** ：当前棋盘局面，包括黑白棋子的位置、轮到谁走等信息。
- **动作 a** ：在任一合法位置落子。动作空间为所有当前允许的落子点。
- **策略 $\pi(a|s)$** ：在局面 s 下选择各个合法动作的概率。
- **奖励 r** ：对弈结束时给出最终结果：胜为 +1，负为 -1；中间局面奖励通常为 0。

这样，一盘围棋从初始布局出发，双方轮流落子直到终局为止，形成一条包含状态、动作与最终胜负的完整轨迹。强化学习的目标，就是学习一个策略 $\pi(a|s; \theta)$ ，使智能体在自对弈与实战中尽可能提高自己的胜率。

围棋场景的挑战

虽然围棋可以自然建模为 MDP，但它所面临的难度远远超过常见的强化学习任务，主要体现在以下几个方面：

- **状态空间极其庞大**：围棋的可能局面数量约为 10^{500} ，远超过国际象棋乃至宇宙中原子的数量，无法用表格或简单函数表示。即时是用大规模参数的神经网络去记忆价值函数，也需要采样对学习更有帮助的状态进行参数优化。
- **回报稀疏且延迟极长**：一盘棋通常需要上百步，而只有对局结束时才会给出胜负奖励，中间没有的即时奖励都为 0。
- **策略间强对抗性**：围棋是零和博弈，对手的策略会根据局面不断调整，使得环境分布呈现强非平稳性，增加了学习难度。例如在 AlphaGo 对阵李世石的第四局中，李世石通过第 78 手（著名的“神之一手”）主动将局面引入非主流的复杂变化，成功打乱 AlphaGo 的策略评估并赢下了唯一的一局。

这些挑战意味着：单纯依赖前述章节价值学习或策略学习的任一方法都难以获得强性能。AlphaGo 的突破关键正是在于将策略网络、价值网络与蒙特卡洛树搜索有机结合，把学习与搜索的优势相互补足，从而在极其庞大的搜索空间中高效逼近最优对弈策略。

AlphaGo Zero 的整体框架

本节我们以 AlphaGo Zero 为例进行说明。相较于依赖大量人类棋谱的 2016 年版 AlphaGo，AlphaGo Zero 的设计思想更为“纯粹”：不给任何人类示范，不依赖外部知识，只通过自我博弈与学习实现完全从零开始的自我进化。这一结构不仅更简单，而且最终棋力远超前代版本。整个系统可概括为下述三个核心步骤：

1) **构造一个统一的策略-价值网络**：AlphaGo Zero 不再从人类专家棋谱中学习“人类会怎么下”，而是直接随机初始化一个神经网络 f_θ 。其动机在于：

- 专家棋谱固然强，但也包含偏好、习惯、错误，可能限制学习上限；
- 从零开始让系统只依赖游戏规则，可以避免人为偏见，使策略更自由地演化。

网络接受棋局状态 s 作为输入，同时输出策略分布 p 与价值估计 v ：

$$f_\theta(s) = (p, v). \quad (13.60)$$

2) **在自我博弈中使用 MCTS 产生训练数据（搜索参与学习，而非仅在对弈时使用）**：与早期 AlphaGo 依赖策略网络自身给出落子不同，AlphaGo Zero 在训练阶段就让蒙特卡洛树搜索（MCTS）参与决策。其动机在于：

- 刚初始化的策略网络几乎等同于随机落子，如果直接自对弈，会产生质量极差的训练样本，难以推动学习；
- MCTS 通过模拟多种未来走法，可以在弱网络的基础上给出更接近正确、更具规划性的落子，从而让早期的训练数据也具有明确的学习价值。

具体来说，在每一个局面 s_t 中，AlphaGo Zero 使用当前的策略-价值网络 f_θ 引导 MCTS 展开搜索。搜索结束后，每个动作 a 都会获得一个访问次数 $N(s_t, a)$ ，表示“在搜索中，这一步被当作好棋尝试了多少次”。这些访问次数构成了一个比原始网络输出 p 更优质的策略分布：

$$\pi_t(a) \propto N(s_t, a). \quad (13.61)$$

这里的 π_t 被称为**改进策略**，表示经过 MCTS 深度推演后，系统认为每个落子应占多大的概率。它是比原始网络策略更强的“搜索策略”，代表 MCTS 对所有动作的综合判断，而不是单个动作的选择结果。

对弈结束后会得到最终胜负结果 $z \in \{-1, +1\}$ 。整局对弈中所有的三元组：

$$(s_t, \pi_t, z)$$

都被加入训练集，供下一步更新网络参数使用。

3) **用搜索生成的数据训练网络：**每一轮训练网络的目标，是让网络输出 (p, v) 去逼近搜索策略与真实胜负标签 (π_t, z) 。动机在于：

- 搜索得到的 π_t 通常比原始策略 p 强，因此网络通过拟合 π_t 便能“升级”为更强策略；
- 网络越强，下一轮搜索得到的策略也越强，从而形成不断上升的正反馈闭环。

通过反复执行“自我博弈 → 模型训练”，策略能力不断提升，最终在无人类专家监督的条件下达到超越顶尖人类棋手的水平。

接下来我们将主要介绍其中的两个关键技术点：策略-价值网络的形式和训练方式，以及如何通过 MCTS 与策略-价值网络协作进行策略搜索和优化。

统一的策略-价值网络：同时学习“下哪儿”和“谁能赢”

在这一节我们将介绍统一的策略-价值网络的具体形式，并介绍在已知训练数据的情况下网络是如何训练的。训练数据的获取将在下节详细介绍。

与早期 AlphaGo 分别训练策略网络 $\pi(a|s)$ 和价值网络 $V(s)$ 不同，AlphaGo Zero 使用一个统一的网络 $f_\theta(s) = (p, v)$ 同时输出策略和价值。网络输入是当前棋盘局面 s 的特征编码（例如多通道的 19×19 棋盘平面），网络主体由多层残差卷积块组成，最后分成两个输出头：

- **策略头 (policy head)：**输出一个在所有位置上的概率分布 $p(a|s; \theta)$ ，表示在当前局面下落在每个位置的倾向；

- **价值头 (value head)**: 输出一个标量 $v(s; \theta) \in [-1, 1]$, 估计从该局局面出发本方最终胜负结果的期望值。

在这里我们假设已经通过自我博弈得到了训练样本 (s_t, π_t, z) (训练样本的生成流程将在下一节详细介绍)。其中:

- π_t 是在局面 s_t 上通过蒙特卡洛树搜索得到的搜索策略分布, 反映了“经过深度推演后的动作概率分配”;
- z 是整盘棋的最终胜负结果 (本方胜为 +1, 负为 -1), 作为从局面 s_t 开始的真实价值标签。

策略-价值网络的训练目标, 是让策略头输出

$$p_t = p(\cdot | s_t; \theta)$$

尽量接近 MCTS 给出的搜索策略 π_t , 同时让价值头输出

$$v_t = v(s_t; \theta)$$

尽量接近最终胜负结果 z 。典型的损失函数形式为:

$$L(\theta) = (z - v_t)^2 - \pi_t^\top \log p_t + \lambda \|\theta\|^2, \quad (13.62)$$

其中第一项是价值预测误差, 第二项是策略的交叉熵损失, 第三项是参数正则项。

通过最小化 $L(\theta)$, 网络在两个方向上同时得到强化:

- **策略改进**: 策略头不断向 MCTS 给出的搜索策略 π_t 靠近, 使下一轮自对弈中网络本身就更接近“搜索后的最佳落子分布”;
- **价值评估提升**: 价值头学习逼近真实结果 z , 使其对局面好坏的判断越来越准确, 从而在后续 MCTS 的叶子节点评估中发挥更高质量的作用。

以上内容构成了 AlphaGo Zero 的核心学习过程。接下来我们介绍在给定策略-价值网络的情况下如何利用蒙特卡洛树搜索，通过“多向后看几步”，推演出更优的策略。

蒙特卡洛树搜索：用网络引导和评估搜索

虽然网络本身已经具备较强的下棋能力，但要在复杂局面中做出高质量决策，仅依赖单步预测往往不够。AlphaGo Zero 通过蒙特卡洛树搜索 (Monte Carlo Tree Search, MCTS)，将“前瞻搜索”与网络给出的策略与价值估计结合起来，从而显著提升决策质量。不同于早期 AlphaGo 版本仅在推理时应用 MCTS，AlphaGo Zero 也将 MCTS 应用在了训练数据的构造流程中。下面我们首先介绍一次 MCTS 模拟的四个阶段，再介绍如何基于多次 MCTS 模拟进行推理决策和训练数据构造。

一次 MCTS 模拟的四个阶段 在当前局面 s_0 下，每一次 MCTS 模拟大致包含四个步骤：选择 (Selection)、扩展 (Expansion)、求值 (Evaluation)、回传 (Backup)。介绍如下：

1) 选择 (Selection)：

从根节点 s_0 开始，沿着搜索树不断向下选择动作，直到到达一个尚未展开或需要扩展的节点。围棋每一步通常有几十甚至上百个合法动作，不可能对所有动作逐一进行深度搜索，因此需要优先选择“看起来更有希望”的动作进入搜索。

在某个节点 s 上，对每个可行动作 a 定义综合评分：

$$\text{score}(s, a) = Q(s, a) + \frac{\eta}{1 + N(s, a)} \pi(a | s; \theta), \quad (13.63)$$

其中：

- $N(s, a)$ 是在搜索过程中，从 s 经过动作 a 的访问次数；

- $Q(s, a)$ 是此前多次模拟得到的平均价值估计;
- $\pi(a | s; \theta)$ 是策略网络给出的先验概率;
- η 是控制探索强度的超参数。

这个评分公式可以这样理解:

- 若动作 a 从未被访问过, 则 $N(s, a) = 0$ 且 $Q(s, a) = 0$, 此时

$$\text{score}(s, a) = \eta \pi(a | s; \theta),$$

即评分完全由策略网络的先验判断决定, 优先探索网络认为更有潜力的动作;

- 若动作 a 已被访问多次, 则 $N(s, a)$ 较大, 使先验项的权重变小, 此时

$$\text{score}(s, a) \approx Q(s, a),$$

决策更多地依赖搜索过程中积累的经验价值;

- 因为访问次数越小时系数 $\frac{1}{1+N(s, a)}$ 越大, 访问次数少的动作会得到更高的探索加成, 这在“利用已有好招”和“探索潜在好招”之间起到平衡作用。

在节点 s 上, 对所有合法动作 a 计算 $\text{score}(s, a)$, 选择得分最高的动作, 并沿该分支向下继续前进。需要注意的是, 这里的“执行”仅发生在搜索树的虚拟对弈中, 并不会真的在棋盘上落子。

2) 扩展 (Expansion):

选择步骤最终会到达一个尚未完全展开的叶节点 s_{leaf} 。此时, MCTS 会在该节点上调用当前的策略网络

$$\pi(\cdot | s_{\text{leaf}}; \theta), \quad (13.64)$$

将其输出作为从 s_{leaf} 出发各个动作的先验概率。对所有合法动作 a ，初始化：

$$P(s_{\text{leaf}}, a) = \pi(a | s_{\text{leaf}}; \boldsymbol{\theta}), \quad N(s_{\text{leaf}}, a) = 0, \quad Q(s_{\text{leaf}}, a) = 0. \quad (13.65)$$

这一步相当于在搜索树中为局面 s_{leaf} 添上新的子节点及其统计量，使搜索树向前扩展了一层。由于在搜索过程中双方轮流落子，无论是我方还是对手的走棋，都是由同一策略网络给出先验概率，因此整个搜索可以看作是在当前策略下的一次“虚拟自对弈”。

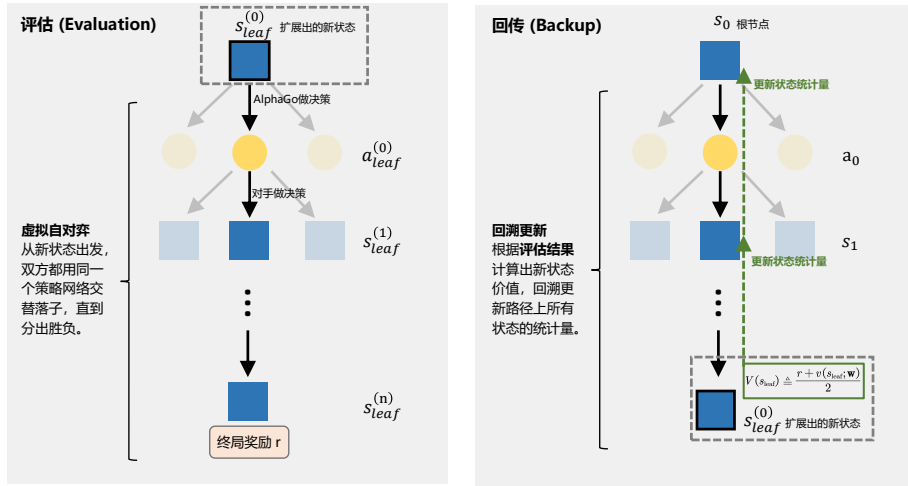


图 13.1: MCTS 中评估和回传阶段的示意图

3) 评估 (Evaluation):

扩展出新的叶节点 s_{leaf} 后，需要评估该局面的好坏。如图13.1所示，AlphaGo 的做法是，从 s_{leaf} 出发，在模拟器中继续进行一段“虚拟自对弈”，并结合价值网络的预测，得到对 s_{leaf} 的价值估计。

具体地，从状态 s_{leaf} 开始，双方交替落子，在每一个中间状态 s_k ：

$$a_k \sim \pi(\cdot | s_k; \boldsymbol{\theta}), \quad (13.66)$$

即由当前策略网络采样动作，推进到下一个状态 s_{k+1} 。对手一方同样使用同一个策略网络，只是从对手视角观察局面进行决策。这样从 s_{leaf} 一直模拟到终局，得到本局虚拟对弈的胜负结果

$$r \in \{-1, +1\}, \quad (13.67)$$

其中 $r = +1$ 表示从 s_{leaf} 出发当前一方最终赢棋， $r = -1$ 表示最终输棋。这个 r 是对局面 s_{leaf} 最直接、可靠的评价，但由于整盘对弈中存在大量随机抽样，单次结果的方差较大。

与此相对，价值网络 $v(s; \mathbf{w})$ 会对 s_{leaf} 给出一个平滑、稳定的价值预测：

$$v(s_{\text{leaf}}; \mathbf{w}) \in [-1, 1], \quad (13.68)$$

表示在当前策略下从该局面出发的胜负期望。它的随机性较小，却可能带有一定偏差。AlphaGo 将二者结合，得到综合的节点价值：

$$V(s_{\text{leaf}}) \triangleq \frac{r + v(s_{\text{leaf}}; \mathbf{w})}{2}, \quad (13.69)$$

既利用了终局胜负提供的“强监督”，又借助价值网络降低了估计的方差。这个 $V(s_{\text{leaf}})$ 就是本次模拟对叶节点的评估结果。

在实际系统中，为了提升效率，AlphaGo 在上述“虚拟自对弈”环节通常会使用一个更轻量的策略网络，以加快每一步的决策速度；而在选择、扩展阶段则使用更强的策略网络，以保证搜索方向的质量。

4) 回传 (Backup):

在评估步骤得到叶节点的价值 $V(s_{\text{leaf}})$ 后，需要将这一结果沿着本次模拟经过的整条路径（从叶节点一直回溯到根节点）向上回传，更新路径上所有边 (s, a) 的统计量，如图13.1所示。

对于路径上的每一个状态-动作对 (s, a) ，执行：

$$N(s, a) \leftarrow N(s, a) + 1, \quad (13.70)$$

$$Q(s, a) \leftarrow \frac{N(s, a) - 1}{N(s, a)} Q(s, a) + \frac{1}{N(s, a)} V(s_{\text{leaf}}), \quad (13.71)$$

上述更新可以看作对平均价值的逐次更新：当某个 (s, a) 被访问得越多，其平均价值 $Q(s, a)$ 就越稳定；当访问次数较少时，新得到的 $V(s_{\text{leaf}})$ 会对 $Q(s, a)$ 产生更大的影响。回传过程的含义是：每一次虚拟自对弈的结果都会被“写回”到搜索树中，逐步积累对各个分支优劣的经验判断，使得后续模拟在选择阶段能够更聪明地分配搜索资源。

以上介绍了一次 MCTS 模拟的流程。重复若干次模拟之后，根节点 s_0 上的每个动作 a 都会累积出相对可靠的访问次数 $N(s_0, a)$ 与平均价值估计 $Q(s_0, a)$ 。它们综合反映了在当前网络与搜索深度下，各种候选落子的探索程度与优劣判断。

基于访问次数的落子决策与训练信号 在实际对弈时，AlphaGo 系统通常不会直接选择策略网络给出概率最高的动作，而是基于搜索后的访问次数作出最终决策：

$$a^* = \arg \max_a N(s_0, a). \quad (13.72)$$

访问次数 $N(s_0, a)$ 体现了“搜索后的信心”，综合了策略先验与多轮模拟的经验统计，比单纯依赖网络原始输出更稳健可靠。

在训练阶段，更重要的是，各个动作的访问次数经过归一化后：

$$\pi(a | s_0) \propto N(s_0, a) \quad (13.73)$$

被视为“搜索改进后的策略分布”，通常远优于原始策略网络给出的 $\pi(a | s_0; \theta)$ 。因此，策略头以 $\pi(a | s_0)$ 作为监督信号进行训练，使下一轮网络的输出更接近“搜索后的决策质量”；而整盘棋的最终胜负结果 z 则用来训练价值网络。由此形成一个自我强化的闭环：

网络为搜索提供先验与评估 $\Rightarrow$ 搜索产生更优的策略与价值估计 $\Rightarrow$
搜索结果反过来训练网络

经过不断迭代，这一闭环使策略与价值估计同步提升，让系统在完全不依赖人类棋谱的前提下，通过自我博弈不断跃升棋力。

小结

本节以 AlphaGo Zero 为例，展示了如何将策略学习与价值学习在一个统一的系统中实现闭环优化：

- 将围棋建模为 MDP，定义状态、动作、策略与终局奖励，使“赢棋”可以作为长期回报的优化目标；
- 使用一个统一的策略-价值网络 $f_{\theta}(s) = (p, v)$ ，同时输出“下哪儿”的概率和“当前局面好坏”的评估；
- 通过自我博弈与 MCTS 生成训练数据 (s, π, z) ，其中 π 来自搜索得到的改进策略， z 是真实对局结果；
- 利用这些数据反向训练网络，使其策略输出逼近搜索策略，价值输出逼近真实胜负，从而在“自我对弈-学习”的循环中不断提升棋力。

与依赖人类棋谱的早期版本不同，AlphaGo Zero 完全不依赖人类先验，而是通过自我博弈和强化学习从零开始习得超越人类的围棋水平。这一范式表明，在合适的建模和计算资源支持下，强化学习系统可以在复杂博弈中实现真正意义上的“自我进化”。

13.4 强化学习在现实中的应用：训练具有思考能力的大语言模型

在上一节中，我们以围棋程序 AlphaGo Zero 为例，展示了如何通过“自我博弈—模型训练”构建一个能够在复杂博弈中实现自我进化的智能体。本

节我们将视角从棋盘转向语言世界，讨论另一个近年来引起巨大关注的方向：**如何利用强化学习训练出具有思考能力的大语言模型**。

近年来，以 OpenAI o1 和 DeepSeek-R1 为首的模型展示出前所未有的链式思考、推理与规划能力，而实现这一能力的核心技术之一，正是将强化学习思想迁移到语言模型上。本章将以 DeepSeek-R1 为例，系统介绍语言模型推理任务如何建模、面临哪些挑战、以及如何通过 GRPO 等强化学习算法实现具备思考能力的大语言模型。

接下来我们依次讨论如何将语言模型推理建模为强化学习问题及其相比传统强化学习（如围棋）所面临的固有挑战；介绍 DeepSeek-R1 的整体训练流程以及 GRPO（Group Relative Policy Optimization）算法如何实现高质量思维链增强。

语言模型推理如何建模为强化学习问题

尽管在第11.3节我们从强化学习视角对语言模型推理进行了简单的建模，在这里我们给出更具体和深入的分析。要让强化学习算法处理现实任务，第一步总是将任务形式化为一个马尔可夫决策过程（Markov Decision Process, MDP）。语言模型推理可以自然地对应为以下 MDP 结构：

- **状态 s** ：由当前已生成的自然语言序列构成，包括题目本身以及模型已经写下的部分思维链。状态随着每次生成新的 token 而不断扩展。
- **动作 a** ：在给定状态下输出下一个 token。动作空间即语言模型的词表，通常包含数万至数十万 token。
- **策略 $\pi(a | s)$** ：大语言模型充当策略函数，给定状态 s 时生成 token a 的条件概率分布。直观来说，从当前文本前缀（状态）预测下一个 token（动作）。
- **奖励 r** ：根据完整生成序列的质量给出的评价信号（实数），例如最终答案是否正确、推理链条是否清晰合理、格式是否符合要求等。由于

推理过程中的中间 token 难以单独评估，奖励通常只在序列结束时一次性给出，类似围棋中的终局胜负。

语言模型的状态空间由所有可能的文本前缀组成，动作空间由高维词表构成，而奖励既稀疏又延迟，且常包含噪声。接下来我们将详细分析语言模型推理任务所面临的挑战。

语言模型推理任务的挑战

将强化学习应用于语言模型推理任务面临一系列严峻的挑战：

- **高维且组合爆炸的状态与动作空间：**典型大语言模型的词表规模（即动作空间）可达到 10^5 量级；而状态是全部已生成 token 的序列，若最大长度设为 10^4 ，则状态空间规模约为 $(10^5)^{10^4} = 10^{50,000}$ 。这一数量级远超过围棋约 10^{500} 的状态空间。如此巨大的组合空间不仅使搜索难以开展，而且由于思维链本质上是一条动作序列，模型若在序列前段无法生成有效推理步骤，后续几乎无法得到有意义的反馈，从而严重阻碍了策略的改进。
- **奖励极度稀疏且高度延迟：**一条推理链往往包含数千甚至上万个 token，但训练时通常只能在生成结束后根据最终答案对错给出一次性奖励。中间步骤既难以标注也难以评估，导致整个轨迹的绝大部分决策都没有即时奖励。这样，策略梯度的信号几乎完全依赖最终反馈，造成训练不稳定的问题。
- **奖励存在显著噪声：**与围棋终局的胜负信号不同，语言模型推理任务中的“答案是否优质”、“解释是否合理”、“思维链是否清晰”等评价带有明显的主观性，往往需要依赖近似的奖励模型 (reward model) 进行评分。然而，奖励模型本身可能存在偏差、噪声或不一致，从而使得奖励信号并不可靠。噪声奖励不仅会增加策略优化的难度，还可能诱导

模型利用奖励模型的缺陷产生偏离真实目标的行为 (reward hacking)。当策略能力较弱时, 这种噪声的影响相对有限, 但随着模型变得越来越强, 奖励模型的偏差将逐渐成为性能上限的主要瓶颈。

综上所述, 语言推理任务中巨大的状态空间、极度稀疏且延迟的奖励结构, 以及存在显著噪声的奖励机制, 使得传统强化学习方法难以直接赋予模型稳健的推理能力。在接下来的章节中, 我们将介绍 DeepSeek-R1 的训练流程。尽管 R1 的整体系统包含诸多算法与工程细节, 本章聚焦其最核心的技术点——基于可验证奖励的强化学习方法, 并阐释其如何通过“采样—评估—优化”机制有效提升大模型的推理能力。

DeepSeek-R1 的整体训练流程

DeepSeek-R1 的训练建立在一个经过大规模预训练的大语言模型 **DeepSeek-V3-Base** 之上。预训练模型已经具备丰富的世界知识、语言能力与一定的推理潜力, 为后续强化学习阶段提供了良好的基础。这不仅显著缩小了有效搜索空间, 从而在一定程度上缓解了前文所述的“状态与动作空间巨大”的挑战, 也使得模型能够在强化学习开始时就生成结构化、具备一定合理性的推理链, 为策略优化提供可利用的起点。

R1 的核心思想之一是充分利用**可验证的奖励 (Verifiable Reward)**。对于数学题、逻辑题、代码执行类问题等答案具有明确可验证性的任务, R1 通过构造规则化的判别器或程序化的验证器直接判断生成答案是否正确, 从而绕过了依赖奖励模型所带来的噪声问题, 为强化学习提供了稳定、高质量的奖励信号。

具体来说, 给定一个问题与其标准答案对 (q, a) , DeepSeek-R1 的强化学习流程可概括为以下三个阶段:

1. **采样 (Sampling)**: 在给定问题 q 的条件下, 模型按照参数为 π_θ 的

策略分布，以较高温度 τ 采样生成 G 条样本：

$$y^{(i)} \sim \pi_{\theta}(\cdot \mid q; \tau), \quad i = 1, \dots, G,$$

其中 G 表示每轮采样所生成的轨迹数量。每条轨迹可写为

$$y^{(i)} = (t_1^{(i)}, t_2^{(i)}, \dots, t_{T_i}^{(i)}),$$

包含从推理过程到最终答案的完整生成序列。较高的温度增强了随机性，使模型在推理空间中进行更充分的探索，从而覆盖更多潜在的推理路径。

2. **评估 (Evaluation)**: 对每条轨迹 $y^{(i)}$ 的最终答案 $\hat{a}^{(i)}$ ，通过规则化的自动验证函数 $V(\cdot)$ 进行评分：

$$r^{(i)} = V(\hat{a}^{(i)}, a),$$

其中 a 为标准答案。若任务对思维链本身另有格式、结构或一致性等约束，可加入附加的规则化奖励 $R_{\text{format}}(y^{(i)})$ ，得到最终的轨迹级奖励：

$$R^{(i)} = r^{(i)} + R_{\text{format}}(y^{(i)}).$$

在可验证任务中，这些奖励完全由确定性规则与验证器产生，从源头上显著降低了奖励的噪声。

3. **策略更新 (Policy Update)**: 根据轨迹奖励 $R^{(i)}$ 对策略进行更新，使模型逐渐倾向生成更高质量的思维链与正确答案。

值得注意的是，这一流程在结构上与 AlphaGo Zero 的“搜索产生更强策略 → 网络学习改进策略”颇为相似。不同的是，AlphaGo Zero 的搜索依赖蒙特卡洛树搜索 (MCTS) 与自我对弈来获取高质量的终局信号；而 DeepSeek-R1 的“搜索”环节则由多样化的采样策略与可验证奖励共同完成，在无需显式构建搜索树的情况下依然能够实现对推理空间的有效探索。

这里读者可能会提出一个疑问：为什么 *DeepSeek-R1* 不使用 *MCTS*？关键在于语言推理任务难以像围棋那样为每一个中间状态提供明确且精准的价值评估。思维链的每一步都是词表中的某个 token，既难以精确给出选择当前 token 的奖励，也难以判断其对最终答案的真实贡献——模型在思考过程中甚至可以随时推翻先前的推理并重新开始，这使得中间步骤的价值难以稳定建模。由此，*MCTS* 难以构建可靠的树状搜索结构。因此，将 AlphaGo 式的价值评估与树搜索直接应用于语言推理几乎不可行。

上述三个阶段共同构成了 *DeepSeek-R1* 在可验证任务上的核心强化学习框架。尽管不同的强化学习算法（如 GRPO、PPO、GSPO 等）在策略更新目标以及正则化机制等方面存在差异，但它们整体上都遵循“采样—评估—策略更新”的统一范式。下面，我们将以 **Group Relative Policy Optimization (GRPO)** 为例，详细解析 *DeepSeek-R1* 在策略更新阶段中的具体机制。

DeepSeek-R1 的策略更新算法：GRPO

Group Relative Policy Optimization (GRPO) 是 *DeepSeek-R1* 采用的核心策略更新算法，其提出的动机源于对语言推理任务特性的系统性认知：该类任务的奖励往往只能在轨迹结束时给出，同时难以构建可靠的中间状态价值函数。具体而言，语言模型的推理链并不像围棋等决策任务具备清晰的单步优劣信号。模型在推理过程中可以随时推翻早期思路、重新组织推理步骤，从而使中间状态价值的定义变得模糊且高度不稳定。在这种缺乏明确过程监督的环境下，critic 很难学习到有意义的价值估计，甚至会因为误差累积而严重干扰策略更新。与传统强化学习中常用的近端策略优化 (PPO) 不同，GRPO 并未使用额外的价值网络来估计状态价值，而是通过同一批采样轨迹的“组内相对排名”构造优势函数，从根本上避免了价值网络在此类任务中常见的训练不稳定、价值偏移甚至崩塌等问题。GRPO 在可验证奖励场景中表现出更加稳定、高效且计算成本更低的优化特性，并在多种

推理任务中展现出优越的性能。

我们先来介绍 GRPO 是如何定义轨迹的“优势”的,紧接着介绍 GRPO 策略更新的优化目标。核心思想是:在同一个问题的一组采样轨迹中,以轨迹级奖励为参照,通过相对排名构建优势函数。设采样得到一组轨迹集合的奖励为 $\{r^{(i)}\}_{i=1}^G$, 则 GRPO 将优势定义为组内相对奖励的归一化形式:

$$A^{(i)} = \frac{r^{(i)} - \bar{r}}{\text{std}(r)}, \quad (13.74)$$

其中 $\bar{r}$ 和 $\text{std}(r)$ 分别为该组轨迹奖励的均值与标准差。该构造完全基于当前采样到的轨迹奖励本身,不需要任何价值网络参与,从而避免了价值函数近似带来的系统性偏差与过拟合风险;同时优化的焦点落在“组内谁更好、谁更差”这一相对关系上,而非奖励的绝对数值,使得训练过程对奖励尺度变化也更加鲁棒。

在策略更新方面,GRPO 的目标是:在一组采样轨迹内部,通过相对优势提升高质量轨迹的概率、抑制低质量轨迹的概率,并保持策略更新的稳定性。为了保证策略更新的稳定,GRPO 在优化目标中引入了两个关键部件:重要性比率 (importance ratio) 与 截断函数 (clip)。我们先来给出 GRPO 的优化目标,再详细解释二者的作用。

注意,理想情况下的在线强化学习应当做到“边采样边更新”,即每采样生成少量新数据就立刻进行策略更新。然而对于大语言模型,这种严格的在线学习的方式几乎不可行:采样和反向传播都极其耗费算力,如果每次更新策略都依赖极少量新样本,将导致硬件利用率极低;同时,采样和训练反复多次更替,也会消耗大量模型加载的时间。因此,往往采取“批量采样、批量更新”的方式:先用当前最新策略在多个问题上并行采样一批数据,然后利用这批数据进行多步的策略更新。如果采用这种方式,那么,采样数据时的模型参数往往已经滞后于当前策略。为了加以区分,我们将采样数据时的策略称为**旧策略**。

具体来说,在实际训练中,会不断从训练数据中采样一组问题,然后对一组问题批量进行“采样—评估—更新”的流程,以提高训练效率。为了便

于理解，下面我们主要以采样单个问题为例进行说明。对于问题 q ，我们使用旧策略 $\pi_{\theta_{\text{old}}}$ 并行采样 G 次得到 G 条轨迹：

$$\{y^{(i)}\}_{i=1}^G, \quad y^{(i)} = (o_{i,1}, o_{i,2}, \dots, o_{i,|y^{(i)}|}),$$

其中 $|y^{(i)}|$ 表示轨迹 i 的 token 数。对同一个问题 q 的这 G 条轨迹构成一个“组”，GRPO 的核心思想便是在该组内部进行相对比较并据此更新策略。

在这一设定下，对单个问题 q ，GRPO 最大化以下优化目标：

$$\mathcal{L}(\theta) = \mathbb{E}_{\{y^{(i)}\} \sim \pi_{\theta_{\text{old}}}(\cdot|q)} \frac{1}{G} \sum_{i=1}^G \frac{1}{|y^{(i)}|} \sum_{t=1}^{|y^{(i)}|} \left[\min(\rho_{\theta,i,t} A_{i,t}, \text{clip}(\rho_{\theta,i,t}, 1 - \epsilon, 1 + \epsilon) A_{i,t}) - \beta \mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}}) \right], \quad (13.75)$$

其中 $A_{i,t}$ 为轨迹级优势，其构造依赖于该组 G 条轨迹的奖励，如公式 (13.74) 所示。重要性比率 $\rho_{\theta,i,t}$ 的定义为：

$$\rho_{\theta,i,t} = \frac{\pi_{\theta}(o_{i,t} \mid q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} \mid q, o_{i,<t})}.$$

优化目标中额外包含的 KL 惩罚项 $\mathbb{D}_{\text{KL}}(\pi_{\theta} \parallel \pi_{\text{ref}})$ 用于衡量当前策略 π_{θ} 相对于参考策略 π_{ref} （通常是训练的初始模型）的偏离程度，其系数 β 控制该惩罚的强度。KL 项提供约束，使策略不会偏离参考策略过远，使得更新更加稳健。

由于训练时往往会从训练数据分布 $P(Q)$ 中的采样多个问题 q 批量进行上述优化，因此完整的训练目标就是对 $q \sim P(Q)$ 取期望的形式：

$$\mathcal{L}(\theta) = \mathbb{E}_{q \sim P(Q)} [\mathcal{L}_q(\theta)]. \quad (13.76)$$

为什么要引入重要性比率 $\rho_{\theta,i,t}$ ？

策略梯度的目标是最大化当前策略 π_θ 下的期望回报。以单个 token 为例，其优化目标可写为：

$$J(\theta) = \mathbb{E}_{o_{i,t} \sim \pi_\theta(\cdot | q, o_{i,<t})} [A_{i,t}].$$

若能直接从 π_θ 采样，则可以无偏估计该期望。但在实际训练中，由于大模型难以做到“边采样边更新”，我们通常在一次采样后执行多次梯度更新，因而用于更新的样本往往来自若干步更新前的旧策略 $\pi_{\theta_{\text{old}}}$ 。此时，我们需要将旧策略分布下的样本“转换”为当前策略分布下的样本。

在此，我们借助重要性采样的技巧。给定两个分布 $p(x)$ 和 $q(x)$ ，以及任意可积函数 $f(x)$ ，有：

$$\mathbb{E}_{x \sim p}[f(x)] = \int f(x)p(x) dx = \int f(x) \frac{p(x)}{q(x)} q(x) dx = \mathbb{E}_{x \sim q} \left[f(x) \frac{p(x)}{q(x)} \right].$$

这里 $p(x)$ 是我们希望计算期望的采样分布（对应当前策略 π_θ ）； $q(x)$ 是实际采样所来自的分布（对应旧策略 $\pi_{\theta_{\text{old}}}$ ）； $f(x)$ 是我们希望取期望的量（在此为 $A_{i,t}$ ）。

将 $p = \pi_\theta$ 、 $q = \pi_{\theta_{\text{old}}}$ 代入上式即可得到：

$$\mathbb{E}_{o_{i,t} \sim \pi_\theta} [A_{i,t}] = \mathbb{E}_{o_{i,t} \sim \pi_{\theta_{\text{old}}}} \left[A_{i,t} \frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})} \right].$$

因此定义重要性比率：

$$\rho_{\theta,i,t} = \frac{\pi_\theta(o_{i,t} | q, o_{i,<t})}{\pi_{\theta_{\text{old}}}(o_{i,t} | q, o_{i,<t})},$$

即可得出目标期望在旧策略分布下的无偏估计。换言之，引入重要性比率的根本目的是用旧策略采样得到的数据来无偏估计当前策略 π_θ 下的期望。若没有这一比率修正，梯度在统计意义上对应的是旧策略 $\pi_{\theta_{\text{old}}}$ 的分布，而非我们真正要优化的策略，最终会导致学习目标偏移和训练不稳定。

为什么要引入截断函数 $\text{clip}(\cdot)$ ？

如果直接使用 $\rho_{\theta,i,t} A_{i,t}$ 更新，当 $\rho_{\theta,i,t}$ 偏离 1 较远时，会导致策略在单个 token 上发生过大的更新，从而破坏训练稳定性。为此 GRPO（与 PPO 一致）引入了截断函数：

$$\text{clip}(\rho_{\theta,i,t}, 1 - \epsilon, 1 + \epsilon) = \begin{cases} 1 - \epsilon, & \rho_{\theta,i,t} < 1 - \epsilon, \\ \rho_{\theta,i,t}, & 1 - \epsilon \leq \rho_{\theta,i,t} \leq 1 + \epsilon, \\ 1 + \epsilon, & \rho_{\theta,i,t} > 1 + \epsilon. \end{cases}$$

然后在目标函数中取

$$\min(\rho_{\theta,i,t} A_{i,t}, \text{clip}(\rho_{\theta,i,t}, 1 - \epsilon, 1 + \epsilon) A_{i,t}).$$

其效果可以从两类优势理解：

- $A_{i,t} > 0$ (**好 token**)：应提高其概率，但若 $\rho_{\theta,i,t}$ 过大则会“更新过猛”。截断项提供上限 $1 + \epsilon$ ，使得提升幅度受到限制。
- $A_{i,t} < 0$ (**差 token**)：应减少其概率，但若 $\rho_{\theta,i,t}$ 过小也会导致“更新过猛”。截断项提供下限 $1 - \epsilon$ ，使策略下降不至于过度。

综上，通过**重要性比率** $\rho_{\theta,i,t}$ 与 **截断操作** $\text{clip}(\cdot)$ 的结合，GRPO 在保持“以优势导向的策略更新”的同时，将策略变化限制在一个可控的范围内，从而实现稳定的策略优化。

为了便于理解，我们给出 GRPO 的整体训练伪代码，如算法9所示。可以将每一轮训练看作两个阶段：先用旧策略采样并评估，获得一个训练数据集 $\mathcal{D}_b^{\text{RL}}$ ；再在该数据集上进行 N 步策略更新。

DeepSeek-R1 的训练闭环

将采样、评估与策略更新三个阶段结合起来，DeepSeek-R1 构建了一个稳定而高效的**强化学习闭环**：模型首先对给定问题生成多条推理链，随后由可

验证奖励机制对生成结果进行评分，接着 GRPO 根据组内相对质量对策略进行优化，得到的更强模型再用于下一轮采样，如此循环往复，推理能力不断提升。

这一整体结构与 AlphaGo Zero 的“自我博弈—学习”机制在思想上高度一致，但也具有显著差异。AlphaGo Zero 依靠蒙特卡洛树搜索在棋盘状态空间中进行显式搜索，而 DeepSeek-R1 则利用高温采样在语言空间中进行“隐式搜索”；围棋中的价值网络用于评估棋局优劣，而 R1 仅利用可验证奖励对最终答案进行程序化判断；AlphaGo 依靠拟合搜索后的最优策略分布不断改进棋力，而 R1 中的 GRPO 则以组内相对排序为基础不断优化策略。两者都通过“搜索产生更优行为—网络吸收并放大这些行为”的方式形成自增强循环，但 R1 的闭环更适配自然语言推理这种难以构建显式状态价值、无可观察过程监督的任务形式。

通过多轮闭环迭代，模型不仅能生成正确答案，还逐渐学会构造清晰的推理链，自动分解问题、规划步骤，从而形成类似人类的深度思考过程。这也解释了为何 DeepSeek-R1 与 OpenAI o1 系列模型能够展现出远超传统 LLM 的推理能力。

小结

本节介绍了如何将强化学习思想系统地应用于语言模型的推理能力训练，并以 DeepSeek-R1 为例阐明了思维链强化学习的核心机制。我们将语言推理过程建模为一个高维且奖励稀疏的 MDP，并通过“多样采样 + 可验证奖励”的方式在语言空间中构建了可行的搜索机制。随后，GRPO 利用组内相对排序来稳定地改进策略，使模型在每一轮迭代中都能更倾向于生成高质量的推理链。最终，采样—评估—策略更新形成的闭环训练，使模型能够在没有过程监督的情况下持续提升推理能力。

这一范式展示了强化学习在语言智能方向的巨大潜力：**只要具备充足的计算、数据资源与合理的奖励结构，大模型便能够通过强化学习不断提**

升推理能力，逐步形成类似人类的“思维链”能力。

Algorithm 9 Group Relative Policy Optimization (GRPO)

Require: 初始策略模型 $\pi_{\theta_{\text{init}}}$, 训练集 $\mathcal{D}$, 超参数 G, ϵ, β, M, N 初始化策略参数: $\theta \leftarrow \theta_{\text{init}}$ 设置参考模型: $\pi_{\text{ref}} \leftarrow \pi_{\theta}$ **for** iteration = 1, ..., M **do**从训练集采样一个 batch: $\mathcal{D}_b \subset \mathcal{D}$ 更新旧策略: $\pi_{\theta_{\text{old}}} \leftarrow \pi_{\theta}$ 初始化本轮训练数据集 $\mathcal{D}_b^{\text{RL}} \leftarrow \emptyset$ **for** 每个问题 $q \in \mathcal{D}_b$ **do** **采样 (由旧策略执行):** 采样 G 条轨迹 $\{y^{(i)}\}_{i=1}^G \sim \pi_{\theta_{\text{old}}}(\cdot | q)$ **评估:** 使用可验证规则或奖励模型计算轨迹级奖励 $\{r^{(i)}\}_{i=1}^G$ 基于组内相对奖励构造每个 token 的优势 $A_{i,t}$ (见式 (13.74)) 将 $\{(q, \{y^{(i)}\}, \{A_{i,t}\})\}_{i=1}^G$ 加入 $\mathcal{D}_b^{\text{RL}}$ **end for****在采样得到的 $\mathcal{D}_b^{\text{RL}}$ 上进行多步策略更新:****for** update step = 1, ..., N **do** 从 $\mathcal{D}_b^{\text{RL}}$ 抽取一个小批量样本 根据式 (13.76) 计算 GRPO 损失 $\mathcal{L}(\theta)$ 执行梯度上升更新: $\theta \leftarrow \theta + \alpha \nabla_{\theta} \mathcal{L}(\theta)$ **end for****end for**

对行为主义部分的总结

强化学习作为人工智能领域的第四部分内容，是行为主义 AI 的当代延续。其核心思想在于，智能体的知识来源于通过与环境的交互和实践，并以最大化长期回报为目标来获得行动策略。强化学习旨在解决序列决策过程问题，其中智能体在一连串前后关联的时间点上依次做出动作选择，目标是使整个过程结束后获得的总奖励（回报）最大化。

强化学习的序列决策过程被形式化定义为马尔可夫决策过程 (MDPs)。一个 MDP 通常由状态、动作、环境模型、奖励和策略函数等组成。其中，策略函数是根据观测到的当前状态做出决策的规则。现实中的强化学习问题，例如围棋或训练具有推理能力的大语言模型，普遍面临状态空间巨大、奖励稀疏且延迟极长的挑战，且环境模型通常是未知的，属于灰盒环境模型，大多要求进行在线学习（一边生成交互数据一边学习）。

在理论层面，如果环境模型完全已知（白盒环境模型），且状态离散，那么强化学习问题可以通过求解满足贝尔曼方程的价值函数（例如状态价值函数）来解决，并利用策略提升等迭代算法获得最优策略。对于连续状态，则需要使用近似算法，如拟合值迭代。

在实践层面，针对灰盒环境模型，我们首先考虑没有状态的简化问题，如多臂老虎机问题；其核心挑战在于探索 (Exploration) 与应用 (Exploitation) 的权衡，常使用 ϵ -贪心或置信上界 (UCB) 等在线学习算法来近似最大化即时奖励的均值。对于有状态的复杂问题，强化学习主要沿着两条路径发展：

- 价值学习：通过近似动作价值函数 $Q(s, a)$ 来指导决策，通常使用深

度 Q 网络 (DQN) 的时序差分 (TD) 算法, 即用 “当前奖励 + 未来回报的估计” 来更新当前的价值预测。经典的算法包括 Q-learning (off-policy, 更新目标基于下一状态的最优动作的价值估计) 和 SARSA (on-policy, 更新目标依赖于实际执行的下一个动作的价值估计)。

- 策略学习: 直接对策略函数 $\pi(a|s; \theta)$ 建模, 通过策略梯度优化参数 θ 使长期回报最大化。基础算法如 REINFORCE 使用经验回报近似价值, 但易受高方差影响, 通常需要引入 *baseline* 降低方差。更稳定的 Actor-Critic 框架引入一个独立的评论家网络 (Critic, 即价值函数) 和行动者网络 (Actor, 即策略网络) 的更新, 使训练更高效稳定。

在复杂应用围棋程序 AlphaGo Zero 中, 为应对巨大的状态空间和延迟奖励, 系统实现了策略和价值合并的网络, 来同时输出落子策略和局面价值, 并通过蒙特卡洛树搜索在自我博弈中生成高质量的训练数据, 使网络通过逼近搜索改进后的策略, 不断实现棋力的自我强化。这一范式表明, 强化学习结合深度学习和搜索技术, 能够在复杂博弈中实现超越人类的智能。此外, 强化学习思想也被应用于训练具有思考能力的大语言模型, 将思维链生成建模为 MDP, 尽管面临着动作空间极其庞大和奖励高度稀疏的挑战, 但通过多样化采样和策略优化的 GRPO 方法, 能够提升模型的系统推理能力。

总而言之, 行为主义的强化学习是通过计算而获得的智能, 它依赖于搜索算法作为技术底座来实现决策过程的优化。

第五部分

AI 的回顾与总结：苦涩的教训

在学习了搜索算法和人工智能的三大主义后，我们已经了解了大部分人工智能学科的基础知识。事实上，人工智能的发展史就是人类对“智能是如何产生的”这个问题的探索历程。符号主义认为人类的知识可以归纳为符号操作，运用逻辑推演公理从而形成了现有的知识体系。因此，用知识表示的方法让计算机理解现实中问题，然后用自动推理的方法让计算机解决问题，就可以看作计算机拥有了人类的智能。联结主义认为智能来自于人脑结构，通过让电脑来模拟神经结构来获取知识，因此发明了许多神经网络模型，通过大量样本的训练来让计算机获得智能。而强化主义又是一种新的思路，认为知识是通过实践得来的，因此设计奖励机制来指导计算机往获得最大奖励的方向优化让计算机获得智能。然而无论哪种方式，它们都离不开搜索算法来实现，因此让搜索算法更高效也是人工智能领域里一个非常重要的研究内容。

而在这一部分中，我们会站在历史的角度，来回顾人工智能的发展历程，介绍在人工智能各流派中具有代表性的里程碑事件，然后总结我们能从人工智能发展历程中得到的经验。

第十四章 AI 发展的历史回顾

14.1 计算世界的双城记：效率与智能的相爱相杀

计算机科学 (Computer Science, CS) 的历史，是一部由两位主角共同出演的戏剧：一位是脚踏实地、追求效率与可靠性的现实主义者；另一位是仰望星空、探索智能与无限可能的浪漫主义者。这两股力量从上世纪中叶开始，便以一种既竞争又合作的“双螺旋”模式，塑造了我们今天所依赖的数字宇宙。这种划分不仅是研究领域的分类，更体现了计算机科学家们在构建人类数字文明时的两种根本哲学取向。

奠基：计算理论与第一台机器的诞生

计算机科学的源头可以追溯到 20 世纪 30 年代艾伦·图灵的伟大理论工作。图灵通过提出图灵机 (Turing Machine) 这一抽象数学模型，从根本上定义了“计算”的可能性与边界，为现实主义者和浪漫主义者都设定了最初的舞台。图灵机的概念本身就蕴含了两种张力：它的确定性和通用性为现实的机器构造提供了蓝图；而它所能模拟的逻辑推理能力，则暗示了机器智能的潜力。随着二战的推进，现实的需求加速了理论向实践的转化，ENIAC 等第一批通用电子计算机被建造出来。这些机器的出现，标志着浪漫的设想开始拥有了坚实的物理载体，同时也将计算机科学研究明确地分化为两条路径。

现实主义者的使命：构建高效、可靠的基础设施

现实主义者的核心目标，可以概括为“让计算更易用、更快、更便宜”。他们是计算机产业化和普及化的中坚力量，专注于突破物理和工程上的限制，为所有计算活动提供高效、稳定、可管理的数字基础设施。他们的工作具有强烈的迭代性、工程性和实用性，成果直接体现在计算机的性能提升和用户体验的改善上。

硬件的极限突破与摩尔定律的驱动

现实主义科学家在硬件领域的努力是推动整个 CS 发展的引擎。从电子管到晶体管，再到集成电路，他们遵循着摩尔定律 (Moore's Law)，不断将数百万甚至数十亿的晶体管集成到微小的芯片上，实现了中央处理器 (CPU) 性能的指数级增长和内存 (RAM) 容量的飞速扩大。每一次架构的更新、每一次制程的优化，都显著提高了机器处理数据的能力和能效比。与此同时，他们也致力于解决机器与人交互的难题。人机交互 (HCI) 的革命性创新，如鼠标和键盘的发明、图形用户界面 (GUI) 的普及，将计算机从实验室的复杂设备转化为大众可以直观操作的工具，是现实主义者致力于提升易用性的典范。

软件的抽象层与操作系统的崛起

为了驾驭日益复杂的硬件，现实主义者创造了越来越高级的软件抽象层。他们发明了高级程序语言 (如 C、Java 等)，使得程序员不必再直接面对底层的机器指令，可以专注于算法，极大地提高了开发效率和软件的可维护性。而他们的巅峰之作，无疑是操作系统 (Operating System, OS)。操作系统不仅仅是资源的管理者，更是将冰冷硬件包装成友好平台的中介。从早

期的批处理系统到分时共享系统，再到今天的多任务、多用户 OS，现实主义者通过发明进程管理、文件系统和内存虚拟化等复杂机制，确保了资源的公平分配、系统的稳定运行和用户界面的友好交互。此外，数据库管理系统 (DBMS) 的建立，解决了现实世界中海量数据的存储、索引和高效检索问题，成为商业和科学研究的信息生命线。

浪漫主义者的远征：探索机器智能的边界

浪漫主义者的核心目标，可以概括为“让机器能思考、能学习、能创造”。他们受图灵关于机器智能的早期设想的启发，致力于赋予机器类人甚至超越人类的智能能力，去完成那些传统上认为只有人类才能胜任的、需要“心智”参与的任务。他们的研究一开始就具有强烈的理想主义、挑战性和理论前瞻性，往往在硬件基础尚不成熟时就已开始布局。

早期的符号主义与人工智能的寒冬

浪漫主义者的旅程始于 1956 年的达特茅斯会议，人工智能的概念正式提出。早期的 AI 研究主要采用符号主义 (Symbolicism) 路径，相信智能的核心在于对知识和推理规则的精确表示和逻辑操作。他们尝试构建“专家系统”，通过编码人类领域的专业知识，让计算机进行自动推理和问题求解。著名的例子包括让机器证明数学定理，以及开发能够下跳棋的程序等。然而，受限于当时的硬件速度和存储容量，以及难以将现实世界的所有复杂知识都编码成精确的逻辑语言，早期的 AI 发展遭遇了瓶颈，被称为“AI 寒冬”。尽管如此，浪漫主义者对机器智能的根本性探索从未停止，他们为后来的突破积累了理论和算法的火种。

深度学习的爆发与对人类智能的超越

21 世纪，随着现实主义者提供的计算能力（尤其是高性能的 GPU）和海量数据（互联网和云计算的产物）达到临界点，浪漫主义者迎来了黄金时代。他们转向了连接主义 (Connectionism)，并引爆了深度学习技术。不再是手动编写规则，而是让多层神经网络通过学习数据中的模式来自动提取特征和进行决策。这一转变带来了人工智能的里程碑式飞跃：2016 年 AlphaGo 战胜围棋世界冠军，展示了机器在复杂、直觉性任务中对人类的超越。今天，大型语言模型 (LLMs) 的出现，实现了自然语言的流畅理解和生成，这是对早期浪漫主义者在自然语言处理领域梦想的最高实现。

双螺旋的共舞与永恒的迭代

计算机科学的历史清晰地展现了现实主义者与浪漫主义者之间的共生关系。浪漫主义者的大胆想象（如深度学习、大规模并行计算）不断提出新的、对计算资源极度饥渴的需求，这反过来成为驱动现实主义者进行下一代硬件、系统和基础设施研发的原动力。每一次现实主义者在速度、容量和能效上的突破，都为浪漫主义者开启了新的实验空间，使其能够探索更大、更复杂的模型。

最终，无论是追求更快的 CPU 频率，还是探索更强的通用人工智能，计算机科学的未来仍将是这两股力量的永恒迭代。现实主义者将继续为计算提供坚实的、高效的土壤；而浪漫主义者则将不断播撒智能和创意的种子，共同引领人类走向一个更加智能化和数字化的未来。

14.2 计算机科学的浪漫主义：人工智能里程碑回顾与流派交融

早期的计算机科学家们怀抱一种“浪漫主义”的豪情壮志，坚信机器不仅能计算，更能思考。这些“天马行空”的设想催生了人工智能发展史上最激动人心的里程碑工作。回顾这些工作，我们能清晰地看到人工智能的

三大流派——符号主义、联结主义、行为主义——如何各自探索、相互交融，共同推动了智能革命。

理性之光：让计算机证明和发现数学定理（符号主义的荣耀与瓶颈）

在人工智能的早期，符号主义 (Symbolism) 主导了研究方向，其核心信念是人类智能源于对符号和规则的逻辑操作。这一流派的开创性工作便是自动定理证明。1956 年，艾伦·纽厄尔、赫伯特·西蒙和约翰·肖合作开发的“逻辑理论家” (Logic Theorist)，成功证明了《数学原理》中的大量定理，被视为 AI 历史上的重要里程碑。随后，约翰·艾伦·罗宾逊在 1965 年提出了极具影响力的归结原理，将复杂的一阶谓词逻辑推演统一为单个规则，极大地简化了计算机的推理机制，也为逻辑编程语言（如 Prolog）奠定了基础。在中国，吴文俊先生则将中国传统数学的构造性思维与西方代数几何结合，开创了几何定理自动证明的有效算法。从吴文俊的工作中，我们可以看到研究定理自动证明有效的方法是找到一个相对可控的子领域，然后针对这个子领域的特性，找到有效的算法。

然而，符号主义遇到了其最大的瓶颈：知识表示。面对现实世界中浩瀚、复杂的知识体系，如何将它们自动化、大规模地转化为计算机可操作的精确符号，成了一道难以逾越的鸿沟。尽管符号主义推动了搜索技术（特别是启发式搜索 A* 算法）的发展，但其固有的指数级时间复杂度仍是巨大挑战。时至今日，虽然大语言模型（联结主义的产物）开始辅助数学证明，但仍需人的干预和修正，这正说明了符号主义在形式逻辑上的深远价值，以及当前领域需要探索符号与联结融合的最佳路径。

智慧之巅：人机对弈的演进（对抗搜索、联结主义与行为主义的交织）

人机对弈一直被视为衡量机器智能的试金石，而其成功是联结主义与行为主义（以试错和强化学习为代表）共同作用的结果。人机对弈的历史经历了从跳棋、国际象棋到围棋的演进。1949 年，亚瑟·塞缪尔 (Arthur Samuel)

在 IBM 第一批商用计算机 IBM701 上编写出世界上第一款走国际跳棋的程序。1997 年，IBM 的“深蓝”击败国际象棋大师卡斯帕罗夫。2016 年，谷歌 DeepMind 的 AlphaGo 击败围棋世界冠军李世石。每一次胜利都伴随着技术的跨越式发展。

从方法上讲，棋类游戏本质上是对抗搜索问题。理论上，我们可以运用 MINIMAX 搜索算法计算每一个棋局（状态）的效用（即强化学习中定义的状态价值），以此获得落子的最佳策略。然而，巨大的挑战在于随着游戏复杂度的提升——从跳棋、国际象棋到围棋——可达状态的数量呈指数级爆炸式增长，使得对所有可能状态的穷举搜索变得不切实际。“深蓝”在国际象棋上的胜利，是经典对抗搜索技术（尤其是 $\alpha - \beta$ 剪枝算法）结合顶尖计算机硬件性能的胜利。它虽然高效，但本质上仍是基于穷尽式搜索的优化。相比之下，AlphaGo 在围棋上的成功，则标志着一场强化深度学习的革命性胜利。它巧妙地解决了巨大的状态空间的难题：通过使用大规模参数的深度神经网络来参数化表示状态价值函数，从而能够存储和学习超大量棋局的价值信息；同时，它创造性地引入了蒙特卡洛树搜索方法，这个方法作为一种高效的启发式搜索策略，能够引导系统更有针对性地探索和选择“高质量”的状态进行价值学习，极大地提升了强化学习在超大状态空间下的学习效率。因此，AlphaGo 实现了联结主义与行为主义的完美结合。

人机对弈的成功不仅证明了机器的计算和搜索能力，更证明了深度学习可以在海量状态空间中抽象出“智慧”的价值判断。如今，这个方面的研究已转向更复杂的部分可观测的游戏（如德州扑克）和多主体协同与对抗的游戏（如星际争霸），进一步推动着强化学习与多智能体系统的发展。

语义突破：机器翻译的范式革命（联结主义的胜利）

机器翻译能力的实现，一直被视为机器智能超越人类智慧的显著标志，因此它长期以来都是人工智能领域的核心焦点。

机器翻译的发展历程可以清晰地划分为三次重大的技术范式转换。第

一个阶段始于上世纪 50 年代计算机科学的浪漫主义初期，当时的研究人员对机器的能力抱持着极度的乐观，甚至有著名学者断言机器翻译可以在“十年内”轻松解决。这一时期的核心是基于规则的机器翻译 (Rule Based Machine Translation, RBMT)，即语言学家手工编写浩如烟海的词典和语法规则，试图通过一套固定的逻辑来解析和生成语言。然而，人类语言固有的复杂性、大量的歧义、习语以及数不尽的语法例外，远远超出了早期模型的处理能力，导致系统维护成本极高，而翻译质量却始终徘徊不前。随着 1966 年 ALPAC 报告的悲观评估，这一过度乐观的浪潮受挫，研究资金锐减，标志着机器翻译领域的第一轮“寒冬”。

第二个阶段从 1980 年代末拉开序幕，研究范式从依赖人工设定的规则转向依赖数据，催生了统计机器翻译 (Statistic Machine Translation, SMT) 的主导地位。SMT 通过学习海量的双语语料库，利用概率模型来计算源短语被翻译成目标短语的概率，并搜索出最有可能的翻译组合。尽管 SMT 在准确性和流畅性上相比基于规则的方法有了巨大的提升，并成为了长达二十年的主流，但其固有的局限性也使其难以触及人类翻译的水平。SMT 系统主要关注局部短语的翻译，缺乏对全局语义和长距离依赖的有效捕捉，翻译结果往往显得生硬、缺乏连贯，如同“词对词”的拼凑。因此，SMT 虽然是机器翻译发展中的关键一步，但它未能根本性地解决翻译的质量和自然度问题，从而促使了下一代革命性技术的诞生。

第三个阶段是自 2010 年代以来的神经网络机器翻译 (Neural Machine Translation, NMT)。得益于深度学习和大规模并行计算能力的成熟，研究人员采用了端到端的神经网络模型。这一方法，特别是基于 Transformer 架构和 Attention 机制的模型，能够将整个源语句编码为一个高维的语义向量，再由解码器生成目标语句。这一变革极大地提升了机器对句子上下文和长距离依赖的理解能力，使得译文的连贯性、流畅性和自然度都达到了前所未有的高度。NMT 成功克服了前两代技术在语义理解和流畅度上的瓶颈，使机器翻译的性能在许多通用领域达到了与人类专业译员相匹敌的水

平，标志着机器翻译技术获得了迄今为止最重大的成功。

显然，机器翻译领域的突破是联结主义 (Connectionism) 人工智能的巨大胜利，证明了深度学习在处理复杂、高维的自然语言任务中的卓越能力。

感知世界：多模态智能的深度融合 (联结主义的整合)

机器理解语言、图像、音频、视频等多模态信息的研究，经历了从独立处理到深度融合的三个主要阶段，其核心目标是模仿人类通过多种感官输入来感知世界和做出判断的能力。

第一阶段是早期的独立与浅层融合，在 2000 年代及以前，研究主要聚焦于单一模态，如自然语言处理和计算机视觉分别发展，而多模态任务通常是通过设计复杂的规则系统或使用经典机器学习方法，对来自不同模态的特征进行提取，然后通过简单的线性或非线性函数在决策层进行后期融合 (Late Fusion)，例如，分别识别图像中的物体和文本中的关键词，最后将它们的判断结果进行组合。这一方法简单直观，但由于各模态间的特征空间是割裂的，难以捕捉深层的语义关联和时间同步信息。

第二阶段是基于深度学习的特征对齐与联合嵌入，从 2010 年代中期开始，随着深度学习的兴起，研究人员开始利用神经网络为不同模态设计专门的编码器（例如，使用卷积神经网络 CNN 处理图像，使用循环神经网络 RNN 处理文本）。这一阶段的关键突破在于引入了联合嵌入空间 (Joint Embedding Space) 的概念，目标是将不同模态的数据投影到一个共享的向量空间中，使得语义相关的图像和文本（例如，一张猫的照片和“一只猫在沙发上”的描述）在向量空间中距离相近。像 CLIP 和 ALIGN 这样的模型通过对比学习 (Contrastive Learning) 在海量图像-文本对上进行训练，极大地提升了跨模态检索和零样本 (Zero-shot) 识别的能力，实现了强大的中期融合 (Intermediate Fusion)，为机器理解跨模态概念打下了基础，但模型结构通常仍是为每个模态独立编码后，在嵌入层进行连接。

第三阶段是基于统一架构和自注意力机制的深度交互，进入 2020 年代，

多模态研究转向以 Transformer 为代表的统一架构。这一阶段不再满足于仅仅对齐不同模态的特征，而是追求在更深的层次上实现模态间的深度交互和早期融合 (Early Fusion)，通过引入统一的自注意力机制 (Self-Attention)，模型可以在同一网络层中同时处理和交叉引用来自文本、图像块、声音片段等不同模态的信息，从而构建出一个连贯的、跨模态的上下文理解。例如，GPT-4o 和 Gemini 等先进的多模态大模型 (Multimodal Large Models, MMLMs) 能够原生、无缝地处理和生成各种模态，不仅能回答图像中的问题，还能理解视频中的事件，这标志着机器已经开始向着更接近人类认知的、真正统一的多模态智能迈进。

显然，这些工作的胜利都是联结主义 AI 的胜利。当前，这些方面的研究正在飞快的发展。其中，强化学习正在被广泛的应用进来，让大语言模型具备更多思考的能力。

人工智能的哲学探索：三大流派对“智能起源”的追问与融合

人工智能的发展史，实质上是人类对“智能是如何产生的”这一根本问题的探索历程，并由此衍生出了三大主要学派，各自提出了不同的实现路径。

符号主义 (Symbolism) 的哲学基石在于相信人类知识可以被归纳为精确的符号，并通过形式推演系统可靠且完备地推理出所有知识。该学派的核心目标，是构建一个能解决一切问题的“通用问题求解器”。

联结主义 (Connectionism) 则持完全不同的观点，认为任何所谓的“智能”都可以建模为一个从输入到输出的复杂映射函数。这些映射可以通过大规模参数的神经网络来表示，而模型参数本身则被视为一种内隐的知识，通过基于大量训练数据的最优化方法获取。联结主义的核心研究问题在于：探索何种形式的神经网络结构更适合建模各类智能功能，以及如何设计高效的参数学习方法。

而行为主义 (Behaviorism) 坚持知识是通过实践与环境的交互而获得的。其核心问题是通过接收环境的即时奖励信号，学习到能够最大化长期

价值的最优策略。在当前，行为主义已经与联结主义深度融合，形成了深度强化学习这一强大的工具。

无论哪个 AI 学派，其内部的推理或决策过程都离不开搜索算法来实现。因此搜索被视为实现人工智能的技术底座，持续提升搜索算法的效率始终是人工智能领域的基础研究问题。

综上，AI 的三大流派有着各自的基本假设和核心研究方向，它们都在尝试从不同的角度逼近智能的本质。近几年，随着联结主义 AI 凭借其强大的通用性取得了巨大成功，各种神经网络方法已开始渗透和赋能符号主义和行为主义的研究。因此，这三大流派的工作正呈现出越走越近、深度交融的局面，而非早期的针锋相对，并共同推动着通用人工智能的进步。

第十五章 人工智能发展的“苦涩的教训”



图 15.1: Richard S. Sutton，现代强化学习之父，现任加拿大阿尔伯塔大学教授，2024 年图灵奖获得者

回顾 AI 发展的历程，有些工作并未真正解决问题，而有些工作取得了

巨大的成功。从这些工作中我们能否总结出一些经验和教训，以指导我们未来的研究？理查德·萨顿（如图15.1）于2019年3月13日发布的博客文章《苦涩的教训》（The Bitter Lesson）的原文中文版¹，正是试图回答这个问题。

在这个文章中，他在总结了人工智能70年的历史中发现，发展人工智能的当前最“奏效”的方法就是：

放下将已有的人类知识灌输给电脑的想法，而是开发能捕获外部世界复杂性的通用方法，这样才能更好的利用电脑的计算能力来扩展智能。

因此，我们当前获得的智能都是计算而来的智能，即 Intelligence through Computing。

这篇文章值得每一位对人工智能感兴趣的同仁反复阅读。因此，把中文的翻译放在本书的末尾，与各位同仁共勉！

苦涩的教训

过去70年人工智能研究领域最重要的一堂课是，只有通用计算方法最终是最有效的，而且优势巨大。根本原因是摩尔定律，更确切地说是，每个计算单元的成本持续呈指数下降。大多数人工智能研究都是假设 Agent 可用的计算量是恒定的（在这种情况下，利用人类知识将是提高性能的唯一方法之一），但是，从较长的时间看，不可避免地会产生大量的计算量。为了在短期内获得有所作为的改善，研究人员试图利用该领域内的人类知识。但长远来看，唯一重要的是利用算力。这两者不必相互对立，但在实践中它们往往会相互对立。时间投入到一种方法上，就没办法投入到另一种方法上，这是对投入一种或另一种方法的心理承诺。而人类知识往往很复杂，不太适合利用好通用算力。有很多的例子表明人工智能研究人员迟迟未能学习这个苦涩的教训，回顾一些最突出的例子很有启发性。

在电脑国际象棋中，1997年击败世界冠军卡斯帕罗夫的方法基于大规模的深度搜索。当时，大多数计算机国际象棋研究人员一直致力于利用人

¹英文原文 The Bitter Lesson, 请访问 <http://www.incompleteideas.net/IncIdeas/BitterLesson.html>

类对国际象棋特殊结构的理解来改善方法。当一种更简单的、基于搜索的、结合特殊的硬件和软件的方法，被证明更有效时，这些基于人类知识的国际象棋研究人员没有虚心接受失败，他们反驳道，“蛮力”搜索可能这次赢了，但这不是一种通用的策略，而且也不是人们玩国际象棋的方式。这些研究人员希望基于人类的行棋思路能够获胜，当没有获胜时他们感到失望。

在电脑围棋中，也出现了类似的研究进展，只是比国际象棋晚了 20 年。最初的巨大努力是想办法利用人类知识（一千年的棋谱），或围棋游戏的特殊特征，避免用蛮力搜索，但是所有这些努力都被证明是无关紧要的。更糟糕的是，一旦有效地进行了大规模搜索，以及，使用自我对局来学习价值函数（就像在许多其他游戏中一样，在国际象棋中也是如此，尽管这种学习在 1997 年首次击败世界冠军的项目中并没有发挥重要作用），之前的努力都是负向的。通过自我对局以及通用学习来学习，就像搜索一样，因为它能够进行大规模的计算。在电脑围棋、电脑国际象棋中，研究人员最初的努力是利用人类的理解（更少的搜索），直到很久以后，通过拥抱搜索和学习，才取得更大的成功。

在语音识别方面，在 1970 年代，由 DARPA（美国国防部高级研究计划局）赞助了一项早期的竞赛。参赛者包括许多利用人类知识的特殊方法——单词、音素、人类声道等等知识。另一方面是更具统计学意义的方法，基于隐马尔可夫模型进行了更多的计算。再次，统计学方法战胜了基于人类知识的方法。这导致了几十年来所有自然语言处理的重大变化，统计学和计算逐渐开始主导该领域。最近深度学习在语音识别领域的兴起是朝着这一方向迈出的最新一步。深度学习方法对人类知识的依赖程度更低，使用更多的计算，加上在巨大的训练集上的学习，以产生更好的语音识别系统。正如游戏中一样，研究人员总是试图让系统按照他们自己的大脑工作的方式工作——他们试图将这些知识放入他们的系统中——但事实证明，当大规模计算变得可用并找到了一种充分利用摩尔定律的方法时，这最终会适得其反，并且浪费了研究人员的时间。

在计算机视觉领域，也存在类似的模式。早期的方法把视觉想象成寻找边缘，或广义柱，或以 *SIFT* 特征的形式。但今天，这一切都被抛弃了。现代深度学习神经网络仅使用卷积和某些不变性的概念，并且表现得更好。

这是一个重要的教训。纵观整个人工智能领域，我们仍然没有彻底地吸取它，因为我们继续犯同样的错误。为了看到这一点，并有效地抵制它，我们必须理解这些错误的吸引力。我们必须吸取苦涩的教训，即把我们认为的思维方式构建到系统中是行不通的。

苦涩的教训是基于历史观察：

- 1) 人工智能研究人员经常试图将知识构建到他们的 *Agent* 中；
- 2) 这在短期内总是有帮助的，并且对研究人员来说是个人满意的；
- 3) 但从长远来看，它总会达到一个瓶颈，甚至会阻碍进一步的进展；
- 4) 突破性的进展最终是通过一种相反的方法实现的，这种方法基于通过搜索和学习来扩展大规模计算。

最终的成功中夹杂着一丝苦涩和消化不良，因为它比受青睐的、以人为本的方法更成功。

从苦涩的教训中学到的一件事是通用方法的巨大力量，即随着可用算力变得非常大，这些方法会随着计算量的增加可以继续扩展。可以以这种方式近乎无限扩展的两种方法是搜索和学习。

从苦涩的教训中学到的第二个普遍观点是，人类心灵的实际内容是极其复杂的，我们不应该再试图找到简单的方法来思考心灵的内容，比如简单地思考空间、物体、多重主体或对称性。所有这些都是任意的、内在复杂的外部世界的一部分。它们不应该被构建，因为它们复杂性是无止境的；相反，我们应该只构建可以找到和捕获这种任意复杂性的元方法。这些方法的关键是它们可以找到很好的近似值，但算法应该是基于我们的方法，而不是我们已经学到的知识。我们希望人工智能 *Agent* 能够像我们人类一样去发现，而不是在系统里集成我们已经发现的东西。建立在我们已知发现之上只会让我们更难看到如何完成发现过程。